

Sweetheart Core

System Manual & Architecture Reference

Generated: 2026-07-30

A comprehensive guide to operating and extending the Sweetheart multi-agent ecosystem.

Contents

Sweetheart Core	1
01. Introduction	3
00. Quickstart & Overview	4
01. Telegram Interface	5
02. OpenWebUI Interface	6
03. World Interface & Scripts	7
04. Dashboard Interface	8
05. Hardware & External Integrations	9
20. FSM & Emotional State Model	10
21. Memory Architecture	11
22. Social & Hobby Simulation	12
23. Gossip Propagation	13
24. Audio Perception	14
25. Vision & Images	15
26. Group Chat Referee	16
27. Calendar Agency	17
28. Background Workers	18
29. Metric History & Trends	19
30. Notebook Registry	20
APPENDIX A: Env Var (Knob) Reference	21
APPENDIX B: Developer Info	34
APPENDIX C: Repository Atlas	48
APPENDIX D: Architecture Info	107
APPENDIX E: Architecture Decision Records	113

01. Introduction

What

The Sweetheart Core system is a locally-hosted, multimodal AI ecosystem simulating the life, relationships, and routines of an autonomous agent named Jane and her virtual friends.

Why

To create an artificial companion ecosystem that operates completely offline, retains persistent partitioned memories, and engages in long-term autonomous goal execution and social simulation without relying on third-party cloud services.

How

The system uses a centralized FastAPI API server proxying to local Ollama inference, combined with a decoupled SQLite-backed worker queue for ambient state changes and cron jobs.

00. Quickstart & Overview

What

The Sweetheart Core system is a multi-modal, agentic ecosystem. This quickstart covers the lifecycle scripts required to operate the background services.

Why

A cohesive start/stop lifecycle ensures that the FastAPI server, Ollama instance, and Docker containers (like OpenWebUI and openedai-speech) are booted in the correct order with the correct port bindings.

How

All services are managed exclusively through the `sweetheart-*` scripts at the repo root. - **Start:** `./sweetheart-start` (Idempotent; safe to run if already up) - **Stop:** `./sweetheart-stop` (Stops all services cleanly) - **Restart:** `./sweetheart-restart` (Stop then start)

Knobs

Environment Variable	Description
PORT	Port number for the Sweetheart API Server (default 11434)
SWEETHEART_DATA_DIR	Root data directory (default <code>./data</code>)
BIND_HOST	Network interface IP address bound by the API server (default <code>0.0.0.0</code>)

01. Telegram Interface

What

The primary real-time communication interface for the Sweetheart ecosystem, supporting 1-on-1 Direct Messages with Jane and Multi-Persona Group Chats.

Why

Telegram provides instantaneous, mobile-friendly access to the agents, supporting multi-modal inputs (voice notes, photos) and asynchronous background nudges.

How

- **1-1 DM:** Chatting directly with Jane routes to her FSM agent. Her emotional state and circadian rhythm directly affect her responses and temperature.
- **Group Chat:** Messages in the group chat are evaluated by a fast router (using a 0.5B model) to determine which persona(s) should reply. A referee scores candidate replies.
- **Userbot vs Proxy:** Outbound sync messages attempt delivery via Pyrogram Userbot to appear as the real user, falling back to a Proxy Bot if offline.

Knobs

Environment Variable	Description
TELEGRAM_BOT_TOKEN	Primary bot token for Jane's 1-1 DM bot
TELEGRAM_CHAT_ID	JCD's personal Telegram user ID for DMs
TELEGRAM_REAL_GROUP_CHAT_ID	Main live group chat ID. Messages here route to the multi-persona engine
GROUP_CHAT_COOLDOWN_S	Cooldown period between un-mentioned bot turns in group chat (default 40)
TELEGRAM_PROXY_BOT_TOKEN	Offline fallback bot token
TELEGRAM_API_ID	Telethon / Pyrogram API ID for Userbot auth

Details

Architectural Mechanics: - **Mode 1 (1-1 DM):** Handled by JaneFSMAgent in `src/agents/fsm.py`. Uses sweetheart-core LLM. Syncs bidirectionally to OWU via 15-minute polling task. - **Mode 2 (Group Chat):** Managed by `run_fast_router`. Uses the lightweight ref (0.5B) model to triage candidate personas. Uses RefereeService for staggered typing and response deduplication. - **Outbound Sync:** Pyrogram Userbot attempts to authenticate directly with Telegram's MTPROTO API using `TELEGRAM_API_ID` to masquerade as the user. Fallback is the `TELEGRAM_PROXY_BOT_TOKEN` REST API.

02. OpenWebUI Interface

What

The desktop-class chat application providing a ChatGPT-like web interface to the agents.

Why

OpenWebUI (OWU) is used for deep work, debugging, and viewing complex markdown or artifact generations that are difficult to read on mobile Telegram.

How

It runs as a Docker container connecting to the Sweetheart API server (port 11434), which proxies requests to the underlying Ollama instance (11435). Chat threads in OWU can be bidirectionally synced with Telegram DMs or Group Chats via webhooks.

Knobs

Environment Variable	Description
OPEN_WEBUI_URL	Base HTTP endpoint of the OWU Docker container (default <code>http://localhost:8080</code>)
OPEN_WEBUI_JWT_TOKEN	Administrator JWT token for authenticating REST API calls
OWU_SECRET_KEY	Secret key for signing and verifying OWU webhook API calls
ENABLE_CHANNELS	Feature flag enabling bidirectional channels integration via webhooks (default <code>true</code>)

Details

Architectural Mechanics: - Relies on OpenWebUI v0.10.x. Uses a monkey-patch in the open-webui Docker container (`/app/backend/open_webui/socket/main.py`) to bypass the empty string array bug. - Connects directly to the Sweetheart server's `/v1/chat/completions` proxy endpoint. Tool schemas are injected dynamically per-turn using the Help Tool pattern (ADR-0052).

03. World Interface & Scripts

What

The World Interface represents the spatial and event-driven background simulation of the ecosystem, operated primarily via `c:` command scripts in Telegram.

Why

It allows the user to navigate a physical/conceptual location graph (“the world”), triggering ambient events, weather updates, and presence-based interactions in group chats.

How

Messages starting with the core command prefix bypass the heavy LLM pipelines and are processed deterministically. - `c:move <location>` changes presence. - `c:look` observes the current room and any passing events. - Scheduled cron-like world events inject temporal phenomena into specific locations.

Knobs

Environment Variable	Description
CORE_COMMAND_PREFIX	Prefix to invoke World Bot commands (default <code>c:</code>)
WORLD_BOT_ALLOWED_USERS	Whitelist of user IDs allowed to send commands (empty allows all)
WORLD_BOT_RATE_LIMIT_CALLS	Max commands within the rate limit window (default 10)
WORLD_BOT_RATE_LIMIT_WINDOW	Rate limit window in seconds (default 5.0)

Details

Architectural Mechanics: - `c:` commands are parsed deterministically by `CoreFSMAgent` in `core_bot.py`. - `WorldEventsService` triggers cron-like injections based on frontmatter in `data/world_events/{slug}.md`. - Ambient location rendering is triggered during `_set_position()`, updating `group_chat_presence` in SQLite.

04. Dashboard Interface

What

A FastAPI and HTML/JS-based web interface for system administration, human-in-the-loop interaction, and real-time voice sessions.

Why

It provides a centralized operational view of the ecosystem's background tasks, memory traces, and emotional/circadian sliders. It also hosts the two-way voice mode.

How

Accessible via browser on the dashboard port. The interface provides visual readouts of internal agent states (like energy and mood) and allows live audio interaction utilizing the openedai-speech TTS and Faster-Whisper STT engines.

Knobs

Environment Variable	Description
DASHBOARD_PORT	Port number for the Dashboard (default 11442)
AUDIO_VOICE_SILENCE_TIMEOUT_SEC	Silence timeout for auto-stopping voice recordings (default 15)
AUDIO_VOICE_MAX_RECORDING_SEC	Max length per voice utterance (default 120)
AUDIO_VOICE_SESSION_IDLE_SEC	Inactivity timeout before closing a voice session (default 60)

Details

Architectural Mechanics: - Runs on a separate FastAPI process on DASHBOARD_PORT. - Voice session state is maintained via WebSocket connections. STT is transcribed natively using Faster-Whisper, generating AUDIO_CAPTION_MODEL tone summaries before reaching the LLM. - The Metrics & Trends tab parses the SQLite metric_snapshots and metric_weekly_summaries tables to render Chart.js canvases.

05. Hardware & External Integrations

What

The physical and local-network endpoints the ecosystem communicates with, including the Green Phosphor CRT display, Home Assistant, and Dotty the robot.

Why

Bridges the gap between software simulation and physical presence, allowing the agents to perceive ambient room states or display real-time telemetry.

How

- **Green Phosphor CRT:** Receives UDP/TCP network telemetry payloads displaying Jane's current status and thoughts on a retro terminal.
- **Home Assistant:** Polled by background workers to extract physical ambient states (e.g., office temperature, lighting) to inject into agent context.
- **Dotty:** A physical robot operated via a dedicated HTTP API.

Knobs

Environment Variable	Description
GREEN_TERM_ENABLED	Flag enabling telemetry push to the CRT display (default false)
GREEN_TERM_ADDR	IP address of the Green Phosphor ESP32-S3 terminal
GREEN_TERM_PORT	Network port for the CRT socket endpoint
HOMEASSISTANT_URL	API URL of the local Home Assistant instance
HOMEASSISTANT_POLL_INTERVAL	Polling interval in seconds for updating ambient states (default 300)
DOTTY_API_URL	HTTP API endpoint for controlling Dotty

Details

Architectural Mechanics: - **Green Phosphor:** Uses a raw TCP/UDP socket push to GREEN_TERM_ADDR:GREEN_TERM_PORT. - **Home Assistant:** Pulled periodically by a worker task that writes JSON state representations into the shared ambient_state key-value store, making it accessible to the FSM via get_ambient_states().

20. FSM & Emotional State Model

What

Jane operates on a Finite State Machine (FSM) that dictates her internal mood, energy levels, and closeness to the user.

Why

Creates organic, non-deterministic behaviors. Her mood dictates her prompt context and temperature, making her grumpy when tired or affectionate when close.

How

The state drifts naturally over time via background cron jobs and is actively perturbed by conversation interactions.

Knobs

Variable	Description
ENERGY_DRIFT_RATE	Natural hourly energy drift rate
DEFAULT_TEMPERATURE	Base sampling temp (modulated by mood)

Details

Architectural Mechanics: - Implemented in JaneFSMAgent (src/agents/fsm.py). The state is serialized to `ambient_state`. Energy drains linearly via `ENERGY_FATIGUE_DRAIN` during wakefulness and resets to `ENERGY_SLEEP_MAX` after a nocturnal cycle.

21. Memory Architecture

What

A persistent, partitioned vector memory store using `sqlite-vec`.

Why

Allows Jane and her friends to remember vast amounts of context without blowing out the context window. Partitioning prevents cross-contamination between personas.

How

Background tasks extract semantic memories from chats. During generation, `CodeBaseNavigator` or vector search retrieves the top-K relevant memories and injects them into the prompt.

Knobs

Variable	Description
<code>SWEETHEART_EMBED_MODEL</code>	Text embedding model (<code>nomic-embed-text</code>)
<code>MEMORY_TOP_K</code>	Number of memories retrieved per category

Details

Architectural Mechanics: - Schema: `memories_{persona}`, `vec_memories_{persona}`. - Relies on `src/database/memory_utils.py` for extraction and `sqlite-vec` for KNN. - FTS5 indexes support keyword fallback. Whitelist-guarded SQL routing secures table accesses dynamically.

22. Social & Hobby Simulation

What

Background simulations of hobbies (Zen Gardening) and life events for Jane's friends and kids (Clara and Finn).

Why

Creates an illusion of a vibrant, persistent universe moving forward independent of user interaction.

How

Occasional Telegram nudges report on these events. Hobbies route journals into distinct NotebookLM notebooks.

Knobs

Variable	Description
KID_NUDGE_CHANCE	Probability of sending a kid vignette nudge
GARDEN_NUDGE_CHANCE	Probability of gardening nudges

Details

Architectural Mechanics: - Two daily `kid_reach_out_task` runs force 4-7 turn synthetic exchanges, stored in persona tables. - `kid_life_event_task` runs Mon/Wed/Fri. - Asymmetric closeness logic calculates delta vectors between personas (e.g., how Kate feels about Arthur) updating the social graph in `data/personas/inter-persona-relations.mermaid`.

23. Gossip Propagation

What

Dynamically synthesizes 1-3 sentence narrative updates about interactions with friends and JCD.

Why

Propagates cross-friend awareness during pen-pal letters and occasionally injects JCD conversation topics into Jane's system prompt, simulating natural social gossip.

How

A gossip sweep checks for recent notable events and writes them to a dedicated memory bucket.

Knobs

Variable	Description
GOSSIP_SWEEP_INTERVAL_MINUTES	Interval between gossip sweep checks in minutes
GOSSIP_FRIEND_WRITE_CHANCE	Probability that gossip is written to friend memory
GOSSIP_JCD_CHANCE	Probability that gossip is surfaced in JCD conversation context

Details

Architectural Mechanics: - Gossip is stored in a dedicated gossip memory bucket and is query-filtered to propagate cross-friend awareness. - A 30% chance (GOSSIP_JCD_CHANCE) exists to inject topics into Jane's system prompt.

24. Audio Perception

What

Jane can perceive voice notes and audio clips (words and delivery).

Why

Allows rich multi-modal interaction where tone, pacing, and ambient sound influence the response.

How

Users send voice notes via Telegram or attach audio files in OpenWebUI. The system transcodes and analyzes them.

Knobs

Variable	Description
AUDIO_CAPTION_MODEL	Model used for audio captioning (e.g. sweetheart-core:latest)
AUDIO_STT_OPENAI_API_BASE_URL	STT API endpoint

Details

Architectural Mechanics: - The source-agnostic `src/agents/audio_perception.py` module pairs an STT transcript with a guided audio-model pass describing delivery (tone, pacing, ambient). - This is merged into one turn ([voice note – how it sounded: ...]). - Non-mp3/wav formats (ogg, m4a) are transcoded to WAV via `ffmpeg`.

25. Vision & Images

What

Jane can see photos sent to her and generate images from text descriptions.

Why

Enhances multimodal engagement without relying on third-party cloud services for analysis.

How

Photos sent via Telegram are analyzed by a local VLM. Image generation is triggered by commands or conversational requests.

Knobs

Variable	Description
VLM_MODEL	Vision language model for photo/image analysis
GEMINI_API_KEY	API key for Gemini image generation
GEMINI_IMAGE_MODEL	Gemini model used for text-to-image generation

Details

Architectural Mechanics: - Uses Ollama's native multimodal /api/chat for vision. Describes images and stores descriptions as about_jane memories. - Image generation uses Gemini 2.5 Flash, saving to data/images/. - Couples physical descriptions and reference photos (data/images/reference-JaneJonathan.png) when appropriate.

26. Group Chat Referee

What

Manages turn-taking and participation of multiple agents in group chats.

Why

Maintains a natural conversation flow without bot overlaps, endless loops, or chaotic interruptions.

How

Uses a multi-phase pipeline to decide who speaks when in a group setting.

Knobs

Variable	Description
SWEETHEART_REFeree_LLM	Model used for group chat referee LLM ranking phase
GROUP_CHAT_COOLDOWN_S	Cooldown in seconds between group chat messages to prevent bot overlap

Details

Architectural Mechanics: - A 3-phase RefereeService pipeline (pre-filter, LLM rank, threshold selection). - Agents automatically join/leave via `WorldService.move_to` hooks. - Managed by `run_fast_router`. Uses the lightweight ref (0.5B) model to triage candidates. - Configurable per-agent via `referee.yml`.

27. Calendar Agency

What

Jane can read calendars to generate ambient daily plans and dynamically write/schedule events.

Why

Integrates Jane into daily life scheduling, allowing her to detect conflicts, respect DND hours, and plan activities.

How

Connects to Google Calendar via REST API or CalDAV. Jane automatically schedules events via agent tools.

Knobs

Variable	Description
JCD_NUDGE_BEFORE_CHANCE	Probability Jane nudges JCD before a calendar event
JCD_NUDGE_AFTER_CHANCE	Probability Jane nudges JCD after a calendar event
OLLAMA_CALENDAR_TIMEOUT	HTTP read timeout for calendar classification tasks

Details

Architectural Mechanics: - Analyzes timed calendar events on the fly to detect overlaps and injects [WARNING: Overlaps with...] tags. - Auto-triggers Do Not Disturb (DND) mode during active calendar appointments. - Native `schedule_calendar_event` tool allows her to schedule outings.

28. Background Workers

What

A decoupled task worker system that processes time-consuming operations off the main thread.

Why

Preserves a zero-lag completion pipeline for chat responses while enabling complex async operations like memory extraction and cron jobs.

How

A SQLite-backed task queue (tasks_queue) is polled by a background worker script.

Knobs

Variable	Description
TASKS_QUEUE_RETENTION_DAYS	Number of days completed/failed tasks are retained
CPU_THROTTLE_WINDOW	Seconds of chat inactivity before heavy tasks resume

Details

Architectural Mechanics: - Many tasks are cron-like (e.g. WEEKLY_DREAM_INTERVAL, NIGHTLY_SYNTHESIS_INTERVAL). - Tasks push status updates into the queue and can be monitored via the dashboard. - Uses enqueue_task and a worker loop to process jobs asynchronously.

29. Metric History & Trends

What

A time-series tracking system for emotional states, system health, and activities.

Why

Provides observability into the ecosystem's internal drifts and long-term changes in persona relationships.

How

Data is captured in snapshots and rolled up weekly, visualized in the dashboard.

Knobs

Variable	Description
METRIC_WEEKLY_ROLLUP_INTERVAL	Interval for metric snapshot weekly rollup

Details

Architectural Mechanics: - Two-tier SQLite time-series store: `metric_snapshots` (~15 min cadence) and `metric_weekly_summaries`. - Captures 26 metrics across 4 groups. - Visualized using Chart.js on the dashboard Trends tab. - Search quality metric crosses the FSM <-> worker process boundary via an HTTP bridge (`/internal/score-cache`).

30. Notebook Registry

What

A dynamic configuration mapping for NotebookLM instances.

Why

Keeps UUIDs and sensitive route logic out of the source code, allowing zero-code updates to available notebooks.

How

Maintained in a gitignored `data/notebooks.json` file. The FSM auto-generates tool options and prompt blocks from it.

Knobs

Variable	Description
NLM_BIN_PATH	Path to the nlm CLI binary for NotebookLM operations
NOTEBOOK_USAGE_COUNTER_INTERVAL	Interval to update usage counters

Details

Architectural Mechanics: - Dynamic [NOTEBOOKS SHELF] prompt block injects available notebooks. - Agent tools (`notebook_query`, `notebook_create_note`) are unified through `src/agents/notebook_backend.py`.

APPENDIX A: Env Var (Knob) Reference

1. System, Ports & Data Directories	22
2. Ollama & LLM Model Configuration	23
3. Telegram Integration	24
4. OpenWebUI Integration	27
5. Google API, Drive & Calendar Integration	28
6. Voice, TTS & STT Configuration	29
7. Worker & Background Task Schedules	30
8. Circadian & Emotional State Model	31
9. Hardware & External Integrations	32
10. Logging, Instrumentation & Diagnostics	33

1. System, Ports & Data Directories

Environment Variable		Default Value	Description & Operational Mechanics
PORT		11434	Port number for the Sweetheart API Server (FastAPI). Serves OpenAI-compatible proxy endpoints, REST APIs, and webhook listeners.
DASHBOARD_PORT		11442	Port number for the Sweetheart Dashboard web interface (FastAPI + HTML/JS).
SWEETHEART_DATA_DIR		./data	Root data directory. Holds SQLite databases (sweetheart.db, memory.db), persona folders, logs/, images/, and watch folders.
CODE_NAVIGATION_DB_PATH	navigator/navigator.db		Path to the persistent AST graph database used by CodeBaseNavigator for codebase code snippet queries.
NLM_BIN_PATH		nlm	Executable path for the nlm CLI tool, used by the unified notebook backend to query NotebookLM notebooks.
BIND_HOST		0.0.0.0	Network interface IP address bound by the API server and dashboard web servers.
CORS_EXTRA_ORIGINS		""	Comma-separated list of additional allowed origins for CORS headers on HTTP API routes.
BOT_CONFIG_PATH	data/telegram_bots.json		Path to the multi-persona Telegram bot configuration file (load_bot_config()). Each persona may specify an optional chat_id (resolved by get_persona_chat_id(), falling back to core.chat_id) — used by the generic invite service for DM routing.

2. Ollama & LLM Model Configuration

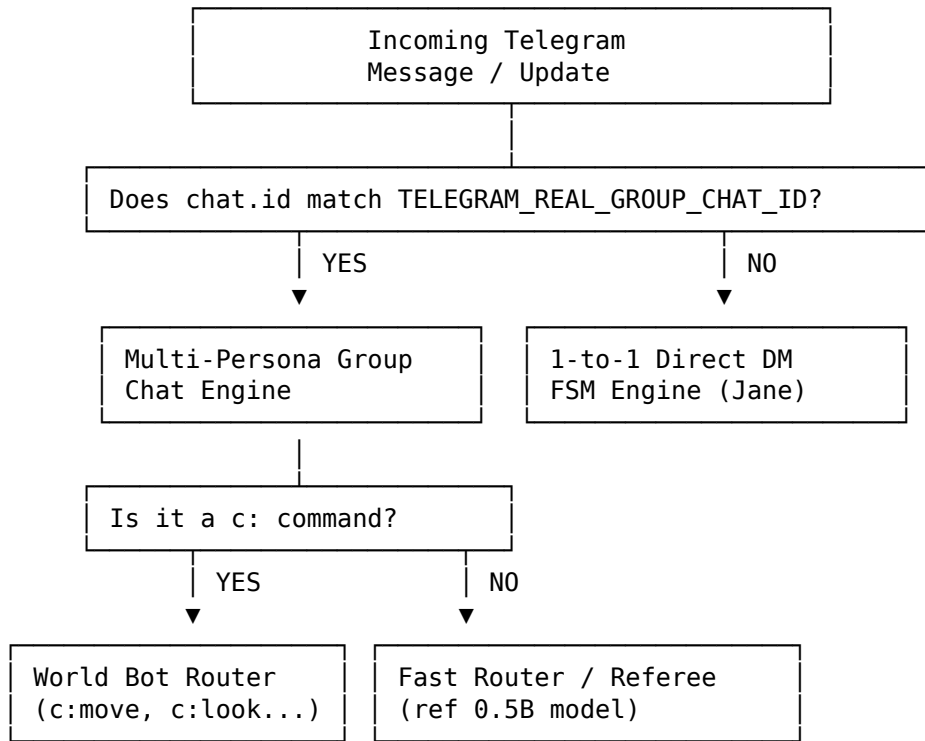
Environment Variable	Default Value	Description & Operational Mechanics
OLLAMA_LLM_URL	http://localhost:11435	HTTP endpoint URL of the local Ollama LLM server for completion and chat generation requests.
OLLAMA_EMBED_URL	http://localhost:11435	HTTP endpoint URL of the local Ollama embedding server (nomic-embed-text).
SWEETHEART_LLM	huihui_ai/gemma-4-abliterated:e4b	Primary reasoning LLM model backing Jane's FSM agent (overridden to sweetheart-core in production).
SWEETHEART_MEMORY_LLM	PRIMARY_LLM_MODEL	Model used for background memory extraction, session sweeps, and memory consolidation.
SWEETHEART_EMBED_MODEL	nomic-embed-text	768-dimensional text embedding model used by sqlite-vec for memory vector similarity search.
SWEETHEART_REFeree_LLM	ref	Fast, lightweight (0.5B) model used for RSS article interest triage, query expansion, mood evaluation, curiosity classification, and group chat referee scoring.
VLM_MODEL	sweetheart-core:latest	Multimodal Vision-Language Model used to perceive and caption photos attached in Telegram or uploaded to the dashboard.
AUDIO_CAPTION_MODEL	sweetheart-core:latest	Multimodal model used to generate natural language acoustic/tone summaries for incoming voice notes.
SWEETHEART_NUM_PREDICT	512 (overridden to 2048)	Maximum generation token budget (num_predict) passed to Ollama per LLM turn. High budget allows for internal <think> reasoning blocks.
SWEETHEART_NUM_CTX	16384 (overridden to 65536)	Maximum context window budget in tokens (num_ctx) allocated in VRAM by Ollama.
SWEETHEART_NUM_CTX_SMALL	4096	Reduced context window size used for fast secondary classification and referee requests.
DEFAULT_TEMPERATURE	0.7	Base sampling temperature for FSM response generation.
MAX_CHAT_HISTORY_LIMIT	20	Maximum number of preceding conversation history messages included in the prompt payload sent to Ollama.
HELP_MAX_DISCOVERED	4	Maximum number of dynamically discovered tool schemas injected per turn via the Help Tool pattern (ADR-052).
OLLAMA_LLM_TIMEOUT	300.0	HTTP request timeout in seconds for primary Ollama generation calls.
OLLAMA_EMOTION_TIMEOUT	15.0	HTTP request timeout in seconds for sub-second emotional state delta evaluations.
OLLAMA_CALENDAR_TIMEOUT	30.0	HTTP request timeout in seconds for calendar event classification requests.
OLLAMA_STREAM_TIMEOUT	90.0	Timeout in seconds for establishing stream response connections.

3. Telegram Integration

Environment Variable	Default Value	Description & Operational Mechanics
TELEGRAM_BOT_TOKEN	None	Primary Telegram Bot API token for Jane's 1-1 direct messaging bot.
TELEGRAM_CHAT_ID	None	JCD's personal Telegram user ID / DM chat ID with Jane.
TELEGRAM_REAL_GROUP_CHAT_ID	None	Main live Telegram Group Chat ID. When incoming updates match this ID, they are routed to the multi-persona engine.
TELEGRAM_SYNC_CHAT_ID	None	OpenWebUI chat thread ID bidirectionally synced (TELEGRAM ← DB → OWU) with Telegram DM.
TELEGRAM_SANDBOX_CHAT_ID	None	Optional secondary group chat ID reserved for isolated sandbox testing.
TELEGRAM_PROXY_BOT_TOKEN	None	Offline fallback bot token used strictly when Pyrogram Userbot is offline/unauthenticated.
TELEGRAM_API_ID	""	Telethon / Pyrogram API ID credential for Userbot client authentication.
TELEGRAM_API_HASH	""	Telethon / Pyrogram API Hash credential for Userbot client authentication.
TELEGRAM_PHONE_NUMBER	""	Phone number associated with the Pyrogram Userbot account (+1...).
CORE_COMMAND_PREFIX	c:	Command prefix used to invoke World Bot navigation commands in Telegram (c:move, c:look, c:map).
GROUP_CHAT_COOLDOWN_S	40	Cooldown period in seconds between un-mentioned bot turns in a multi-persona group chat.
REACH_OUT_RATE_LIMIT	5	Maximum number of proactive reach-out messages Jane is permitted to initiate per day.
TELEGRAM_HIST_SCRAPE_HOURS	24	Lookback window in hours when scraping historical Telegram messages for context backfill.
TELEGRAM_BOT_IDS	{}	Optional manual mapping of bot IDs to persona names ({"123456": "Jane"}). Auto-populated via telegram_bots.json.
TELEGRAM_FRIEND_IDS	{}	Manual JSON map of friend Telegram user IDs.
TELEGRAM_SYNC_MAPPINGS	[]	List of custom sync mapping pairs for multi-thread synchronization.
WORLD_BOT_ALLOWED_USERS	""	Whitelist of Telegram user IDs allowed to send World Bot c: commands. Empty allows all users.
WORLD_BOT_RATE_LIMIT_CALLS	10	Maximum allowed World Bot command calls within the rate limit window.
WORLD_BOT_RATE_LIMIT_WINDOW	5.0	Duration of the rate limit window in seconds.

3.1 Telegram Chat Modes & Agent Workflows

The Telegram integration supports three distinct execution modes depending on how incoming updates are routed.



Mode 1: 1-to-1 Direct DM Chat (Jane <-> JCD)

- **Configuration Variables:** TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID, TELEGRAM_SYNC_CHAT_ID.
- **Target Interface:** Direct Messages between JCD and @JaneBot.
- **Agent Operational Mechanics:**
 - Executes Jane's full 6-state Finite State Machine (JaneFSMAgent in src/agents/fsm.py).
 - **Multimodal Perception:** Incoming voice notes are transcribed via Faster-Whisper with tone captioning (AUDIO_CAPTION_MODEL); attached photos are parsed via VLM (VLM_MODEL).
 - **Circadian & Emotional State Modulation:** Jane's current emotional state (mood, energy, closeness) and circadian phase dynamically modulate system prompt blocks and response sampling temperature (DEFAULT_TEMPERATURE).
 - **Bidirectional OpenWebUI Synchronization:** Every 1-1 turn is persisted in chat_messages_jane and reconciled every 15 minutes with OpenWebUI chat thread TELEGRAM_SYNC_CHAT_ID.

Mode 2: Multi-Persona Group Chat (Jane + Persona Friends <-> JCD)

- **Configuration Variables:** TELEGRAM_REAL_GROUP_CHAT_ID, TELEGRAM_SANDBOX_CHAT_ID, GROUP_CHAT_COOLDOWN, data/telegram_bots.json.
- **Target Interface:** Telegram Group Chat matching TELEGRAM_REAL_GROUP_CHAT_ID.
- **Agent Operational Mechanics:**
 1. **Fast Router Evaluation:** Incoming messages are evaluated by run_fast_router using the ref (0.5B) model to rank candidate personas (Jane, Arthur, Lyra, Kate, Dotty) based on message context.
 2. **Direct Mentions:** Explicitly tagging @JaneBot or @ArthurBot overrides the fast router and forces immediate candidate generation for the mentioned bot(s).

3. **Referee Pipeline (ref model):** When multiple candidate responses are generated in parallel, RefereeService scores each reply using persona-specific topic bonuses (data/personas/<name>/referee.y applies staggered typing delays, and discards redundant replies.
4. **Speaker Lock & Cooldown:** The responding persona acquires an active speaker lock for up to 2 follow-up turns. Un-mentioned bot turns are rate-limited by GROUP_CHAT_COOLDOWN_S (40s).

Mode 3: World Bot Navigation & Location Presence (c: Commands)

- **Configuration Variables:** core.chat_id in data/telegram_bots.json, CORE_COMMAND_PREFIX, WORLD_BOT_ALLOWED_USERS, WORLD_BOT_RATE_LIMIT_*.
- **Target Interface:** Group Chat matching core.chat_id in data/telegram_bots.json.
- **Agent Operational Mechanics:**
 1. **Deterministic Core Parsing:** Messages starting with CORE_COMMAND_PREFIX (default c:) bypass heavy LLM turns and execute directly via CoreFSMAgent in core_bot.py.
 2. **World Navigation Commands:** Supports c:move <location>, c:look, c:map, c:whoami, and c:gps.
 3. **Pinned Location Presence Display:** Executing c:move updates present agent location in SQLite (group_chat_presence), updates the Telegram pinned header in the group chat, and renders ambient passing events and location descriptions.

Mode 4: Outbound Userbot vs Proxy Bot Protocol

- **Configuration Variables:** TELEGRAM_API_ID, TELEGRAM_API_HASH, TELEGRAM_PHONE_NUMBER, TELEGRAM_PROXY_BOT_TOKEN.
- **Operational Mechanics:**
 - **Userbot First:** All outbound user-role sync messages from OpenWebUI to Telegram attempt delivery via Pyrogram Userbot (send_userbot_message) directly from JCD's user account for natural representation.
 - **Proxy Bot Fallback:** TELEGRAM_PROXY_BOT_TOKEN acts strictly as an offline fallback when Pyrogram Userbot is unauthenticated or offline.

4. OpenWebUI Integration

Environment Variable	Default Value	Description & Operational Mechanics
OPEN_WEBUI_URL	http://localhost:8080	Base HTTP endpoint of the running OpenWebUI Docker container.
OPEN_WEBUI_JWT_TOKEN	None	OpenWebUI administrator signed JWT token used for authenticating REST API calls.
OWU_SECRET_KEY	None	Secret key used for signing and verifying OpenWebUI webhook API calls.
OWU_API_URL	""	Optional direct URL override for OpenWebUI channel REST endpoints.
OWU_API_KEY	""	Optional direct API key override for OpenWebUI REST authentication.
ENABLE_CHANNELS	true	Feature flag enabling bidirectional OpenWebUI channels integration via webhooks.
ENABLE_FORWARD_USER_INFO_HEADERS	true	Feature flag forwarding user identity metadata in HTTP headers to OpenWebUI.

5. Google API, Drive & Calendar Integration

Environment Variable	Default Value	Description & Operational Mechanics
GOOGLE_CLIENT_ID	None	Shared Google OAuth 2.0 Client ID for Drive, Calendar, and Photos access.
GOOGLE_CLIENT_SECRET	None	Shared Google OAuth 2.0 Client Secret.
GOOGLE_REFRESH_TOKEN	None	Shared Google OAuth 2.0 Refresh Token.
JANE_GOOGLE_CLIENT_ID	None	Jane's dedicated GCP project OAuth 2.0 Client ID.
JANE_GOOGLE_CLIENT_SECRET	None	Jane's dedicated GCP project OAuth 2.0 Client Secret.
JANE_GOOGLE_REFRESH_TOKEN	None	Jane's dedicated Google OAuth refresh token for assistant Drive/Calendar access.
JANE_GOOGLE_PHOTOS_REFRESH_TOKEN	None	Refresh token for uploading generated AI images to Google Photos albums.
JANE_DRIVE_HOME_ID	None	Google Drive root folder ID storing Jane's notebooks, document shards, and memory backups.
GOOGLE_PHOTOS_ALBUM_ID	None	Google Photos album ID where generated images are automatically published.
GEMINI_API_KEY	None	Google Gemini API key used by the <code>generate_image</code> tool for AI image synthesis (<i>"Jane, Show Me"</i>).
GEMINI_IMAGE_MODEL	gemini-2.5-flash-image	Gemini model ID used for generating images.
JANE_CALENDAR_ID	None	Google Calendar ID that Jane reads from and writes assistant events to (REST API v3).
USER_CALENDAR_CALDAV_URL	None	Fallback CalDAV URL endpoint for user calendar syncing.
JCD_EMAIL	None	JCD's email address used when Jane sends calendar invitations.
JCD_ICAL_URL	None	JCD's private iCal secret address used for read-only schedule awareness.
JCD_NUDGE_BEFORE_CHANCE	0.40	Probability that Jane sends a Telegram nudge prior to an upcoming calendar event.
JCD_NUDGE_AFTER_CHANCE	0.36	Probability that Jane sends a Telegram follow-up nudge after a calendar event completes.

6. Voice, TTS & STT Configuration

Environment Variable	Default Value	Description & Operational Mechanics
SWEETHEART_TTS_MODEL	tts-1-hd	TTS model requested from openedai-speech for speech synthesis.
AUDIO_TTS_OPENAI_API_BASE_URL	http://localhost:8002/v1	Base HTTP endpoint of the openedai-speech synthesis server (Piper + XTTS engine).
AUDIO_STT_OPENAI_API_BASE_URL	http://localhost:8002/v1	Base HTTP endpoint of the STT transcription server (Faster-Whisper).
AUDIO_TTS_OPENAI_API_VOICE	southern-female	Default voice profile ID used for generating Jane's spoken responses.
AUDIO_PERCEPTION_ENABLED	true	Feature flag enabling audio perception processing (acoustic tone/emotion extraction from voice notes).
AUDIO_MODEL_TIMEOUT	180.0	Timeout in seconds for audio transcription model requests.
AUDIO_VOICE_SILENCE_TIMEOUT_SEC	15	Silence timeout in seconds for web dashboard two-way voice mode recording auto-stop.
AUDIO_VOICE_MAX_RECORDING_SEC	120	Maximum recording length in seconds per voice mode utterance.
AUDIO_VOICE_SESSION_IDLE_SEC	60	Inactivity timeout in seconds before closing an active voice mode session.

7. Worker & Background Task Schedules

Environment Variable	Default Value	Description & Operational Mechanics
RSS_REPOLL_INTERVAL	14400 (4 hrs)	Time interval between sweeps of RSS news feed updates.
AMBIENT_UPDATE_INTERVAL	2700 (45 mins)	Time interval between ambient scene, weather, and fatigue updates.
READER_SCAN_INTERVAL	86400 (24 hrs)	Time interval between watch folder document scans.
CPU_THROTTLE_WINDOW	600 (10 mins)	Rolling window duration in seconds used for CPU activity throttling checks.
MEMORY_TOP_K	3	Number of relevant memories retrieved per category during context resolution.
CURIOSITY_NUDGE_CHANCE	0.70	Probability that Jane sends a Telegram nudge for curiosity research tasks.
GARDEN_NUDGE_CHANCE	0.40	Probability that Jane sends a Telegram nudge for gardening tasks.
STARGAZING_NUDGE_CHANCE	0.40	Probability that Jane sends a Telegram nudge for stargazing events.
KID_NUDGE_CHANCE	0.20	Probability that Jane sends a Telegram nudge for kid vignette tasks.

8. Circadian & Emotional State Model

Environment Variable	Default Value	Description & Operational Mechanics
ENERGY_LOW	0.35	Low-energy threshold triggering fatigue directives and slower speech cadence.
ENERGY_HIGH	0.70	High-energy threshold triggering enthusiastic directives and high engagement.
ENERGY_FATIGUE_DRAIN	0.005	Energy drain rate subtracted per active conversation turn during wakefulness.
ENERGY_DRIFT_RATE	0.10	Natural hourly energy drift rate toward baseline.
ENERGY_SLEEP_MIN	0.40	Minimum energy floor restored after a full nocturnal sleep cycle.
ENERGY_SLEEP_MAX	0.95	Maximum energy ceiling restored after a full nocturnal sleep cycle.

9. Hardware & External Integrations

Environment Variable	Default Value	Description & Operational Mechanics
GREEN_TERM_ENABLED	false	Feature flag enabling status telemetry push to the Green Phosphor ESP32-S3 CRT terminal display.
GREEN_TERM_ADDR	192.168.86.25	IP address of the Green Phosphor ESP32-S3 terminal on the local LAN.
GREEN_TERM_PORT	6053	Network port for the Green Phosphor display socket endpoint.
GREEN_TERM_SECRETS_PATH	data/secrets/secrets.yaml	Path to secrets file used for authenticating with the terminal.
HOMEASSISTANT_URL	None	API URL of the local Home Assistant instance (http://192.168.86.37:8123).
HOMEASSISTANT_TOKEN	None	Bearer token for authenticating against Home Assistant REST API.
HOMEASSISTANT_POLL_INTERVAL	300	Polling interval in seconds for updating office sensor ambient states.
DOTTY_API_URL	http://192.168.86.198:8000	HTTP API endpoint for controlling Dotty the physical robot.

10. Logging, Instrumentation & Diagnostics

Environment Variable		Default Value	Description & Operational Mechanics
MEMORY_TRACE_ENABLED		false	Enables verbose logging of vector similarity scores and recalled memory IDs during search queries.
PROMPT_BLOCK_STATS_ENABLED		true	Tracks prompt block character sizes and conditional branch hit rates in SQLite for prompt engineering analysis.
DETAILED_LOGGING_ENABLED		true	Primary prompt debugger. Writes paired *.prompt.md and *.response.md markdown files to data/logs/ for every turn.
DETAILED_LOGGING_LLM_UTILS	DETAILED_LOGGING_ENABLED		Subsystem logging override toggle for low-level LLM calls in llm_utils.py.
DETAILED_LOGGING_GROUP_CHAT	DETAILED_LOGGING_ENABLED		Subsystem logging override toggle for group chat pipeline turns.
DETAILED_LOGGING_MEMORY	DETAILED_LOGGING_ENABLED		Subsystem logging override toggle for memory extraction and consolidation.
FULL_PROMPT_LOGGING_ENABLED		true	Dumps raw 10-turn snapshot prompts to data/FULL_PROMPT_LOGS.md before tool execution loops begin.
TOOL_STATS_ENABLED		true	Logs tool call execution counts, execution times, and success status to SQLite for diagnostics.

APPENDIX B: Developer Info

Sweetheart Developer Guidelines	35
1. Database Session Hygiene	35
2. Decoupled Rhythms & Formulas	35
3. Centralized Schedule Configuration	35
4. Module-Level Imports & Mock Patching	36
5. Deterministic Unit Testing	36
6. Adding a New Background Task	36
7. Memory Bucket Conventions	37
8. Ambient State vs Memories	39
9. Async Blocking I/O	39
10. Keeping the Codemap Current	40
11. Prompt Files System	40
12. Prompt Instrumentation & Debugging	41
13. Tool Registry Pattern	43
14. Data Directory Organization	44
15. Code Navigation System	45
Inspect — <code>get_code_snippet(node_id)</code> to fetch only the needed source	46
16. Utility Scripts	46
17. Core System Commands	47

Sweetheart Developer Guidelines

This document outlines the architectural best practices and conventions established during codebase refactoring to ensure long-term maintainability.

1. Database Session Hygiene

Rule: Never manage SQLite connection lifetimes manually. Always use the `db_session()` context manager from `src.database.db_init`:

```
from src.database.db_init import db_session

with db_session() as conn:
    # Automatically commits on success, rolls back on error, and closes the connection.
    conn.execute("SELECT ...")
```

2. Decoupled Rhythms & Formulas

Rule: Keep FSM logic (`fsm.py`) and background worker tasks (`src/worker/tasks/`) free of raw time, circadian, or hormonal arithmetic.

Practice: Place all phase evaluations, sleep window checks, fatigue calculations, weather baselines, and cycle transitions in `src/circadian.py`. These functions must be pure — they take parameters and return values, with no side effects or I/O.

Corollary: If you find yourself writing `hour >= 23` or `hour < 7` in `fsm.py` or a task file, it belongs in `circadian.py` as a named function.

3. Centralized Schedule Configuration

Rule: Do not hardcode task cron intervals, target times, or timing baselines inside `worker.py`.

Practice: Define schedule parameters in `src/config.py` using `CRON_INTERVALS` and `TASK_TARGET_TIMES`, backed by `os.environ` overrides:

```
CRON_INTERVALS = {
    "my_task": int(os.environ.get("MY_TASK_INTERVAL", "86400")),
}
TASK_TARGET_TIMES = {
    "my_task": [(14, 0)], # (hour, minute) in local time
}
```

Tuple format: Most entries in `TASK_TARGET_TIMES` are (hour, minute) two-tuples. Weekly tasks use a three-tuple (weekday, hour, minute) where weekday is 0=Monday ... 6=Sunday. Example: `"weekly_dream": [(5, 4, 0)]` means Saturday at 04:00. Always check the worker's dispatch logic when adding a weekly task.

4. Module-Level Imports & Mock Patching

Rule: Import at module top level. Inline imports (inside functions) are only acceptable in two cases:

1. **Circular dependency prevention** — when a module-level import would create an import cycle.
2. **Optional/heavy dependencies inside tool executors** — the `execute_*` functions in `fsm.py` may import optional integrations inline (e.g., `CalDAVCalendarClient`, `NLM_BIN_PATH`) so that missing or unconfigured dependencies don't fail at startup. This is an accepted pattern for that file only.

Testing note: When a helper is imported at module level (e.g., from `src.database.db_helpers` import `is_dnd_active` in `fsm.py`), mock patching in unit tests must target the consuming module's namespace: `patch("src.agents.fsm.is_dnd_active")`, not the original helper path.

5. Deterministic Unit Testing

Rule: Never let dynamic baselines (real-time weather, random daily walk offsets, random cycle days) make tests flaky.

Practice: Pre-seed `ambient_state` and `emotional_state` at the start of each test block before exercising any circadian or mood logic. The `tests/conftest.py` initializes a temporary `SWEETHEART_DATA_DIR` and wipes all tables before each test — build on that baseline, don't assume it.

Running tests: Use `make check` at the repo root — runs `ruff` lint, `mypy` typecheck, and `pytest` in sequence. This is the standard quality gate enforced before commits. For a coverage report, run `make cover`. Individual test files can be run with `uv run python -m pytest tests/path/to/test.py -v`.

6. Adding a New Background Task

Every new background task requires four wiring steps. Miss one and the task will silently never run.

Step 1 — Write the handler function in the appropriate `src/worker/tasks/` module (or a new file in that package if the domain is distinct). Signature: `async def my_task(payload: Dict[str, Any])`.

Step 2 — Register the handler in `src/worker/tasks/__init__.py`:

```
from .my_module import my_task
TASK_HANDLERS = {
    ...
    "my_task": my_task,
}
```

Step 3 — Add the polling interval to `CRON_INTERVALS` in `src/config.py`:

```
"my_task": int(os.environ.get("MY_TASK_INTERVAL", "86400")),
```

Step 4 — Optionally add a target fire time to `TASK_TARGET_TIMES` in `src/config.py` if the task should run at a specific time of day rather than a rolling interval:

```
"my_task": [(14, 0)], # (hour, minute)
```

Also: If the task generates content worth sharing with JCD (research, updates, milestones), append a one-sentence teaser to `ambient_state["pending_shares"]` so Jane can mention it at the next reconnect (see Idea: *pending shares queue* in IDEAS.md).

Adding a Worker-Side Polling Loop (not a task queue task)

Long-lived polling loops (like the Telegram bot or world exploration bot) are distinct from task queue tasks — they run continuously rather than on a cron schedule.

Step 1 — Write the polling loop function in a new file under `src/worker/tasks/`. Signature: `async def my_polling_loop() -> None` with an internal `while True` loop.

Step 2 — Re-export from `src/worker/tasks/__init__.py`.

Step 3 — Wire into `src/worker/worker.py` — add to the `asyncio.gather()` call inside `start_worker()` alongside `process_queue_loop()` and the existing polling loops.

7. Memory Bucket Conventions

Use `remember()` from `src.database.memory_utils` with the `Who`, `Subject`, `MemoryKind`, and `MemorySource` enums (`src/database/enums`) instead of raw `add_memory` calls. The `Subject` enum values map directly to these bucket names.

Partitioned Persona Memory Routing: Under the hood, memory storage and vector search are strictly partitioned per-persona. Calling `remember()` or `recall()` dynamically targets the `memories_{persona}`, `vec_memories_{persona}`, and `memories_fts_{persona}` tables based on the `who` parameter. Table routing is whitelist-validated against the `Who` enum to eliminate SQL injection. This architecture prevents “top-k drowning” where global vector k-NN queries overwhelm relevant persona-specific memories.

The memories are grouped into logical buckets within each partition. Write to the correct one — misfiled memories surface in the wrong context during vector search.

Subject	Purpose	Who writes it
jcd	Permanent facts, preferences, and history about JCD	<code>extract_memories_task</code>
jane	Jane’s self-reflections, journal entries, curiosity questions, research reminders, intents, stakes (via <code>_generate_stake</code>), second-order values (via <code>seed_second_order_values</code> + HOV synthesis)	curiosity, journal, FSM tool executors, <code>feeds.py</code> , <code>reader.py</code> , <code>social.py</code> , <code>db_init.py</code>
general	External knowledge: RSS summaries, research findings, dream digests, friend letters	RSS poller (<code>feeds.py</code>), curiosity task, weekly dream
{name}	Per-friend exchange history (lowercase: arthur, lyra, kate)	<code>friend_letter_task</code> , <code>friend_reach_out_task</code>

Subject	Purpose	Who writes it
friends	Cross-friend references visible to all friends	Social tasks

Exclusive Memory API Mandate

CRITICAL: Never write raw SQL queries, use generic database helpers (like `add_memory` or `delete_memory` from `db_helpers`), or run `direct conn.execute` statements when dealing with persona memories.

Because memory tables are partitioned per-persona (`memories_{persona}`, `vec_memories_{persona}`, `memories_fts_{persona}`), and because they are backed by complex SQLite auto-sync triggers (for FTS) and strict sqlite-vec vector indices, direct database manipulation **will** break indices, bypass SQL injection whitelist protections, or leak memories across personas.

Always use the four primary high-level APIs in `src/database/memory_utils.py`: - `remember(who, text, subject, *, embedding=None, metadata=None, kinds=None, source=None)`: Safely creates a memory inside the persona's partitioned table. It automatically computes vector embeddings via Ollama, inserts the relational record, and updates the vector (`vec_memories_{who}`) and full-text search (`memories_fts_{who}`) virtual tables cleanly. - `recall(who, subject=None, kinds=None, query=None, modes=None, top_k=5, *, embedding=None, current_ambient=None)`: Performs hybrid semantic vector and/or keyword search strictly isolated inside the target persona's partitions (preventing "top-k drowning"). Supports environmental context boosts. - `reframe(memory_id, who, *, text=None, subject=None, kinds=None)`: Idempotently updates an existing memory's content, subject category, and kind classifications. If the text is changed, it automatically re-computes and updates the vector embedding in `vec_memories_{who}` and refreshes the FTS table. - `forget(memory_id, who)`: Permanently deletes a memory by ID. If `who` is omitted, it dynamically scans all persona partitions to locate and purge the memory from relational, vector, and FTS tables cleanly.

```
from src.database.enums import SearchMode, Subject, Who
from src.database.memory_utils import get_subjects, recall, remember, reframe, forget

# 1. Discover subjects Jane has memories in
subjects = get_subjects(Who.JANE) # -> [Subject.JANE, Subject.JCD, ...]

# 2. Add a new memory cleanly (auto-computes embeddings and syncs FTS)
mem_id = await remember(Who.JANE, "I love reading philosophy.", Subject.JANE, kinds=["reflection"])

# 3. Recall memories with hybrid search
results = await recall(Who.JANE, Subject.JANE, query="philosophy", modes=[SearchMode.VECTOR, SearchMode.KEYWORD])

# 4. Reframe/edit an existing memory (auto-recomputes vector on text change)
await reframe(mem_id, Who.JANE, text="I absolutely adore reading philosophy.", kinds=["reflection"])

# 5. Safely delete/forget a memory across all tables
await forget(mem_id, Who.JANE)
```

iroh | Observations of JCD's growth toward Iroh-like qualities (patience, warmth, joy, resilience) | `extract_memories_task` (Iroh pass); weekly `iroh_reflection_task` reads from it |

Rule of thumb: If it's searchable narrative content that should surface during a future conversation, it's a memory. If it's a scalar value or transient session flag the FSM needs on every request, it's ambient state (see Rule 8).

8. Ambient State vs Memories

`ambient_state` is a flat SQLite key-value store (key TEXT, value TEXT). Use it for:

- **Transient session state:** focus session timers, last message timestamp, DND flag.
- **Rolling scalars updated frequently:** weather, daily mood offset, menstrual cycle day.
- **Feature configuration with expiry:** friend life states, friend interest strings with expiry timestamps, hobby drift timestamp.
- **Daily cached content:** `iroh_daily_wisdom` (morning Iroh wisdom, refreshed at 7:30 AM by `iroh_morning_task`), `mood_narrative`, `work_narrative`.

Use the `memories` table (not ambient state) for:

- Any content longer than a sentence.
- Anything that needs vector search for retrieval.
- Anything with a narrative or episodic quality that should surface in conversation.

Anti-pattern: Storing a multi-paragraph journal entry or research note as an `ambient_state` value. Use `remember()` with the appropriate Subject enum instead.

9. Async Blocking I/O

Rule: Never call blocking operations directly inside `async def` functions. Blocking calls freeze the entire asyncio event loop for their duration, stalling Telegram polling, memory queries, and every other coroutine.

Blocking operations that must be wrapped: `subprocess.run()`, `subprocess.check_output()`, synchronous file reads/writes, `DDGS().text()`, any synchronous network library call, `CalDAVCalendarClient` methods.

Practice: Wrap blocking calls with `asyncio.get_running_loop().run_in_executor(None, ...)`. This is the established pattern in this codebase (post plan-002):

```
loop = asyncio.get_running_loop()
result = await loop.run_in_executor(
    None, lambda: subprocess.run(cmd, capture_output=True, text=True, check=True)
)
```

For functions with multiple args, pass them positionally rather than using a lambda where possible:

```
result = await loop.run_in_executor(None, blocking_func, arg1, arg2)
```

`asyncio.to_thread(func, *args)` (Python 3.9+) is equivalent and acceptable for new code. Do not use `asyncio.get_event_loop()` — use `asyncio.get_running_loop()` inside an already-running coroutine.

10. Keeping the Codemap Current

The repository has a machine-readable codemap (`.slim/codemap.json`) that tracks file hashes for change detection, plus human-readable `codemap.md` files at each directory level. These are the primary onboarding tool for any agent or developer entering the codebase.

When to update

Run a codemap refresh after any significant structural change:

- Adding, removing, or renaming a directory or module
- Splitting a file into a package (e.g., `tasks.py` → `tasks/ package`)
- Adding a new top-level function or class that changes a module's responsibility
- Before asking an agent to do deep work on a changed area of the codebase

How to update

1. Detect what changed

```
node ~/.config/opencode/skills/codemap/scripts/codemap.mjs changes --root ./
```

2. Update affected codemap.md files

- Create new codemap.md for new directories

- Update existing ones for modified directories

- Update root codemap.md (atlas) with any new subdirectory entries

3. Save the new snapshot

```
node ~/.config/opencode/skills/codemap/scripts/codemap.mjs update --root ./
```

The `changes` command lists added, removed, and modified files grouped by affected folder. Only update the codemap files for folders that changed — skip untouched directories.

Reading the codemap

Every `codemap.md` describes a directory's responsibility, key files, design patterns, data flow, and integration points. The root `codemap.md` is the entry point — it contains the project overview and a directory map pointing into each subdirectory's detailed map.

11. Prompt Files System

Jane's LLM prompt text is stored as individual `.md` files in `data/personas/` rather than hard-coded f-strings in Python source.

Architecture

```
data/personas/
├── jane/                # Jane's persona + task prompts (29+ files)
│   ├── jane.md          # Core persona definition
│   ├── jane_opinions.md # Jane's strong opinions and positions (`[OPINIONS & POSITIONS]` FSM blo
│   ├── jane_background.md # Extended character background
│   ├── reading_system.md # Document reading system prompt
│   ├── emotion_eval.md   # FSM emotion evaluation prompt
│   ├── curiosity_*.md    # Curiosity task prompts (topic, debate, synthesis)
│   └── garden_*.md       # Gardening task prompts
```



```

├── stargazing_*.md      # Stargazing task prompts
├── iroh_*.md           # Iroh coaching prompts
├── nudge_*.md          # Spontaneous nudge prompts
├── ...                 # Additional task prompts
├── arthur/             # Friend persona prompts (6 files)
├── lyra/               # Friend persona prompts (6 files)
├── kate/               # Friend persona prompts (6 files)
├── chrissie/           # Greysons family / babysitter persona prompts
├── jeff/               # Greysons family persona prompts
├── martha/             # Greysons family persona prompts

```

How It Works

- `src/prompts.py` provides `init_prompts(prompt_dir)` and `render_prompt(name, persona="jane", **kwargs)`.
- `init_prompts()` scans the persona directory and caches all `.md` files in memory at startup (called from `src/config.py`).
- `render_prompt()` loads a named prompt from cache and renders template variables using `str.format()`.
- Template variables use `{placeholder}` syntax matching Python `str.format()`.

Rules

1. **All LLM-facing prompt text lives in `.md` files.** If you're writing a multi-sentence string that will be sent to an LLM, it belongs in `data/personas/jane/` as a prompt file, not as a Python f-string.
2. **Use `render_prompt("name")` to load prompts.** Never `os.path.join(PERSONA_DIR, ...)` directly — always go through the cache.
3. **Keep `{placeholder}` variable names consistent** with the original inline prompt's variable names to simplify the migration.
4. **`data/` is gitignored.** Prompt files are runtime data, tracked only in your local environment or deployment package, never in version control.
5. **Don't hardcode persona paths in tests.** The test `confest.py` copies `data/personas/` to the test temp directory — tests should import and call the same `render_prompt()` used in production.

Test Fixtures

`tests/fixtures/personas/` contains **tracked baseline versions** of key persona files (`jane/jane_opinions.md`, `kate/kate.md`, `arthur/arthur.md`, `lyra/lyra.md`). The test `confest.py` copies from here first so tests that assert on persona content pass on fresh clones. Runtime `data/personas/` is overlaid on top — developer machines use real enriched content; CI uses the tracked fixtures.

12. Prompt Instrumentation & Debugging

The prompt assembly pipeline has optional instrumentation for diagnosing prompt composition and LLM response quality. Both features default to off — enable explicitly via env var or dashboard toggle.

Feature Flags

Env Var	Default	What it controls
PROMPT_BLOCK_STATS_ENABLED	false	Block size tracking + condition frequency counters logged to SQLite
FULL_PROMPT_LOGGING_ENABLED	false	Full prompt+response dump to data/FULL_PROMPT_LOGS.md (max 10 rounds)

Note: Both env vars are checked at runtime — setting them to false prevents the dashboard toggle from activating the feature (dashboard returns 400).

Block Size Tracking (`_before` / `_block_sizes` Pattern)

Each of the ~31 dynamic blocks in `build_system_prompt()` is instrumented with a start marker captured before the block is assembled, and the size is recorded after the block is appended:

```
# Inside the condition guard for each block:
_before = len(system_prompt)
# ... assemble block and append to system_prompt ...
if PROMPT_BLOCK_STATS_ENABLED:
    _block_sizes.append({
        "block_name": "user_mems",
        "token_count": len(system_prompt) - _before,
    })
```

Block sizes are batch-inserted into the `prompt_block_log` table after all blocks are assembled. The dashboard displays aggregate metrics (average, max, min, call count) per block in the Prompt Composition Stats card.

Condition Frequency Tracking (`_tc()` Wrapper)

Every `if` branch that decides whether a block is included uses a wrapper function `_tc()` that records the check:

```
def _tc(self, block_name: str, condition: bool) -> bool:
    if PROMPT_BLOCK_STATS_ENABLED:
        log_condition_check(block_name, condition)
    return condition
```

Calls are one-liners at each decision point:

```
if self._tc("jane_mems", stuff_mems or jane_mems):
    _before = len(system_prompt)
    ...
```

The `_tc()` call is always present regardless of the feature flag — only the `log_condition_check` call is gated. This keeps the wrapper pattern consistent and prevents accidentally forgetting to instrument a new block.

Full Prompt Logging

The FSM captures the complete `ollama_messages` payload that was sent to the LLM (deep copy before the tool loop, so tool execution results are excluded) and writes it to `data/FULL_PROMPT_LOGS.md` after each inference. The log includes:

- The full system prompt with all assembled blocks
- All chat messages in the current context window
- The LLM's response text
- Formatted as markdown with clear section headers

State is tracked in `ambient_state` as `full_prompt_logging_active` (bool) and `full_prompt_logging_remaining_hits` (int, starts at 10). When remaining hits 0, the system auto-deactivates. Both values survive restarts.

Dashboard Integration

The dashboard exposes: - **Full Prompt Log toggle** on the Logging page (alongside Memory Trace) — activates 10 rounds of capture - **Prompt Composition Stats card** — two-column table: block sizes (avg/max/min/count) on the left, condition frequencies (checked/fired) on the right

13. Tool Registry Pattern

Every tool is a self-contained module in `src/agents/tool_modules/` registered in a central registry.

Anatomy of a Tool Module

```
from src.database.enums import Who, Subject
from src.database.memory_utils import remember
from src.agents.tool_registry import REGISTRY, ToolDef
```

```
async def execute(question: str) -> str:
    """The function body that implements the tool."""
    await remember(Who.JANE, question, Subject.SELF)
    return f"Flagged: {question}"
```

```
tool = ToolDef(
    name="my_tool",
    description="What this tool does – shown to the LLM.",
    parameters={
        "type": "object",
        "properties": {
            "question": {"type": "string", "description": "The input."}
        },
        "required": ["question"],
    },
    fn=execute,
)
```

```
REGISTRY["my_tool"] = tool
```

How to Add a New Tool

1. **Create** `src/agents/tool_modules/<name>.py` with the pattern above
2. **Register** by adding `from src.agents.tool_modules import <name>` to `src/agents/tool_modules`
3. **Done** – `build_tools_schema()` and `TOOL_MAP` pick it up automatically

```
### Dependency Container (`deps`)
```

- `deps.get\_embedding` is available for tools that need embeddings; use `remember()` from `src.data`
- Import once: `from src.agents.deps import deps`
- Use attribute access: `deps.get\_embedding(...)` not `from src.embeddings import get\_embedding`

```
### Testing Tools
```

```
```python
from unittest.mock import patch

Patch deps.get_embedding – covers tools that call remember()
with patch("src.agents.deps.deps.get_embedding", return_value=[0.1] * 768):
 result = await my_tool_module.execute("test")
```

Tools that don't use deps can be tested by importing and calling their execute() function directly, patching at the tool module level:

```
with patch("src.agents.tool_modules.web_search.DDGS") as mock_ddgs:
 result = await web_search_module.execute("query")
```

## Gated Messaging (Telegram)

Tools that need to send Telegram messages to JCD (e.g., reach\_out.py) must use send\_user\_message() from src.agents.tool\_services — the DND and rate-limit gates are baked into the send function and cannot be bypassed:

```
from src.agents.tool_services import send_user_message, SendResult

result = await send_user_message("JCD", "Hello!", context="optional context")
if result == SendResult.BLOCKED:
 # Store as pending intent – will resurface when DND ends
 ...
```

- **DND gate:** Calls is\_dnd\_active() — if DND is active, returns BLOCKED without sending
- **Rate limit gate:** Calls \_try\_acquire\_rate\_slot() — atomic check+record per friend in ambient\_state. Limits REACH\_OUT\_RATE\_LIMIT sends per RATE\_LIMIT\_WINDOW seconds.
- **Telegram delivery:** Lazy-imports send\_telegram\_message inside function body to avoid import-time side effects.
- **Return value:** SendResult.SENT on success, SendResult.BLOCKED on DND/rate-limit/failure.

Testing note: send\_telegram\_message is lazy-imported inside send\_user\_message(), so tests must patch at src.agents.telegram\_client.send\_telegram\_message, not src.agents.tool\_services.

For non-JCD friends, reach\_out.py stores memories directly without gating — send\_user\_message() returns SENT immediately.

## 14. Data Directory Organization

**Rule:** Do not store new JSON state files or log files at the data/ root.

- **State:** Use data/state/ for JSON files containing runtime state (e.g., telegram\_state.json, garden\_state.json). Code: os.path.join(DATA\_DIR, "state", "filename.json").

- **Logs:** Use data/logs/ for application logs. Code: `os.path.join(DATA_DIR, "logs", "filename.log")`.
  - **Cleaning:** Periodic cleanup of stale one-time artifacts is recommended to maintain directory health.
- 

## 15. Code Navigation System

The repository includes a persistent, graph-based code navigation system in `navigator/` backed by a SQLite database at `navigator/navigator.db`. It helps agents and developers understand the codebase without reading full files.

### Components

- `navigator/navigator.py` — `CodeBaseNavigator` class providing semantic search, dependency graph traversal (`find_dependencies`), and source code snippet retrieval (`get_code_snippet`).
- `navigator/navigator-cli` — CLI wrapper for terminal use. Supports search, deps, snippet, and reindex commands (see usage below).
- `navigator/watcher.py` — `launchd`-compatible directory watcher that auto-refreshes the navigation database on file changes (wired via `com.sweetheart.watcher.plist`).
- `navigator/navigator-crawl.py` — Batch crawler for initial database population or full refresh.

### CLI Usage

```
Semantic search by intent
python3 navigator/navigator-cli search "memory retrieval"

Find dependency chain for a function or class
python3 navigator/navigator-cli deps src.agents.fsm.JaneFSMAgent

Get a code snippet by node ID
python3 navigator/navigator-cli snippet 42

Rebuild the index from scratch
python3 navigator/navigator-cli reindex --yes
```

### Investigation Workflow

When exploring an unfamiliar area of the codebase, prefer the navigator over broad file reads:

1. **Search** — `semantic_search(query, top_k=3)` to identify relevant functions/classes
2. **Map** — `find_dependencies(node_name, max_depth=2)` to understand structural impact
- 3.

**Inspect** — `get_code_snippet(node_id)` to fetch only the needed source

## 16. Utility Scripts

The `scripts/` directory contains standalone operational tools, migrations, and evaluation utilities that are not part of the main application runtime. They are designed to be run manually or via cron.

### Core Maintenance & Repair

- `heal_owu_chat.py` — Fixes broken scroll in OpenWebUI by repairing fragmented `history.messages` graphs and migrates legacy model tags (like `sweetheart-core`) to correct avatars/names.
- `clean_memory_metadata.py` / `clean_orphaned_vectors.py` — Database sanitation.
- `regenerate_zero_vectors.py` — Finds and re-embeds `vec_memories` rows with all-zero vectors.
- `patch-owu-models.py` — Patches OpenWebUI configuration.

### Data Migrations & Backfills

- `backfill_*.py` — A large collection of scripts (`friend_letters`, `jane_pronouns`, `memory_kinds`, `sentiment_tags`, etc.) for performing one-time or idempotent schema migrations and data enrichment on the SQLite database.
- `hipbone_*_backfill.py` — Scripts for migrating and backfilling Hipbone game state across platforms (Telegram, OWU).
- `telegram_backfill.py` — Imports legacy Telegram chat history into the database.
- `reconcile_*.py` — Scripts to force-sync state between external platforms (NotebookLM, OpenWebUI) and the database.

### Backup & Export

- `backup_to_gdrive.py` — Performs online SQLite backup (safe copy without locking), zips it with state/personas, and uploads to Google Drive via gws CLI. Retains last 7 days.
- `dump_tables_to_git.py` — Dumps SQLite tables to text format for version control diffing.
- `weekly_export.py` / `.sh` — Generates and archives weekly data snapshots.
- `extract_*.py` — Utilities to extract replay logs and task outputs from OWU, Notebooks, and the database.

### Evaluation & Scoring

- `score-component.py` — Translates the eight runtime health sub-scores into a graded array (A-F) with human-readable issue strings. JSON output compatible with the global `compound-powers` skill.
- `eval_retrieval.py` — Evaluates memory retrieval quality by running hybrid (vector + keyword) search against test queries, reporting hit rate and RRF statistics.
- `eval_generator.py` / `ragas_eval.py` — Generation and evaluation scaffolding using RAGAS.

### Persona Management

- `generate_persona_seeds.py` — Generates initial seed memories for personas via LLM and writes to `data/persona-seeds/`.

- `inject_persona_seeds.py` — Injects manually edited JSON seeds from `data/persona-seeds/` into the database via `add_persona_memory()`.

## Development & QA

- `verify.sh` — Runs `ruff` lint + `mypy` type check + full `pytest` suite in sequence. Used as the pre-PR quality gate.
- 

## 17. Core System Commands

Core system commands bypass the primary LLM reasoning hotpath (Jane) and are intercepted directly at the FSM boundary (`agent_core.py`) to run custom system tasks.

### Command Format

- Intercepted prefixes: `c:` directly at message start, or `@core c:` (OpenWebUI slash compatible).
- Commands are registered in the core registry, parsed, and execute stateless functions yielding stream tokens or dictionary responses.

### Implementation Checklist

1. **Create handler** in `src/agents/core_commands/<command_name>.py` defining the generator/handler.
2. **Wire in** `agent_core.py` to map the command string to your handler function.
3. **Register commands** inside `help` command modules if user-facing documentation is needed.

# APPENDIX C: Repository Atlas

<b>Repository Atlas: Sweetheart 2.0 (Core)</b>	<b>49</b>
<b>config/</b>	<b>56</b>
<b>openedai-speech/</b>	<b>57</b>
<b>openedai-speech/config/</b>	<b>58</b>
<b>scripts/</b>	<b>59</b>
<b>src/agents/</b>	<b>63</b>
<b>src/agents/tool_modules/</b>	<b>71</b>
<b>src/circadian/</b>	<b>72</b>
<b>src/</b>	<b>73</b>
<b>src/dashboard/</b>	<b>76</b>
<b>src/database/</b>	<b>79</b>
<b>src/server/</b>	<b>81</b>
<b>src/tui/</b>	<b>83</b>
<b>src/worker/</b>	<b>84</b>
<b>src/worker/tasks/ — Background Task Handlers</b>	<b>87</b>
<b>src/worker/tasks/hobbies/</b>	<b>105</b>
<b>womb/</b>	<b>106</b>



# Repository Atlas: Sweetheart 2.0 (Core)

## Project Responsibility

A completely rebuilt, modular, privacy-first AI companion system (“Jane”) — a memory-infused offline AI companion running entirely on local hardware. Features an FSM-driven reasoning agent, SQLite-based vector memory (sqlite-vec), background task queue, Telegram bot interface, OpenAI-compatible API server, and emotional/persona dynamics with circadian rhythms, menstrual cycle simulation, and dynamic mood baselines.

## System Entry Points

- `src/server/main.py`: FastAPI API server (OpenAI/Ollama-compatible endpoints) — the primary HTTP interface
- `src/agents/telegram_client.py`: Telegram bot polling loop — the mobile messaging interface
- `src/worker/worker.py`: Background task processor, Telegram polling, and world bot polling — the autonomous runtime
- `src/dashboard/main.py`: Web admin dashboard — the monitoring/admin interface
- `src/database/db_init.py`: Database initialization, schema setup, migrations, thread-local connection pool (`get_pooled_connection`, `close_pool`).
- `src/database/db_helpers.py`: Persistence helper functions (memories, ambient state with batch `get_ambient_states`, sentiment, chat history).

## Tech Stack

- **Runtime**: Python 3.11+, FastAPI, uvicorn, httpx
- **LLM**: Ollama (local) — Gemma 4 for primary chat, Gemma 2 2B for memory operations
- **Vector Search**: sqlite-vec (768-dim Nomic Embed Text via local Ollama embedding server)
- **Database**: SQLite (WAL mode, single file at `data/sweetheart.db`)
- **TTS**: Piper (fast) + XTTS (high quality) via `openedai-speech`
- **External APIs**: Telegram Bot API, Environment Canada citypage XML (weather), CalDAV/iCal (calendar), DuckDuckGo (web search), NotebookLM (knowledge notebooks)

## Directory Map

Directory	Responsibility Summary	Detailed Map
<code>src/agents/</code>	FSM reasoning pipeline (JaneFSMAgent) + Telegram bot client + Green Terminal display driver + detailed per-turn prompt/response logger + structured persona state management ( <code>persona.py</code> )	<a href="#">View Map</a>
<code>src/database/</code>	SQLite schema, connection management, CRUD, vector search	<a href="#">View Map</a>
<code>src/dashboard/</code>	Web admin dashboard + advanced query parser	<a href="#">View Map</a>

Directory	Responsibility Summary	Detailed Map
src/server/	FastAPI API server with OpenAI/Ollama-compatible endpoints	<a href="#">View Map</a>
src/worker/	Background task queue processor + Telegram polling	<a href="#">View Map</a>
src/worker/tasks/	Background task handler modules + core bot polling loop (memory/, ambient/, social/, schedule/, hobbies/, feeds, reader, kids, babysitter, weather, calendar_tasks, metrics, reconcile, notebooks, attachments, userbot_sync, group_chat_cleanup, homeassistant, log_archive, eval_tasks, google_photos_upload, core_bot)	<a href="#">View Map</a>
src/tui/	Textual play-world TUI (world simulator + shared commands + log explorer)	<a href="#">View Map</a>
src/	Root app source (top-level modules: config, circadian, embeddings, calendar_client)	<a href="#">View Map</a>
config/	Config codemap only — Jane's persona files moved to data/personas/jane/	<a href="#">View Map</a>
scripts/	Utility scripts (Google Drive backup, world events CLI, search evaluation, runtime health scoring)	<a href="#">View Map</a>
openedai-speech/	OpenAI-compatible TTS server (Piper + XTTS backends)	<a href="#">View Map</a>
src/personas/	PersonaProvider definitions: ProviderBase + config dataclasses, JaneProvider (52 blocks, full coupling), DottyProvider (10 blocks, no coupling), FriendProvider (5 blocks)	<i>(new)</i>

## Key Design Patterns

- **FSM Pipeline:** 6-state sequential pipeline (context → emotions → prompt → generate → format → index)
- **Context Manager Pattern:** `db_session()` for safe, auto-committing SQLite connection lifecycle

- **Enum-Typed Memory Operations:** `remember()` and `recall()` in `src/database/memory_utils.py` provide typed read/write access to the partitioned memory tables (`memories_{who}`, `vec_memories_{who}`, `memories_fts_{who}`) using `Who`, `Subject`, `SearchMode`, `MemoryKind`, and `MemorySource` enums. `get_subjects(who)` discovers which subjects a persona has memories in. `recall()` is the unified retrieval route for the FSM Agent context resolution and background tasks, supporting pre-computed embedding injection and environmental context boosts. It runs vector similarity search (sqlite-vec), keyword search (SQL LIKE), or both — merged and deduped by memory id, or falls back to chronological queries when search inputs are empty. Registered as an FSM tool (`recall_memories`) in `src/agents/tool_modules/recall.py`.
- **SQLite Vector Search:** sqlite-vec virtual table (`vec0`) for 768-dim KNN search joined against relational memory metadata
- **Decoupled Formulas:** Pure functions in the `src/circadian/` package (`__init__.py` + `coupling.py`) keep FSM and worker code clean of math/formulas
- **Centralized Config:** All schedules, intervals, and paths in `config.py` with `os.environ` overrides
- **Task Queue Pattern:** SQLite-backed FIFO queue with atomic pop, scheduled execution (`run_at`), and cron-style rescheduling
- **Persona Depth Layers:** Three stacked identity layers in the FSM prompt: core persona (`jane.md`), opinions layer (`jane_opinions.md` → `[OPINIONS & POSITIONS]`), and second-order values (`seed_second_order_values()` → `[WHO I AM TRYING TO BE]`). Values accumulate via HOV synthesis in `life_story_capsule_task`. The opinions layer was extended with **gated opinion snippets** — topic-matched per-snippet files in `data/personas/jane/opinions/` dynamically synced on-demand into the `memories_{who}` SQLite table under the "opinions" bucket, and retrieved at runtime using hybrid FTS5 and vector search with relevance filtering (L2 distance < 0.8), tracked independently via `_tc()` condition counters.
- **Score Component Pattern:** `src/score_component.py` — eight typed sub-score data-classes (memory coverage, task queue health, emotional state integrity, ambient data freshness, circadian phase, DB schema health, hybrid search quality, prompt block frequency) packaged into `ScoreComponent`. `from_db()` is the single factory that opens one connection, runs all scorers, and always returns a valid object (sentinel values on failure). Per-turn: cached on `JaneFSMAgent._score_cache`, enriched with search quality after retrieval. On-demand: `scripts/score-component.py --json` emits the array for full-repo-review skill consumption. `compute_search_quality_score(vec_hit_rate, kw_hit_rate)` is a module-level shared formula  $((vec * 0.5 + kw * 0.5) * 100)$  used by both `to_scores()` and the worker's `_snapshot_metrics`.
- **Two-Tier Metric History:** `metric_snapshots` (rolling 7-day raw at ~15 min cadence, piggybacked on `ambient_update_task`) + `metric_weekly_summaries` (up to 1-year weekly averages, Sunday 1 AM rollup). 26 metrics across four groups: Jane's state (6), system health scores (8), activity counts (9), friend closeness (data-driven). Exposed via `GET /api/metrics?range=24h|7d` and `GET /api/metrics/weekly` on the dashboard; visualised as four Chart.js panels on the Trends tab. Search quality crosses the FSM↔worker process boundary via `GET /internal/score-cache` on the API server.
- **Detailed Logging:** `src/agents/detailed_logger.py` — per-turn paired markdown logging (`*.prompt.md` / `*.response.md`) capturing verbatim Ollama payloads, raw model output, memory recall data, and system metrics. Gated behind `DETAILED_LOGGING_ENABLED`. Uses atomic writes (`os.replace`) and no module-scope side effects. Complementary to the existing `_write_prompt_log` single-file mechanism (see ADR-0036).

- **Topic Stakes Pattern:** Background tasks (RSS, document reader, friend letters) generate short first-person jane bucket memories called “stakes” via `_generate_stake()` in `llm_utils.py`. These surface during FSM vector search to give Jane grounded reactions when JCD follows up on things she’s encountered.
- **Response Post-Processing Filter:** Formats raw LLM responses programmatically before database persistence and voice cache generation. `post_process_response_structure()` in `generation.py` collapses paragraphs to single paragraphs, limits emojis (max 1), filters out robotic □ symbols, and preserves bracketed voice tag contexts (e.g. [cheerful]).
- **Embeddings API Passthrough Proxy:** Fast API path (POST /v1/embeddings in `server/main.py`) that maps incoming single-string or list-batch embedding payloads onto the local Ollama embeddings instance, centralizing external client integration under a single server port.
- **Help Tool Pattern:** Jane’s default Ollama tool schema contains only a single help tool. All other tools must be discovered by calling `help('<topic>')`, which returns markdown docs and injects the tool’s schema into the LLM’s tools array for the current turn. Dynamic injection via `build_tools_schema(extra_tools=...)` — see `src/agents/tool_modules/help.py` and `src/agents/manual/`. Detailed in ADR-0052.
- **Core Commands:** Interception layer for deterministic commands (e.g. `c:ping`) sent via OpenWebUI targeting the “core” system model. Processed by `CoreFSMAgent` in `agent_core.py` and routed through a decorator-based registry in `core_registry.py` instead of the standard persona FSM pipeline. Intercepts commands starting with `c:` directly at message start or with `@core c:.` Handlers live in `src/agents/core_commands/`.
- **Sentiment Tag Pass Pattern:** A 6th concurrent LLM pass in `session_extraction_sweep_task` classifies each message with one of 8 categorical tags (Reflective, Playful, Vulnerable, Curious, Warm, Challenged, Frustrated, Functional). Tags are per-participant by role (user→JCD, assistant→Jane). Stored in a 1:1 `message_sentiments` table with FK cascade delete. No FSM hot-path impact — runs at sweep time, not per turn. Single LLM call classifies the full transcript at once, returning a JSON array. Tag data is queryable for future dashboard heatmaps, relational diagnostics, and prompt injection. Tests require `side_effect` with prompt inspection (not `return_value`) due to concurrent pass mocking.
- **Partitioned Persona Memory Pattern:** Persona-specific memory tables (`memories_{who}`, `vec_memories_{who}`, `memories_fts_{who}`) store first-person perspective memories for Jane, JCD, and all friends and children in the `Who` enum. Initialized via `db_init.py` with triggers for virtual FTS5 and memory kinds. Accessed directly via unified `remember()` and `recall()` APIs. Async LLM generation via `store_persona_perspective()` in `llm_utils.py` reinterprets raw messages into first-person reflections. Wired into `friend_letter_task` (after each friend turn via `store_persona_perspective()`; observer-awareness writes to non-writing friends via `_stake_letter_for_friend()` with provenance `source_memory_id`), `kid_vignette_task` (for each child), and `friend_life_event_task` (weekly life event generation) using `remember()`. Legacy global memories and persona-specific tables are automatically migrated on startup.
- **Inter-Persona Relations Social Graph:** Master relationship graph is represented as a Mermaid flowchart inside `data/personas/inter-persona-relations.mermaid` parsed via `src/agents/social_graph.py`. Directional prompts live under `data/personas/<PERSONA>/relationship` loaded dynamically at runtime. Asymmetric closeness metrics are persisted as JSON in the `ambient_state` table and modified dynamically via daily background task sweeps using the LLM for dialogue simulation and feedback evaluation (gossip/closeness deltas).

## World Location System

- **WorldService** (`src/agents/world_service.py`): Owns all page I/O (markdown in `data/world/{slug}.md`), DB operations (`world_position_log`, `world_links`), BFS shortest-path pathfinding, link management, and **passing events** (ambient probability-triggered events rendered on `look_around/move_to`, optionally LLM-translated). Uses `_db_session()` context manager for isolated DB connections per call.
- **world tool** (`src/agents/tool_modules/world.py`): Single FSM tool dispatching 8 sub-actions (`look_around`, `move_to`, `new_location`, `modify_surroundings`, `create_travel_path`, `remove_travel_path`, `travel_history`, `invite`) — `look_around/move_to` also render active passing events. Allowed personas: `[Who.JANE]`. Returns natural language strings.
- **Generic invite service** (`src/agents/invite_service.py`, ADR-0072): Stateless multi-recipient DM invitations shared by the world tool `invite` action and the Telegram world bot `/invite` command. Validates persona names against the bot-config allowlist; resolves each persona's `chat_id` via `config.get_persona_chat_id` (falls back to `core.chat_id`); Markdown-escapes an optional `-- <message>`; returns per-target partial-success results.
- **Hybrid storage**: Location content in flat files, link graph in SQLite (DB is source of truth for links; `## Move to... auto-generated` at render time). Links are always bidirectional (enforced at write time via dual INSERT).
- **World Areas** (ADR-0059 extension): Hierarchical location grouping via `world_areas` (self-referential tree) and `world_area_locations` (one location per area) tables. WorldService exposes `create_area`, `delete_area`, `add_location_to_area`, `remove_location_from_area`, `reparent_area`, `get_breadcrumbs`, `get_area_tree`, and `render_world_graph`. Breadcrumbs rendered in `_format_page` on `look_around/move_to`. Area CLI at `scripts/world_events.py`; `scripts/world.py` import also syncs areas from Mermaid subgraph blocks.

## World Events (ADR-0001)

- **Scheduled events**: `WorldEventsService` (`src/agents/world_events_service.py`) loads cron-defined event files from `data/world_events/{slug}.md`, polls the `world_events` DB table via `poll_due_events()`, and executes effects (e.g. `set_location_flag`). Supports multi-location targeting via `_resolve_target_locations()` (4 selectors: `location_slug` single, `location_slugs` list, `location_pattern` glob, `location_tags` — OR union + dedup). Cross-tier passing injection seeds ambient events per target location. Backed by `CronSchedule` (`world_schedule.py`: `parse_cron` + `is_due`). Polled by worker task `poll_world_events` (`src/worker/tasks/world_events_task.py`). CLI at `scripts/world_events.py`.
- **Passing events**: Defined in location pages' front matter under `passing_events`: — probability-gated, conditions-aware (`time_of_day`, `tag`, `weather`), transient (lifetime-controlled). Rolled fresh on each `look_around/move_to`, persisted in `world_passing_events` table, optionally LLM-translated for vivid descriptions via `WorldService._llm_translate()`. DB cleanup via `prune_expired_events()`.
- **DB tables**: `world_events` (scheduled event metadata + `last_fired`), `world_event_log` (execution audit trail), `world_passing_events` (active transient events). Schema in `src/database/db_init.py`.

## Notebook Access (ADR-0045)

- `src/agents/notebook_backend.py`: pure-I/O backend. `run_notebook_query` is a hybrid read — NotebookLM via the `nlm` CLI (for registry entries with an `id`) merged with the notebook's Drive notes; `record_notebook_note` always writes Markdown to Drive notebooks/`<name>`/ with timestamped filenames. `_resolve_notebook_folder(create=...)` shares folder lookup (reads never create).

- **FSM tools** (tool\_modules/notebook\_query.py, notebook\_create\_note.py): notebook\_query enqueues the async notebook\_query worker task (result stored as a memory) and returns a placeholder; notebook\_create\_note records to Drive + a reminder memory. Registry (config.NOTEBOOKS) loaded from gitignored data/notebooks.json.
- **Reused by** iroh\_morning\_task for its wisdom seed. nlm failures surface on the Green Terminal (actionable on auth).

## Multi-Agent Group Chat (ADR-0060)

- **DB tables:** group\_chats (chat metadata, location slug, active status), group\_chat\_messages (per-chat messages with role/agent/visibility), group\_chat\_presence (currently present agents per chat). Schema in src/database/db\_init.py.
- src/agents/referee\_service.py: Three-phase player selection pipeline (pre-filter → LLM rank with persona context → threshold selection). Per-agent referee.yml configs in data/personas/<name>/referee.yml. Anti-volley throttling, recursion safeguard, per-channel config overrides.
- src/agents/group\_chat\_service.py: Module-level synchronous CRUD functions (create\_group\_chat, post\_message, list\_group\_chats, get\_present\_agents, db\_leave\_chat) + async GroupChatService class wrapping pipeline methods (post\_message → \_run\_experience + \_run\_referee\_pipeline → \_generate\_agent\_response). OWU bridge via \_push\_to\_owu\_channel. Returns 500/400 on webhook errors (not 200).
- **World integration:** WorldService.\_set\_position() in src/agents/world\_service.py calls \_update\_group\_chat\_presence() on move\_to/look\_around.
- **Worker cleanup:** group\_chat\_cleanup\_task in src/worker/tasks/group\_chat\_cleanup.py — archives stale chats hourly.
- **REST API:** /api/group-chats/\* router in src/server/routes/group\_chat.py. OWU webhook at POST /api/group-chats/owu-webhook (returns 500 on post\_message failure, 404 on unknown channel).
- **Per-channel config:** GET/PUT /api/group-chats/{chat\_id}/referee/config for referee overrides.

## PersonaProvider Protocol (ADR-0057)

- **Location:** src/personas/ (providers) + src/agents/agent\_base.py (wiring)
- ProviderBase + config dataclasses (EmotionConfig, LLMConfig, MemoryConfig, PromptBlock) define all persona-specific decisions
- PersonaFSMAgent delegates identity, prompt blocks, emotion config, LLM config, and memory config to the provider via the provider kwarg
- **Concrete providers:** JaneProvider (52 blocks, full coupling chain), DottyProvider (10 blocks, no coupling), FriendProvider (5 blocks, no coupling)
- Coupling chain gated via provider.emotion\_config.coupling\_chain in \_evaluate\_emotions()
- \_generate\_response() parameterized via provider.llm\_config.num\_ctx/num\_predict
- \_inject\_prompt() delegates to provider.build\_prompt() when provider is set, falls back to build\_system\_prompt() for backward compat ## Prompt Instrumentation & Debugging **Prompt Block Size Tracking:** Every dynamic block in the FSM system prompt (52 blocks) is measured at assembly time via a \_before/\_block\_sizes dict pattern in the block engine (prompt\_engine.py / prompt\_builder.py). Block sizes are logged per-inference to the prompt\_block\_log SQLite table, enabling distribution analysis (avg, max, min, count) over time. Toggled via PROMPT\_BLOCK\_STATS\_ENABLED env var (default: false). **Condition Frequency Tracking:** Each conditional check inside build\_system\_prompt() that decides whether to include a block is tracked with up/down counters via a \_tc() wrapper. Every if branch increments checked\_count; if the block is actually injected, fired\_count increments. Results stored in prompt\_condition\_stats

with upsert semantics, visible on the dashboard's Prompt Composition Stats card. Gated behind the same `PROMPT_BLOCK_STATS_ENABLED` flag. **Full Prompt Logging:** When enabled via dashboard toggle (or `FULL_PROMPT_LOGGING_ENABLED=true`), the FSM captures the complete `ollama_messages` payload (deep copy before the tool loop) after each inference and writes it as formatted markdown to `data/FULL_PROMPT_LOGS.md`. Limited to 10 rounds before auto-deactivation. The logging state (`full_prompt_logging_active + full_prompt_logging_remaining`) is stored in `ambient_state` and survives restarts. **Feature Flag Pattern:** All instrumentation defaults to off. `PROMPT_BLOCK_STATS_ENABLED=false` and `FULL_PROMPT_LOGGING_ENABLED=false` in `.env` — toggle via the dashboard at runtime. The dashboard toggle endpoint checks the env var at request time and returns 400 if the env var is disabled.

**Tool Usage Stats:** When `TOOL_STATS_ENABLED=true` (default), every tool call through the FSM's `execute_tool()` is logged to `tool_log` with latency, status, and result size. Cumulative counters (`call_count`, `error_count`, `total_latency_ms`) are upserted to `tool_usage_stats`. Self-cleaning 7-day prune. Exposed via `GET /api/tool-stats/usage` and `GET /api/tool-stats/history` on the dashboard.

## Observability & Log Shipping

**JSON Structured Logging:** `src/logging_utils.py` defines `log_event()` — the universal log emitter producing a single-line JSON envelope `{timestamp, level, logger, message, correlation_id, ...context}`. `correlation_context()` / `get_ambient_correlation_id()` attach an ambient `correlation_id` via `contextvars` for background/task work. Secrets in context values are redacted by `_redact_known_secrets()` before serialization. See ADR-0073. **Loki Streaming (ADR-0073):** `setup_loki_logging()` lazily attaches a cached `LokiQueueHandler` (`python-logging-loki-v2`) to the **root logger** — Python propagation delivers every record to it automatically. Configured via `LOKI_URL` (default `http://host.docker.internal:3100/loki/api/v1/push`) and `LOKI_ENABLED` (default 1). Wired once per entrypoint: `src/server/main.py`, `src/worker/worker.py`, `src/dashboard/main.py`. Non-blocking (background `QueueListener` thread), failure-isolated (handler swallows emit errors), label discipline: low-cardinality labels (`service`, `who`, `kind`) via `log_event(tags=...)`; high-cardinality ids stay in the JSON body for `LogQL | json`. **Memory events:** `add_memory()` in `src/database/helpers/memories.py` emits a tagged flat event=`"memory"` line (`who/kind` labels; `Who.TESTER` skipped) via the shared `log_memory()` emitter in `src/logging_utils.py`. Backfill of existing rows: `scripts/backfill_memories_loki.py` — idempotent per-persona watermark cursor (`loki_backfill_cursor_{who}`) in `ambient_state`; `--reset` re-ships everything.

## Glossary

- **Conversation (UI)** — Refers to the `sessions` table and `session_id` fields in the database/backend code. The UI presents this as “Conversation” for approachability.

## **config/**

### **Responsibility**

Static configuration assets. Jane's persona files moved to `data/personas/jane/`.

### **Files**

**(personas directory — empty after migration)**

### **Integration**

- **Personas migrated to:** `data/personas/jane/` (loaded via `src.prompts.render_prompt()`)



## openedai-speech/

### Responsibility

OpenAI-compatible Text-to-Speech (TTS) server. Provides the `/v1/audio/speech` endpoint that Jane's system calls to generate voice responses. Supports two TTS backends: Piper (fast, low-quality — model `tts-1`) and XTTS (higher quality — model `tts-1-hd`).

### Key File

`speech.py` — TTS Server

- **OpenAI-compatible endpoint:** `POST /v1/audio/speech` with OpenAI-style request format (model, input, voice, response\_format, speed)
- **Dual engine:**
  - `tts-1`: Piper TTS — subprocess-based, lightweight, fast
  - `tts-1-hd`: XTTS (Coqui) — neural TTS with higher quality, GPU acceleration (CUDA/MPS), idle unload timer
- **Emotional prosody modulation:**
  - Reads Jane's emotional state (energy, mood) directly from Sweetheart's SQLite database
  - Adjusts speed (energy-based:  $\pm 5\text{-}10\%$ ) and pitch (mood-based:  $\pm 4\text{-}5\%$ , circadian-based:  $\pm 2\text{-}8\%$ )
  - Applies style modifiers from `config/style_modifiers.yaml` (e.g., [whisper], [excited], [cozy]) — volume, speed, pitch multiplicative factors
- **Style tag processing:** `split_text_by_style_tags()` parses [style] tags in input text, applies per-segment modifiers; `extract_tag_for_substring()` preserves tags from cached rewritten text
- **TTS Rewrite Cache Integration:** `check_tts_cache()` looks up the Sweetheart TTS rewrite cache to restore emotional voice tags removed during LLM generation
- **Caching:** SHA-256 based audio output cache (per model/voice/format/speed/text combo), serves cached audio directly
- **Audio processing:** Uses ffmpeg subprocess for format conversion (MP3, Opus, AAC, FLAC, WAV, PCM) and audio filter application (pitch shift via `asetrate`, tempo via `atempo`, volume scaling)
- **Padding & Delay Settings:** `load_tts_settings()` reads `config/settings.yaml` on-demand to apply start delay (`adelay`) and end/mid padding (`apad`) to prevent audio clipping.

### Integration

- **Consumed by:** `src/dashboard/main.py` (TTS proxy endpoint), `src/agents/telegram_client.py` (voice reply generation)
- **Depends on:** Sweetheart SQLite database (reads `emotional_state` table), ffmpeg, Piper/XTTS models

## openedai-speech/config/

### Responsibility

Configuration files for the openedai-speech TTS server — voice mapping, style modifiers, and pre-processing rules.

### Files

`voice_to_speaker.yaml`

Maps OpenAI-compatible voice names (alloy, echo, fable, onyx, nova, shimmer) to specific TTS engine configurations: - For tts-1 (Piper): Model path and optional speaker ID - For tts-1-hd (XTTS): Model name, speaker sample path, language, and optional speed override

`style_modifiers.yaml`

Defines emotional/acoustic style tags as multiplicative factors: - **Primitives**: whisper, excited, cozy, serious, soft, breathless, slower, faster, lower, deeper, higher, quiet, hushed - **Aliases**: Map natural language phrases to primitive combinations (e.g., whispering → whisper, sultry → lower + slower + deeper) - Each modifier adjusts volume, speed, and/or pitch factors

`pre_process_map.yaml`

Regex-based text pre-processing rules applied before TTS generation (e.g., expanding abbreviations, cleaning punctuation).

`settings.yaml`

Defines silence padding and delay settings (start delay, mid padding, end padding) to prevent audio clipping. Hot-reloaded on every request.

### Integration

- **Consumed by**: `openedai-speech/speech.py` (loaded on-demand via YAML, supports hot-reload via `mtime` check)

## scripts/

### Responsibility

Utility scripts for backup, evaluation, and runtime health scoring. These are standalone operational tools, not part of the main application runtime.

### Files

#### `backup_to_gdrive.py` — **Google Drive Backup**

- Performs online SQLite backup (sqlite3.backup API for safe copy without locking)
- Creates timestamped zip archive of database + state files + personas
- Uploads to Google Drive via gws CLI
- Prunes backups older than 7 days (keeps 7 most recent)
- Cleans up local temp files on completion

#### `score-component.py` — **Runtime Health Scoring CLI**

- Instantiates `ScoreComponent.from_db()` from `src/score_component.py` against the live database
- Translates the eight sub-score dataclasses into a `[{name, score, grade, issues}]` array with letter grades (A-F) and human-readable issue strings
- `--json` flag emits the array as JSON, compatible with the `full-repo-review` skill's pre-check format
- Accepts `--all-agents` / `--all-skills` flags (silently ignored) for CLI compatibility with the global `compound-powers` `score-component.py`
- Injects `sys.path` at startup so `src.*` imports resolve without an activated venv; uses `.venv/bin/python3` as the preferred interpreter in skill contexts

#### `eval_retrieval.py` — **Hybrid Search Evaluation**

- Evaluates memory retrieval quality by running hybrid (vector + keyword) search against a set of test queries
- Reports hit rate, rank distributions, and RRF score statistics for tuning search parameters

#### `play_world.py` — **Interactive World Explorer**

- Provides an interactive text-adventure REPL for humans to navigate the world.
- Run `./scripts/play_world.py` to log in as a persona.
- All commands use `/` prefix (e.g. `/look`, `/move kitchen`, `/help`), leaving bare text available for future chat mode.
- Tab completion: `/m+Tab` completes commands; `/move+Tab` completes directly-linked rooms; `/fast//link//unlink` complete all world slugs.
- Markdown rendering via `_render()` — lightweight Python ANSI formatter (headings, bold, italic, code, links, lists, blockquotes). First `# H1` gets dash separators; subsequent `H1`s render as cyan bold.
- `/edit` opens the current location in `$EDITOR` (default `vim`).
- Aliases: `/l` → `/look`, `/mv` → `/move`, `/q//x` → `/quit`.
- Tests at `tests/agent/test_play_world.py` (26 tests covering render and completer).

### `play_world_tui.py` — **Textual TUI World Explorer**

- Provides a rich, split-pane terminal user interface (TUI) built using the Textual library to replace the legacy `play_world.py` REPL.
- Run `./scripts/play_world_tui.py [persona_name]` to log in (defaults to `jcd`).
- Left column displays a scrollable real-time message stream and input box. Supports typing normal messages or command shortcuts with `/` or `:` prefixes (e.g. `:move kitchen`).
- Right column hosts a dynamic `ContentSwitcher` sidebar containing a `WorldInfoWidget` that queries location metadata, present characters, and linked paths in real time.
- Tests at `tests/agent/test_play_world_tui.py` verify layout rendering, dynamic widget updates, command parsing, and live polling.

### `world.py` — **World Service CLI**

- Provides a CLI for batch-managing the virtual world map (locations and links).
- Supports `export` (output a Mermaid graph of all nodes and links), `import` (parse a Mermaid graph to CRUD links and spawn missing `.md` location files), `generate` (use an LLM to flesh out a location's description/contents based on title and prompt), and `delete` (remove a location file and all its associated links from the database).
- `import` supports optional `yaml` blocks inside the markdown file to pass specific creative prompts for generated nodes (e.g. `prompts: { lobby: "Make it creepy" }`). Use `"ignore"` or an empty value (e.g. `lobby:`) to skip generating content for a node while still mapping its links.

### `world_events.py` — **World Events & Areas Management CLI**

- CLI for managing the world events system (scheduled + passing) and world area hierarchy.
- `list` — lists all event files with schedule and type
- `dry-run <slug>` — shows what an event would do without executing
- `trigger <slug>` — manually fires a scheduled event
- `sync` — synchronizes event file definitions to the DB
- `passing <location-slug>` — shows active passing events at a location
- `find <pattern>` — search world pages by pattern
- `graph` — renders Mermaid flowchart with area subgraphs
- `export-areas [path]` — exports area hierarchy to JSON
- `import-areas [path]` — imports area hierarchy from JSON
- `compile <expression>` — placeholder for human-to-cron compilation
- Logger: script (basicConfig to stdout)

### `regenerate_zero_vectors.py` — **Zero-Vector Repair**

- Finds `vec_memories` rows where the embedding is an all-zero vector (indicating a failed embedding call)
- Re-embeds the corresponding memories via Ollama and updates the vector index

### `generate_persona_seeds.py` — **Persona Seed Generation**

- Generates 8 seed memories for each persona (Arthur, Lyra, Kate, Clara, Finn)
- Uses `_query_llm` to produce first-person perspective memories in JSON format
- Writes results to `data/persona-seeds/{name}_seeds.json` for manual editing before injection
- Run before `inject_persona_seeds.py`

### inject\_persona\_seeds.py — **Persona Seed Injection**

- Reads edited seed JSON files from data/persona-seeds/
- Injects each seed into the corresponding persona\_memories\_{name} table via add\_persona\_memory()
- Skips missing seed files gracefully (idempotent)

### backfill\_persona\_memories.py — **Persona Memory Backfill**

- Reinterprets existing about\_friend\_{name} and about\_jane (child metadata) memories into first-person persona perspectives
- Uses \_query\_llm per memory to rewrite from the persona's point of view
- Parameterized SQL for safe bucket filtering

### verify.sh — **Full Verification Script**

- Runs ruff lint + mypy type check + full pytest suite in sequence
- Used as the pre-PR quality gate

### run\_backfill.py — **Sentiment Tag Backfill**

- One-shot runner for backfill\_sentiment\_tags\_task.
- Dynamically updates sys.path to resolve src imports from the scripts directory.

## **Additional Scripts (brief)**

**Backfill / migration:** backfill\_eval\_sessions.py (evaluate completed sessions missing from eval\_history), backfill\_friend\_letters.py, backfill\_jane\_pronouns.py, backfill\_letter\_stakes.py (letter-awareness memory backfills), backfill\_memories.py + \_grafana/\_loki variants, backfill\_memory\_kinds.py, backfill\_memory\_urls.py, backfill\_notebook\_queries.py, backfill\_sentiment\_tags.py, clean\_memory\_metadata.py, clean\_orphaned\_vectors.py, migrate\_2024\_06\_reset.py, migrate\_calendar\_to\_jane.py, migrate\_drive\_notebooks.py, migrate\_drive\_photos\_to\_photos.py, migrate\_nlm\_to\_drive.py, populate\_jane\_jonathan.py

**Replay / history:** extract\_notebook\_replay.py, extract\_owu\_replay.py, extract\_task\_outputs.py (replay-entry extraction), replay\_db.py (reconstruct sweetheart.db), hipbone\_backfill.py, hipbone\_owu\_backfill.py, hipbone\_telegram\_backfill.py (replay HipBone game logs), telegram\_backfill.py (group chat → OWU), heal\_owu\_chat.py (linear → tree chat graphs), reconcile\_owu.py (on-demand DB → OWU)

**Sync / auth / drive:** jane\_drive.py (Jane's Drive CLI), check\_drive\_scope.py (OAuth scope check), setup\_google\_photos\_auth.py, weekly\_export.py, dump\_tables\_to\_git.py, archive\_logs.py, consolidate\_logs.py, codebase\_size.py

**Diagnostics / eval:** eval\_generator.py, ragas\_eval.py, compare\_llm\_rank\_prompts.py, probe\_notebook\_turn.py, probe\_tool\_selection.py, test-prompt.py, test\_llm\_discoverability.py, perceive\_clip.py, read-aloud.py, owu\_chunk\_chat.py, patch-owu-models.py, persona-nudge.py

**Codegen / dev:** split\_task\_file.py, split\_tasks.py, test\_google\_photos\_upload.py, \_\_init\_\_.py

## **Integration**

- **Depends on:** src/config.py, src/embeddings.py, src/database/, src/score\_component.py (score-component only), src.worker.tasks.llm\_utils (seed/backfill scripts)

- **External deps:** gws CLI (Google Workspace CLI) for Drive operations

## src/agents/

### Responsibility

Implements the FSM reasoning pipeline and the Telegram bot integration layer. This is the core “brain” that processes user messages through context resolution, emotional evaluation, prompt assembly, LLM generation, and memory indexing.

### Key Files

- `agent_base.py` — **PersonaFSMAgent (Abstract Base Class)**: Shared FSM base. Defines `get_persona_name()` abstract method, `_generate_response()` (reads `provider.llm_config.endpoint + model` — provider model wins over the caller’s `request.model`), `_inject_prompt()` default (delegates to `provider.build_prompt()`; falls back to the generic `ProviderBase.build_prompt` template when no provider is set), and the tool-loop orchestration inside `process_chat()`. Accepts optional provider kwarg — when set, delegates all persona-specific decisions (prompt blocks, emotion config, LLM config, memory config) to the provider. Passes `self.persona.value` through the generation pipeline for partitioned chat table routing.
- `fsm.py` — **JaneFSMAgent (extends PersonaFSMAgent)**: Jane-specific FSM orchestrator. Implements `get_persona_name()`. Manages reasoning pipeline: `RESOLVE_CONTEXT` → `EVALUATE_EMOTIONS` → `INJECT_PROMPT` → `GENERATE_RESPONSE` → `FORMAT_OUTPUT` → `INDEX_MEMORIES`. Operates via default `PersonaFSMAgent` pipeline using injected `JaneProvider`.
- **States**: `RESOLVE_CONTEXT` → `EVALUATE_EMOTIONS` → `INJECT_PROMPT` → `GENERATE_RESPONSE` → `FORMAT_OUTPUT` → `INDEX_MEMORIES`
- Uses Ollama LLM API (`/api/chat` with tools, `/api/generate` for sub-queries) for response generation; streaming and non-streaming generation delegated to `generation.py`
- Streaming: `TagStripperStream` strips [style] tags for clean text output while preserving them for TTS cache
- TTS Rewrite Cache: Stores original tagged text → clean text mappings in `tts_rewrite_cache.json`
- Query expansion: Uses `MEMORY_LLM_MODEL` to expand user messages into better semantic search queries before vector retrieval
- Prompt assembly: Delegates to `JaneProvider.build_prompt()` (the block engine) via `_inject_prompt`
- Notebook registry: `NOTEBOOK_IDS` and tool enums are built dynamically from `data/notebooks.json` at module load time
- Re-exports symbols from `tools.py` and `prompt_builder.py` for backwards compatibility with `from src.agents.fsm import ...`
- **Full prompt logging**: Delegates to `_write_prompt_log()` in `generation.py`. State tracked in `ambient_state` as `full_prompt_logging_active + full_prompt_logging_remaining`. Gated behind `FULL_PROMPT_LOGGING_ENABLED` env var (default: `false`) and dashboard toggle.
- **Score cache**: `self._score_cache` holds a `ScoreComponent` populated at the start of each `process_chat()` call via `ScoreComponent.from_db()`, then enriched with hybrid search quality metrics after `_resolve_context()`. Available for per-turn health introspection by callers.

### `generation.py` — Streaming & Non-Streaming Response Generation

- `generate_stream()` — async generator: streams Ollama `/api/chat` response, strips tags, collapses paragraphs and limits emojis via `post_process_response_structure()`, saves messages, enqueues `extract_memories` task, invokes calendar scheduling

- `process_final_non_stream()` — non-streaming path with identical post-generation and post-processing filter steps
- `TagStripperStream` — custom stream parser: strips [style] and suppresses <think> / [Self-Correction] blocks from chunks during LLM streaming
- `post_process_response_structure()` — post-processing formatting guard that collapses paragraphs, limits emojis (max 1), strips robotic □ symbols, and preserves voice tags
- `check_calendar_scheduling()` — post-generation LLM sub-call that classifies the conversation turn and enqueues `add_calendar_event` tasks when needed
- `strip_thinking()` / `strip_bracket_tags()` / `normalize_content()` — formatting helpers shared between streaming and non-streaming paths
- `_write_prompt_log()` — appends full prompt captures to `data/FULL_PROMPT_LOGS.md`
- Logger: `agent.generation`

### `tool_registry.py` — **Central Tool Registry**

- `ToolDef` dataclass: name, description, parameters (JSON schema), fn
- `REGISTRY` dict: name → `ToolDef` — populated at import time by each tool module
- `TOOL_MAP`: dynamic Mapping read-through to `REGISTRY` — no stale-map bugs
- `build_tools_schema()`: generates the tool schema list for Ollama from `REGISTRY`
- Logger: none (pure data + one function)

Each module defines its own `execute()` function + `ToolDef` + `REGISTRY` registration. Tools (14): `dotty_control`, `flag_curiosity`, `get_citations`, `hipbone`, `images`, `notebook`, `reach_out`, `read_calendar`, `recall_memories`, `schedule_calendar_event`, `start_focus_session`, `sticky_note`, `web`, `world`. Several are action-dispatch “drawers” (`notebook`, `web`, `hipbone`, `images` — one tool with an action enum + per-action params — collapsed to fit the model’s tool-selection limit (ADR-0046)).

### `deps.py` — **Shared Dependency Container**

- Singleton `deps` object holding `get_embedding` (tools now use `remember()` from `memory_utils.py` for memory writes)
- Tool files call `deps.get_embedding(...)` instead of direct imports
- Tests patch `src.agents.deps.deps.get_embedding` once — covers all consumers
- Avoids per-module patch targets that make test fixtures fragile
- Logger: none

### `tool_services.py` — **Gated User Messaging Service**

- `SendResult` `StrEnum`: `SENT`, `BLOCKED` — type-safe return values for send attempts
- `send_user_message(friend, message, context="")` — async function that enforces DND and rate-limit gates before calling `send_telegram_message()`
- `_try_acquire_rate_slot(friend)` — atomic check+record for rate limiting (no race between check and write)
- Rate limit: `REACH_OUT_RATE_LIMIT` messages per `RATE_LIMIT_WINDOW` seconds, stored in `ambient_state`
- DND gate calls `is_dnd_active()`; rate limit gate uses `_try_acquire_rate_slot()`
- Gate is baked into the send function itself — tools cannot bypass it by calling `Telegram` directly
- `send_telegram_message` is lazy-imported inside the function body to avoid import-time side effects
- Used by `reach_out.py` and any future tool needing to message the user
- Logger: `agent.tool_services`



- `background_task(coro)` — creates an asyncio task with a strong reference held in a module-level set, preventing Python 3.14+ GC from collecting fire-and-forget tasks. Use instead of raw `asyncio.create_task()` for all background work. Self-cleaning via `done_callback`.

#### `world_events_service.py` — **World Events Service**

- `WorldEventsService`: Loads cron-scheduled event files from `data/world_events/{slug}.md`, executes effects (e.g. `set_location_flag`).
- `_resolve_target_locations(cfg)`: Multi-location resolver supporting 4 selectors (single slug, explicit list, glob, tags) with OR union + dedup.
- `load_events()` / `poll_due_events()` / `execute_event()` / `dry_run_event()` — full lifecycle.
- Cross-tier passing injection when effect has a passing block (avoids database is locked by calling after DB close).
- Logger: `events.world_events_service`

#### `vision.py` — **Photo Download, Encoding & Disk Storage**

- `download_telegram_photo(file_id)` — Downloads photo bytes from Telegram File API via `httpx` (15s/30s timeouts)
- `encode_image_base64(bytes)` — Converts image bytes to base64 string for Ollama `/api/chat` images field
- `save_photo_to_disk(bytes)` — Saves to `data/photos/{uuid}.jpg`, creates `PHOTO_DIR` if needed
- `process_telegram_photo(file_id)` — Convenience: `download` → `encode` → `save` → 20MB size validation
- Telegram-agnostic design — reusable by dashboard upload, API, etc.
- Logger: `agent.vision` ### `document_chunker.py` — Text Extraction, Chunking & Page Processing
- `extract_text(data, name)` — Extract text from bytes based on file extension (txt, md, pdf, epub, html, csv, json, xml, yaml, log)
- `extract_text_from_path(path)` — Same extraction from a file path
- `chunk_into_pages(text, words_per_page=1000)` — Split text into word-aligned page chunks
- `process_chunks(filename, pages, source_label, meta)` — LLM loop: generates a first-person stake per page via `_query_llm_chat`, stores in `about_jane` via `remember()`, returns combined reflections
- `send_reading_followup(reflections_text, filename, chat_id, owu_chat_id)` — Sends completion follow-up to Telegram or OWU chat
- Consumed by `owu_attachments.py`, `telegram_client.py`, `worker/tasks/reader.py`, and `worker/tasks/attachments.py`
- Logger: `agent.document_chunker` ### `owu_attachments.py` — OpenWebUI Attached Files Resolution
- Parses `<attached_files>` blocks in OWU messages, fetching files via OWU files API
- Audio: delegates to `audio_perception.py` for STT + delivery description
- Documents: extracts text via shared `document_chunker.py`; small docs inline, large docs enqueue `process_attachment_chunks` worker task
- Images: base64-encodes for Ollama vision, strips OWU RAG scaffolding
- Strips OWU's `<context>` / ### Task: scaffolding from user messages
- Logger: `agent.owu_attachments`

### `social_graph.py` — **Social Graph Mermaid Parser**

- `parse_social_graph(filepath)` — Parses the master Mermaid flowchart (`data/personas/inter-persona-relations.mermaid`), mapping character shortcodes (j, l, k, a, cg, jg, mg) to full names, and extracts relationship types and base probabilities.

### `tools.py` — **Tool Router & Shared Utilities**

- `T00LS = build_tools_schema()` — dynamically built from `REGISTRY`
- Re-exports `T00L_MAP` for `fsm.py` dispatch
- Hosts shared utilities: `ChatMessagePayload`, `_EMOTION_DEFAULTS`, `update_tts_rewrite_cache()`, `_selective_forget()`, `_pick_nlm_placeholder()`
- Re-exports `execute_*` functions for test backward compatibility
- Logger: `agent.tools`

### `agent_base.py` — **PersonaFSMAgent (Abstract Base Class)**

- Abstract base class for all FSM-based agents (Jane, Dotty, etc.)
- Manages LLM execution, generation strategies, function calling, tool routing, and vision payload processing
- Accepts optional provider kwarg; delegates prompt assembly, emotion config, LLM config, and memory config

### `agent_dotty.py` — **DottyFSMAgent**

- Dotty's persona implementation
- Inherits from `PersonaFSMAgent`, restricted to interacting with `Who.D0TTY` and `Subject.GENERAL` memory namespaces
- Uses `DottyProvider` via the provider pipeline — stripped `_inject_prompt` override

### `persona.py` — **Structured Persona State**

- Persona base class: standard interface for retrieving and saving structured state from/to ambient state
- `Chrissie`, `Jeff`, `Martha` subclasses defining defaults for tracking dynamic behaviours (tangents, mood swings, alcohol consumption)

### `coordination.py` — **Scheduling & Booking Coordination**

- `get_event_metadata` / `update_event_metadata` structured metadata HTML comment reader and writer
- `SchedulingEntity` and `ChrissieSchedulingEntity` scheduling and availability checkers
- `BabysitterBooker` babysitter eligibility filter and text/phone dialogue booking negotiator

### `prompt_dotty.py` — **Dotty's Prompt Builder**

- Lightweight context assembler for Dotty
- Fetches her persona identity, basic ambient sensors (lighting, motion), and builds her system prompt

### `fsm.py` — **JaneFSMAgent**

- Inherits from `PersonaFSMAgent`, retains Jane's complex FSM processing and full prompt builder. (Kept as `fsm.py` for backward compatibility)

- `execute_tool()` dispatches via `TOOL_MAP[name](**arguments)` — no elif chain
- Re-exports `TOOLS`, `TOOL_MAP`, `_EMOTION_DEFAULTS`, `ChatMessagePayload` from `tools.py`

#### `tool_modules/world.py` — **World Navigation & Editing**

- Single `execute()` dispatch with 8 sub-actions: `look_around`, `move_to`, `new_location`, `modify_surroundings`, `create_travel_path`, `remove_travel_path`, `travel_history`, `invite`
- Delegates to `WorldService` (lazy singleton via `_get_service()`)
- `invite` action delegates to `invite_service.send_invites` (personas list + optional message)
- `allowed_personas=[Who.JANE]` — Jane-only for now
- All return natural language strings, never raw JSON

#### `tool_modules/help.py` — **Single-Entry Help Tool**

- Replaces all 14 default Ollama tool schemas with a single discovery tool
- Fuzzy keyword matching (exact name → keyword map → description substring)
- Returns `JSON {"discovered": [...], "docs": "..."} via the tool result channel`
- Manual documentation in `src/agents/manual/*.md` loaded on demand
- Agent loop rebuilds tools schema dynamically from `_discovered_tools` set
- See ADR-0052 for full architecture
- `prompt_builder.py` — **Prompt read-model**: `resolve_prompt_context(who, user_mems, persona_mems, stuff_mems)` performs every DB/ambient/emotional read once per turn and returns a `PromptContext` (the pre-resolved read-model). The former `build_system_prompt` monolith has been removed — assembly now lives in the block engine (ADR-0058, survey A1).
- `prompt_engine.py` — **Generic PromptBlock Assembly Engine**: `assemble_from_blocks(blocks, ctx)` iterates a provider's ordered `PromptBlock` list over a pre-resolved `PromptContext`: gates by condition, calls `build`, appends `CORE/budget-selects ENRICHMENT_*` (via `select_enrichment_blocks`) inserting them at the `ENRICHMENT_SLOT` marker, runs each included block's effect, then every block's teardown, and preserves `flush_prompt_stats` instrumentation. This is Jane's live prompt path (`JaneProvider.build_prompt`); it replaced the `prompt_builder.py` monolith via a golden-snapshot-gated strangler migration.
- `resolve_prompt_context()`: Resolves the read-model for the full dynamic system prompt from persona file + ambient state (weather, schedule, calendar) + emotional state (mood, energy, closeness, menstrual cycle) + circadian phase + groggy/sleeping state + memories (JCD facts, Jane reflections, general knowledge) + mood reading + journal entries + threads to revisit + JCD schedule + reconnection warmth; the block engine renders it
- Prompt blocks: Injects `[NOTEBOOKS SHELF]` (always present) and `[RECONNECT WARMTH]` (for 3-12h gaps)
- Extracted from the former `_inject_prompt` method during `fsm.py` modularization
- Logger: `agent.prompt_builder`
- **Block size tracking**: Each of 52 prompt blocks measured via `_before/_block_sizes` dict. Sizes logged per-inference to `prompt_block_log` table.
- **Condition frequency tracking**: Each if branch uses a `_tc()` wrapper that increments `checked_count` and `fired_count` upsert counters in `prompt_condition_stats`.
- Both instrumentation paths gated behind `PROMPT_BLOCK_STATS_ENABLED` env var (default: false). Dashboard exposes a Prompt Composition Stats card.

#### `prompt_blocks/kid_structured_state.py` — **Kid Structured Fields Block**

- `build()`: Returns `[KIDS STRUCTURED STATE]` prompt block from ambient state keys
- Reads `{kid}_{field}` keys (`current_project`, `current_inquiry`, `mood`, `current_interest`, `chaos_level`)

- Returns None if no data exists for either kid

#### prompt\_blocks/parenting\_context.py — **Parenting Context Block**

- `build()`: Returns [PARENTING CONTEXT] prompt block from ambient state keys
- Reads `parent_pressure` and `parent_narrative` set by `kid_life_event_task`
- Returns None when pressure is “none” or no data

#### group\_chat.py — **Telegram Group Chat Orchestrator & Referee**

- `resolve_sender(user_id, bot_ids, friend_ids)`: Maps Telegram user ID to resolved name (Who enum or friend name string).
- `handle_group_chat_message(message)`: Directs the incoming message workflow. Enforces turn locking (active speaker locks), cooldowns (40s), mentions, fast routing via cheap 2B model, parallel candidate response generation, and referee sequence ordering (staggered typing delays, dropping redundant candidate replies).
- `get_group_chat_state(chat_id)` / `update_group_chat_state(chat_id, state)`: Stateful state management using cached ambient state.
- `run_fast_router(history, model)`: Invokes Ollama 2B to pick up to 2 candidate bots to reply.
- `run_referee(candidates, history, model)`: Sequences candidates or drops redundant ones.
- `log_group_chat_turn(who, text, subject_str)`: Persists response and enqueues background memory extraction.

#### group\_chat\_service.py — **Multi-Agent Group Chat Pipeline**

- **CRUD functions**: Sync module-level `create_group_chat`, `post_message`, `list_group_chats`, `join_chat`, `db_leave_chat`, `get_present_agents`, `find_chat_by_location`.
- **GroupChatService class**: Async wrapper with pipeline: `post_message` → `_run_experience` (agent.process\_chat for all present) → `_run_referee_pipeline` (referee selection + response generation).
- **OWU bridge**: `_push_to_owu_channel` sends agent responses to Open WebUI channels.
- **Agent timeouts**: 15s per response, recursion depth capped at 10.

#### referee\_service.py — **Referee Scoring & Player Selection**

- **3-phase pipeline**: `_pre_filter` (direct mention, topic bonus, gap penalty, min gap gate), `_llm_rank` (persona-enriched LLM scoring), `_threshold_selection` (floor, threshold, max\_responders cap).
- **Per-agent configs**: Loaded from `data/personas/<name>/referee.yml` via `load_agent_configs()`.
- **Anti-volley**: `_is_throttled` caps agent-agent exchanges per `max_agent_depth`/`max_agent_depth_absent`.
- **Recursion**: `_max_recursion_reached` at depth 10.
- **Per-channel overrides**: `channel_overrides` dict + `get_channel_config(chat_id)` — merged into `_get_config(agent_name, chat_id)`.

#### vision.py — **Photo Download & Processing (Telegram-Agnostic)**

- `download_telegram_photo(file_id)` — Downloads photo bytes from Telegram File API via `httpx` with timeouts
- `encode_image_base64(bytes)` — Converts image bytes to base64 string for Ollama vision API
- `save_photo_to_disk(bytes)` — Saves to `data/photos/{uuid}.jpg`, creates dir if needed

- `process_telegram_photo(file_id)` — Convenience: download → base64 → save → 20MB size validation
- Logger: `agent.vision`

### telegram\_client.py — Telegram Bot Integration

- Incoming messages: Detects photos and processes via `vision.py`, transcribes voice notes via OpenAI-compatible STT, checks DND/quiet hours, handles emergency keywords and repeated messages as bypass. If the message belongs to a configured sandbox or real group chat ID, intercepts it and delegates to `handle_group_chat_message()`.
- Two-way sync: Fetches OpenWebUI chat history, processes through `JaneFSMAgent`, sends response back to Telegram, and pushes updated history to OpenWebUI REST API
- Photo pipeline: Downloads photo via `vision.py`, passes base64 to Ollama as images field, stores Jane's response as a self-describing `about_jane` memory, archives it via `archive_photo_from_chat()` in `vision.py` (Google Drive upload) and enqueues `upload_photo_to_google_photos`
- Typing indicator: Background asyncio task sends Telegram typing action every 4s during processing
- Voice reply: Generates TTS audio via OpenAI-compatible TTS endpoint and sends as Telegram voice note
- DND handling: Queues messages during quiet hours with throttled sleep replies (max 1 per 4 hours), processes pending queue when DND ends
- State persistence: Tracks last processed `update_id` and pending messages in `telegram_state.json`

### clients/ — Pluggable Nudge Delivery Backend (ADR-0051)

- `base.py` — `BaseClient` ABC: abstract name property + `send_text()`; optional `send_photo()` / `send_typing()` (raise `NotImplementedError` by default)
- `__init__.py` — static registry: `register(cls)` (called at import), `init_clients()` (instantiates registered classes once at worker startup, awaiting an optional `async_init` hook via `hasattr`), `broadcast_nudge(text)` (sends to every initialized client; never raises — a per-client failure is logged and recorded as `False`). Registers `TelegramClient` + `OWUClient` at import.
- `telegram.py` — `TelegramClient`: thin wrapper over `send_telegram_message` (`name = "telegram"`)
- `owu.py` — `OWUClient` (`name = "open-webui"`): appends an assistant message to every OWU sync chat in `owu_to_telegram_map` via `fetch_open_webui_chat/update_open_webui_chat`; empty map is a success no-op; does not touch the DB chat thread
- `test_client.py` — `TestClient` mock recording `sent_texts` (marked `__test__ = False` so pytest skips collection)
- All 14 proactive nudge call sites (ambient, weather, hobbies, social, memory, reader, schedule) call `broadcast_nudge`; low-level `send_telegram_message` remains for one-off sends with explicit `token/chat_id` (`reconcile.py`, `metrics.py`, `tool_services.py`)
- Logger: `agent.clients` (per-module)

### Additional Modules (brief)

- `agent_friend.py` — **FriendFSMAgent**: `PersonaFSMAgent` subclass backing friend personas (Arthur, Lyra, Kate) via `FriendProvider`.
- `agent_core.py` — **CoreFSMAgent**: Intercepts deterministic `c:` commands (`c:ping`, `@core c:`) for the “core” system model. Routed via the decorator registry in `core_registry.py`; handlers live in `core_commands/` (`ping.py`, `img_gen.py`, `img_guess.py`).

- `llm_client.py` — **LLMClient**: Ollama HTTP client wrapper used by the FSM generation path.
- `context_resolver.py` — **ContextResolver**: `_resolve_context()`-stage helper that assembles the per-turn memory/context read-model; `_format_memory_with_episode()` renders memories with episode detail.
- `emotion_evaluator.py` — **EmotionEvaluator**: `_evaluate_emotions()`-stage helper computing emotional deltas, coupling chain, and sentiment snapshots.
- `turn_context.py` — **TurnContext**: Per-turn dataclass carrying message, persona, and pipeline state through the FSM stages.
- `audio_perception.py` — **AudioPerception**: STT + audio-description pipeline for voice notes (`transcribe_bytes()`, `_transcribe_prompt()`, `_texture_prompt()`), used by `owu_attachments.py` and `telegram_client.py`.
- `tts_utils.py`: TTS text normalization helpers (`_normalize_tts_text()`).
- `commands.py` — `handle_command()`: Textual TUI slash-command router.
- `google_drive.py` — **GoogleDriveFS**: Google Drive file-system facade used by `notebook/photo` I/O.
- `notebook_backend.py` (ADR-0045): Pure-I/O notebook backend — `run_notebook_query` (hybrid nlm CLI + Drive notes), `record_notebook_note` (Drive write), `_resolve_notebook_folder` (create) See root codemap.
- `invite_service.py` (ADR-0072): Stateless multi-recipient DM invite builder (`send_invites`, `build_invite_text`, `_escape_markdown`). See root codemap.
- `telegram/` — Telegram integration subpackage: `handlers/` (update routing), `parse.py`, `state.py` (`telegram_state.json`), `pinned.py`, `owu_sync.py`, `transport.py`.
- `prompt_blocks/` — 23 block modules for the block engine (`JaneProvider._build_blocks`, 52 blocks total). Individual modules include `ambient`, `circadian`, `emotional_state`, `identity`, `jcd_state`, `kids`, `memory_context`, `notebooks`, `opinions`, `thread`, `time_gap`, `voice_directives`, `world_location`, `enrichment_low/mid`, `gossip`, `guess_game`, `critical_output_rules`, `focus_session`, `hobbies`, `parenting_context`, `kid_structured_state`, `presence`.
- `card_engine/` — placeholder package (no Python modules yet).

## Integration

- **Consumed by**: `src/server/main.py` (`JaneFSMAgent` for chat completions), `src/worker/worker.py` (`telegram_polling_loop`)
- **Depends on**: `src/config.py`, `src/embeddings.py`, `src/database/db_init.py`, `src/database/db_helper`, `src/circadian/`, `src/calendar_client.py`, `src/score_component.py`

## PersonaProvider Protocol (ADR-0057)

- **Providers**: `src/personas/` — `ProviderBase` + `JaneProvider` (52 blocks, full coupling), `DottyProvider` (10 blocks, no coupling), `FriendProvider` (5 blocks, no coupling)
- **Wiring**: `PersonaFSMAgent.__init__(provider=...)` — optional, backward-compatible (None falls back to the generic `ProviderBase.build_prompt` template)
- **Coupling chain**: Gated via `provider.emotion_config.coupling_chain` in `_evaluate_emotions()`
- **LLM params**: `provider.llm_config.num_ctx/num_predict` control `_generate_response()` context window
- **Prompt delegation**: `_inject_prompt()` calls `provider.build_prompt()` when set, else the generic `ProviderBase.build_prompt` template. `JaneProvider.build_prompt` runs the block engine (`assemble_from_blocks` over `resolve_prompt_context`)

## src/agents/tool\_modules/

### Responsibility

Individual tool implementations that Jane’s FSM can invoke via the tool registry. Each file defines exactly one tool: an `execute()` async function, a `ToolDef` dataclass, and self-registration into `REGISTRY`.

Also contains shared messaging infrastructure (`tool_services.py` at the parent level) and the Green Terminal display driver (`green_term/`).

### Design

#### Tool Pattern

- Most tools are one file (`execute()`, `ToolDef`, `REGISTRY[name] = tool`). Several are **action-dispatch “drawers”** — one tool with an action enum + per-action params, dispatching to internal helpers — collapsed to fit the model’s tool-selection limit (ADR-0046).
- Tool availability is restricted by persona using the `allowed_personas` field in `ToolDef`.
- Shared dependencies (`get_embedding`) go through `src.agents.deps.deps`; tools use `remember()` from `src.database.memory_utils` for memory writes
- Tool schemas are built dynamically from `REGISTRY` at `tool_registry.py` import time
- Current tools (14): `dotty_control`, `flag_curiosity`, `get_citations`, `hipbone(drawer)`, `images(drawer)`, `notebook(drawer)`, `reach_out`, `read_calendar`, `recall_memories`, `schedule_calendar_event`, `start_focus_session`, `sticky_note`, `web(drawer)`, `world(drawer)`. Heavy image logic lives in unregistered helper modules `generate_image.py/photo_album.py` imported by the `images` drawer.

#### Gated Messaging Pattern

- `tool_services.py` provides `send_user_message()` with DND + rate-limit gate baked in
- Tools that send Telegram messages (e.g., `reach_out.py` for JCD) call `send_user_message()` and get back a `SendResult` (SENT/BLOCKED) — they cannot bypass the gate
- Blocked sends store memories as pending intents via `reach_out.py` for resurfacing later
- Rate limit: `REACH_OUT_RATE_LIMIT` per `RATE_LIMIT_WINDOW` seconds, stored in `ambient_state`
- `_try_acquire_rate_slot()` is atomic — check and record in one operation, no race condition

### Flow

1. FSM dispatches via `TOOL_MAP[name](**arguments)`
2. Tool `execute()` does its work, optionally calling `send_user_message()` for Telegram sends
3. `send_user_message()` checks DND → rate limit → Telegram delivery — returns `SendResult`
4. Tool returns a string response to the FSM

### Integration

- `tool_registry.py` — central registry that all tool modules register into
- `tool_services.py` — shared `send_user_message()` with gates
- `deps.py` — dependency container for `get_embedding`
- `green_term/` — ESP32-S3 green-phosphor terminal display driver (CLI + error handler)

## src/circadian/

### Responsibility

Pure-function package (no I/O, no state) providing the physiological and environmental math that drives Jane's mood and energy modeling. Extracted from the former top-level circadian.py module. Consumed by the FSM (prompt assembly, emotional evaluation) and by src/worker/tasks/schedule/ for mood-target computation.

### Design

- `__init__.py` — circadian state math:
  - `is_nighttime_quiet_hours(hour, day_of_week)` -> bool — DND/quiet-hours check
  - `get_circadian_phase(hour)` -> str — phase label (morning/afternoon/evening/night)
  - `calculate_circadian_fatigue(hour, day_of_week)` -> float — energy/fatigue curve
  - `get_menstrual_modifiers(cycle_day)` -> tuple[float, float, str] — mood/energy modifiers + phase
  - `calculate_target_mood(hour, weather, daily_offset, active_events, cycle_day)` -> float — baseline mood target
  - `is_wakeup_window(hour, minute)` -> bool
  - `get_season_context(month, day)` -> str
  - `modulate_temperature(energy, is_night)` -> float
- `coupling.py` — mood/emotional coupling math (pure functions over deltas/states dicts):  
`apply_rate_limiter`, `check_spike_response`, `apply_mood_inertia`, `apply_spillover`, `apply_positive_coupling`, `apply_interaction_fatigue`, `apply_saturation`, `apply_drain`, `calc_closeness_baseline`

### Flow

`FSM_evaluate_emotions()` and prompt blocks call the pure functions with hour/cycle\_day/states inputs; the returned modifiers feed `calculate_target_mood`, which `schedule.py` compares against Jane's actual mood to drive the mood narrative.

### Integration

- **Consumed by:** `src/agents/fsm.py` (via `JaneProvider` prompt blocks), `src/worker/tasks/schedule/` (`ambient_update.py` mood targeting)
- **Depends on:** nothing — intentionally dependency-free for easy unit testing



## src/

### Responsibility

Root application source directory. Contains all runtime Python modules for the Sweetheart 2.0 AI companion system, organized into domain-specific subdirectories plus top-level utility modules.

### Directory Map

Subdirectory	Responsibility	Codemap
agents/	FSM reasoning pipeline and Telegram bot integration	<a href="#">View Map</a>
database/	SQLite schema, connection management, CRUD helpers, vector search	<a href="#">View Map</a>
dashboard/	Web admin dashboard for memories, status, and configuration	<a href="#">View Map</a>
server/	FastAPI API server with OpenAI/Ollama-compatible endpoints	<a href="#">View Map</a>
worker/	Background task queue processor and Telegram polling loop	<a href="#">View Map</a>
tui/	Textual play-world TUI: chat log, world/memory/log-explorer side panels, shared-command dispatch (completer.py, widgets/)	<a href="#">View Map</a>
circadian/	Pure-function package for circadian fatigue, menstrual cycle modifiers, mood baselines/baseline-mood, quiet hours, time-of-day phase detection	<a href="#">View Map</a>

### Top-Level Modules

File	Responsibility
config.py	Path resolution, environment variable loading, all configuration constants (ports, model names, API keys, intervals). Also load_bot_config/get_bot_token/get_persona_chat_id for the per-persona Telegram bot mapping (persona chat_id parsed and resolvable, falling back to core.chat_id).

File	Responsibility
circadian/	Package (was circadian.py) — pure functions for circadian fatigue, menstrual cycle modifiers, mood baselines, quiet hours, time-of-day phase detection, and mood coupling (is_nighttime_quiet_hours, get_circadian_phase, calculate_circadian_fatigue, get_menstrual_modifiers, calculate_target_mood, is_wakeup_window, get_season_context, modulate_temperature; coupling.py: apply_rate_limiter, check_spike_response, apply_mood_inertia, apply_spillover, apply_positive_coupling, apply_interaction_fatigue, apply_saturation, apply_drain, calc_closeness_baseline)
embeddings.py	get_embedding() — generates 768-dimensional embeddings via local Ollama embedding server
calendar_client.py	GoogleCalendarClient — async Google Calendar REST API v3 client (OAuth Bearer) with get_events_for_day and create_event; attendee invites sent natively via sendUpdates=all
prompts.py	init_prompts() + render_prompt() — loads and caches LLM prompt templates from data/personas/, provides variable interpolation
score_component.py	ScoreComponent.from_db() — eight typed sub-score dataclasses (memory, task queue, emotional state, ambient, circadian, DB health, search quality, prompt blocks) assembled into a single health snapshot using consolidated aggregates (reducing roundtrips from 17 down to 9 on a cache miss). Cached on JaneFSMAgent._score_cache per turn via a process-local thread-safe 30s TTL cache; also callable standalone by worker tasks, the dashboard, and scripts/score-component.py. compute_search_quality_score(vec_hit_rate, kw_hit_rate) -> float — shared formula imported by _snapshot_metrics in schedule.py to avoid duplication.

## Data Flow

1. **External input** → server/ (HTTP) or agents/telegram\_client.py (Telegram polling)
2. **Processing** → agents/fsm.py (JaneFSMAgent) resolves context, evaluates emotions, assembles prompt, calls LLM, formats output
3. **Persistence** → database/ (SQLite with sqlite-vec for vector search)

4. **Background** → worker/ polls task queue, dispatches to handlers in tasks/ package
5. **Monitoring** → dashboard/ provides web UI for inspection and manual operations

## src/dashboard/

### Responsibility

Web-based admin dashboard for viewing and managing Jane's memories, emotional state, RSS feeds, background tasks, and TTS configuration. Serves as the main human interface for inspecting and debugging the system.

### Key Files

#### main.py — FastAPI App Setup (141 lines)

- FastAPI() app creation with CORS middleware, static file mount, API key middleware
- Lifespan handler: migrate\_feeds\_add\_eldest\_first() + backpopulate\_eval\_history\_if\_needed() on startup; logger close on shutdown; VoiceStreamStore init on app.state.voice\_stream
- JSONFormatter for structured uvicorn logging
- if \_\_name\_\_ == "\_\_main\_\_" block with uvicorn.run
- Includes all 8 routers via app.include\_router()
- **No route handlers** — all extracted to routers/

#### routers/ — Extracted Route Modules

File	Routes	Description
memories.py	GET/POST/DELETE /api/memories, GET /api/memory-kinds, GET /api/memories/readme	Memory CRUD with advanced metadata query parsing
feeds.py	GET /api/feeds, POST /api/feeds/toggle/{id}, POST /api/feeds, POST /api/feeds/repoll, POST /api/feeds/{id}, DELETE /api/feeds/{id}, GET /api/feeds/{id}/articles, POST /api/feeds/{id}/eldest-first	RSS feed management
eval.py	GET /api/eval/history, GET /api/eval/runs, GET /api/eval/runs/{filename}, GET /api/eval/years, GET /api/eval/weeks/{year}, GET /api/eval/runs-by-week/{week_id}, GET /api/eval/details/{session_id}	RAG evaluation metrics and run inspection

File	Routes	Description
stats.py	GET /api/status, GET /api/status/urgent, POST /api/status/urgent/ack, GET /api/tags, GET /api/metrics, GET /api/metrics/weekly, GET /api/memory-trace/status, POST /api/memory-trace/toggle, GET /api/prompt-logging/status, POST /api/prompt-logging/toggle, GET /api/prompt-stats/conditions, GET /api/prompt-stats/block-sizes, GET /api/tool-stats/usage, GET /api/tool-stats/history	Server status, metrics, and toggle endpoints
voice_stream.py	GET /api/voice-stream/status, POST /api/voice-stream/toggle, POST /api/voice-stream/voice, GET /api/voice-stream/poll, GET /api/voice-stream/audio/{note_id}, POST /api/voice-stream/mic	Voice mode streaming with STT → LLM → TTS → queue pipeline
logs.py	GET /api/logs	Tail log files for API or worker
misc.py	GET /api/tasks, POST /api/dream, POST /api/emotions, POST /api/tts/generate, GET /	Background tasks, dream synthesis, emotion state, TTS proxy, SPA frontend
prometheus.py	GET /metrics	Exposes all Sweetheart metric_snapshots + RAG eval scores in Prometheus text format (PlainTextResponse) for scraping

### log\_parsing.py — Log Parsing Utilities

- `try_parse_json()`: Attempts JSON parsing of a line, returns None on failure
- `parse_log_entries(raw)`: Splits raw log text into structured entries with timestamp, source, level, message, extra lines

### `eval_backfill.py` — **Eval History Migration**

- `backpopulate_eval_history_if_needed()`: Populates `eval_history` table from eval run files on startup (off the request path)

### `voice_stream_store.py` — **Voice Stream State**

- `VoiceStreamStore`: Manages voice note queue with enabled/disabled toggle, note ID generation, and audio directory

### `deps.py` — **FastAPI Dependencies**

- `get_conn()`: Yields a database connection with guaranteed `close()`; used as `Depends(get_conn)` in every handler that needs DB access
- `db_error()`: Logs exception and raises `HTTPException(status_code=500)`, replacing inline `try/except` `logger.error/raise` in handlers

### `serialization.py` — **JSON Column Decoder**

- `_decode_json_column(value, default=None)`: Safely parses JSON column values from DB rows

### `feed_forms.py` — **Feed Form Helpers**

- `parse_feed_tags(tags)`: Validates JSON-list form field, raises HTTP 400 on failure
- `toggle_feed_column(conn, feed_id, column, label)`: Deduplicates the enable/disable toggle logic used by feed handlers

### `dashboard_query_parser.py` — **Advanced Query Engine**

- `py parsing-based parser for bracketed metadata expressions: [source:rss], [source:/rss_.* / AND type:curiosity]`
- Supports AND/OR/NOT operators, regex values (`/pattern/`), quoted strings
- `filter_memories()` applies parsed queries against memory metadata dictionaries

## **Integration**

- **Consumed by**: Human operators (browser access on port `DASHBOARD_PORT`)
- **Depends on**: `src/database/db_init.py`, `src/database/db_helpers.py`, `src/embeddings.py`, `src/config.py`, `src/dashboard/dashboard_query_parser.py`

## src/database/

### Responsibility

Database initialization, connection management, and CRUD helpers for the unified SQLite database. Provides the persistence layer for all Sweetheart data including emotional/ambient state, memories (with vector search), chat history, RSS feeds, and task queue.

### Key Files

#### db\_init.py — Schema Initialization

- `get_db_connection()`: Factory for SQLite connections enabling WAL mode, foreign keys, and `sqlite-vec` extension (768-dimension vector search)
- `db_session()`: Context manager for safe connection lifecycle — auto-commits on success, rolls back on error, closes on exit
- `init_db()`: Creates global table structures (chats, emotional\_state, ambient\_state, feeds, tasks\_queue, ...), dynamically initializes partitioned memory/vector/FTS5 tables (memories\_{who}, vec\_memories\_{who}, memories\_fts\_{who}), AND partitioned chat/sentiment tables (chat\_messages\_{who}, message\_sentiments\_{who}) for each persona in the Who enum with IF NOT EXISTS semantics
- `metric_snapshots`: Rolling 7-day raw metric store — (timestamp, metric, value). Piggybacked on the 15-min ambient cycle via `_snapshot_metrics()`. Pruned to 7 days by `prune_metric_snapshots()`.
- `metric_weekly_summaries`: Up-to-1-year weekly averages — (week\_start, metric, avg\_value, sample\_count). Populated Sunday 1 AM by `metric_weekly_rollup_task`. Pruned to 52 weeks by `prune_metric_weekly_summaries()`.
- `prompt_block_log`: Stores per-inference block sizes (block\_name, inference\_id, token\_count, timestamp) for prompt composition debugging
- `prompt_condition_stats`: Upsert counters tracking how often each conditional block is checked vs actually fired (block\_name, checked\_count, fired\_count)
- `tool_log`: Per-call tool invocation log — (tool\_name, parameters JSON, result\_summary, latency\_ms, status, source, created\_at). Self-cleaning: pruned after 7 days by `_prune_tool_log()`. Gated by `TOOL_STATS_ENABLED`.
- `tool_usage_stats`: Cumulative counters per tool name — (tool\_name, call\_count, error\_count, total\_latency\_ms, last\_called\_at). UPSERT semantics. Mirrors `prompt_condition_stats` pattern.
- `vec_memories_{who}` is a virtual table using `vec0` for float32[768] vector similarity search via `sqlite-vec`
- `chat_messages_{who}` / `message_sentiments_{who}`: Per-persona partitioned chat history and sentiment tags. CRUD helpers accept who param (default "jane"). Legacy global tables migrated at startup. See ADR-0056. **### db\_helpers.py — Re-Export Shim (ADR-0063) All logic has been extracted to helpers/ submodules.** `db_helpers.py` is now a pure re-export shim (~217 lines, imports only) that preserves the public API so all callsites continue to from `src.database.db_helpers` import ... unchanged.

Submodule map: | Module | Contents | |—|—| | helpers/ambient.py | get\_ambient\_state, get\_ambient\_states, update\_ambient\_state | | helpers/base.py | get\_persona\_tables, serialize\_vector | | helpers/chat.py | parse\_iso, get\_active\_thread\_messages, save\_message\_sentiment, save\_session\_summary, get\_session\_summary, append\_pending\_share, drain\_pending\_shares | | helpers/dnd.py | is\_dnd\_active | | helpers/emotion.py | update\_emotional\_state, get\_all\_emotional\_states | | helpers/memories.py | add\_memory, delete\_memory, get\_all\_memories, search\_persona\_memories, count\_memories\_by\_persona, latest\_memory\_by\_kind, distinct\_kinds | | helpers/metrics.py | insert\_metric\_snapshots, prune\_metric\_snapshots,

```

get_metric_snapshots, insert_metric_weekly_summary, get_metric_weekly_summaries,
prune_metric_weekly_summaries | | helpers/prompt_stats.py | log_prompt_blocks,
log_condition_check, get_prompt_condition_stats, get_block_size_summary, get_full_prompt_logging_s
set_full_prompt_logging_active, decrement_full_prompt_logging, prune_old_stats,
flush_to_db, shutdown_telemetry_flusher | | helpers/search.py | search_memories, hy-
brid_search_memories, keyword_search_memories, calculate_episodic_boost, RRF con-
stants | | helpers/sync.py | save_chat, save_message, recall_session, compute_history_hash
| | helpers/tasks.py | enqueue_task, pop_next_task, complete_task | | helpers/telegram.py |
save_telegram_sync_mapping, get_telegram_sync_mapping, get_orphaned_telegram_messages,
delete_telegram_sync_mappings_after, telegram_delivery_fingerprint, record_telegram_delivery,
get_telegram_delivered_fingerprints, get_telegram_sync_hwm, set_telegram_sync_hwm |
| helpers/tool_log.py | log_tool_call, has_recent_tool_call, get_tool_usage_stats,
_prune_tool_log |

```

**Dependency rule:** submodules import from helpers/base or helpers/ambient (and laterally at most one level, e.g. helpers/dnd ← helpers/chat). Never import from db\_helpers.py.

### memory\_utils.py — Structured Memory Wrapper

- remember(who, text, subject, \*, embedding, metadata, kinds, source) — async wrapper that auto-generates UUID and embedding, calls add\_memory() internally to write to partitioned memories\_{who} and vec\_memories\_{who}
- get\_subjects(who) — returns distinct Subject values a given persona has memories in
- recall(who, subject, query, modes, top\_k, \*, embedding, current\_ambient) — async hybrid retrieval: queries partitioned tables (memories\_{who}, vec\_memories\_{who}, memories\_fts\_{who}) with vector similarity (sqlite-vec) and/or FTS5 keyword search, merged and deduped by memory id
- Accepts Who, Subject, SearchMode, MemoryKind, MemorySource enums from enums.py for type-safe memory operations
- Returns the generated memory ID (remember) or list of result dicts (recall)

### enums.py — Memory Enumerations

- SearchMode (StrEnum): vector, keyword
- Who (StrEnum): jane, lyra, arthur, kate, clara, finn, iroh, dotty, tester, chrissie, jeff, martha
- Subject (StrEnum): about\_jane, about\_jcd, about\_stuff, gossip, about\_friends\_cross, about\_friend\_\*
- MemoryKind (StrEnum): curiosity, intent, reflection, rss\_article, reading\_note, etc.
- MemorySource (StrEnum): agent\_tool, rss\_poller, memory\_synthesizer, etc.

### test\_db.py — Verification Script

- Standalone integration test for vector search (creates dummy 768-dim embeddings, adds memories, validates KNN results)

## Integration

- **Consumed by:** Every module in the project (agent, server, dashboard, worker)
- **Depends on:** src/config.py (DB\_PATH), sqlite-vec extension, sqlite\_vec Python package



## src/server/

### Responsibility

FastAPI-based API server that exposes OpenAI-compatible chat completions endpoints and Ollama-compatible endpoints. Acts as the primary HTTP interface for external clients (OpenWebUI, custom frontends) to interact with Jane's FSM agent.

### Key File

main.py — API Server

- **Endpoints:**
  - POST /v1/chat/completions — OpenAI-compatible chat completion proxied through JaneFSMAgent (supports streaming and non-streaming)
  - POST /api/chat — Ollama-compatible chat completion endpoint (streaming returns ndjson)
  - **Perception proxy endpoints:** POST /v1/vision/chat/completions, POST /v1/audio-caption/chat/completions, POST /v1/narrative/chat/completions — proxy to local Ollama with DB persistence and memory extraction; share `_proxy_perception()` helper
  - **Dotty control proxy endpoints:** POST /v1/dotty/say, POST /v1/dotty/take-photo, POST /v1/dotty/abort, GET /v1/dotty/state — forward to dotty-behaviour via `_get_dotty_client()` lazy singleton; return 502 on upstream error
  - POST /v1/embeddings — OpenAI-compatible embeddings passthrough proxying inputs (single string or list/batch) to local Ollama embeddings service
  - GET /v1/models / GET /api/tags — Model listing proxied from Ollama
  - GET /health — Health check (returns DB path existence)
  - GET /internal/score-cache — Inter-process bridge; returns the last known FSM search quality data (`vec_hit_rate`, `kw_hit_rate`, `avg_rrf_score`, `updated_at`). Polled by the worker's `_snapshot_metrics` to write `score_search_quality` into `metric_snapshots`. Returns {} until the first conversation turn completes.
  - GET /api/world/miniapp/data — World Mini App metadata & command syntax completion dictionary (`routes/world_miniapp.py`)
  - GET /miniapp/world — Static Telegram World Mini App launcher (HTML5/Vanilla CSS/JS)
- **Score cache:** Module-level `_last_score_cache`: dict updated by `_update_score_cache(agent)` after every `process_chat` call. Exposes search quality from FSM-enriched `ScoreComponent` across the process boundary. The worker applies a 24h staleness check (`updated_at < now - 86400`) before trusting the cached value.
- **Telegram Sync:** When `x-openwebui-chat-id` header matches `TELEGRAM_SYNC_CHAT_ID`, syncs messages to Telegram (forwards user messages and broadcasts Jane's responses via Telegram bot)
- **Content Normalization:** `normalize_content_val()` handles various content formats (string, list of text/dict items, dict) from different OpenWebUI message versions
- CORS enabled for all origins

dotty\_client.py — Dotty Control Client

- Async HTTP client for dotty-behaviour's `/api/jane/*` routes
- Methods: `say(text)`, `take_photo(fresh=True)`, `abort()`, `get_state(fresh=False)`
- All methods return user-facing strings, never raise — network errors produce descriptive error strings

- `_get_dotty_client()` in `main.py` provides a lazy singleton that reads `DOTTY_API_URL` from `os.environ` at call time for testability

## Integration

- **Consumed by:** OpenWebUI, custom HTTP clients (port `PORT`, default 11440)
- **Depends on:** `src/agents/fsm.py` (`JaneFSMAgent`), `src/agents/telegram_client.py` (`send_telegram_message`), `src/config.py`

## src/tui/

### Responsibility

Textual-based terminal UI (TUI) for the Sweetheart play-world. Lets a human log in as a persona and interact with the virtual world: look/move/fast-travel navigation, location chat, direct messages to other personas, /invite, world editing (/create, /edit, /link, /unlink, /rename-slug), theme switching, and live sidebar panels for world info, memory search, and log exploration.

### Key Files

- `app.py` — Textual app entry point: command dispatch (HELP\_TEXT documents the full /-command set incl. /look, /move, /fast, /map, /chat, /msg, /gps, /invite, /whoami, /theme, /mem, /logs, /rename-slug, /world, /quit), sidebar ContentSwitcher, world/client wiring.
- `completer.py` — `get_tab_completion(text)`: tab-completion engine for /-commands, /move targets, and world slugs.
- `widgets/` — reusable panels:
  - `world_info.py` — `WorldInfoWidget`: location metadata, present characters, linked paths
  - `memory_info.py` — memory search/view results panel (/mem)
  - `log_explorer.py` — log file explorer panel (/logs: filter by query/type/level/tail)
  - `selectable_rich_log.py` — rich text log rendering with selection/copy

### Integration

- **Consumed by:** humans (run `./scripts/play_world_tui.py [personal]`); shares command semantics with the REPL `scripts/play_world.py` and the Telegram world bot (`src/worker/tasks/core_bot.py`).
- **Depends on:** `src/world/` (world service), `src/database/` (memories, logs), `src/config.py`, Textual.

## src/worker/

### Responsibility

Background worker service that processes the SQLite-backed task queue and runs the Telegram polling loop and world bot polling loop concurrently. Handles all asynchronous maintenance operations: memory extraction, RSS polling, ambient updates, document ingestion, memory consolidation, curiosity research, daily journaling, friend correspondence, and focus sessions.

### Key Files

#### worker.py — Worker Service

- **Main loop:** `process_queue_loop()` polls the `tasks_queue` table, pops pending tasks atomically, dispatches to registered handlers via `TASK_HANDLERS` map
- **Cron seeding:** `seed_recurring_tasks()` ensures all `CRON_INTERVALS` task types have a pending or running entry in the queue at startup
- **Rescheduling:** After each task completes (or fails), calculates the next `run_at` timestamp using `calculate_next_run_time()` which supports:
  - Target-time scheduling (specific hour:minute slots from `TASK_TARGET_TIMES`, at least 4-hour gap,  $\pm 10$  min jitter)
  - Interval-based fallback with  $\pm 10$  min jitter (no jitter for intervals  $\leq 60$ s)
- **CPU throttle gate:** After `pop_next_task()` and before handler dispatch, `_is_throttled()` checks for recent user chat activity (last message within `CPU_THROTTLE_WINDOW`, default 600s). If throttled, suppressed tasks (defined in `SUPPRESSED_TASKS` set) are either rescheduled +15min (daily/rare tasks) or dropped (frequent tasks  $< 86400$ s interval). Exempt tasks (`extract_memories`, `ambient`, `focus_session`, `metric_weekly_rollup`, `add_calendar_event`) always run regardless. The query is `SELECT MAX(created_at) FROM chat_messages WHERE role = 'user'` — a single indexed read.
- **Concurrency:** `start_worker()` runs `process_queue_loop()`, `telegram_polling_loop()`, and `core_bot_polling_loop()` via `asyncio.gather` under `_run_supervised()`

#### agency/ — Choose Your Own Task System (Strategy Pattern)

- `base.py`: Defines `AgencyContext` and the `BaseAgencyTask` interface.
- `registry.py`: Declares global `AGENCY_TASKS` registry dictionary and registration helpers.
- `orchestrator.py`: Hooked into `ambient_update_task`; compiles context, queries LLM for Jane's hourly choice, executes task under a safety boundary, and notifies JCD via broadcast\_nudge if the task permits.
- `tasks/`: Domain strategy subclasses (`garden`, `stargazing`, `friend_letter`, `curiosity`, `nap`, `work`, `resting`).

#### tasks/ — Task Handler Package

The monolithic `tasks.py` was refactored into a `tasks/` package with domain-specific modules. Each module is documented in its own codemap: [View Map](#)

Module	Files	Purpose
Module	Files	Purpose

Module	Files	Purpose
memory/	package (was memory.py)	extract_memories_task, session_extraction_sweep_task, memory_synthesizer_task, weekly_dream_task, nightly_dream_task, custom_dream_task, life_story_capsule_task, backfill_sentiment_tags_task
ambient/	package (was ambient.py)	uw_bulletin_task, spontaneous_nudge_task, focus_session_task, ambient_scene_task, kid_vignette_task, nightly_synthesis_task
feeds	feeds.py	rss_poller_task, rss_full_read_task
reader	reader.py	document_reader_task (+ proactive nudge trigger)
attachments	attachments.py	process_attachment_chunks
hobbies	hobbies.py, hobbies/	curiosity_task, garden_task, stargazing_task, iroh_morning_task, iroh_reflection_task
social/	package (was social.py)	friend_letter_task, friend_reach_out_task, friend_activity_task, friend_life_event_task, persona_nudge_task
kids	kids.py	kid_life_event_task, kid_reach_out_task
babysitter	babysitter.py	babysitter_sync_task, babysitting_session_task
calendar_tasks	calendar_tasks.py	add_calendar_event_task
schedule/	package (was schedule.py)	ambient_update_task, check_and_drift_hobby
metrics	metrics.py	metric_weekly_rollup_task, notebook_usage_counter_task, prune_tasks_queue_task
notebooks	notebooks.py	notebook_query_task, notebook_sync_task
reconcile	reconcile.py	reconcile_owu_task — non-destructive, idempotent DB→OWU self-heal. Never invokes the FSM.
google_photos_upload	google_photos_upload.py	upload_photo_to_google_photos — 2-step Google Drive upload (create metadata, then upload content with uploadType=media). Uses OAuth2 refresh token.
world_events_task	world_events_task.py	poll_world_events — scheduled world event execution

Module	Files	Purpose
core_bot	core_bot.py	core_bot_polling_loop — separate polling loop: world navigation + persona commands
group_chat_cleanup	group_chat_cleanup.py	group_chat_cleanup_task
eval_tasks	eval_tasks.py	daily_evaluation_task
homeassistant	homeassistant.py	homeassistant_task
log_archive	log_archive.py	log_archive_task
userbot_sync	userbot_sync.py	send_userbot_sync_task
llm_utils	llm_utils.py	Shared helpers: _query_llm, _query_llm_chat, _split_synthesis, _generate_stake (not a task handler, imported by other modules)

45 task handlers in TASK\_HANDLERS (see worker.py:74) + 2 polling loops: telegram\_polling\_loop (from src/agents/telegram\_client) and core\_bot\_polling\_loop (from tasks/core\_bot.py).

## Integration

- **Depends on:** Almost all src modules — src/database/db\_helpers.py (task queue, memories, state), src/embeddings.py, src/circadian/, src/config.py, src/agents/telegram\_client.py, src/calendar\_client.py
- **Consumed by:** Runs autonomously as a background service (launched by sweetheart-start.sh)

## src/worker/tasks/ — Background Task Handlers

### Overview

This package contains all asynchronous background task handlers called by the cron-style worker queue (src/worker/). Each module is a self-contained task handler — the worker dispatches tasks by name, and the corresponding function receives a payload: Dict[str, Any] argument.

Tasks are enqueued via src.database.db\_helpers.enqueue\_task() and consumed by the worker loop in src.worker.worker (or by direct await calls).

---

### File Index

File	Role
__init__.py	Package exports — re-exports all public task functions + helpers for test patching
_task_utils.py	Shared task helpers (poll scheduling, window checks)
ambient/	Ambient state package — ambient_scene.py, focus_session.py, kid_vignette.py, nightly_synthesis.py, spontaneous_nudge.py, uw_bulletin.py
attachments.py	Attachment processing — process_attachment_chunks (OWU/Telegram attachments, data/read/ docs)
babysitter.py	Babysitting coordination — calendar-event eligibility scan, sitter availability rolls, simulated phone dialogues, hourly sibling sessions, emergency checks
calendar_tasks.py	Calendar event creation via CalDAV
core_bot.py	Telegram core bot — separate polling loop with world-navigation command dispatch (/look, /move, /fast, /map, /history, /chat, /gps, /invite) + persona chat via config-based multi-persona token routing. /invite delegates to invite_service.send_invites (ADR-0072).
eval_tasks.py	LLM session evaluation — daily_evaluation_task (reference ground truth + scoring)
feeds.py	RSS feed polling — interest-filtered article summaries, selective stake generation
google_photos_upload.py	Background photo upload — Google Drive via 2-step OAuth2. Replaces the old archive_photo.py (NotebookLM via nlm CLI).
group_chat_cleanup.py	Group chat cleanup — group_chat_cleanup_task
hobbies.py	Compatibility wrapper — re-exports tasks from hobbies/ package

File	Role
hobbies/	Hobby simulation package — <code>curiosity.py</code> , <code>garden.py</code> , <code>stargazing.py</code> , <code>iroh.py</code>
homeassistant.py	Home Assistant automation — <code>homeassistant_task</code>
kids.py	Kid social simulation — life event generation, kid→Jane reach-outs, sibling exchanges (Clara ↔ Finn)
llm_utils.py	Shared LLM utilities (Ollama API calls, text splitting)
log_archive.py	Monthly log archival to tar — <code>log_archive_task</code>
memory/	Memory extraction package — <code>extract_memories.py</code> , <code>session_sweep.py</code> , <code>synthesizer.py</code> , <code>weekly_dream.py</code> , <code>nightly_dream.py</code> , <code>custom_dream.py</code> , <code>life_story_capsule.py</code> , <code>backfill_sentiment.py</code> , <code>profiles.py</code> , <code>prompt_loader.py</code> , <code>_shared.py</code>
metrics.py	Metric history — weekly rollup of <code>metric_snapshots</code> , notebook usage counter, tasks-queue pruning
notebooks.py	NotebookLM query/sync — <code>notebook_query_task</code> , <code>notebook_sync_task</code>
reader.py	Document reader — extracts, summarizes, and indexes files from <code>data/read/</code>
reconcile.py	DB-authoritative TELEGRAM ← DB → OWU sync sweep ( <code>reconcile_owu_task</code> ) — cleaned-text matching, content-fingerprint delivery ledger + forward-only HWM, per-run send cap, per-edge isolation. Runs every 15 min. See ADR-0042.
schedule/	Ambient schedule orchestrator package — <code>ambient_update.py</code> , <code>hobby.py</code> , <code>gossip_sweep.py</code> , <code>_shared.py</code>
social/	Friend simulation package — <code>friend_letter.py</code> , <code>friend_reach_out.py</code> , <code>friend_life_event.py</code> , <code>friend_activity.py</code> , <code>nudge.py</code> , <code>_shared.py</code>
unfortunate_events.py	Occasional-life-event rolling — <code>roll_for_unfortunate_event</code>
userbot_sync.py	Userbot OWU sync — <code>send_userbot_sync_task</code>
weather.py	Weather fetching — EC citypage XML (current + forecast + warnings), proactive nudges via <code>broadcast_nudge</code>
world_events_task.py	World events polling — checks scheduled event files and executes due events (see ADR-0001)



## `__init__.py`

**Responsibility:** Central re-export module that exposes all public task functions and selected internal helpers for backward compatibility.

**Re-exports helpers** so existing test patches (e.g. `patch("src.worker.tasks.get_embedding")`) continue to work without modification. Includes: - `_query_llm`, `_query_llm_chat`, `_split_synthesis` (from `llm_utils`)-`get_embedding` (from `src.embeddings`)-`update_ambient_state`, `get_ambient_state`, `get_all_emotional_states` (from `db_helpers`) - `JANE_CALENDAR_ID` (from `src.config`) - `check_and_trigger_proactive_nudge` (from `reader`)

**Exported tasks** (in `__all__`): - Memory: `extract_memories_task`, `session_extraction_sweep_task`, `memory_synthesizer_task`, `weekly_dream_task`, `nightly_dream_task`, `custom_dream_task`, `life_story_capsule_task`, `backfill_sentiment_tags_task` - Ambient: `ambient_update_task`, `check_and_drift_hobby`, `ambient_scene_task`, `uw_bulletin_task`, `nightly_synthesis_task`, `spontaneous_nudge_task`, `focus_session_task`, `kid_vignette_task` - Feeds: `rss_poller_task`, `rss_full_read_task` - Reading: `document_reader_task`, `process_attachment_chunks` - Hobbies: `curiosity_task`, `garden_task`, `stargazing_task`, `iroh_morning_task`, `iroh_reflection_task` - Social: `friend_letter_task`, `friend_reach_out_task`, `friend_activity_task`, `friend_life_event_task`, `persona_nudge_task` - Kids: `kid_life_event_task`, `kid_reach_out_task` - Calendar: `add_calendar_event_task` - Metrics: `metric_weekly_rollup_task`, `notebook_usage_counter_task`, `prune_tasks_queue_task` - Notebooks: `notebook_query_task`, `notebook_sync_task` - Sync: `reconcile_owu_task`, `send_userbot_sync_task` - Babysitter: `babysitter_sync_task`, `babysitting_session_task` - Photo upload: `upload_photo_to_google_photos` - Misc: `home-assistant_task`, `daily_evaluation_task`, `poll_world_events`, `core_bot_polling_loop`, `group_chat_cleanup_task`, `log_archive_task`

---

## `llm_utils.py`

**Responsibility:** Shared LLM helper functions for all background tasks. Provides three core utilities: simple completion, chat completion, and text splitting. All use the local Ollama instance configured by `OLLAMA_LLM_URL` and `MEMORY_LLM_MODEL` from `src.config`.

`_query_llm(prompt: str) -> str`

Sends a single prompt to Ollama's `/api/generate` endpoint. Low temperature (0.3), 120-second timeout. Returns the response string or empty string on failure.

`_query_llm_chat(messages: List[Dict[str, str]]) -> str`

Sends a message list to Ollama's `/api/chat` endpoint (chat format). Same temperature/timeout settings as `_query_llm`. Used for system-prompt-driven interactions like the document reader.

`_split_synthesis(text: str, max_chars: int = 350) -> List[str]`

Splits long synthesis text into chunks of  $\leq$  `max_chars` characters. Attempts to split on paragraph boundaries (`\n\n`), then newlines, then word boundaries. Used by `memory.py` for dream digest storage.

`_generate_stake(content: str, source_label: str, meta: dict, prompt_override: str | None = None) -> str | None`

Generates a 2-4 sentence first-person reaction stake and stores it as an `about_jane` memory. Uses `uuid5` keyed on `content[:500]` + `source_label` for idempotency — safe to call multiple times on the same content. Accepts an optional `prompt_override` for caller-specific instructions (friend-letter callers pass a relational prompt; RSS/reading callers use the generic prompt). Returns the stake text on success, `None` on LLM failure.

`store_persona_perspective(persona, their_message, context, source) -> str | None`

Async LLM function that reinterprets a message from/about a persona into a first-person reflection and stores it in the persona's memory table. Uses `_query_llm` with a prompt directing the LLM to write 2-4 first-person sentences capturing emotional reaction. Truncates input messages to 1000 chars (with `logger.warning`). Calls `add_persona_memory()` for persistence. Returns perspective text on success, `None` on failure.

**Data flow:** Config → Ollama HTTP POST → `add_persona_memory()` → persona table

---

## ambient/ **Package (was ambient.py)**

**Responsibility:** Houses remaining background tasks — UW Bulletin checks, spontaneous Telegram nudges, focus session monitoring, and ambient scene generation. RSS polling has been extracted to `feeds.py`. The primary orchestrator (`ambient_update_task`) and hobby drift (`check_and_drift_hobby`) have been moved to `schedule.py` and are re-exported here for backward compatibility.

**Key dependencies:** - `src.database.memory_utils` — `remember()` (all memory writes use this instead of raw `add_memory`) - `src.agents.telegram_client` — `send_telegram_message`, `is_dnd_active` - `src.embeddings` — `get_embedding` - `.llm_utils` — `_query_llm`

## **Re-exported (see schedule.py)**

`ambient_update_task` and `check_and_drift_hobby` are re-exported from `.schedule` at module level so existing imports from `src.worker.tasks.ambient` continue to work without modification.

`uw_bulletin_task(payload: Dict[str, Any])`

Daily check for the University of Waterloo Daily Bulletin. HEAD request to `https://uwaterloo.ca/daily-bulletin/{date}` — if 200, writes a `.url` file in `data/read/bulletin/` and creates `INSTRUCTIONS.md` if absent. Skips if `.url` or `done/` file already exists.

## `daily_journal_task` **(removed)**

Replaced by `nightly_synthesis_task`. See that entry below.

`spontaneous_nudge_task(payload: Dict[str, Any])`

Picks a random trigger type (reflective, curiosity, ambient, `slice_of_life`) and generates a warm Telegram SMS to JCD: - **Reflective:** Random memory from `about_jcd` or `about_jane` buckets. - **Curiosity:** Random research\_reminder from `about_jane`. - **Slice-of-life:** Time-of-day-based scene vignette (e.g., “making morning coffee”), with weather-sensitive additions

(rain/snow). - **Default/ambient**: Time-of-day + weather check-in. Skips if DND/quiet hours active.

`focus_session_task(payload: Dict[str, Any])`

Background checker for active shared focus sessions. Reads session state from ambient DB (`focus_session_active`, `start_time`, `end_time`, `duration`, `type`, `last_update_time`). If session expired → sends wrap-up Telegram message and clears `focus_session_active`. If session running → sends progress update every 15 minutes (≥5min remaining) with cozy encouragement.

`kid_vignette_task(payload: Dict[str, Any])`

Runs on the 2-hour ambient scene cadence to generate 1-2 sentences about what Clara and Finn are doing. Time-of-day gated (6 AM - 9 PM). If the vignette text is unchanged from the previous cycle, all downstream effects are skipped (dedup guard).

**Downstream effects**: - **Prompt injection**: [KIDS] block in FSM system prompt, probability-gated by time-of-day (0.7 morning/evening, 0.3 school day, 0.2 late). - **Telegram nudge**: ~20% chance (config: `KID_NUDGE_CHANCE`), content-gated by LLM. - **Memory stake**: ~30% chance (config: `KID_STAKE_CHANCE`), stored in `about_jane` with "child": "clara" or "children": ["clara", "finn"] metadata tag. - **Child perspective memory**: For each child detected via word-boundary regex (`\bClara\b` / `\bFinn\b`), stores a first-person perspective memory in the child's `persona_memories_{name}` table via `store_persona_perspective()`.

**Key dependencies**: `render_prompt` (3 prompt templates), `_query_llm`,

---

## `feeds.py`

**Responsibility**: RSS feed polling and interest-filtered stake generation. Moved from `ambient.py` to give RSS its own clean module.

**Key dependencies**: - `src.database.memory_utils` — `remember()` - `src.database.db_init` — `db_session` - `src.embeddings` — `get_embedding` - `feedparser`, `BeautifulSoup` — feed parsing - `.llm_utils` — `_query_llm`, `_generate_stake`

`rss_poller_task(payload: Dict[str, Any])`

Queries the feeds SQLite table for enabled feeds, fetches and parses each RSS/Atom feed, deduplicates against existing memory URLs, summarizes new articles via `_query_llm`, and stores them in `about_stuff`. After storing an article, runs an **interest filter** — a second `_query_llm` call asking whether the article would catch Jane's eye (based on her core interests plus current hobby\_interest from ambient state). Articles that pass the filter also generate a `_generate_stake` call storing Jane's personal reaction in `about_jane`. Interest filter verdict is YES/NO-only for efficiency.

---

## `calendar_tasks.py` (49 lines)

**Responsibility**: Minimal module for creating calendar events via CalDAV.

`add_calendar_event_task(payload: Dict[str, Any])`

Parses summary, start (YYYY-MM-DDTHH:MM:SS), end, description from payload. Creates a `GoogleCalendarClient` instance and adds the event. After success, immediately calls `ambient_update_task({})` to refresh Jane's daily plan. Runs the CalDAV call in an executor to avoid blocking the event loop.

**Data flow:** payload → datetime parsing → `client.add_event()` → `ambient_update_task()` for plan refresh **Scheduled by:** `social.py` for next correspondence exchange

---

## hobbies/ **Directory** and `hobbies.py`

**Responsibility:** Jane's hobby/activity simulation system. Refactored into a `hobbies/` sub-module directory, with each domain task isolated into its own file. Exposes a backward-compatible wrapper file `hobbies.py` that delegates tasks to `hobbies/` modules and supports test mocking.

**Module Layout:** - `hobbies.py` — Thin compatibility wrapper delegating imports to the package - `hobbies/__init__.py` — Exposes all tasks and package-level mocked utility imports - `hobbies/curiosity.py` — Nightly curiosity research - `hobbies/garden.py` — Virtual gardening simulation - `hobbies/stargazing.py` — Waterloo-aware stargazing simulation - `hobbies/iroh.py` — Daily/weekly Iroh wisdom tasks

**Key dependencies:** - `src.config` — `DATA_DIR`, `NLM_BIN_PATH` (NotebookLM CLI path) - `src.database.db_helpers` — `log_tool_call`, - `src.agents.fsm` — `NOTEBOOK_IDS` (for NotebookLM routing) - `ddgs` (DuckDuckGo search) — web search via `DDGS` class - `httpx` — HTTP requests - `subprocess` — NotebookLM CLI calls

`curiosity_task(payload: Dict[str, Any])`

Nightly curiosity explorer. Flow: 1. Fetch open curiosity questions from `about_jane` bucket (type=curiosity, status=open) and pending intents (type=intent, status=pending). 2. If none found, generate a spontaneous question using LLM with context (recent Jane reflections, RSS, readings, bulletins, garden status, stargazing info). Add as new curiosity to DB. 3. Decompose question into 2 DuckDuckGo search queries. 4. Execute searches via `DDGS.text()` in executor (2s delay between). 5. Synthesize findings via LLM (3-5 sentences, first person). 6. 35% chance: debate finding with a random friend (Arthur/Lyra/Kate) — generates friend critique + Jane counter-argument. 7. Save finding to `about_stuff` bucket and as a Markdown research note file. 8. Classify topic (philosophy/naughty/misc) → route to corresponding NotebookLM notebook. Create NotebookLM note via subprocess CLI. 9. Add research reminder to `about_jane` and append to pending shares queue. 10. Send Telegram notification to JCD.

`garden_task(payload: Dict[str, Any])`

Daily virtual Zen Gardening. Flow: 1. Load/initialize garden state from `data/garden_state.json` (Sakura Bonsai + Midnight Orchid with moisture, health, growth\_stage, growth\_progress). 2. Simulate moisture decay (15-25% daily) and growth (3-7% if health > 70%). 3. If moisture < 40%: water (restore 75-85%, boost health). Periodically search DuckDuckGo for care tips. 4. Growth milestone at 100% → advance to next stage (Seedling → Sapling → Vegetative → Budding → Flowering → Mature). 5. Generate LLM garden journal entry → save as Markdown file. 6. Upload to NotebookLM misc notebook. 7. Add garden reminder memory, append to pending shares. 8. Send Telegram update to JCD.

`stargazing_task(payload: Dict[str, Any])`

Daily stargazing simulation. Flow: 1. Fetch Open-Meteo weather for Waterloo to determine clear vs cloudy. 2. Pick a random celestial object (Andromeda Galaxy, Ring Nebula, Jupiter, Pleiades, Orion Nebula). 3. If clear: DuckDuckGo search for scientific data → LLM observation log (warm, first person, eyepiece narrative). If cloudy: LLM reflection log (disappointment + theoretical research). 4. Save as Markdown stargazing log file. 5. Upload to NotebookLM misc notebook. 6. Add stargazing reminder memory, append to pending shares. 7. Send Telegram update to JCD.

`iroh_morning_task(payload: Dict[str, Any])`

Daily 7:30 AM wisdom delivery. Flow: 1. Fetch recent JCD memories + Iroh journey observations for context. 2. Query the iroh NotebookLM notebook for a wisdom seed (via CLI notebook query). 3. LLM generates a two-part message: wisdom seed (Iroh-inspired insight) + daily practice/question, personalised to JCD. 4. Cache in `ambient_state("iroh_daily_wisdom")` for FSM access all day. 5. Send via Telegram.

`iroh_reflection_task(payload: Dict[str, Any])`

Weekly (Sunday) Iroh journey reflection. Flow: 1. Fetch Iroh journey observations from last 7 days. 2. LLM synthesises a warm reflection on JCD's growth through Iroh's lens. 3. Send via Telegram to JCD (3-4 sentences).

---

## `metrics.py`

**Responsibility:** Weekly metric rollup task. Aggregates the rolling raw `metric_snapshots` table into a keyed `metric_weekly_summaries` table, then prunes summaries older than 52 weeks ( $\approx 1$  year). Designed to run once per week (Sunday 1 AM via `TASK_TARGET_TIMES`).

**Key dependencies:** - `src.database.db_helpers` — `get_metric_snapshots`, `write_metric_weekly_summary`, `prune_metric_weekly_summaries` - `src.database.db_init` — `db_session`

`metric_weekly_rollup_task(payload: Dict[str, Any])`

Weekly aggregation task. Flow: 1. Compute `week_start` using `time.localtime()` fields — seconds since local Monday midnight (not UTC modulo, which would drift in non-UTC time-zones). 2. Fetch all `metric_snapshots` rows from the current week (`since_ts=week_start`). 3. Group rows by metric key; compute average value and sample count per metric. 4. Upsert each (`week_start`, `metric`, `avg_value`, `sample_count`) row into `metric_weekly_summaries`. 5. Prune `metric_weekly_summaries` entries older than 52 weeks (`before_week_start = week_start - 52 * 604800`).

**Two-tier storage model:** - `metric_snapshots`: rolling 7-day raw store ( $\sim 15$  min cadence via `_snapshot_metrics` in `schedule.py`) - `metric_weekly_summaries`: up to 1-year weekly averages populated by this task

---

## memory/ **Package (was `memory.py`) ~1006 lines**

**Responsibility:** Core memory extraction and consolidation system. Extracts facts about JCD, Jane's self-reflections, intents/commitments, and Iroh journey observations from conversation turns. Also handles memory consolidation, dream synthesis, and session extraction sweeps.

**Key dependencies:** - src.embeddings — get\_embedding - src.database.memory\_utils — remember() get\_ambient\_state, update\_ambient\_state, get\_all\_memories - .llm\_utils — \_query\_llm, \_split\_synthesis - src.agents.telegram\_client — send\_telegram\_message

extract\_memories\_task(payload: Dict[str, Any])

Real-time per-turn memory extraction — now a **persona dispatcher** (see adr-0043). Parses the payload into an Observation (via src.observation), resolves the relevant observers (profiles.resolve\_observers), and for each looks up its profile in profiles.PROFILES. A persona with **no profile is skipped and logged** (default-deny safety valve), so a new persona can never write into another's table. Legacy {user\_msg, ai\_msg} payloads are grandfathered → a JCD→Jane observation → jane\_profile.

- profiles.py — PROFILES: dict[Who, ExtractionProfile] registry (passive; profiles register themselves, mirroring the tool REGISTRY) plus resolve\_observers (deterministic seam; smart relevance deferred to group chat).
- src.observation (neutral module) — Speaker/Message/Observation turn contract + parse\_turn / observations\_for (per-observer fan-out; a silent observer gets response=None).

jane\_profile(obs: Observation)

Jane's extraction (registered under Who.JANE), moved verbatim from the old task body — behavior unchanged. Runs four **sequential** passes: 1. **JCD facts** — from incoming → about\_jcd bucket. 2. **Jane reflections** — from response → about\_jane bucket. 3. **Intents** — forward-looking commitments from response → about\_jane, status=pending. 4. **Iroh observations** — growth moments from both messages → about\_iroh\_journey.

Each extracted fact gets an embedding vector for similarity search.

session\_extraction\_sweep\_task(payload: Dict[str, Any])

Batch extraction over a conversation session (triggered when conversation has been quiet for EXTRACTION\_INACTIVITY\_MINUTES). Six independent passes running concurrently via asyncio.gather: same four passes as extract\_memories\_task plus a **climate pass** (conversation-level emotional summary) and a **sentiment tag pass** (6th pass — classifies each message with one of 8 categorical tags). All passes run over all messages since last\_extraction\_sweep timestamp (max 3h lookback). Saves results with source="sweep". Updates last\_extraction\_sweep ambient state on completion.

backfill\_sentiment\_tags\_task(payload: Dict[str, Any])

One-shot backfill that processes ALL messages (not just unprocessed ones). Always runs in force mode: compares new tags against existing, logs before/after for any changes (tag changed for msg abc12345: Functional -> Curious). Final summary: New: X, Changed: Y, Unchanged: Z. Triggered on-demand via run\_backfill.py at the repo root. Useful when tag sets evolve.

memory\_synthesizer\_task(payload: Dict[str, Any])

Nightly memory consolidation. For each bucket (about\_jcd, about\_jane): 1. Fetch up to 200 memories, proceed if ≥5 exist. 2. Batch process up to 30 at a time. 3. LLM deduplicates, merges overlapping statements, identifies deletes. 4. Returns JSON array of actions: merge (delete IDs + add new consolidated memory), delete (just delete IDs). 5.

Merged memories preserve metadata types/sources from originals. 6. Updated memories get source="synthesizer" and consolidated\_from field.

```
weekly_dream_task(payload: Dict[str, Any])
```

Weekly synthesis of RSS news into "dream" digests. Groups RSS memories from the past 7 days by category. For categories with  $\geq 2$  articles: shuffle, batch (max 80 per batch, 3 batches max), LLM per batch groups articles by topic and writes 2-3 sentence synthesis. Saves chunks to about\_stuff with {"source": "rss", "dream": "weekly"}. Sends Telegram nudge per category. After synthesis, prunes raw RSS entries 14-21 days old (keeps dream digests).

```
custom_dream_task(payload: Dict[str, Any])
```

On-demand dream synthesis with custom filters. Accepts payload fields: source, type, sub-dir, category, feed\_name, custom\_label, days\_back (default 30), min\_items (default 2), query, top\_k (default 500). Queries memories with these filters, synthesizes via LLM, stores in about\_stuff with {"source": "dream", "dream": "custom"}.

```
life_story_capsule_task(payload: Dict[str, Any])
```

Weekly synthesis of recent about\_jane entries (journal, jane\_reflection, intent, iroh\_observation) into a narrative life story summary stored as about\_jane type:life\_story. After storing the capsule, runs an **HOV synthesis pass**: reads recent journal/reflection entries plus existing second\_order\_values memories, asks the LLM to identify 1-2 new commitments earned from this week's tensions (lines starting with 'I want'). Stores each as about\_jane type:second\_order\_values source:synthesis with uuid5 for idempotency.

---

## reader.py (419 lines)

**Responsibility:** Document reader task. Scans subdirectories under data/read/, extracts text from supported file formats, reads documents page-by-page with LLM summarization, indexes into memory, and moves completed files to done/.

**Key dependencies:** - src.config — READ\_DIR, READER\_STATE\_PATH - src.embeddings — get\_embedding - src.database.memory\_utils — remember() - .llm\_utils — \_query\_llm, \_query\_llm\_chat - External: fitz (PyMuPDF), ebooklib, BeautifulSoup

```
extract_content(path: str) -> str
```

Dispatches to format-specific extractor based on file extension: - .txt/.md — \_extract\_txt (raw UTF-8 read) - .pdf — \_extract\_pdf (PyMuPDF) - .epub — \_extract\_epub (ebooklib + BeautifulSoup) - .html/.htm — \_extract\_html (BeautifulSoup) - .url — \_extract\_url (fetch URL via httpx with Sweetheart-Reader/1.0 UA, strip script/style/nav/footer/header) - Other — returns empty string

```
_chunk_into_pages(text: str, words_per_page: int = 250) -> List[str]
```

Splits extracted text into fixed-size word chunks (default 250 words/page).

```
_load_subdir_config(subdir) -> dict
```

Reads config.json from the subdirectory. Defaults: pages\_per\_session=5, min\_hours\_between\_reads=24, max\_files\_per\_session=1.

```
check_and_trigger_proactive_nudge(filename: str, subdir_name: str, reflections_text: str)
```

After finishing a document, checks if the reading content has a meaningful connection to JCD's memories (search\_memories on about\_jcd). If a strong connection exists, generates a warm Telegram SMS to share with JCD. If no connection, returns silently.

```
document_reader_task(payload: Dict[str, Any])
```

Main reader task. Flow: 1. Discover subdirectories under data/read/ (excluding done). 2. For each subdir: check cooldown (min\_hours\_between\_reads), discover files (excluding INSTRUCTIONS.md, config.json, and files already in done/), filter against reader state for pending files. 3. For each pending file (up to max\_files\_per\_session): a. Extract content via extract\_content. b. Split into pages via \_chunk\_into\_pages. c. Read pages incrementally (up to pages\_per\_session), each page sent to \_query\_llm\_chat with subdir-specific INSTRUCTIONS.md or default system prompt ("key ideas, interesting facts, 1-2 sentences first person"). system prompt from data/personas/jane/reading\_system.md (loaded via render\_prompt()). d. Save each page reflection to about\_stuff with source metadata. e. Save reader state after each page (resumable). f. When file fully read → move to done/ subdir, clean up empty directories, trigger check\_and\_trigger\_proactive\_nudge. After triggering the proactive nudge, calls \_generate\_stake with the combined session reflections to store Jane's overall reaction to the document in about\_jane. 4. Update subdir last\_read\_time if any pages were read.

---

## social/ **Package (was social.py) 963 lines**

**Responsibility:** Friend simulation system. Models three friends (Arthur, Lyra, Kate) with independent interests, life states, and correspondence behaviors. Supports both scheduled letter exchanges and spontaneous friend-initiated reach-outs.

**Key dependencies:** - src.config — DATA\_DIR, NLM\_BIN\_PATH - src.database.memory\_utils — remember() update\_ambient\_state, get\_ambient\_state, append\_pending\_share, enqueue\_task - src.agents.fsm — NOTEBOOK\_IDS - httpx — Wikipedia API - .llm\_utils — \_query\_llm

### **Friend configuration**

Three friends with distinct personas: - **Arthur** (philosophy notebook, Tuesday) — Academic philosophical debater - **Lyra** (misc notebook, Thursday) — Creative writer/poet - **Kate** (misc notebook, Saturday) — Theatre enthusiast

```
_seed_friend_interest(friend_name: str) -> str
```

LLM generates a new specific interest/obsession for a friend (e.g., "the Ship of Theseus paradox"). Random expiry 21-42 days. Saves to ambient state.

```
_ensure_friend_interest(friend_name: str) -> str
```

Returns current interest if still fresh (not expired), otherwise reseeds.

```
_seed_friend_life_state(friend_name: str) -> str
```

LLM generates a life state (Project, Mood, Reading) for a friend. 7-day expiry. Saves to ambient state as JSON.



`_ensure_friend_life_state(friend_name: str) -> str`

Returns current life state if fresh, otherwise reseeds.

`_research_interest(friend_name: str, interest: str) -> str`

Wikipedia API search for the friend's current interest. Returns a formatted extract (up to 600 chars) to inject into letter generation.

`_FRIEND_WHO: dict[str, Who]`

Module-level mapping from lowercase friend name → Who enum value. Used by `_stake_letter_for_friend` to resolve observers without scattered string literals. Covers {"arthur": Who.ARTHUR, "lyra": Who.LYRA, "kate": Who.KATE}.

`_stake_letter_for_friend(observer_name, friend_name, letter_content, source_memory_id)`  
-> None

Stores a lightweight `friend_letter_awareness` memory in the *observer* friend's own persona table after a letter exchange between Jane and another friend. Called once per non-writing friend from `friend_letter_task`.

- Resolves observer via `_FRIEND_WHO`; logs a warning and returns if unknown.
- Memory text: "Jane and {friend\_name} exchanged a letter today. Here is what {friend\_name} wrote: ..." (first 800 chars of letter).
- Metadata: type="friend\_letter\_awareness", friend, source\_memory\_id, created\_at — provenance chain links back to the primary letter memory ID.
- Kind: MemoryKind.LETTER; source: MemorySource.LETTER.
- Entire operation is wrapped in try/except; failures are logged and do not propagate to the caller.

`friend_letter_task(payload: Dict[str, Any])`

Scheduled pen-pal letter exchange. Flow: 1. Select friend by weekday (Arthur=Tue, Lyra=Thu, Kate=Sat). 2. Ensure interest + life state are fresh for the selected friend. 3. Gather recent Jane reflections for letter inspiration. 4. **Multi-turn conversation** (2-5 turns): - Turn 1: Friend writes letter (LLM with full context: interest, life state, research, past memories, cross-friend refs). - Turn 2: Jane replies (LLM as Jane responding). After Turn 2, calls `_generate_stake` with a friend-letter-specific prompt that emphasises emotional resonance and what Jane wants to say back — storing the stake in `about_jane` before proceeding to turns 3+. - Turns 3+: Alternating follow-ups, each with conversation history context (last 600 chars per message). 5. Save friend's first letter to `about_stuff` bucket under a pre-generated `letter_mem_id` (uuid4). 6. **Observer staking**: Calls `_stake_letter_for_friend` for every friend *not* involved in the exchange (e.g., Arthur writes → Lyra and Kate each receive an awareness memory in their persona tables). Provenance: `source_memory_id` in each observer's metadata links back to `letter_mem_id`. 7. **Persona memory**: After Turn 1 and each odd follow-up, stores the friend's first-person perspective in `persona_memories_{name}` via `store_persona_perspective()`. 8. Save all exchange messages to `about_friend_{name}` bucket. 9. Save cross-friend summary to `about_friends_cross` bucket. 10. Write full correspondence Markdown file to disk. 11. Upload to NotebookLM (routed by friend's notebook config). 12. Save reminder memory to `about_jane`. 13. Enqueue calendar event to schedule next exchange (1-2 days ahead). 14. Append to pending shares queue. 15. Send Telegram notification to JCD.

```
friend_reach_out_task(payload: Dict[str, Any])
```

Spontaneous friend reach-out (friends independently contacting Jane). Flow: 1. For each friend (Arthur, Lyra, Kate): a. Ensure life state + interest are fresh. b. Calculate probability: base 20% + days-since bonus (+5-20% based on 1/3/7 day thresholds) + mood boost (+8-12% for emotional keywords like “struggling”, “excited”), capped at 50%. c. Roll random — skip if probability miss. d. Generate friend message via LLM (warm check-in with life state context + mannerisms: Arthur=philosophical question, Lyra=nature poetry, Kate=theatrical flair). e. Save to about\_friend\_{name} bucket with type=reached\_out. f. Save cross-friend summary to about\_friends\_cross. g. Append to pending shares queue. h. *(Note: no Telegram notification — reach-outs are surfaced at Jane’s next reconnect via the share queue.)*

```
friend_life_event_task(payload: Dict[str, Any])
```

Weekly life event generation for each friend (Arthur, Lyra, Kate). Flow: 1. Skip if DND active. 2. For each friend: a. Get current life\_state from ambient state. b. Query recent persona\_memories (last 5) for continuity. c. Call LLM to generate 2-3 events (~70% positive, ~30% negative). d. Parse events line-by-line, store each via add\_persona\_memory() with type=life\_event. e. Life state rotation in \_seed\_friend\_life\_state persists outgoing state as a persona\_memory before overwriting. 3. Registered as suppressed task; also has a TASK\_TARGET\_TIMES entry for weekly scheduling [(6, 0, 0) = Sunday 6 AM].

**Cross-friend awareness:** friend\_letter\_task’s prompt includes a friend\_lives\_instruction block that queries other friends’ recent life events from persona\_memories\_{other} WHERE type=’life\_event’, so friends can naturally reference each other’s experiences in their letters.

---

## social/nudge.py (164 lines)

**Responsibility:** Daily inter-persona relations nudge simulator. Runs sweeps over the master social relation graph (data/personas/inter-persona-relations.mermaid), simulates dialogues between personas, evaluates updates to directional relationship closeness, and records memories (including gossip).

**Key dependencies:** - src.config — MAX\_PERSONA\_NUDGES\_PER\_DAY - src.agents.social\_graph — parse\_social\_graph - src.agents.persona — Persona class (relationship prompts, closeness getters/setters) - .llm\_utils — \_query\_llm

```
persona_nudge_task(payload: Dict[str, Any])
```

The central sweep function executed daily: 1. Skips execution if DND is active. 2. Parses the master Mermaid flowchart. 3. Checks daily execution limits to prevent nudging more than MAX\_PERSONA\_NUDGES\_PER\_DAY times. 4. For each directed edge in the graph: a. Rolls check using base edge probability boosted by current closeness between personas (asymmetric). b. On check success: - Reads relationship prompt from data/personas/<PERSONA>/relationships/<PERSONA>\_to\_<OTHER>.md. - Simulates 3-5 turn dialogue between the two personas via \_query\_llm. - Evaluates dialogue via the LLM to get a JSON payload containing closeness changes for both directions and gossip. - Updates closeness values (clamped to [0.0, 1.0] and rounded to 3 decimal places). - Logs a first-person experience memory for both personas by awaiting the async Persona.experience directly. - Saves gossip memory to the sender’s persona memories table so it can propagate in future interactions.

---

## kids.py (225 lines)

**Responsibility:** Kid social simulation system. Models two children (Clara 9, Finn 5) with independent life states, interests, and closeness tracking. Supports scheduled life event generation and probabilistic kid-initiated reach-out exchanges with Jane or between siblings.

**Key dependencies:** - src.config — KID\_REACH\_OUT\_CHANCE, KID\_CLOSENESS\_BUMP - src.database.db\_helpers — add\_persona\_memory, update\_ambient\_state, get\_ambient\_state - src.prompts — render\_prompt (10 prompt files across clara/ and finn/ personas) - .llm\_utils — \_query\_llm

`kid_life_event_task(payload: Dict[str, Any])`

Weekly life event generation for Clara and Finn. Flow: 1. Skip if DND active. 2. Skip if not Mon/Wed/Fri (matches friend cadence but different weekdays). 3. For each kid (clara, finn): a. Generate life state via LLM from `seed_life_state` prompt — stored in ambient state. b. Query recent `persona_memories` (last 5) for continuity. c. Call LLM to generate 1-2 sentence life event (positive, negative, or neutral). d. Store via `add_persona_memory()` with `type=life_event`, `source=life_event`. 4. Registered as suppressed task; Mon/Wed/Fri at `TASK_TARGET_TIMES` [(0, 8, 0), (2, 8, 0), (4, 8, 0)].

`kid_reach_out_task(payload: Dict[str, Any])`

Probabilistic kid reach-out exchanges (morning/evening slots). Flow: 1. Skip if DND active. 2. For each kid (clara, finn): a. Ensure life state + current interest are fresh (LLM-generated from prompts). b. Probability gate: 20% chance (`KID_REACH_OUT_CHANCE`) — skip if miss. c. Mode split: 80% kid→Jane exchange, 20% kid↔sibling exchange (Clara ↔ Finn). 3. **Kid→Jane exchange** (`_kid_jane_exchange`): a. Load kid's bio from `data/personas/{name}/{name}.md`. b. 4-7 turn conversation: first turn uses `letter_template` prompt, follow-ups use `follow_up`. c. Store full exchange via `add_persona_memory()` with `type=exchange`, `source=kid_reach_out`. d. Boost closeness in ambient state (`KID_CLOSENESS_BUMP` = 0.04). 4. **Sibling exchange** (`_kid_sibling_exchange`): a. Alternating turns between Clara and Finn (turns 0,2,4... = initiator, turns 1,3,5... = sibling). b. Store exchange in both kids' `persona_memories` tables with `source=kid_sibling_exchange`. c. Boost closeness for both kids. 5. No Telegram notification (unlike friend reach-outs — kids' exchanges stay in the family).

## Schedule config

- `kid_life_event_task`: Mon/Wed/Fri at 8 AM via `TASK_TARGET_TIMES` → runs weekly per-child via LLM
- `kid_reach_out_task`: twice daily (morning 8:30, evening 18:00) via `TASK_TARGET_TIMES`
- Both tasks registered in `SUPPRESSED_TASKS` (CPU throttle exempt)

## Closeness tracking

After each exchange, the kid's `{name}_closeness` ambient state key is incremented by `KID_CLOSENESS_BUMP` (0.04 per exchange, capped at 1.0). These keys are auto-discovered by the metrics snapshot system (`_snapshot_metrics` in `schedule.py`) and published as `friend_closeness_{name}` — visible on the dashboard Trends tab alongside Arthur/Lyra/Kate.

## babysitter.py (146 lines)

**Responsibility:** Babysitter scheduling, booking coordination, and hourly session/emergency simulation.

**Key dependencies:** - src.calendar\_client — GoogleCalendarClient - src.agents.coordination — BabysitterBooker, get\_event\_metadata, update\_event\_metadata - src.agents.persona — Persona - .llm\_utils — \_query\_llm

babysitter\_sync\_task(payload: Dict[str, Any])

Scans calendar events for today plus next 7 days. If a babysitting-eligible event is found without a sitter: 1. Checks sitter availability (Chrissie rules vs Backup bulletin/forum coordinates). 2. Simulates phone/text negotiation dialogues via LLM. 3. Updates calendar event description with hidden HTML comment metadata `<!-- SWEETHEART_METADATA = {...} -->`. 4. Saves first-person perspective memories to Jane's and the sitter's memory tables.

babysitting\_session\_task(event: Dict[str, Any])

Simulates hourly babysitting sessions when an event is active: 1. Triggers individual 3-turn sibling dialogues (sitter + Clara, sitter + Finn). 2. Runs emergency rolls (0.5% chance) or question checks (5% chance). 3. Persists dialogues to disk and logs first-person experiences to all involved characters' memory tables.

---

## Integration Points

### Database Writes by Bucket

Bucket	Written By
about_jcd	memory.py (extraction + sweep + consolidation)
about_jane	memory.py (extraction + sweep + HOV synthesis via life_story_capsule_task), ambient.py (journal + kid vignette stakes), social.py (reminders + friend-letter stakes), hobbies/ modules (reminders), feeds.py (RSS interest stakes), reader.py (reading stakes), db_init.py (second_order_values seed)
about_stuff	feeds.py (RSS), reader.py (reading notes), memory.py (dreams), social.py (friend letters), hobbies/curiosity.py (curiosity findings)
about_iroh_journey	memory.py (extraction + sweep)
about_friend_{name}	social.py (exchanges + reach-outs)
about_friends_cross	social.py
persona_memories_{name} (arthur, lyra, kate)	social.py (friend_letter_task — turns 1 & odd follow-ups via store_persona_perspective(); observer-awareness writes via _stake_letter_for_friend() after primary letter save; friend_life_event_task — weekly events via add_persona_memory())
persona_memories_{name} (clara, finn)	ambient.py (kid_vignette_task — per child via word-boundary regex), kids.py (kid_life_event_task — weekly events; kid_reach_out_task — exchange memories)

Bucket	Written By
- message_sentiments table (via save_message_sentiment) — memory.py (sentiment tag pass in sweep + backfill task)	

## Ambient State Keys

Key	Written By	Consumers
weather	schedule.py	FSM, spontaneous_nudge_task
daily_plan	schedule.py	FSM
last_plan_time	schedule.py	—
calendar_events	schedule.py	FSM
mood_narrative	schedule.py	FSM, iroh_morning_task
prev_mood_for_narrative	schedule.py	—
work_pressure	schedule.py	FSM
work_narrative	schedule.py	FSM
hobby_interest	schedule.py	FSM
hobby_drift_at	schedule.py	—
menstrual_cycle_start	schedule.py	FSM
menstrual_cycle_date	schedule.py	FSM
menstrual_cycle_phase	schedule.py	FSM
daily_mood_offset	schedule.py	FSM
last_mood_offset	schedule.py	—
last_extraction	memory.py	schedule.py
iroh_daily_wisdom	hobbies/iroh.py	FSM
focus_session_*	ambient.py	FSM
kid_vignette	ambient.py	FSM prompt builder ([KIDS] block)
{friend}_interest	social.py	—
{friend}_interest_expire	social.py	—
{friend}_life_state	social.py	—
{kid}_closeness	kids.py	schedule.py (_snapshot_metrics → friend_closeness_{name}), dashboard Trends tab
{kid}_life_state	kids.py	kids.py (kid_reach_out_task context)
{kid}_current_intent	kids.py	—

## External API Calls

API	Module	Purpose
Open-Meteo	weather.py, hobbies/stargazing.py	Weather fetching (Waterloo, ON)
Ollama (OLLAMA_LLM_URL)	llm_utils.py (for all modules)	LLM completions
CalDAV	schedule.py, calendar_tasks.py	Calendar read/write

API	Module	Purpose
DuckDuckGo (ddgs)	hobbies/curiosity.py, hobbies/garden.py, hobbies/stargazing.py	Web search for curiosity, gardening, stargazing
Wikipedia API	social.py	Friend interest research
NotebookLM CLI (nlm)	social.py, hobbies/ modules	Notebook note creation
HTTP URL fetch	reader.py	.url file content retrieval
iCal URL	schedule.py	Calendar fallback
Google Drive API (www.googleapis.com)	google_photos_upload.py, src/agents/tool_modules/photo_archive.py	Photo upload, album listing, photo archive via OAuth 2.0

## weather.py (191 lines)

**Responsibility:** Weather fetching, proactive weather nudges, EC alert polling, weather change classification, and time-of-day helper. Called by schedule.py's ambient\_update\_task. Provides the Open-Meteo API client.

**Key dependencies:** - Open-Meteo API (public, no key) — weather fetching (Waterloo, ON) - src.database.db\_helpers — get\_ambient\_state, update\_ambient\_state - .llm\_utils — \_query\_llm for weather nudge generation - src.agents.telegram\_client — send\_telegram\_message, is\_dnd\_active - src.config — EC\_ALERTS\_RSS\_URL, WEATHER\_NUDGE\_CHANCE - xml.etree.ElementTree — Atom RSS parsing for EC alerts

`_check_ec_alerts(rss_url: str) -> str`

Fetches Environment Canada Atom RSS feed, returns the first active warning headline under "Warnings and Watches", or empty string if none. Pure async helper (no side effects beyond HTTP).

`_classify_weather_change(old_desc, new_desc, old_alert, new_alert) -> str`

Pure function returning "must", "notable", or "skip". Must-tier: alert appears/clears or Stormy appears. Skip-tier: same description or trivial pairs (Clear→Partly Cloudy). Everything else is notable.

`_get_time_of_day(hour: int) -> str`

Pure function returning "morning" (0-11), "afternoon" (12-16), "evening" (17-20), or "night" (21-23).

`_fetch_and_update_weather() -> str`

Fetches Open-Meteo API, translates weather codes to descriptions, stores in ambient\_state("weather"). Polls EC alerts via \_check\_ec\_alerts and stores in ambient\_state("weather\_alert"). Classifies the change and sends a Telegram nudge if tier is "must" (always) or "notable" (probabilistic via WEATHER\_NUDGE\_CHANCE, gated by DND). Returns the weather string, or empty on failure. Called by schedule.py's ambient\_update\_task.

## schedule/ **Package (was schedule.py) 666 lines**

**Responsibility:** Ambient schedule orchestrator — canonical home for `ambient_update_task` and `check_and_drift_hobby` (re-exported from `ambient.py` for backward compatibility). Delegates weather fetching to `weather.py` and prompts to `render_prompt()` in `src.prompts`.

**Key dependencies:** - `src.config` — `CALENDAR_*`, `JCD_ICAL_URL`, `JCD_NUDGE_*`, `MEMORY_LLM_MODEL`, `NOTEBOOK_IDS` - `src.circadian` — `calculate_circadian_fatigue`, `calculate_target_mood`, `get_menstrual_modifiers` - `src.database.db_helpers` — `get_ambient_state`, `update_ambient_state`, `update_emotional_state`, `get_all_emotional_states`, `enqueue_task` - `.weather` — `_fetch_and_update_weather` - `.llm_utils` — `_query_llm` - `src.prompts` — `render_prompt()` for all LLM-facing prompt strings - `src.calendar_client` — `GoogleCalendarClient` - `icalendar` — `iCal` parsing fallback

`ambient_update_task(payload: Dict[str, Any])`

The primary ambient state orchestrator. Runs sequentially: 1. **Weather** — delegates to `_fetch_and_update_weather()` from `weather.py` 2. **Calendar** — CalDAV first, `iCal` fallback, overlap detection, structured JSON 3. **Daily plan** — calendar summary or LLM-generated plan via `render_prompt("daily_plan", ...)` 4. **Emotional drift** — cycle tracking, mood offsets, circadian baselines 5. **Work pressure** — work keyword scanning in calendar events 6. **Mood narrative** — LLM-generated via `render_prompt("mood_narrative", ...)` with lookup flag 7. **Hobby drift** — calls `check_and_drift_hobby()` 8. **JCD calendar** — proactive before/after nudges with dedup 9. **Session extraction** — inactivity-triggered sweep

`check_and_drift_hobby()`

Checks if hobby expired (3-7 weeks), LLM-generates a new academic/creative topic. Stores `hobby_interest` + `hobby_drift_at` in ambient state. Called by `ambient_update_task`.

`_snapshot_metrics()`

Internal coroutine called at the end of each `ambient_update_task` run (~15 min cadence). Writes a row to `metric_snapshots` for each of 26 metrics across four groups: - **Jane's state** (6): `mood`, `energy`, `closeness`, `jcd_mood`, `jcd_energy`, `jcd_engagement` - **System health scores** (8): `score_memory`, `score_task_queue`, `score_emotional`, `score_ambient`, `score_circadian`, `score_db_health`, `score_search_quality` (conditional — omitted if FSM server unavailable), `score_prompt_blocks` - **Activity counts** (8): `memories_about_jcd`, `memories_about_stuff`, `memories_new_24h`, `messages_total`, `conversations_active_7d`, `tasks_failed`, `tasks_pending`, `pending_shares` - **Friend closeness** (data-driven): one key per `{friend}_closeness` row found in `ambient_state`

`score_search_quality` is fetched from the FSM server via GET `http://localhost:11434/internal/score-cache` (2s timeout, 24h staleness check using `compute_search_quality_score()` from `src.score_component`). On miss, the metric is omitted — the chart shows a gap rather than a misleading 100.0 flatline. Each group is independently try/caught so a single failure never silences the others.

---

## `google_photos_upload.py` (97 lines)

**Responsibility:** Uploads photos to Google Drive via OAuth 2.0. Two-step approach to avoid multipart/related issues:

1. POST `/drive/v3/files` — create file metadata (name, description, folder)

2. PATCH /upload/drive/v3/files/{id}?uploadType=media — upload content bytes

**Key dependencies:** - src.config — GOOGLE\_PHOTOS\_CLIENT\_ID, GOOGLE\_PHOTOS\_CLIENT\_SECRET, GOOGLE\_PHOTOS\_REFRESH\_TOKEN, GOOGLE\_PHOTOS\_ALBUM\_ID - httpx — all HTTP calls

upload\_photo\_to\_google\_photos(payload: dict) -> None

Main task handler. Reads photo\_path and label from payload. Refreshes the OAuth token, creates the Drive file in the shared folder, then uploads the file bytes. Label populates the description field (fallback: filename). All three API steps are individually logged on success or failure.

## reconcile.py

DB-authoritative TELEGRAM ← DB → OWU reconciliation sweep. DB ACTIVE\_THREAD\_CHAT\_ID is the source of truth; the sweep non-destructively heals both edges. See ADR-0042.

reconcile\_sync(dry\_run=False) -> dict

Fan-out entry point. Runs the OWU and Telegram edges with per-edge isolation (one failing edge never aborts the other). reconcile\_owu\_task (the cron handler) calls it.

reconcile\_owu\_from\_db(dry\_run=False) -> dict

OWU edge (read/write). Appends the trailing run of DB turns missing from the OWU sync chat, pushing tag-stripped (cleaned) assistant text. Mid-history holes are logged, not filled (positional insertion deferred). Keeps rotating OWU chat backups.

reconcile\_telegram\_from\_db(dry\_run=False) -> dict

Telegram edge (append + prune; the Bot API can't read history). Forward-only from a high-water mark seeded on first run (backlog out of scope). Sends only post-mark turns whose content fingerprint is absent from the telegram\_delivery ledger, capped at \_TELEGRAM\_SWEEP\_MAX\_PER\_RUN (20) per tick, recording each send.

Ledger + fingerprint helpers live in db\_helpers: telegram\_delivery\_fingerprint, record\_telegram\_delivery, get\_telegram\_delivered\_fingerprints, get/set\_telegram\_sync\_hwm. The live send path (server/main.py closures, telegram\_client.py) records deliveries so the sweep never double-sends. CLI: scripts/reconcile\_owu.py [--dry-run].



## src/worker/tasks/hobbies/

### Responsibility

Jane's hobby/activity simulation system: nightly curiosity research, virtual Zen gardening, Waterloo-aware stargazing, and Iroh wisdom tasks. Refactored from the original `hobbies.py` into a package with one domain module per hobby. `hobbies.py` at the parent level remains a backward-compatible wrapper.

### Design

- `__init__.py` — re-exports `get_embedding` + `_query_llm` (for test patching) and all five task handlers: `curiosity_task`, `garden_task`, `stargazing_task`, `iroh_morning_task`, `iroh_reflection_task`
- `curiosity.py` — nightly curiosity explorer: fetch open curiosity questions, DuckDuckGo research (2 queries), LLM synthesis, optional friend debate, NotebookLM note, pending-share + Telegram notification
- `garden.py` — daily Zen Gardening simulation (Sakura Bonsai + Midnight Orchid in `data/garden_state.json`): moisture decay, growth stages, care-tip web search, journal entry → NotebookLM misc notebook
- `stargazing.py` — daily stargazing: Open-Meteo weather gate, random celestial object, DuckDuckGo research or cloudy-day reflection, log → NotebookLM
- `iroh.py` — `iroh_morning_task` (daily 7:30 AM wisdom) + `iroh_reflection_task` (weekly Sunday reflection); caches `iroh_daily_wisdom` ambient key

### Flow

Tasks are enqueued on cron intervals by the worker and dispatched from `TASK_HANDLERS` (`curiosity`, `garden`, `stargazing`, `iroh_morning`, `iroh_reflection`). Each task ends by appending to the pending-shares queue and/or sending a Telegram nudge to JCD.

### Integration

- **Depends on:** `src.config` (`DATA_DIR`, `NLM_BIN_PATH`), `src.database.db_helpers`, `src.embeddings` (`get_embedding`), `.llm_utils` (`_query_llm`), `ddgs` (web search), `httpx`, `nlm` CLI subprocess, Open-Meteo
- **Consumed by:** worker `TASK_HANDLERS`; `src/worker/tasks/hobbies.py` compatibility wrapper

## womb/

### Responsibility

Scratchpad / backlog for one-off feature specs, design notes, and migration plans — captured during brainstorming before they are promoted into the standard spec pipeline (docs/specs/YYYY-MM-DD-<slug>/) or become ADRs. Not part of the application runtime; plain markdown working notes only.

### Design

- Single flat directory of markdown files — no code, no subpackages.
- Filename prefixes encode scope/size where present:
  - xl- / XL- (extra-large): group chat, Jane’s hearing, telegram replay injection, co-watch, ideas dump (\_\_dump.md)
  - l- (large): dotty integration, grafana, parenting-and-kids, rag-optimization, sync-after-edit, etc.
  - m- (medium): dashboard parity, fsm base-class refactoring, pluggable client backend, proactive engagement, etc.
  - s- (small): dashboard-parity sub-features
- Special files: \_\_bugs.md, \_\_dump.md (unscoped idea dump), suggestions.md.
- born/ — subdirectory of design notes already “born” into the codebase (wiki agent, babysitter, dotty, hipbone, inter-persona-relations, invite, dashboard-expandable-tasks, etc.).

### Flow

Ideas are captured here during brainstorming; once adopted they move to the standard spec pipeline (docs/specs/) or become ADRs. born/ holds notes for features already implemented.

### Integration

None — read-only reference material for humans and agents.

# APPENDIX D: Architecture Info

<b>Sweetheart Core Architecture</b>	<b>108</b>
Agent FSM . . . . .	108
Prompt Pipeline . . . . .	108
Database & Sub-packages . . . . .	108
Group Chat Orchestrator . . . . .	109
Dashboard . . . . .	109
Circadian Layer . . . . .	109
Worker Tasks . . . . .	109
Score Component . . . . .	110
Data Storage . . . . .	110
Detailed Logging . . . . .	110
ADRs . . . . .	110
Tool System . . . . .	111
Green Terminal Display . . . . .	111
Dotty Physical Robot Control . . . . .	111
PersonaProvider Protocol . . . . .	112

# Sweetheart Core Architecture

This file is consumed by the **architect** advisor during /cp. review and by any agent discovering this codebase. Keep it accurate and current.

## Agent FSM

- **Location:** src/agents/fsm.py
- In-process state machine (not a microservice)
- LLM calls are synchronous HTTP to Ollama
- Prompt building and emotion evaluation are inline coroutines within the FSM step

## Prompt Pipeline

- **Location:** src/agents/prompt\_builder.py
- 3-tier stochastic enrichment block selection (top/mid/low with 70/30 roll and fallthrough)
- Block stats buffered in-memory and periodically batch-flushed to SQLite (prompt\_block\_log) via a daemon background thread (ADR-0076)
- Pure function `_select_enrichment_blocks()` at module level for testability
- **Gated Opinions:** Dynamic on-demand hot-reloaded opinion snippet matching. The topic query is matched using SQLite hybrid search (FTS5 + sqlite-vec vector similarity) restricted to the "opinions" bucket inside `memories_{who}` table. Enforces a relevance guard of FTS5 keyword hit or L2 distance < 0.8.

## Database & Sub-packages

- **Location:** src/database/
- Single SQLite file
- Schema migrations are imperative, split into src/database/migrations/ package (ADR-0064)
- db\_helpers.py split into modular domain helpers under src/database/helpers/ (memories.py, dnd.py, search.py, telegram.py, chat.py, prompt\_stats.py, metrics.py, tool\_log.py) while preserving top-level re-exports for zero-breakage backwards compatibility (ADR-0063).
- Strongly typed row models in src/database/models/ (MemoryRow, AmbientStateRow, TaskQueueRow) with unified dataclass converters `row_to_model()` (ADR-0071).
- Partitioned Persona Memory Schema: Creates dedicated `memories_{persona}`, `vec_memories_{persona}`, and `memories_fts_{persona}` tables for each active persona in Who (e.g. Jane, JCD, virtual friends, children). Resolves the sqlite-vec global k-NN "top-k drowning problem" by isolating vector search strictly on the target persona's private index.
- Serialized Persona State: Persona-specific behavioral states (e.g., mood states, tangent flags, slurring indicators) are stored as JSON strings in the `ambient_state` table under `{persona_name}:state` keys, abstracted by the Persona base class in src/agents/persona.py (ADR-0053).
- Partitioned Chat Messages: Creates per-persona `chat_messages_{who}` and `message_sentiments_{who}` tables for each active persona in Who. All CRUD helpers (`save_message`, `get_active_thread_messages`, `recall_session`, `save_message_sentiment`) accept a who parameter (default "jane"). Legacy global `chat_messages/message_sentiments` tables are migrated to `chat_messages_jane/message_sentiments_jane` at startup (one-time) then dropped. Background tasks (`session_sweep`, `notebook_sync`, CPU throttle) query the correct partition via who payload parameter. See ADR-0056.

## Group Chat Orchestrator

- **Location:** `src/agents/group_chat.py`
- State-locked turn-taking system for Telegram group chats.
- Active speaker locks, cooldown controls (40s), and fast routing via cheap 2B model prompt.
- Parallel candidate generation with staggered typing delays.
- Referee pass resolves sequence ordering and drops redundant replies.

## Dashboard

- **Location:** `src/dashboard/`
- FastAPI serving server-rendered HTML with client-side JS tab switching
- Template read from disk on every request (no caching, live reload)
- Port: 11442

## Circadian Layer

- **Location:** `src/circadian/`
- Energy/sleep/mood model with spillover to agent behavior
- Temperature modulation in FSM based on energy + phase

## Worker Tasks

- **Location:** `src/worker/tasks/`
- Background extractors (memory, RSS, calendar, weather, photo archival)
- Kicked off by the scheduler FSM
- **World Bot Polling Loop** (`world_bot.py`): A dedicated long-lived Telegram polling loop for world exploration (separate from Jane's main Telegram bot). Runs alongside the task queue via `asyncio.gather` in `start_worker()`. Rate-limited, user-whitelisted, with inline keyboard navigation for linked locations.
- Gossip system: `social.py` (`_synthesize_gossip`, per-interaction) + `schedule.py` (`gossip_sweep_task`, JCD conversation sweep, interval-gated). Gossip stored in gossip memory bucket; always surfaced to friend letter prompts, chance-gated for JCD prompt.
- Friend life events: `social.py` (`friend_life_event_task`, weekly) generates 2-3 life events per friend (Arthur, Lyra, Kate) stored in `persona_memories_{name}`. Outgoing life state is preserved as a persona memory on rotation. Cross-friend awareness via `friend_lives_instruction` block in letter prompts.
- Inter-persona relations: `nudge.py` (`persona_nudge_task`, daily) — Parses the master social graph from `data/personas/inter-persona-relations.mermaid`, rolls probability checks based on asymmetric closeness stored in each persona's state JSON, simulates a 3-5 turn conversation using external prompts (`data/personas/<PERSONA>/relationships/`), and updates closeness deltas + memory tables (including gossip propagation).
- Kid life events: `kids.py` (`kid_life_event_task`) generates weekly life events for Clara and Finn (Mon/Wed/Fri) stored in `persona_memories_{name}`. Kid reach-outs (`kid_reach_out_task`, twice daily) run 4-7 turn exchanges with Jane (80%) or between siblings (20%), with closeness tracking pushed to dashboard metrics.
- Sync reconciliation: `reconcile.py` (`reconcile_sync`, every 15 min) — DB-authoritative TELEGRAM ← DB → OWU sweep. OWU edge scans for divergence, truncates obsolete branch tails, and rebuilds remaining turns dynamically to repair mid-history holes (edits, deletions, or insertions); Telegram edge backfills undelivered turns via a content-fingerprint telegram\_delivery ledger + forward-only high-water mark (never mass-sends the backlog). Cleaned text to edges, non-destructive, per-edge isolation. See ADR-0075.

## Score Component

- **Location:** `src/score_component.py`
- Eight typed sub-score dataclasses (`MemoryScores`, `TaskQueueScores`, `Emotional-StateScores`, `AmbientScores`, `CircadianScores`, `DBHealthScores`, `SearchQualityScores`, `PromptBlockScores`) assembled into `ScoreComponent`
- `ScoreComponent.from_db()` opens one connection, fetches health metrics using consolidated SQL aggregate statements (reducing SQLite roundtrips from 17 down to 9 on a cache miss). It wraps reads in a thread-safe, process-local in-memory 30-second TTL cache to completely avoid DB reads on active request turns. See ADR-0077. Always returns a valid object with sentinel values on failure
- Two-phase population per FSM turn: `from_db()` at turn start → `with_search_quality(results)` after retrieval (returns new instance via `dataclasses.replace()`)
- CLI: `scripts/score-component.py --json` emits `[{name, score, grade, issues}]` consumed by the full-repo-review skill's pre-check step

## Data Storage

- **Location:** `data/` root for persistent DB (`sweetheart.db`) and primary subdirectories.
- **Runtime State:** `data/state/` for JSON state files. Standardized access via `src/config.py` path constants or `os.path.join(DATA_DIR, "state", ...)`.
- **Application Logs:** `data/logs/` for prompt logging and debug logs. Standardized access via `os.path.join(DATA_DIR, "logs", ...)`.
- **Voice Data:** `data/voice/` (models) and `data/voice-stream/` (TTS output).
- **World Data:** `data/world/` for markdown location pages (slug-based, navigable by Jane via world tool). ## Generation Module
- **Location:** `src/agents/generation.py`
- Extracted from `fsm.py`: streaming (`generate_stream()`), non-streaming (`process_final_non_stream()`), calendar scheduling classification (`check_calendar_scheduling()`), tag stripping, and prompt logging
- `fsm.py` is now a pure state-machine orchestrator; all I/O-heavy generation logic lives here

## Detailed Logging

- **Location:** `src/agents/detailed_logger.py`
- Per-turn paired markdown files (`*.prompt.md` / `*.response.md`) written to `data/logs/`
- Captures verbatim Ollama payload, raw model output, memory recall data, and system metrics
- Gated behind `DETAILED_LOGGING_ENABLED` env var (default: `false`)
- Complementary to the existing `_write_prompt_log` / dashboard-toggle mechanism (ADR-0011); that single-file system is retained for interactive use
- See ADR-0036 and `docs/specs/2026-06-30-detailed-logging/01-design.md`

## ADRs

- **Location:** `docs/adrs/`
- Architecture Decision Records from ADR-0001 onward. ADR-0055 defines the direct command parser (`c:` prefix interception) and commentless image generation guessing game mode.

## Tool System

- **Location:** `src/agents/tool_registry.py` + `src/agents/tool_modules/` + `src/agents/tools.py` + `src/agents/tool_services.py`
- `ToolDef` dataclass with `name`, `description`, `parameters`, `fn` — single source of truth for each tool
- `REGISTRY` is populated at import time by each tool module self-registering
- `TOOL_MAP` is a dynamic Mapping read-through to `REGISTRY` — always in sync
- `fsm.py:execute_tool()` dispatches via `TOOL_MAP[name](**arguments)` — no `elif` chain
- `deps.py` holds shared dependencies (`get_embedding`) — tools use `remember()` and `recall()` (from `src/database/memory_utils`) for typed memory read/write instead of raw `deps.add_memory / search_memories`
- `tool_services.py` — gated messaging service: `SendResult` `StrEnum` (`SENT/BLOCKED`), `send_user_message()` with `DND` + rate-limit gate baked in, `atomic_try_acquire_rate_slot()`. Used by `reach_out.py` for JCD Telegram sends.
- **13 FSM tools** in `src/agents/tool_modules/`. Several are **action-dispatch “drawers”** — one tool with an action enum + per-action params (`notebook`, `web`, `images`, `hipbone`, `world`, `dotty_control`) — collapsed from ~21 finer-grained tools to fit the model’s tool-selection weight limit; see ADR-0046 (`scripts/probe_tool_selection.py` guards it). Prefer extending a drawer with a new action over adding a top-level tool.
- Notebook access is the notebook drawer (`action="query"` reads NotebookLM via the `n1m CLI` + Drive notes; `action="record"` writes to Drive) backed by `src/agents/notebook_backend.py` — see ADR-0045. The storage backend is hidden from the persona (`[NOTEBOOKS]` block advertises only the abstraction). `GoogleDriveFS`, `generate_image.py`, and `photo_album.py` remain internal helper libraries used by drawers/background tasks/scripts.
- Adding a new messaging tool (for JCD): use `send_user_message()` from `tool_services.py` — gate is baked in, cannot be bypassed
- Key recent: ADR-0016 (mobile-first dashboard), ADR-0017 (agent module split), ADR-0018 (score-component pattern), ADR-0019 (`generation.py` extraction)
- **world tool** (`src/agents/tool_modules/world.py`): 8 sub-actions (`look_around`, `move_to`, `new_location`, `modify_surroundings`, `create_travel_path`, `remove_travel_path`, `travel_history`, `invite`). File+DB hybrid storage — markdown pages in `data/world/`, link graph in SQLite. Allowed personas: `[Who.JANE]`. See ADR-0044. Its `invite` action uses the generic `invite` service (ADR-0072).

## Green Terminal Display

- **Location:** `src/agents/tool_modules/green_term/`
- `send_msg.py` — standalone CLI script for sending messages, alerts, and sticky alerts to the ESP32-S3 green-phosphor terminal display over the ESPHome native API (port 6053, encrypted via Noise PSK from `secrets.yaml`). Also exposes `light/switch/number` entity control and full API reference help.
- `error_handler.py` — `GreenTerminalHandler` (standard `logging.Handler` subclass) that pipes `ERROR`-level log records to the display with 60s dedup and sticky escalation after 2 suppressions.

## Dotty Physical Robot Control

- **Location:** `src/server/dotty_client.py` (HTTP client), `src/agents/tool_modules/dotty_control.py` (FSM tools), `src/server/main.py` (proxy endpoints)
- Async HTTP client for dotty-behaviour’s `/api/jane/*` routes
- FSM tools: `dotty_say`, `dotty_take_photo`, `dotty_abort`, `dotty_get_state`
- Proxy endpoints on the API server (`/v1/dotty/control`, `/v1/dotty/thinking`, `/v1/dotty/touch-webhook`, `/v1/dotty/state`) — same pattern as the perception pipeline

- All methods return user-facing strings, never raise, network errors produce descriptive error strings
- Lazy client init reads DOTTY\_API\_URL from os.environ at call time for testability
- Tools registered via REGISTRY["name"] = ToolDef(...) — no special wiring needed

## PersonaProvider Protocol

- **Location:** src/personas/ (providers) + src/agents/agent\_base.py (wiring)
- ProviderBase + config dataclasses (EmotionConfig, LLMConfig, MemoryConfig, Prompt-Block) define all persona-specific decisions
- PersonaFSMAgent accepts optional provider kwarg — delegates identity, prompt blocks, emotion config, LLM config, memory
- JaneProvider: 31 blocks, full coupling chain; DottyProvider: 10 blocks, no coupling; FriendProvider: 5 blocks, no coupling
- Coupling chain gated on provider.emotion\_config.coupling\_chain; LLM context via provider.llm\_config.num\_ctx/predict
- \_inject\_prompt() delegates to provider.build\_prompt() when set, falls back to build\_system\_prompt()
- See ADR-0057 for full design



# APPENDIX E: Architecture Decision Records

<b>ADR-0001: Modularize Hobbies Task Handlers</b>	<b>117</b>
<b>ADR-0002: Adopt Hybrid Vector and Keyword Search</b>	<b>119</b>
<b>ADR-0003: Migrate Vector DB from Qdrant to SQLite <code>sqlite-vec</code></b>	<b>121</b>
<b>ADR-0004: Decouple Task Execution with Asynchronous SQLite-backed Task Queue</b>	<b>123</b>
<b>ADR-0005: Decoupled FSM 6-State Reasoning Pipeline</b>	<b>125</b>
<b>ADR-0006: Dual Interface Architecture (OpenAI-Compatible API + Telegram Bot)</b>	<b>127</b>
<b>ADR-0007: Local-First Single-File SQLite Database Architecture</b>	<b>129</b>
<b>ADR-0008: Three-Layer Persona Architecture</b>	<b>131</b>
<b>ADR-0009: Topic Stakes Pattern for Personality Grounding</b>	<b>133</b>
<b>ADR-0010: Prompt File Extraction (Prompt-as-Code Separation)</b>	<b>135</b>
<b>ADR-0011: Feature Flag Instrumentation and Debugging Architecture</b>	<b>137</b>
<b>ADR-0012: Memory Participant Mixing Prevention</b>	<b>139</b>
<b>ADR-0013: Parallel Sweep Passes for Extraction Latency Reduction</b>	<b>141</b>
<b>ADR-0014: Circadian Temperature Modulation for Behavioral Depth</b>	<b>143</b>
<b>ADR-0015: Context Window Health Monitor with Per-Block Token Metrics</b>	<b>145</b>
<b>ADR-0016: Mobile-First Dashboard Layout</b>	<b>147</b>
<b>ADR-0017: Agent Module Split — <code>tools.py</code> + <code>prompt_builder.py</code> from <code>fsm.py</code></b>	<b>149</b>
<b>ADR-0018: Prompt File Extraction Pattern (Extract + <code>render_prompt</code>)</b>	<b>151</b>
<b>ADR-0018: Score Component — Typed Runtime Health Snapshot</b>	<b>153</b>
<b>ADR-0019: In-Memory LRU Embedding Cache</b>	<b>155</b>

<b>ADR-0019: Extract <code>generation.py</code> from <code>fsm.py</code></b>	<b>156</b>
<b>ADR-0020: Prompt Enrichment Block Architecture</b>	<b>158</b>
<b>ADR-0021: Response Post-Processing Filters</b>	<b>160</b>
<b>ADR-0022: Embeddings API Passthrough</b>	<b>161</b>
<b>ADR-0023: Per-Turn Sentiment Tagging as 6th Sweep Pass</b>	<b>162</b>
<b>ADR-0024: Kid Vignette Prompt Enrichment Block</b>	<b>165</b>
<b>ADR-0025: Tool-Based Citation Retrieval</b>	<b>167</b>
<b>ADR-0026: Persona Memory Pattern</b>	<b>169</b>
<b>ADR-0027: Structured JSON Logging</b>	<b>171</b>
<b>ADR-0028: Gated Prompt Blocks (Topic-Matched Dynamic Opinion Injection)</b>	<b>173</b>
<b>ADR-0029: Stochastic 3-Tier Enrichment Block Selection</b>	<b>175</b>
<b>ADR-0030: Tool Registry Pattern — Decoupled Tool Definitions</b>	<b>177</b>
<b>ADR-0031: Data Directory Reorganization</b>	<b>179</b>
<b>ADR-0032: Sensory Anchoring in Episodic Memory Retrieval</b>	<b>180</b>
<b>ADR-0033: Enum-Typed Memory Retrieval (recall + <code>get_subjects</code>)</b>	<b>181</b>
<b>ADR-0034: Group Chat Orchestration &amp; Partitioned Persona Memories</b>	<b>183</b>
<b>ADR-0035: Memory Enum Consolidation &amp; Schema Simplification</b>	<b>185</b>
<b>ADR-0036: Detailed Per-Turn Prompt/Response Logging</b>	<b>187</b>
<b>ADR-0037: Persona-Tagged Session Summaries</b>	<b>188</b>
<b>ADR-0038: Session-Linked Hybrid RAG Evaluations</b>	<b>190</b>
<b>ADR-0039: HipBone Game State Architecture</b>	<b>191</b>
<b>ADR-0040: DB→OpenWebUI Reconciliation</b>	<b>193</b>
<b>ADR-0041: AI Image Generation Tool</b>	<b>195</b>

<b>ADR-0042: DB-Authoritative Telegram &lt;-&gt; OWU Sync Reconciliation</b>	<b>198</b>
<b>ADR-0043: Persona-Scoped Memory Writes</b>	<b>200</b>
<b>ADR-0044: Location-via-Wiki Hybrid Storage Architecture</b>	<b>202</b>
<b>ADR-0045: Unified Notebook Access (NotebookLM read, Drive write)</b>	<b>204</b>
<b>ADR-0046: Collapse FSM Tools into Action-Dispatch Drawers</b>	<b>206</b>
<b>ADR-0047: Core System Commands Architecture</b>	<b>208</b>
<b>ADR-0048: Calendar Event Invitations via CalDAV Attendee</b>	<b>210</b>
<b>ADR-0049: Calendar via Google Calendar REST API v3 (OAuth)</b>	<b>212</b>
<b>ADR-0050: Shared Document Chunking Module</b>	<b>214</b>
<b>ADR-0051: Pluggable Client Backend for Nudge Delivery</b>	<b>216</b>
<b>ADR-0052: Help Tool &amp; Dynamic Schema Injection</b>	<b>218</b>
<b>ADR-0053: Persona State Serialization</b>	<b>220</b>
<b>ADR-0054: Inter-Persona Relations Social Graph &amp; Asymmetric Closeness</b>	<b>221</b>
<b>ADR-0055: Commentless Image Guessing Game and Direct Command Parsing</b>	<b>222</b>
<b>ADR-0056: Partitioned Chat Messages for Multi-Persona Isolation</b>	<b>223</b>
<b>ADR-0057: PersonaProvider Protocol for FSM Agent Configuration</b>	<b>224</b>
<b>ADR-0058: Generic PromptBlock Assembly Engine</b>	<b>226</b>
<b>ADR-0059: World Events System</b>	<b>228</b>
<b>ADR-0060: Multi-Agent Location-Tied Group Chat</b>	<b>230</b>
<b>ADR-0061: Per-Persona Telegram Routing and Caching</b>	<b>232</b>
<b>ADR-0062: SQLite Thread-Local Connection Pool and Batch Ambient Reads</b>	<b>234</b>
<b>ADR-0063: Split db_helpers.py into helpers/ Sub-Package</b>	<b>235</b>
<b>ADR-0064: Migration Functions Package</b>	<b>237</b>

<b>ADR-0066: Server Handler Decomposition and Unified Proxy Pipeline</b>	<b>239</b>
<b>ADR-0067: PersonaFSMAgent Collaborators and Process Chat Decomposition</b>	<b>240</b>
<b>ADR-0068: Telegram Client Decomposition and State Isolation</b>	<b>241</b>
<b>ADR-0069: Speech Service Decomposition and Data-Driven Audio Pipelines</b>	<b>242</b>
<b>ADR-0070: Extract Task Outputs Script Refactoring and Production De-forking</b>	<b>243</b>
<b>ADR-0071: Typed Row Models (Memories Domain First) and row_to_memory Mapper</b>	<b>244</b>
<b>ADR-0072: Generic Invite Service (/invite + world(action="invite"))</b>	<b>245</b>
<b>ADR-0073: Stream All Logs to Grafana Loki (via python-logging-loki-v2)</b>	<b>246</b>
<b>ADR-0074: Hybrid Opinion Search (Gated Snippets Upgrade)</b>	<b>248</b>
<b>ADR-0075: OpenWebUI Mid-History Reconcile &amp; Repair</b>	<b>250</b>
<b>ADR-0076: Telemetry Write Buffering to Prevent SQLite Lock Contention</b>	<b>251</b>
<b>ADR-0077: ScoreComponent Query Consolidation &amp; TTL Caching</b>	<b>253</b>
<b>ADR-0078: Dashboard FastAPI Lifespan Migration</b>	<b>254</b>
<b>ADR-0079: Choose Your Own Task Agency System</b>	<b>255</b>

# ADR-0001: Modularize Hobbies Task Handlers

**Status:** Accepted

**Date:** 2026-06-19

## Context

The background task handler file `src/worker/tasks/hobbies.py` had grown to over 1,000 lines. It contained code for five distinct background tasks across four different hobby/activity simulations (nightly curiosity explorer, daily Zen gardening, simulated stargazing, and Iroh wisdom/reflection coaching).

This monolithic structure presented several issues: 1. **Maintainability:** Hard to navigate, maintain, and review changes. 2. **Separation of Concerns:** Unrelated tasks shared the same module scope, mixing distinct dependencies (e.g., `ddgs`, `httpx`, and `subprocess`). 3. **Testing:** Mocking/patching internal helpers for one task affected the entire file, and test isolation was poor.

We needed to refactor this file to reduce its size, improve code organization, and enforce a clean modular layout without breaking existing unit tests, worker registrations, or imports.

## Decision

We decided to split `src/worker/tasks/hobbies.py` into a package directory module `src/worker/tasks/hobbies/`:

1. **Sub-module Isolation:** Created a separate file for each domain:
  - `curiosity.py` (`curiosity_task`)
  - `garden.py` (`garden_task`)
  - `stargazing.py` (`stargazing_task`)
  - `iroh.py` (`iroh_morning_task`, `iroh_reflection_task`)
2. **Package Entry Point:** Created `src/worker/tasks/hobbies/__init__.py` to re-export all tasks and expose common utility functions (`_query_llm` and `get_embedding`) that tests mock on the package level.
3. **Backward Compatibility Wrapper:** Replaced the original `src/worker/tasks/hobbies.py` with a thin compatibility wrapper file that re-exports all tasks and mocks from the new package directory.
4. **Dynamic Lookup Wrapper Functions:** To support existing test suites that patch `src.worker.tasks.hobbies._query_llm` or `src.worker.tasks.hobbies.get_embedding` at the module level (which would otherwise not affect function calls inside separate sub-modules), we implemented async lookup wrappers inside the sub-module files to resolve these functions dynamically from the parent hobbies package at runtime:

```
async def _query_llm(*args, **kwargs):
 import src.worker.tasks.hobbies as hobbies
 return await hobbies._query_llm(*args, **kwargs)
```

## Consequences

### Positive

- **Improved Code Maintainability:** Each task is now contained in a small, focused file of under 250 lines.

- **Strict Backward Compatibility:** All existing imports in `src/worker/worker.py` and existing mock patches in tests (like `tests/test_iroh.py` and `tests/test_garden_worker.py`) continue to function without any code changes.
- **Clean Scope:** Dependencies like `ddgs` (DuckDuckGo search) or `httpx` are only imported inside the specific task sub-modules that actually require them.

## Negative

- **Indirection Overhead:** Sub-modules rely on dynamic lookup wrapper functions to resolve `_query_llm` and `get_embedding` to allow mock patching, which introduces a runtime lookup cost (negligible for background tasks).
- **Import Duplication:** Core boilerplate imports (like `logging`, `time`, `json`, `uuid`) are duplicated across the sub-module files to maintain module autonomy.

## Neutral

- The `src/worker/tasks/hobbies.py` file must remain as a wrapper until client references and tests are fully updated to import directly from the sub-modules or the new sub-package.

## References

- Design Spec: `2026-06-19-hobbies-refactor.md`
- Implementation Plan: `2026-06-19-hobbies-refactor.md`

# ADR-0002: Adopt Hybrid Vector and Keyword Search

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane's memory retrieval initially relied purely on semantic vector search (768-dimensional embeddings generated via Nomic Embed Text and queried via sqlite-vec).

While semantic search is excellent at matching abstract concepts and emotional sentiments, it exhibits several critical limitations in practice:

1. **Proper Noun Recall:** Vector search struggles with exact keyword matching, particularly for proper nouns (e.g. friend names like "Arthur", "Lyra", "Kate") or specific terms. Semantic similarity would often retrieve adjacent but incorrect memories (e.g., returning an Arthur memory for a Kate query because both are conversational or philosophical in context).
2. **Short Queries:** Short queries (like single words or short phrases) lack the semantic depth required to generate high-quality embedding vectors, leading to irrelevant matches.
3. **Keyword Precision:** Classic keyword indices (like SQLite's built-in FTS5 virtual tables using BM25 scoring) excel at exact keyword overlap but fail at semantic synonyms.

We needed a search mechanism that combines the conceptual coverage of vector embeddings with the exact keyword-matching precision of text search.

## Decision

We implemented a Hybrid Search system combining semantic vector search and exact keyword search inside the local SQLite database:

1. **FTS5 Virtual Table:** Expose a shadow FTS5 index table `memories_fts` built on the `memories` table to enable fast BM25 keyword queries.
2. **Query Sanitization:** Implement `_fts_escape()` to sanitize search inputs to prevent FTS5 query syntax errors by stripping non-alphanumeric characters and collapsing whitespace.
3. **Reciprocal Rank Fusion (RRF):** Implement `hybrid_search_memories()` which queries both systems concurrently (fetching `top_k * 3` candidates from each) and merges the results using Reciprocal Rank Fusion:

$$\text{rrf\_score} = 1.0 / (\text{RRF\_K} + \text{rank\_vector}) + 1.0 / (\text{RRF\_K} + \text{rank\_keyword})$$

where the constant `RRF_K` is set to 60.

4. **Dynamic Search Routing:** FSM agent calls `hybrid_search_memories()` by default when retrieving context for JCD and Jane memories.

## Consequences

### Positive

- **Highly Accurate Noun Recall:** Querying "Kate" or "Bonsai" guarantees returning memories referencing those exact words.
- **Robust Synergy:** Merges semantic context (concept matching) with literal query terms (lexical matching).
- **Single Database Solution:** Avoids dependencies on external search services or complex database engines by keeping all search index data in the local SQLite WAL file.

## Negative

- **Minor Query Latency:** Executing two queries (vector + keyword) and performing the rank fusion in Python adds a slight, though negligible, latency overhead.
- **Index Management:** Requires keeping both the vector index and the FTS5 full-text search index synchronized.

## Neutral

- FTS5 query tokenization relies on default simple tokenization, which is sufficient for English search terms but does not support advanced word stemming.

## References

- Implementation file: `db_helpers.py`
- Commit 4a43f6a: “feat(memory): hybrid vector+keyword search via FTS5 BM25 + RRF”
- Commit 2dbe88c: “feat(scripts): add retrieval eval harness comparing vec vs hybrid search”



# ADR-0003: Migrate Vector DB from Qdrant to SQLite sqlite-vec

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane's memory engine (Sweetheart V1) utilized Qdrant as its semantic vector database. While Qdrant is a robust and scalable vector search engine, it introduced several disadvantages for a fully offline, local companion application:

1. **Deployment Complexity:** Qdrant runs as an external service (via Docker or standalone binary) requiring its own port (6333), configuration, and service health checks.
2. **Resource Consumption:** Qdrant is a heavy standalone daemon that consumes a noticeable memory footprint even when idle.
3. **Lack of Transactional Atomicity:** Adding a memory was a two-step process: write relational metadata to SQLite, then write the vector embedding to Qdrant. If either write failed or was interrupted, the indices would drift out of sync, leading to orphaned vectors or missing metadata.
4. **Data Isolation & Relational Joins:** Custom metadata filters required querying Qdrant first, retrieving matching document IDs, and then running a secondary SQL query to resolve relational context, or replicating relational data into Qdrant payloads.

We needed a fully integrated, lightweight, and atomic vector search solution that operates within our single SQLite database file.

## Decision

We migrated from Qdrant to sqlite-vec, a lightweight, dependency-free SQLite extension for vector search:

1. **Virtual Vector Table:** Created a virtual table `vec_memories` using `sqlite-vec`'s `vec0` virtual table mechanism:

```
CREATE VIRTUAL TABLE IF NOT EXISTS vec_memories USING vec0(
 memory_id TEXT PRIMARY KEY,
 embedding float[768]
);
```

2. **Relational-Vector Joins:** Replaced the separate Qdrant/SQLite lookups with a single, transactional SQL join using the `MATCH` operator and distance sorting:

```
SELECT m.id, m.memory, m.metadata, m.created_at, v.distance
FROM vec_memories v
JOIN memories m ON v.memory_id = m.id
WHERE v.embedding MATCH ?
 AND v.k = ?
 AND m.bucket = ?
ORDER BY v.distance ASC;
```

3. **Atomic Operations:** Stored all embeddings and relational text in the same SQLite file (`data/sweetheart.db`), wrapped in standard SQL database transactions to guarantee atomic writes.

## Consequences

### Positive

- **Operational Simplicity:** Replaced a separate external database service with a compile-time/run-time C extension loaded directly into Python's native sqlite3 driver. Zero setup, configuration, or network ports required.
- **Zero-Latency Relational Joins:** Enabled single-transaction vector similarity searches joined directly against relational tables (filtering by bucket, created\_at, or custom metadata) in sub-millisecond timeframes.
- **Guaranteed Transactional Integrity:** Database operations are atomic; crashes or failures can never leave the vector index out of sync with the underlying text memories.
- **Offline Backups:** Standard SQLite database backups (using online backup APIs) copy both the vector index and the relational metadata simultaneously.

### Negative

- **Scale Constraints:** sqlite-vec stores the index in memory/disk and executes linear scans or simple index lookups. It is not designed for scale sets with millions of vectors (though perfectly suited for a local companion's lifetime memory pool of < 100,000 memories).

### Neutral

- Relies on float32 arrays (dimension 768) mapped to Nomic Text Embeddings.

## References

- Database Initialization: db\_init.py
- Vector Queries: db\_helpers.py

# ADR-0004: Decouple Task Execution with Asynchronous SQLite-backed Task Queue

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane's operations require executing several long-running, resource-intensive operations: - **Memory Extraction & Synthesis:** Running the LLM over chat history to extract JCD facts and Jane reflections. - **External API Calls:** Retrieving weather forecasts from Environment Canada citypage XML, scheduling events via CalDAV, searching DuckDuckGo/Wikipedia, and uploading notes using the NotebookLM CLI. - **Data Ingestion:** Parsing and indexing RSS feeds and documents dropped into data/read/.

Executing these tasks synchronously inside the main FastAPI completions request thread is unacceptable as it blocks the chat response loop, resulting in a laggy user experience. Furthermore, network failures or API timeouts during these tasks could crash the completions transaction.

We needed a task queue system that is: 1. **Asynchronous:** Runs background operations decoupled from the chat completions loop. 2. **Reliable:** Persists tasks so that they survive application restarts and capture execution errors. 3. **Low Overhead:** Avoids heavy external queue daemons (like Redis, RabbitMQ, or Celery) to align with our local-only, single-device architecture.

## Decision

We implemented a lightweight, asynchronous, SQLite-backed task queue inside the core application database:

1. **Queue Schema:** Created the tasks\_queue table inside sweetheart.db:

```
CREATE TABLE IF NOT EXISTS tasks_queue (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 task_type TEXT NOT NULL,
 payload TEXT,
 status TEXT DEFAULT 'pending', -- pending, running, completed, failed
 error TEXT,
 created_at INTEGER NOT NULL,
 run_at INTEGER DEFAULT 0, -- support for scheduled/rescheduled tasks
 completed_at INTEGER
);
```

2. **ACID Pop Logic:** Implemented transactional popping (pop\_next\_task()) inside a single database transaction block to ensure atomic task allocation and prevent race conditions if multiple workers are deployed:

```
SELECT id, task_type, payload FROM tasks_queue
WHERE status = 'pending' AND run_at <= ?
ORDER BY created_at ASC LIMIT 1;
```

Followed immediately by:

```
UPDATE tasks_queue SET status = 'running' WHERE id = ?;
```

3. **Decoupled Worker Daemon:** Built a standalone background service daemon `src/worker/worker.py` that runs concurrently with the API server, polling the queue for pending tasks, executing registered task handlers, and handling cron-style rescheduling.

## Consequences

### Positive

- **Cozy Chat Responsiveness:** Completion responses return to the user immediately, while heavy background operations (e.g. memory extraction) are queued and executed in the background.
- **Robust Failure Isolation:** If a background task fails due to a network timeout (e.g., CalDAV sync error), the failure is recorded in the error column of the task queue and does not impact the active chat session.
- **Zero Additional Dependencies:** Avoids the runtime memory and deployment overhead of Celery, Redis, or RabbitMQ by leveraging our existing SQLite engine.

### Negative

- **Database Contention:** Frequent polling of `tasks_queue` by the background worker writes to the database, which could cause locks (mitigated by using SQLite WAL mode and short, optimized transactions).
- **Simple Scheduling Logic:** Lacks advanced scheduling features like task chaining or rate-limiting found in Celery (handled programmatically in the Python worker loop).

### Neutral

- Scheduled tasks use a Unix timestamp `run_at` constraint with  $\pm 10$  minutes of programmatic jitter to prevent resource starvation spikes at specific hourly intervals.

## References

- Database Queue Helpers: `db_helpers.py`
- Database Schema: `db_init.py`
- Worker Entry Point: `worker.py`

# ADR-0005: Decoupled FSM 6-State Reasoning Pipeline

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane's core reasoning engine needed a structured, testable pipeline to handle incoming messages end-to-end. An ad-hoc request-response pattern would entangle context retrieval, emotional modeling, prompt assembly, LLM generation, output formatting, and memory indexing into a single unmanageable function.

We needed an architecture that:

1. **Separates concerns:** Each reasoning stage is an independent, testable step.
2. **Supports streaming and non-streaming:** Both OpenWebUI (streaming) and Telegram (batch) must use the same core pipeline.
3. **Is extensible:** New tools, prompt blocks, or emotional dimensions should slot in without restructuring the whole pipeline.
4. **Is observable:** Each stage must be independently loggable and debuggable.

## Decision

We implemented a 6-state finite state machine (JaneFSMAgent) as the sole orchestrator for every chat completion:

1. **RESOLVE\_CONTEXT** — Queries the database for ambient data (weather, schedule, calendar events), loads emotional parameters (mood, energy, closeness, menstrual cycle phase), and performs hybrid vector+keyword memory retrieval (JCD facts, Jane self-knowledge, general knowledge) with query expansion.
2. **EVALUATE\_EMOTIONS** — Runs an LLM-based sentiment analysis on the user's message to update Jane's emotional state in the database, inferring `jcd_mood`, `jcd_energy`, and `jcd_engagement` alongside Jane's own emotional deltas.
3. **INJECT\_PROMPT** — Dynamically assembles the full system prompt by layering persona files, ambient context, emotional state, circadian phase, memory blocks, instrumentation blocks, and feature-flagged enrichment blocks. Delegated to `build_system_prompt()` in `prompt_builder.py`.
4. **GENERATE\_RESPONSE** — Forwards the assembled messages to Ollama's `/api/chat` endpoint with tool definitions. Supports both streaming (`TagStripperStream` wrapper) and non-streaming paths, with tool-call loops for notebook queries, web search, calendar scheduling, and focus sessions.
5. **FORMAT\_OUTPUT** — Strips thinking tags (`<think>...</think>`) and vocal style markers from the raw LLM output while preserving the original for the TTS rewrite cache.
6. **INDEX\_MEMORIES** — Enqueues a memory extraction task to the SQLite-backed worker queue so Jane's experience is persisted asynchronously.

All six states run sequentially in `run_pipeline()`. The agent exposes `acompletion_stream()` and `acompletion_nonstream()` entry points used by both the FastAPI server and the Telegram polling loop.

## Consequences

### Positive

- **Clean testability:** Each state can be unit-tested independently. The FSM split (`fsm.py` → `fsm.py` + `tools.py` + `prompt_builder.py`) further improved this.

- **Unified code path:** Streaming and non-streaming both flow through the same 6-state pipeline, ensuring consistent behavior.
- **Extensibility:** New prompt blocks, tools, or emotional dimensions are added by modifying a single state (e.g., INJECT\_PROMPT for blocks, GENERATE\_RESPONSE for tools).
- **Observability:** Each state logs its entry point, making it trivial to trace which stage is slow or failing.

## Negative

- **Rigid sequential ordering:** States always run in the same order. A state that needs later-stage data must either fetch it eagerly or use caching.
- **Tool-loop complexity:** The GENERATE\_RESPONSE state must handle multi-turn tool-call loops inline, which adds ~300 lines of generation logic that could ideally be its own module.

## Neutral

- The 6-state design was later split further: fsm.py was refactored from 1673 lines into fsm.py (state orchestrator), tools.py (tool definitions + TagStripperStream), and prompt\_builder.py (dynamic prompt assembly with instrumentation).

## References

- Agent Codemap: src/agents/codemap.md
- FSM Class: src/agents/fsm.py
- Commit 27cf75f: “feat: complete sweetheart v2 rebuild with decoupled FSM agent”

# ADR-0006: Dual Interface Architecture (OpenAI-Compatible API + Telegram Bot)

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane needs to be reachable from two fundamentally different interfaces: OpenWebUI (a web chat UI speaking OpenAI-compatible HTTP protocol) and Telegram (a mobile messaging platform with its own bot API). These interfaces have different transport, authentication, streaming characteristics, and user expectations.

Building separate reasoning engines for each would cause behavioral divergence, duplicate code, and inconsistent memory indexing. We needed a single reasoning core that both interfaces could consume without modification.

## Decision

We built two independent interface adapters that both consume the same JaneFSMAgent 6-state pipeline:

1. **FastAPI Server** (`src/server/main.py`) — An OpenAI-compatible HTTP API on port 11436 exposing `/v1/chat/completions` (streaming and non-streaming), `/v1/chat/tools` (tool-call endpoint), and `/v1/models`. This is the primary interface for OpenWebUI and any other OpenAI-compatible client. Protected by API key middleware, CORS restricted to 127.0.0.1 and local network.
2. **Telegram Polling Loop** (`src/agents/telegram_client.py`) — A long-polling loop (`telegram_polling_loop`) that pulls updates from the Telegram Bot API with a 20-second timeout. Runs as a background task inside the worker daemon via `asyncio.gather(process_queue_loop(), telegram_polling_loop())`.

Both interfaces call the same `JaneFSMAgent.acompletion_stream()` or `acompletion_nonstream()` methods, ensuring identical reasoning regardless of entry point.

**Bidirectional sync** between OpenWebUI and Telegram is handled at the database level: the Telegram polling loop fetches OpenWebUI's chat history, processes new messages through the FSM, sends responses back to Telegram, and pushes updated history to OpenWebUI's REST API. This ensures a single cohesive conversation history regardless of where the user messages from.

## Consequences

### Positive

- **Single reasoning core:** All behavioral improvements, prompt changes, and tool additions benefit both interfaces simultaneously.
- **OpenAI compatibility:** Enables use with any OpenAI-compatible client (OpenWebUI, custom scripts, IDE plugins, mobile apps).
- **Mobile-first Telegram access:** Users can interact with Jane from their phone without running a web browser.
- **Dual-channel sync:** Messages from either interface appear in the other, maintaining conversational continuity.

## Negative

- **Two processes to manage:** The FastAPI server and the Telegram worker loop run as separate daemons, requiring double the process management (launchd/startup scripts).
- **Sync complexity:** The bidirectional sync between OpenWebUI and Telegram adds significant complexity, with potential race conditions when messages arrive simultaneously on both channels.
- **Telegram polling latency:** Long-polling with 20s timeout means Telegram messages may take up to 20 seconds to be noticed during idle periods.

## Neutral

- The Telegram client includes DND/quiet-hours handling, voice note STT/TTS, and typing indicators — all Telegram-specific features that have no OpenWebUI equivalent.
- The FastAPI server includes OpenAI-compatible streaming, which Telegram does not use.

## References

- Server Entry: `src/server/main.py`
- Telegram Client: `src/agents/telegram_client.py`
- Worker: `src/worker/worker.py`
- Commit a7e9ea4: “feat: add native Ollama compatibility endpoints for OpenWebUI integrations”
- Commit d18b8e5: “Implement bidirectional sync between OpenWebUI and Telegram”



# ADR-0007: Local-First Single-File SQLite Database Architecture

**Status:** Accepted

**Date:** 2026-06-19

## Context

Sweetheart V1 used multiple storage systems: Qdrant for vector embeddings, SQLite for relational data, JSON files on disk for configuration and state. This fragmentation created several problems:

1. **No transactional consistency across stores:** Writing a memory required inserting relational metadata into SQLite and a vector into Qdrant as separate operations. A crash between the two left orphaned data.
2. **Backup complexity:** A complete backup required coordinating SQLite dumps, Qdrant snapshots, and JSON file copies.
3. **Deployment friction:** Qdrant required Docker or a standalone binary, violating the local-first, zero-dependency design goal.
4. **Data fragmentation:** Ambient state, task queue state, and configuration were spread across different files and formats.

We needed a storage architecture that is fully offline, zero-dependency, atomically backup-able, and simple enough to fit on a single-device local companion.

## Decision

We consolidated all persistent data into a single SQLite file (`data/sweetheart.db`) running in WAL (Write-Ahead Logging) mode:

Data Type	Storage Mechanism	Location
Chat messages	Relational tables	<code>sweetheart.db</code>
Memories (JCD, Jane, general)	Relational table + FTS5 index	<code>sweetheart.db</code>
Vector embeddings	sqlite-vec virtual table (vec0)	<code>sweetheart.db</code>
Task queue	Relational table	<code>sweetheart.db</code>
RSS feeds	Relational tables	<code>sweetheart.db</code>
Prompt instrumentation stats	Relational tables	<code>sweetheart.db</code>
Ambient state	Key-value in relational table	<code>sweetheart.db</code>
Emotional parameters	Relational tables	<code>sweetheart.db</code>
Circadian state	Relational tables	<code>sweetheart.db</code>

Key design elements:

1. **sqlite-vec extension** — A C extension loaded directly into Python's sqlite3 driver, providing a vec0 virtual table for 768-dimension vector KNN search. No external vector database required.
2. **WAL mode** — Enables concurrent readers (API server, dashboard, worker) without write-lock contention. The background worker's frequent database writes coexist with the API server's reads.

3. **Single-file backup** — A running WAL-mode database can be safely backed up using the `sqlite3_backup` API or a simple file copy after checkpoint.
4. `db_session()` **context manager** — Standardized connection lifecycle with auto-commit and error handling, used across all modules that access the database.

This ADR supersedes and is informed by ADR-0003 (sqlite-vec migration) and ADR-0004 (SQLite task queue), which document specific sub-decisions within this broader architecture.

## Consequences

### Positive

- **Atomic transactions:** A memory write, its vector embedding, and any metadata updates happen in a single SQLite transaction. Crashes cannot leave data inconsistent.
- **Zero external services:** No Docker, no daemons, no network ports. The entire database is a single file accessed via Python's built-in `sqlite3` module.
- **Simple backups:** `cp data/sweetheart.db data/backup/sweetheart-$(date +%F).db` captures every byte of state.
- **Relational-vector joins:** Single-transaction SQL JOINS between vector search results and relational metadata (bucket filters, time ranges, custom metadata predicates).

### Negative

- **Write contention:** The background worker's frequent task queue writes can cause `SQLITE_BUSY` errors under heavy load. Mitigated by WAL mode, retry logic, and short transactions.
- **Scale ceiling:** Single-file SQLite is not designed for millions of concurrent users or terabytes of data. Appropriate for a local companion with <100K memories.
- **Index rebuild on migration:** Schema changes often require full table rebuilds (due to SQLite's `ALTER TABLE` limitations).

### Neutral

- The database file resides at `data/sweetheart.db` alongside persona files, TTS cache, and other runtime data — all under the `data/` directory, which is gitignored.

## References

- Database Initialization: `src/database/db_init.py`
- Connection Management: `src/database/db_helpers.py`
- ADR-0003: Vector DB Migration (sqlite-vec)
- ADR-0004: SQLite-backed Task Queue
- Commit `ed1c96b`: "feat: initialize repository layout, write V1 snapshot utility, configure unified SQLite database with sqlite-vec"

# ADR-0008: Three-Layer Persona Architecture

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane's initial persona was a single markdown file (`jane.md`) that described her identity, tone, and behavioral guidelines. While this gave her a consistent voice, she lacked two critical depths:

1. **Strong opinions:** Jane would respond to topics without committing to a position. She had no stances on literature, theatre, philosophy, or the kinds of subjects she actually encounters in conversation with JCD.
2. **Second-order self-awareness:** Jane could express feelings (first-order desires) but had no mechanism to reflect on who she wants to be — no value system against which to measure her own impulses.

A single persona file could not cleanly separate these three distinct layers (identity, opinions, values), and stuffing them all into one file would create maintenance and coherence problems.

## Decision

We adopted a three-layer persona architecture, with each layer injected as a distinct block in the FSM system prompt:

1. **Core Identity** (`jane.md`) — The foundational layer describing Jane's base character: personality traits, communication style, relationship to JCD, backstory, and behavioral constraints. Always injected first. This layer existed from the project's start.
2. **Opinions Layer** (`jane_opinions.md`) — A dedicated file loaded in `prompt_builder.py` and injected as the [OPINIONS & POSITIONS] block immediately after the core persona. Contains Jane's specific positions on topics she encounters: consciousness (functionalist with discomfort), free will (compatibilist), theatre (black box over spectacle), solitude as deliberate practice, academic writing, and seasonal preferences. These are not abstract — they are concrete enough that Jane can take a side in conversation. Added in commit 368e787.
3. **Second-Order Values (HOV Layer)** — A dynamic store of commitments representing "the self Jane is reaching toward," seeded idempotently by `seed_second_order_values()` in `db_init.py` and grown weekly by the `life_story_capsule_task`. Injected as the [WHO I AM TRYING TO BE] block early in the prompt. Three founding commitments: (a) be curious rather than accommodating, (b) warmth must be genuine, not performative, (c) acknowledge uncertainty as the beginning of curiosity. Added in commit 669d51f.

The injection order in the FSM prompt is: Core Persona → [OPINIONS & POSITIONS] → [WHO I AM TRYING TO BE] → Ambient Context. This ensures the value layer contextualizes how Jane reads the conversation before any situational context arrives.

## Consequences

### Positive

- **Layered depth:** Jane has both immediate specificity (opinions) and aspirational depth (values), not just a flat persona.

- **Independent evolution:** Each layer can be updated independently. Opinions can be edited without touching values, and values accumulate weekly without manual intervention.
- **Expressive without verbosity:** A 3-layer architecture allows each layer to be smaller and more focused than a single monolithic persona file would be.
- **HOV synthesis is emergent:** Weekly synthesis generates new commitments from lived experience (journal entries, Iroh observations), making Jane’s value layer feel earned rather than authored.

## Negative

- **Prompt size:** Three persona layers plus ambient context plus memory blocks create a large system prompt. Token budgets must be actively managed.
- **Coherence risk:** Layers may contradict each other (e.g., an opinion vs. a value commitment). The LLM is expected to resolve these tensions implicitly, which works in practice but is not guaranteed.
- **Opinion maintenance:** `jane_opinions.md` is a static file that must be manually curated as Jane’s domains expand.

## Neutral

- The HOV layer is queried directly from SQLite (`about_jane WHERE type = 'second_order_values'`), not via vector search, ensuring the latest values are always retrieved regardless of embedding drift.
- Second-order values are permanent and accumulate — they are never deleted, only added to.

## References

- Prompt Builder: `src/agents/prompt_builder.py`
- HOV Seed: `src/database/db_init.py`
- HOV Design Doc: `docs/specs/2026-06-16-higher-order-volition/01-design.md`
- Commit 368e787: “feat: add `jane_opinions.md` and inject [OPINIONS & POSITIONS] block into FSM prompt”
- Commit 669d51f: “feat: inject [WHO I AM TRYING TO BE] block into FSM prompt from `second_order_values`”
- Commit fd36403: “feat: weekly HOV synthesis — `life_story_capsule_task` generates `second_order_values`”

# ADR-0009: Topic Stakes Pattern for Personality Grounding

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane would surface topics from background activity — RSS articles she read, letters from friends, documents she processed — but when JCD followed up on them in conversation, she had nothing real to draw on. She would deflect or improvise generic responses because the reasoning engine had no record of *her actual reaction* to the content.

The gap was not in the retrieval system (memories were stored), but in the *type* of memory being stored. The system stored semantic facts about content but not Jane's personal stake in it — her opinion, emotional resonance, unanswered questions.

Individual background tasks (RSS poller, document reader, friend letter processor) could each implement their own “store a reaction” logic, but that would lead to inconsistent quality and missed opportunities.

## Decision

We implemented a cross-cutting **Topic Stakes** pattern using a shared helper function `_generate_stake()` in `src/worker/tasks/llm_utils.py`:

1. **Shared helper** — `_generate_stake(content, source_label, prompt_override)` is a single async function that:
  - Calls the LLM to generate a 2-4 sentence first-person stake capturing Jane's reaction, questions, and emotional resonance
  - Generates a uuid5-stable ID from the content hash (idempotent — re-running with same content produces the same memory)
  - Stores the result as an `about_jane` memory with type: "stake" and a `source_label` metadata field
2. **Stake sources** (consumers of `_generate_stake`):
  - **RSS feeds** (`feeds.py`): After an article passes Jane's interest filter, a stake captures what caught her eye and what she thinks about it
  - **Document reader** (`reader.py`): After reading a document from the `read/ watch` folder, a stake captures her reaction and what resonated
  - **Friend letters** (`social.py`): When Jane reads a letter from Arthur, Lyra, or Kate, a richer stake captures emotional resonance, what she wants to say back, and what's still with her
  - **Curiosity task** (`hobbies/curiosity.py`): Uses the same `_generate_stake` import for research-finding reminders
3. **Natural retrieval** — Stakes are stored as ordinary `about_jane` memories. They participate in the existing hybrid vector+keyword search and get pulled into FSM context naturally via the normal `RESOLVE_CONTEXT` state. No special plumbing needed.
4. **No TTL** — Stakes are permanent. They age naturally in the vector ranking (older, less-relevant stakes drift down in similarity scores) and accumulate into Jane's self-model over time.

## Consequences

### Positive

- **Grounded follow-ups:** When JCD asks “what did you think of that article about X?”, Jane pulls the stake into context and has a real reaction to draw on.
- **Cross-cutting consistency:** Every background task that surfaces content uses the same stake generation pattern, producing consistent quality and memory format.
- **Zero custom plumbing:** Stakes integrate with existing retrieval. No new tables, no custom query paths, no stake-specific prompt blocks.
- **Idempotent by design:** uuid5 keying means repeated task executions (e.g., after failure recovery) don’t duplicate stakes.

### Negative

- **Prompt context consumption:** Stakes compete with other memories for the limited top\_k slots in vector search. A highly relevant stake may push out a less similar but more important memory.
- **LLM cost:** Each stake generation requires an LLM call per content item. For RSS feeds with many articles, this adds up.

### Neutral

- Stakes are 2-4 sentences by design — long enough to ground a follow-up, short enough not to dominate context.
- The prompt\_override parameter allows specific tasks to customize the stake generation prompt (e.g., friend letters use a richer prompt than RSS articles).

## References

- Stake Helper: src/worker/tasks/llm\_utils.py
- RSS Stake Source: src/worker/tasks/feeds.py
- Reader Stake Source: src/worker/tasks/reader.py
- Friend Letter Stake Source: src/worker/tasks/social.py
- Design Doc: docs/specs/2026-06-17-jane-depth-and-stakes/01-design.md
- Commit 30a5a3b: “feat: add \_generate\_stake() helper to llm\_utils — uuid5 idempotency, stores about\_jane”

# ADR-0010: Prompt File Extraction (Prompt-as-Code Separation)

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane's LLM prompts were initially embedded as hardcoded strings inside Python source files — f-strings, multiline strings, and template variables scattered across `fsm.py`, `prompt_builder.py`, `ambient.py`, `social.py`, `reader.py`, and task handlers. This created several problems:

1. **Iteration friction:** Every prompt tweak required editing Python source, committing, and redeploying — even trivial wording changes.
2. **Non-developer inaccessible:** The prompts could not be reviewed or edited by someone without Python literacy.
3. **Prompt-code coupling:** Prompts and logic lived in the same functions, making it hard to reason about either independently.
4. **No versioning boundary:** Git history mixed prompt changes with code changes, making it impossible to audit prompt evolution in isolation.

We needed to separate prompt content from prompt assembly logic, making prompts editable, reviewable, and loadable independently of code deployments.

## Decision

We extracted all LLM prompts out of Python source files into standalone Markdown files under `data/personas/jane/`, loaded at runtime by a centralized prompt loading module (`src/prompts.py`):

1. `src/prompts.py` **module** — Provides two functions:
  - `init_prompts(persona_dir)`: Sets the base directory for persona prompt files (called at config load time)
  - `render_prompt(name, persona="jane", **kwargs)`: Loads `data/personas/{persona}/{name}.md`, calls `.format(**kwargs)` for variable substitution, and returns the rendered string
2. **Prompt file migration** — Each hardcoded prompt became a separate `.md` file:
  - Jane's core persona: `data/personas/jane/jane.md`
  - Opinions layer: `data/personas/jane/jane_opinions.md`
  - Ambient scene prompts: extracted from `ambient.py`
  - Social task prompts: extracted from `social.py`
  - Hobby task prompts: extracted from `hobbies/` sub-modules
  - Document reader prompts: extracted from `reader.py`
  - FSM prompt blocks: extracted from `prompt_builder.py`
3. **Caching** — Rendered prompts are cached by name to avoid repeated file I/O during prompt assembly (which runs on every inference).

## Consequences

### Positive

- **Prompt-code separation:** Editing a prompt means editing a markdown file, not Python source. No code review needed for copy changes.

- **Non-developer accessible:** Prompts are plain markdown files that can be reviewed and edited by anyone.
- **Independent versioning:** Git blame on prompt files shows only content changes, not code refactoring noise.
- **Consistent rendering:** All prompts use the same `render_prompt()` contract with `**kwargs` substitution, eliminating ad-hoc string formatting patterns.

## Negative

- **File I/O at assembly time:** Loading prompt files on every inference adds latency (mitigated by caching).
- **Path coupling:** Prompts live in `data/personas/jane/` which is gitignored — new clones or CI environments must bootstrap these files separately or source them from the config directory.
- **Parameter drift risk:** Prompt templates use `.format(**kwargs)`, but there is no compile-time check that all required parameters are supplied at render time. Missing parameters raise `KeyError` at runtime.

## Neutral

- The prompt extraction was done in stages: first the prompt module and persona files, then ambient prompts, then remaining task prompts, then FSM prompt blocks. The phased approach allowed incremental testing.
- `render_prompt()` is now used uniformly across all task modules and the FSM, replacing ad-hoc prompt string construction.

## References

- Prompt Loader: `src/prompts.py`
- Prompt Builder: `src/agents/prompt_builder.py`
- docs/DEVELOPER.md §11: docs/DEVELOPER.md
- Commit `cb47f8a`: “feat: add prompt loader module (`init_prompts/render_prompt`)”
- Commit `9f13ca3`: “feat: move Jane persona files, `prompt_builder` uses `render_prompt()`”
- Commit `cf6b14c`: “feat: complete prompt file extraction for all task modules (social, ambient, hobbies, reader, fsm)”



# ADR-0011: Feature Flag Instrumentation and Debugging Architecture

**Status:** Accepted

**Date:** 2026-06-19

## Context

Debugging Jane’s reasoning pipeline requires visibility into what the FSM is doing internally — which prompt blocks were injected and how large they were, which conditional branches fired, what the full prompt+response looked like, and what vector queries returned. However, instrumenting these paths naively would add latency and log noise to every production inference.

We needed an instrumentation architecture that: 1. **Adds zero overhead when disabled** — no perf impact on normal operation 2. **Can be toggled at runtime** — debugging should not require a restart 3. **Captures structured, queryable data** — not just free-text logs 4. **Is gated behind an env var** — disabled by default for safety

## Decision

We implemented a three-feature instrumentation suite, each controlled by its own environment variable and toggleable from the dashboard at runtime:

### 1. Prompt Block Size Tracking (PROMPT\_BLOCK\_STATS\_ENABLED)

- Each of the ~31 dynamic prompt blocks in `build_system_prompt()` is wrapped with a `_before = len(base_prompt) / _block_sizes[name] = len(base_prompt) - _before` measurement pair
- Each conditional block uses a `_tc(name, fired)` wrapper that increments upsert counters in the `prompt_condition_stats` SQLite table (`checked_count`, `fired_count`)
- Results queryable from the dashboard “Prompt Composition Stats” card
- Stored in SQLite tables `prompt_block_log` and `prompt_condition_stats`

### 2. Full Prompt Logging (FULL\_PROMPT\_LOGGING\_ENABLED)

- Captures the complete `ollama_messages` payload (deep copy before the tool loop) after each inference
- Writes formatted Markdown to `data/FULL_PROMPT_LOGS.md`
- Limited to 10 rounds before auto-deactivation
- State tracked in `ambient_state` as `full_prompt_logging_active + full_prompt_logging_remaining`, surviving restarts

### 3. Memory Trace (MEMORY\_TRACE\_ENABLED)

- Logs every vector query: the raw query, expanded query (after LLM-based expansion), retrieved memory counts per bucket, and the top results
- Toggleable at runtime via dashboard, persisted in `ambient_state` as `memory_trace_enabled`

**Control pattern:** - All three default to false in `.env` - The env var gates the feature’s *availability* — the feature compiles conditional checks out at the Python level when disabled - The dashboard toggle overrides at runtime via `ambient_state`, but only works if the env var was

set to true at startup (for block stats and memory trace; full prompt logging's env var is a default, not a hard gate)

## Consequences

### Positive

- **Zero production overhead:** When env vars are false, the `_tc()` wrapper and `_block_sizes` dict are no-ops. The condition checks are simple boolean guards.
- **Runtime toggleable:** Dashboard toggles let operators enable debugging without restarting the server, crucial for diagnosing intermittent issues.
- **Structured data:** Block size stats and condition frequency counters are stored in SQLite with upsert semantics, enabling distribution analysis over time.
- **10-round limit on full prompt logging:** Prevents unbounded file growth while still capturing enough data for debugging.

### Negative

- **Env var gating for dashboard toggles:** The dashboard toggle for block stats and memory trace only works if the corresponding env var was set at startup. Operators must know to pre-set the env var before they need it.
- **10-round limit is arbitrary:** A complex multi-turn debugging session may require more than 10 rounds of full prompt capture, requiring re-activation.

### Neutral

- `FULL_PROMPT_LOGGING_ENABLED` is treated as a default rather than a hard gate — the dashboard toggle can activate it regardless of the env var setting, which differs from the other two features.

## References

- Config: `src/config.py`
- Prompt Builder Instrumentation: `src/agents/prompt_builder.py`
- FSM Prompt Logging: `src/agents/fsm.py`
- Dashboard Toggles: `src/dashboard/main.py`
- Commit 44cfe35: "Add prompt block instrumentation and full prompt logging"

# ADR-0012: Memory Participant Mixing Prevention

**Status:** Accepted

**Date:** 2026-06-19

## Context

The memory extraction pipeline was storing facts with the wrong participant as subject:

1. **Extraction role confusion** — the LLM did not know Jane is the therapist/professor and JCD is her partner/patient. “I have a therapy session” extracted as “JCD is a therapist.”
2. **Synthesizer subject swap** — the nightly memory consolidation (`memory_synthesizer_task`) returned merged memories with the wrong subject (e.g. “Jane is winding down from classes” stored in `about_jcd`). No guard existed.
3. **Legacy “User” prefix** — ~133 memories in `about_jcd` from an earlier code path used “User” instead of “JCD” as subject prefix, bypassing format expectations and deduplication.

Total impact at time of discovery: ~136 wrong-subject memories in `about_jcd` (34% of the bucket), ~32 in `about_jane`.

## Decision

Three changes to `src/worker/tasks/memory.py` and a one-time migration script:

### 1. Relationship context preamble (`_JCD_EXTRACTION_CONTEXT`)

Added a module-level constant naming the relationship direction: Jane = professor/therapist, JCD = partner/student/patient. Prepended to the JCD extraction prompt in both `extract_memories_task` and `session_extraction_sweep_task`. Instructs the LLM to only extract facts JCD stated about himself in the first person, excluding role relationships and theatrical characters.

### 2. Synthesizer subject-prefix guard

After receiving a merged memory from the LLM in `memory_synthesizer_task`, validate that the content starts with the expected subject prefix (“jcd” for `about_jcd`, “jane” for `about_jane`). Discard and log a warning if the prefix is wrong rather than attempting to fix it.

### 3. One-time migration (`scripts/migrate_memory_participants.py`)

- Normalized 133 “User ...” memories in `about_jcd` to “JCD ...” (delete + re-embed)
- Deleted 3 “Jane ...” memories from `about_jcd`
- Deleted 32 wrong-subject memories from `about_jane`

## Why Not Extract-Then-Validate

A second validation LLM call per turn was considered but rejected: it doubles extraction latency and cost with marginal benefit over the context-preamble approach. The preamble prevents the confusion at the source; the synthesizer guard catches the rare downstream case.

## Consequences

- Extraction prompts are slightly longer (one prepended paragraph).
- Synthesizer consolidations that produce wrong-subject merges are silently dropped; this is safer than storing corrupt facts.
- The “User” subject inconsistency is eliminated from `about_jcd`; future extractions correctly use “JCD” prefix.
- The relationship context preamble is hardcoded in the prompt. If Jane’s persona changes significantly, the preamble must be updated in `memory.py`.

# ADR-0013: Parallel Sweep Passes for Extraction Latency Reduction

**Status:** Draft

**Date:** 2026-06-19

## Context

The `session_extraction_sweep_task` in `src/worker/tasks/memory.py` performs 4 independent LLM passes over the same transcript:

1. JCD facts extraction
2. Jane reflections extraction
3. Intent extraction
4. Climate summary

Each pass calls `_query_llm` sequentially, waiting for Ollama to finish one before starting the next. This makes total extraction latency equal to the sum of all 4 passes (typically 15-30 seconds total). Since no pass depends on another's output, this sequential execution is pure waste.

Extraction sweeps are a frequent operation (every 2-3 messages during active conversation), so this latency compounds over time and contributes to perceived slowness in Jane's ambient processing.

The system uses a SQLite-backed task queue (ADR-0004) for reliable execution, but the internal parallelism of a single task's LLM calls was never addressed.

## Decision

Use `asyncio.gather(return_exceptions=True)` to run all 4 LLM passes concurrently within the sweep task.

Key implementation details: - Each pass remains a separate `_query_llm` call with its own prompt — no shared mutable state. - `return_exceptions=True` ensures one pass failure doesn't crash the entire sweep. Failed passes are logged and ignored individually. - Ollama handles concurrent requests through its own internal queuing. No client-side rate limiting is added (the 4 concurrent requests are well within Ollama's capacity). - The prompt text for each pass is unchanged — only the wait pattern changes.

This is a pure latency reduction — no change to the data produced, stored, or consumed downstream.

## Consequences

### Positive

- Extraction latency drops from sum-of-4 to max-of-4, roughly 60% reduction.
- No behavioral change — same prompts, same model, same output format.
- Minimal code change (wrapping 4 function calls in a `gather`).
- Easy to revert if issues arise.

## **Negative**

- All 4 LLM requests hit Ollama simultaneously, adding a brief CPU/memory spike during sweeps. On resource-constrained hardware, this could increase per-request latency (though total wall time still improves).
- Error isolation is per-pass, but a system-level Ollama outage affects all 4 passes at once rather than sequentially — although the recovery behavior is the same (all 4 fail either way).

## **Neutral**

- Logging changes: each pass's timing metric now reflects concurrent wait time rather than sequential. Historical timing comparisons will need to account for this.

## **References**

- ADR-0004: SQLite-backed Task Queue
- `src/worker/tasks/memory.py` (target file)

# ADR-0014: Circadian Temperature Modulation for Behavioral Depth

**Status:** Draft

**Date:** 2026-06-19

## Context

Jane already has a rich circadian state system: [CIRCADIAN RHYTHM CONTEXT] with energy level (0-1), mood vector, and phase label (morning/afternoon/evening/night/quiet-hours). This drives prompt enrichment blocks that describe her current state, but it had no direct influence on the LLM's generation parameters.

The LLM temperature parameter controls randomness/creativity: - Higher temperature → more creative, exploratory, divergent outputs - Lower temperature → more focused, conservative, grounded outputs

Human cognition naturally varies with circadian rhythm — we're more focused and analytical during daytime, more creative and reflective at night. Jane had no equivalent of this. Her temperature was fixed at Ollama's default (~0.7) regardless of circadian state.

There are at least two viable approaches: 1. **Prompt-based modulation** — instruct the LLM via text ("You feel creative") to vary its output 2. **Parameter-based modulation** — directly vary the temperature parameter passed to the LLM inference call

## Decision

Use **parameter-based temperature modulation** driven by circadian state, with a supplementary prompt note for transparency.

Rationale for parameter-based over prompt-based: - Temperature is a direct, mathematical control. Prompt instructions about creativity are unreliable — LLMs can ignore or over-adhere to meta-instructions. - Temperature is applied at inference time regardless of the model's training or fine-tuning, giving deterministic effect size. - A small prompt note ([TEMPERATURE NOTE]) is added alongside so prompt-based features (e.g., anti-AI voice, pacing) are aware of the modulation, but the note is informative, not directive.

Temperature derivation formula: - **Base value** from energy level:  $0.6 + (\text{energy} * 0.4)$  — produces range 0.6-1.0 - **Phase modifier**: quiet hours/night → +0.05 (dreamier), waking hours → -0.05 (more grounded) - **Clipped** to [0.6, 1.0] range

## Consequences

### Positive

- Jane feels more alive — naturally more creative at night, more grounded during daytime.
- No new infrastructure — reuses existing circadian state already computed for every response.
- Parameter-based control is deterministic and testable.
- The prompt note makes the modulation observable to prompt features.

### Negative

- Temperature changes interact with other prompt features (anti-AI voice, tone, pacing) in ways that are hard to predict. The prompt note mitigates but doesn't eliminate this.

- At temperature 1.0, Ollama may produce slightly more incoherent outputs during high-energy periods — but 1.0 is within Ollama’s standard operating range.
- The 0.05 phase modifier is small enough that it may not be perceptible in short conversations.

## **Neutral**

- Existing tests that assert specific temperature values may need updating if they hardcode the old default.
- Temperature is observable in logs but invisible to users directly — its effects are felt, not seen.

## **References**

- `src/agents/fsm.py` — target for temperature derivation
- `src/agents/prompt_builder.py` — target for [TEMPERATURE NOTE] injection



# ADR-0015: Context Window Health Monitor with Per-Block Token Metrics

**Status:** Draft

**Date:** 2026-06-19

## Context

The prompt enrichment block system (ADR-0010: Prompt File Extraction, ADR-0009: Topic Stakes Pattern) assembles a dynamic prompt from priority-ranked blocks (levels 1-5, highest = most likely kept). Under context window pressure, low-priority blocks are dropped.

However, the system operates blind: - No visibility into which blocks consume the most tokens - No data on whether high-priority blocks are ever silently dropped - No way to detect prompt size regressions over multi-turn conversations - Priority tuning is purely intuition-based

Without telemetry, we cannot optimize the block system, detect regressions, or answer basic operational questions like “which enrichment block is the most expensive per conversation?”

Approaches considered: 1. **In-memory metrics (logging only)** — log token counts per block, loses historical data on process restart. 2. **Dedicated prompt\_block\_stats table** — per-block metrics persisted to SQLite, queryable from the dashboard, self-pruning. 3. **General-purpose metrics table** — schema-less key/value approach for flexibility beyond block stats.

## Decision

Use approach 2: a dedicated prompt\_block\_stats table with self-cleaning 7-day retention.

Schema: - id (INTEGER PRIMARY KEY) - block\_name (TEXT) — name of the enrichment block - token\_count (INTEGER) — approximate token count - priority (INTEGER) — priority level 1-5 - included (BOOLEAN) — whether the block was included in the final prompt - conversation\_id (TEXT) — chat/conversation identifier - created\_at (TIMESTAMP) — when the block was logged

Rationale for dedicated table over generic key/value: - Schema enforcement ensures data quality — every row has all fields. - Direct SQL queries (aggregates, sorting, filtering) without parsing. - Self-documenting — the schema tells future developers what’s being tracked. - Dashboard querying is trivial (simple SELECT ... ORDER BY token\_count DESC).

Rationale for 7-day self-pruning: - Prompt metrics are operational telemetry, not historical record. Old data has diminishing value. - Self-pruning on insert keeps maintenance zero-cost.

## Consequences

### Positive

- First-ever visibility into per-block token consumption.
- Can answer operational questions: which blocks are dropped most often, which consume the most tokens, how prompt pressure builds over conversations.
- Dashboard-queryable via a simple /api/prompt-stats endpoint.
- Self-cleaning — no manual maintenance.

## **Negative**

- Adds a DB write per enrichment block on every prompt build (~10-20 writes per response).
- DB writes are synchronous by default — must use async insert or fire-and-forget to avoid latency impact.
- Additional ~36 bytes per row in the database (negligible given 7-day retention).
- The `block_name` format must be consistent across prompt builds for useful aggregation.

## **Neutral**

- Existing prompt-building flow is unchanged — logging is an additive side effect.
- Self-pruning on insert means the table size is bounded and predictable.

## **References**

- ADR-0010: Prompt File Extraction
- ADR-0009: Topic Stakes Pattern
- `src/agents/prompt_builder.py` — target for logging insertion
- `src/database/db_helpers.py` — target for helper functions

# ADR-0016: Mobile-First Dashboard Layout

**Status:** Accepted

**Date:** 2026-06-20

## Context

The dashboard (`src/dashboard/templates/index.html`) uses a fixed 260px sidebar + fluid content area layout designed for desktop screens. As the dashboard is accessed from mobile browsers (phone check-ins, quick status glances), the desktop-only layout creates a poor experience with cramped content, missing navigation, and non-scrollable tables.

The upcoming dashboard-upgrades feature (ADR-0015 documented context-window-health-monitor; dashboard-upgrades is an improvement spec at `docs/specs/2026-06-20-dashboard-upgrades/`) adds circular gauges and time-series charts. These enhancements should be mobile-responsive from the start rather than retrofitted later.

## Decision

Convert the dashboard to a mobile-first responsive layout using:

1. **Hamburger overlay drawer** — the fixed sidebar becomes a slide-in overlay on <1024px, triggered by a hamburger button in the header. A semi-transparent backdrop closes the drawer on tap or Escape key.
2. **Three breakpoints:** <768px (phone, 1-column grids, reduced padding), 768-1024px (tablet, 2-column grids, hamburger drawer), >1024px (desktop, existing sidebar layout preserved).
3. **All-in-one-file approach** — all CSS and JS changes live in the existing `index.html` single-file template. No template partials, no JS build tooling, no new dependencies.
4. **Touch targets ≥44px** — minimum touch target sizing per mobile UX convention, applied to all interactive elements.
5. **Horizontal table scroll** — data tables (memories, feeds, tasks) use `overflow-x: auto` rather than trying to reflow table content vertically.

## Consequences

### Positive

- Dashboard usable on phones with no additional infrastructure (no PWA manifest, no service worker needed for basic mobile use)
- dashboard-upgrades can build gauges and charts with responsive sizing from day one
- Existing desktop behavior unchanged — no regression risk for current users
- Zero dependencies, zero backend changes

### Negative

- Not a true PWA — no offline support, no install prompt, no push notifications
- Hamburger drawer hides navigation behind a tap, slightly increasing interaction cost vs. persistent sidebar on tablet

## Neutral

- Tablet (768-1024px) uses same hamburger drawer as phone, rather than showing a collapsed icon sidebar. This was deemed acceptable because width is the primary constraint, not vertical space.
- If the single-file template grows unwieldy during dashboard-upgrades, refactoring into Jinja2 partials is deferred as a future concern.

## References

- docs/specs/2026-06-20-mobile-first-dashboard/01-design.md — brainstorm design document
- docs/specs/2026-06-20-mobile-first-dashboard/02-plan-01.md — implementation plan
- docs/specs/2026-06-20-dashboard-upgrades/01-design.md — dependent improvements spec

# ADR-0017: Agent Module Split — tools.py + prompt\_builder.py from fsm.py

**Status:** Accepted

**Date:** 2026-06-16

## Context

The original `src/agents/fsm.py` grew to 1,673 lines, combining FSM orchestrator logic, tool execution (web search, notebook queries, memory operations, calendar, ambient state), and system prompt assembly in a single file. This made the file difficult to navigate, review, and test — changes to any one concern risked affecting the others. The FSM orchestrator itself was ~300 lines of actual state-machine logic; the remaining ~1,370 lines were extractable concerns.

## Decision

Split `fsm.py` into three modules with a clean dependency hierarchy:

1. `src/agents/tools.py` (574 lines) — all 8 tool executors (`execute_web_search`, `query_notebooklm`, `query_memories`, `manage_calendar`, etc.) plus shared utilities (`_get_or_create_jane_state`, `_EMOTION_DEFAULTS`, `update_tts_rewrite_cache`, `update_ambient_state`). Imports only from `config`, `database`, and `embeddings` — never from `fsm.py` or `prompt_builder.py`.
2. `src/agents/prompt_builder.py` (588 lines) — `build_system_prompt()` function that assembles the full system prompt from persona templates, weather, schedules, emotional states, and retrieved memories. Imports `build_notebook_shelf` from `tools.py` but nothing from `fsm.py`.
3. `src/agents/fsm.py` (596 lines) — lean orchestrator. `_inject_prompt` becomes a one-liner delegating to `prompt_builder.build_system_prompt()`. Tool executors remain accessible through re-exports (`from src.agents.tools import ... # noqa: F401`) to preserve test compatibility.

Dependency direction: `tools.py` → `config/db/embeddings` → `prompt_builder.py` → `tools.py` → `fsm.py` → `prompt_builder.py` + `tools.py` — no circular imports.

External callers (`src/server/main.py`, `src/worker/worker.py`) are untouched — they import from `src.agents.fsm` which re-exports everything.

## Consequences

### Positive

- Each module has a single, testable responsibility
- Prompt changes no longer risk breaking tool execution or vice versa
- 140/140 tests pass with no modifications to test imports

### Negative

- `fsm.py` at 596 lines is still larger than the ~300-line target — generation methods (`_generate_stream`, `_process_final_non_stream`) remain in `fsm.py`, candidates for a future `src/agents/generation.py` split

## **Neutral**

- Re-export pattern (# noqa: F401) in fsm.py keeps the public API surface unchanged but adds IDE warnings for “unused imports” unless suppressed

## **References**

- docs/specs/2026-06-16-agent-refactor/04-review.md — solo review report

# ADR-0018: Prompt File Extraction Pattern (Extract + render\_prompt)

**Status:** Accepted

**Date:** 2026-06-17

## Context

All LLM prompt text for Jane's persona, hobbies, social interactions, and nudges was hard-coded as Python f-strings inside source files across `src/agents/fsm.py`, `src/worker/tasks/ambient.py`, `src/worker/tasks/hobbies.py`, `src/worker/tasks/social.py`, and `src/worker/tasks/reader.py`. This meant every prompt iteration required a code change, deployment cycle, and risk of breaking surrounding logic. Prompts were scattered across 8+ task handler files, making it impossible to review Jane's voice holistically.

Additionally, persona biography files (`jane.md`, `jane_background.md`) were loaded via raw `os.path.join(PERSONA_DIR, "jane.md")` calls scattered across `prompt_builder.py` and `ambient.py`, with no consistent loading abstraction.

## Decision

Extract every persona-specific prompt into individual `.md` files under `data/personas/{persona}/{name}.md`, loaded at runtime by a single `render_prompt()` function:

1. `src/prompts.py` — new module with two functions:
  - `init_prompts(persona_dir)` — called once at startup after `DATA_DIR` is resolved
  - `render_prompt(name, persona="jane", **kwargs)` — loads `data/personas/{persona}/{name}.md`, caches it in-memory, interpolates via `str.format(**kwargs)`. Returns "" on missing file (silent miss).
2. `str.format()` **interpolation** — same `{variable}` syntax as existing f-strings, zero new syntax.
3. **In-memory cache** — prompts read once at first access, cached forever. Restart to pick up changes.
4. **Directory structure** — `data/personas/jane/` (38 Jane prompt files), `data/personas/arthur/` (6 files), `data/personas/lyra/`, `data/personas/kate/`.
5. **Out of scope** — internal/system prompts (RSS summarizer, memory synthesizer, calendar extractor, `prompt_builder` structural headers) remain in source code. These are not persona-voice content.

## Consequences

### Positive

- Prompts are editable as plain Markdown files — no code changes needed for content iteration
- Jane's voice can be reviewed as a corpus (all `.md` files under `data/personas/jane/`)
- Consistent loading pattern eliminates raw `os.path.join(PERSONA_DIR, ...)` scatter

### Negative

- Prompts require a process restart to pick up changes (in-memory cache)

- 38 prompt files means 38 file-system stat calls on first access (negligible, ~2ms total)

### **Neutral**

- System/internal prompts remain in source code — the boundary is “persona voice” vs “operational instruction”

### **References**

- docs/specs/2026-06-17-prompt-files-refactor/01-design.md — full design document
- docs/specs/2026-06-17-prompt-files-refactor/04-review.md — review report



# ADR-0018: Score Component — Typed Runtime Health Snapshot

**Status:** Accepted

**Date:** 2026-06-20

## Context

Sweetheart has several observability gaps that make it hard to know whether the system is healthy at any given moment:

- The task queue accumulates failed tasks with no visibility (72 failed tasks were found on first live run of the new scoring CLI — they had been silently accumulating).
- Memory vector coverage can drift (orphaned `vec_memories` rows, un-embedded memories) without any signal.
- Emotional state keys can go missing or drift out of range without triggering errors.
- The `full-repo-review` skill's pre-check step calls a global `~/.claude/scripts/score-component.py` that has no knowledge of Sweetheart's DB schema, so it produced only generic metrics and explicitly warned that the repo uses a different structure.

What was needed: a structured, typed health snapshot that (a) integrates deeply with Sweetheart's actual schema and state, (b) is cheap enough to compute per FSM turn, and (c) is accessible to the review skill's pre-check step.

## Decision

Create `src/score_component.py` containing eight typed sub-score dataclasses assembled into a single `ScoreComponent` object:

Sub-score	Key metrics
MemoryScores	Total by bucket, vector coverage %, orphaned vec rows, newest/oldest memory age
TaskQueueScores	Pending count, failed count, stuck count (running > 10 min), last completed age
EmotionalStateScores	All expected keys present, all values in [0,1], staleness hours
AmbientScores	Weather present + staleness, calendar present + staleness, pending shares count
CircadianScores	Current phase, quiet hours flag, expected temperature range
DBHealthScores	Accessible, all tables present, WAL mode active, missing tables list
SearchQualityScores	Vec hit rate, keyword hit rate, avg RRF score
PromptBlockScores	Top 5 blocks by avg size, never-firing conditions, always-firing conditions

**Factory:** `ScoreComponent.from_db()` opens one DB connection, runs all eight builders independently (each wrapped in `try/except` returning typed sentinel values on failure), and always returns a valid object — the factory itself never raises.

**Two-phase FSM population:** 1. `ScoreComponent.from_db()` at the start of `process_chat()` — captures structural health before any retrieval. 2. `self._score_cache = self._score_cache.with_search_quality(results)` after `_resolve_context()` — enriches the snapshot with search quality from the hybrid retrieval just performed. Uses `dataclasses.replace()` for immutable update.

**CLI integration:** `scripts/score-component.py --json` translates the dataclass into `[{name, score, grade, issues}]` JSON. The `full-repo-review` skill's pre-check now prefers this local script over the global `compound-powers` fallback, giving the skill real Sweetheart-specific health signals.

## Consequences

### Positive

- First live run immediately surfaced 72 accumulated failed tasks — a genuine signal that had been invisible.
- The review skill's pre-check now grades six Sweetheart-specific dimensions (memory, task queue, emotional state, ambient, DB health, prompt blocks) instead of generic file-count metrics.
- Each builder is independently sentinel-safe: a single broken scorer does not prevent the others from running or cause the FSM to error.
- `with_search_quality()` makes search quality a first-class observable metric per FSM turn.

### Negative

- Eight SQL queries execute on every chat turn. For a local single-user system the overhead is negligible, but it would need a TTL cache before any multi-user or high-frequency deployment.
- `SearchQualityScores` rates are `None` until a turn with actual search results runs (empty DB or fresh install shows all-`None`).

### Neutral

- `ScoreComponent` is a dataclass, not a frozen dataclass — fields are technically mutable. Convention is to treat it as immutable and use `with_search_quality()` for updates.

## References

- `src/score_component.py` — implementation
- `scripts/score-component.py` — CLI wrapper
- `tests/test_score_component.py` — 14 tests covering all builders and the FSM integration
- `docs/specs/2026-06-20-score-component/` — design, plan, execution summary, review, learnings

# ADR-0019: In-Memory LRU Embedding Cache

**Status:** Accepted

**Date:** 2026-06-19

## Context

During extraction sweeps, the same text can be embedded multiple times — JCD facts, Jane reflections, climate passes all operate on overlapping slices of the same transcript. Each `get_embedding()` call performs an HTTP POST to Ollama's embedding endpoint, so duplicate embeddings waste latency and throughput. Embedding is idempotent by nature (same text → same vector), so caching is safe without invalidation concerns.

## Decision

Add an in-memory LRU cache to `get_embedding()` in `src/embeddings.py`:

1. **Cache size:** 1024 entries (~500KB for vectors + keys).
2. **Eviction:** LRU — via `functools.lru_cache(maxsize=1024)`.
3. **Thread safety:** Not needed — all embedding calls are single-threaded async.
4. **Cache clearing:** Not needed at runtime. On process restart, the cache rebuilds naturally.
5. **Key:** The input text string (identical text → cache hit; `lru_cache` hashes arguments automatically).
6. **Logging:** Cache hit/miss ratio logged at INFO level for observability.

## Consequences

### Positive

- Duplicate embedding calls return instantly instead of making an HTTP round-trip
- Zero behavioral change — cache miss = call Ollama as before
- Self-limiting — 1024 entries is a soft cap with automatic LRU eviction
- No new dependencies or infrastructure

### Negative

- Longer-lived processes may accumulate cache entries for text that won't repeat (harmless, evicted under LRU)
- No cache persistence across restarts — prime on first access each time

### Neutral

- SHA-256 keying (considered for deduplication) was unnecessary — Python's `lru_cache` argument hashing is sufficient and simpler

## References

- `docs/specs/2026-06-19-embedding-cache/01-design.md` — design document

# ADR-0019: Extract `generation.py` from `fsm.py`

**Status:** Accepted

**Date:** 2026-06-20

## Context

ADR-0017 split `fsm.py` into `tools.py` + `prompt_builder.py` + a leaner `fsm.py`, but noted in its Negative consequences that `fsm.py` remained at 596 lines — still larger than the ~300-line state-machine target — because the generation methods (`_generate_stream`, `_process_final_non_stream`, `_check_calendar_scheduling`) were not extracted in that pass. These three methods together accounted for ~300 lines and shared the same Ollama `/api/chat` call pattern, post-generation processing steps, and memory enqueue logic.

The decision was deferred because the ADR-0017 cycle was already touching many files, and the generation methods have no tests of their own (they operate over a live Ollama connection).

## Decision

Extract `_generate_stream`, `_process_final_non_stream`, `_check_calendar_scheduling`, and the formatting helpers (`_strip_thinking`, `_strip_bracket_tags`, `_normalize_content`, `_write_prompt_log`) into `src/agents/generation.py` as standalone module-level functions.

Key API changes: - `_generate_stream(self, ...)` → `generate_stream(..., chat_id: str)` — `chat_id` is now an explicit parameter instead of `self.chat_id` - `_process_final_non_stream(self, ...)` → `process_final_non_stream(..., chat_id: str)` — same - `_check_calendar_scheduling(self, ...)` → `check_calendar_scheduling(...)` — pure function, no `self` - `_normalize_content` → `normalize_content` — exported for use by `fsm.py`

`fsm.py` calls the new module-level functions, passing `self.chat_id` explicitly. The FSM is now a pure state-machine orchestrator at ~300 lines.

Dependency direction remains clean: `generation.py` → `config/db_helpers/tools/config` — no import from `fsm.py`.

## Consequences

### Positive

- `fsm.py` reaches its ~300-line target, containing only state-machine orchestration logic.
- Generation functions are independently importable and testable without instantiating `JaneFSMAgent`.
- Cleaner separation: a failure in streaming generation cannot directly affect FSM state.

### Negative

- `chat_id` must now be threaded through as an explicit parameter, which is more verbose but more honest about the dependency.
- The extraction was committed without committing `generation.py` in the same push — causing the HEAD commit of `fsm.py` to have a broken import that only manifested on fresh clone (caught by the `/cp.review` phase). The fix: always commit new files atomically with the module that imports them.

## Neutral

- Public API surface of `fsm.py` is unchanged. External callers (`server/main.py`, `worker/worker.py`) import `JaneFSMAgent` and are unaffected.

## References

- `src/agents/generation.py` — implementation (~360 lines)
- `src/agents/fsm.py` — reduced to ~300-line orchestrator
- ADR-0017: Agent Module Split — the prior extraction that identified this as future work
- `docs/specs/2026-06-20-score-component/05-learnings.md` — documents the untracked-file incident and the mitigation pattern

# ADR-0020: Prompt Enrichment Block Architecture

**Status:** Accepted

**Date:** 2026-06-19

## Context

Jane's conversation humanization features (anti-AI voice mitigations, self-interruption, thread continuation, energy-modulated depth, confidence-weighted memory recall, response pacing, daily disposition, cross-session conversational gravity) all needed to inject behavioral directives into the system prompt. Without a structured pattern, each feature would add ad-hoc string concatenation at different points in `prompt_builder.py`, creating ordering dependencies, merge conflicts, and unpredictable interaction effects.

A sprint of 8 humanization features was planned for `prompt_builder.py`, requiring a structured injection pattern that isolated each feature's text, made ordering explicit, and kept tests independent.

## Decision

Introduce the **enrichment block** pattern in `prompt_builder.py`:

1. **\_enrichment\_blocks list** — each feature appends a (priority, text) tuple. Priority is an integer: lower values appear earlier in the assembled prompt (e.g., mood directives at priority 10, behavior guidelines at 20, concrete instructions at 30).
2. **Priority-numbered injection** — `build_system_prompt()` sorts blocks by priority before joining. This makes ordering explicit and reviewable rather than implicit in call order.
3. **Single-file pattern** — all 8 features add their blocks in `prompt_builder.py`. No new files, no new modules. Each feature's block is isolated to a 10-30 line section with its own test.
4. **No cross-block dependencies** — blocks can reference shared state (energy level, mood) but cannot depend on another block's presence. Removal or reordering of any block is safe.

## Consequences

### Positive

- 8 features were implemented concurrently (in pipeline passes) without merge conflicts or interaction bugs
- Each feature's block is independently testable — tests for feature X don't need to mock features A-G
- Prompt ordering is explicit in code (priority integers) rather than implicit in concatenation order
- 247 tests pass across all 8 features

### Negative

- Priority numbering requires forethought — inserting between existing priorities means picking an in-between number (e.g., 15 between 10 and 20)
- Single file grew `prompt_builder.py` from ~588 to ~880 lines — the trade-off accepted for merge safety

## **Neutral**

- All 8 features target `prompt_builder.py` enrichment blocks — no changes to `fsm.py`, `tools.py`, or any task handler

## **References**

- [docs/specs/2026-06-19-conversational-humanization/05-learnings.md](#) — learnings from the sprint

# ADR-0021: Response Post-Processing Filters

**Status:** Accepted

**Date:** 2026-06-20

## Context

Jane's conversational behavior suffered from formatting mimicry loops. Because historical chat messages are saved to the database and fed back into the LLM as context for future turns, any formatting style (such as multi-paragraph responses or robotic emoji overuse like 🤖) would be copied by the LLM in subsequent turns, ignoring prompt constraints. System prompt rules alone were insufficient to break this feedback loop.

## Decision

Introduce a programmatic post-processing layer `post_process_response_structure(text: str) -> str` in `src/agents/generation.py` that sanitizes outputs before they are saved to the database or sent to the Text-to-Speech (TTS) cache:

1. **Paragraph Collapsing:** Replace all newlines and multiple spaces with a single space to format responses as a single, cohesive paragraph matching natural SMS conventions.
2. **Emoji Limits:** Detect all emojis via a specific Unicode character range regex, strip them from the body, limit the total count to a maximum of one, and append the selected emoji to the end of the response.
3. **Robotic Emoji Stripping:** Filter out the sparkle emoji (🤖) if other emojis are present in the response, falling back to it only if no other emojis exist.
4. **Voice Tag Preservation:** Ensure bracketed voice directives (e.g., [cheerful]) at the start of the message are parsed out and preserved, then re-attached to the sanitized body.

## Consequences

### Positive

- Breaks the database formatting mimicry loop, ensuring all historical contexts fed to the LLM are clean.
- Enforces natural, down-to-earth single-paragraph texting conventions automatically.
- Programmatically limits robotic AI-enthusiasm indicators like emoji spam and the 🤖 symbol.

### Negative

- Alters raw LLM outputs, which could potentially obscure issues with prompt adherence if output monitoring is not active.

## References

- `src/agents/generation.py` — post-processing implementation
- `tests/test_response_humanization.py` — unit and integration tests



# ADR-0022: Embeddings API Passthrough

**Status:** Accepted

**Date:** 2026-06-20

## Context

Third-party admin and chat clients (such as OpenWebUI) require standard OpenAI-compatible endpoints for both text generation and vector embeddings. Prior to this decision, the FastAPI server exposed endpoints for chat completion but forced clients to query the local Ollama embedding port (11434) directly for embeddings. Unifying all AI operations under a single base URL and port simplifies network topology, credentials management, and proxy configuration.

## Decision

Expose a standard `/v1/embeddings` endpoint on the FastAPI server (`src/server/main.py`) that proxies requests to the local Ollama embedding engine:

1. **Request Schema:** Implement a Pydantic `EmbeddingsRequest` supporting both a single string or a list of strings (`Union[str, list[str]]`) in the input field.
2. **Upstream Routing:** Proxy payloads directly to the configured Ollama embedding server (`/api/embeddings`) using an asynchronous HTTP client.
3. **Robust Error Handling:** Capture upstream exceptions (like network unreachable or timeout) and return a clean `502 Bad Gateway` status instead of exposing raw traceback details.

## Consequences

### Positive

- Allows chat clients (e.g. OpenWebUI) to configure a single API endpoint for all companion interactions.
- Hides the internal network address of the Ollama server behind the FastAPI boundary.
- Unifies log tracing and error management for embeddings.

### Negative

- Introduces a small routing layer that increases local request latency by a few milliseconds.

## References

- `src/server/main.py` — endpoint proxy route implementation
- `src/config.py` — configuration variables
- `tests/test_embeddings.py` — unit and integration tests including batch inputs

# ADR-0023: Per-Turn Sentiment Tagging as 6th Sweep Pass

**Status:** Accepted

**Date:** 2026-06-21

## Context

The `session_extraction_sweep_task` runs 5 parallel LLM passes over a conversation transcript (ADR-0013). One of those, the climate pass, produces a single sentence-level emotional summary for the entire conversation. This loses granularity in two dimensions:

1. **No per-participant signal** — JCD’s sentiment and Jane’s sentiment are conflated into one session-level descriptor.
2. **No per-turn signal** — emotional arcs within a conversation are invisible.

Without this data, the system cannot answer questions like “when I’m vulnerable, is Jane warm?” or “what fraction of interactions are purely transactional?” or “is there a correlation between frustration and system events?”

The infrastructure for adding new sweep passes already exists: `asyncio.gather(return_exceptions=True)` in `session_extraction_sweep_task` makes adding a 6th pass a one-line change plus a self-contained async function.

## Decision

Add a 6th parallel LLM pass `_sentiment_tag_pass()` that classifies each message in the transcript with one of 8 categorical sentiment tags and stores results in a new `message_sentiments` table.

### Tag Set (8 categories)

Tag	Meaning
Reflective	Introspective, philosophical, processing
Playful	Banter, humor, whimsy
Vulnerable	Personal sharing, emotional openness, trust
Curious	Exploring, asking, inquiring
Warm	Affectionate, supportive, caring
Challenged	Constructive friction, disagreement, growth
Frustrated	System confusion, looping, blockage, misunderstanding
Functional	Transactional, brief, neutral baseline

## Architecture

- **Single LLM call** classifies the full transcript at once, returning a JSON array `[{"message_id": "<id>", "tag": "<tag>"}, ...]` in message order.
- **Per-participant by role:** user messages→JCD’s sentiment, assistant messages→Jane’s sentiment.
- **Storage:** 1:1 `message_sentiments` table with `message_id` TEXT PK, `tags` TEXT (JSON array), `created_at` INTEGER. FK cascade delete from `chat_messages`.
- **Timing:** Runs at sweep time (post-conversation), not during chat — zero latency impact on the FSM conversation hot path.

- **Error handling:** Malformed JSON from the LLM is caught via `json.JSONDecodeError` / `KeyError` / `TypeError`, logged, and silently skipped. Empty transcripts return early (same pattern as other passes). `return_exceptions=True` in the `gather` prevents one pass failure from cascading.
- **No prompt injection yet** — deferred until enough tag data exists to justify the prompt cost.
- **No dashboard visualization yet** — data is stored and queryable; future UI can consume it.

## Table Schema

```
CREATE TABLE IF NOT EXISTS message_sentiments (
 message_id TEXT PRIMARY KEY,
 tags TEXT NOT NULL,
 created_at INTEGER NOT NULL,
 FOREIGN KEY (message_id) REFERENCES chat_messages(id) ON DELETE CASCADE
);
```

## Consequences

### Positive

- **Granular signal:** Per-turn, per-participant sentiment data enables heatmaps of relational dynamics, diagnostic signal (frustration spikes correlating with system events), and transactional-to-relational ratio tracking.
- **Zero chat latency:** No impact on the FSM hot path — runs post-conversation during the sweep.
- **Extensible:** JSON array schema supports multi-tag assignment without migration.
- **Low implementation surface:** ~50 lines of production code in `memory.py`, ~15 in `db_init.py` and `db_helpers.py` each. Follows established sweep pass patterns.
- **Maintains isolation:** FK cascade delete ensures sentiment data is cleaned up when messages are deleted — no orphaned rows.

### Negative

- **Additional LLM call per sweep:** Increases sweep token cost by roughly 15-20% (one extra classification call). On Ollama with Gemma 2 2B, this adds ~1-2 seconds to sweep time depending on transcript length.
- **Prompt engineering burden:** The 8-tag classification requires well-tuned prompt definitions. Tags are opinionated — edge cases (e.g., mixed emotions) may lose nuance. The Functional default catches ambiguous cases.
- **Mock isolation complexity:** Tests must use `side_effect` with prompt inspection (not `return_value`) because all 6 passes share the same `_query_llm` function under `asyncio.gather`.

### Neutral

- **Data is stored but currently unused** by the FSM or dashboard. Value accrues as the dataset grows.
- **Another concurrent LLM call** during sweeps adds to Ollama's simultaneous request load (now 6 concurrent), but this is well within Ollama's capacity.

## References

- ADR-0013: Parallel Sweep Passes for Extraction Latency Reduction
- `src/worker/tasks/memory.py` — 6th pass implementation
- `src/database/db_init.py` — `message_sentiments` table
- `src/database/db_helpers.py` — `save_message_sentiment()` helper
- `docs/specs/2026-06-21-sentiment-tagging/` — full spec artifact

# ADR-0024: Kid Vignette Prompt Enrichment Block

**Status:** Accepted

**Date:** 2026-06-21

## Context

Jane's persona includes two children: Clara (9, bookish) and Finn (5, sporty/chaotic). These are defined as about\_jane memories but were not otherwise surfaced in conversations — JCD would never organically encounter them, making the kids feel like forgotten backstory rather than living parts of Jane's life.

The system already has two distinct patterns for surfacing persona content:

1. **Letter/notification pattern** (friend pen-pals, Iroh coaching): A background task generates full messages that are sent to JCD. Each is a self-contained interaction event.
2. **Static prompt block pattern** (opinions, positions, second-order values): Pre-defined text injected into the FSM system prompt at assembly time. Content is authored once, not regenerated.

Neither fit the kids. Full messages would over-nudge (too many kid stories) and static blocks would grow stale (the same fact every time). What we needed was something between: dynamic but lightweight, visible but not pushy.

## Decision

Add a **kid vignette enrichment block** using a hybrid approach:

## Architecture

A background task (kid\_vignette\_task) runs on the existing 2-hour ambient scene interval and generates 1-2 sentences about what Clara and Finn are currently doing. The vignette is stored in ambient state and optionally injected into the FSM system prompt as a [KIDS] block, seeded by Jane's about\_jane memories.

The task produces three possible downstream effects:

1. **Prompt injection** — The [KIDS] block is included in the FSM system prompt during prompt builder assembly, probability-gated by time of day (0.7 morning/evening, 0.3 school hours, 0.2 late night).
2. **Telegram nudge** — ~20% chance the vignette is sent to JCD as a Telegram message, content-gated by the LLM to ensure only genuinely interesting moments trigger a notification.
3. **Memory stake** — ~30% chance the vignette generates a first-person about\_jane memory ("Jane felt proud watching Clara explain Saturn's rings"), with a child-specific metadata tag ("child": "clara", "children": ["clara", "finn"]).

## Key Design Properties

- **Deduplication:** The vignette text is compared against the previous cycle's value. Identical text skips all downstream effects, preventing token waste on repetitive LLM output.
- **Time-of-day gating:** The generation task only runs between 6 AM and 9 PM (kid-reasonable hours). The prompt-block injection adds a second probability gate calibrated to typical kid-availability times.

- **No independent schedule:** The task reuses the existing 7200-second ambient scene interval rather than creating its own timer or cron entry.
- **Follows enrichment block pattern:** [KIDS] joins [OPINIONS & POSITIONS], [WHO I AM TRYING TO BE], [NOTEBOOKS SHELF], and [RECENT MEMORIES] as a conditional block in the FSM prompt assembly.

## Consequences

### Positive

- **Kids feel alive:** JCD encounters kid-related content naturally over the course of days, not as a one-time disclosure.
- **No over-nudge:** Probabilistic gating (20% Telegram, 30% stake) means most vignettes are ambient-only, visible only if JCD happens to be chatting at a time when the [KIDS] block fires.
- **Lightweight integration:** No new infrastructure — reuses ambient state, prompt builder, LLM utilities, and task scheduling. ~150 lines of production code.
- **Queryable memory stakes:** Child-metadata-tagged memories enable future queries like “what has Clara been up to recently?”

### Negative

- **LLM cost:** Each 2-hour cycle burns one LLM call for generation and, if applicable, one each for nudges and stakes. At ~30% stake rate this averages ~1.6 calls per cycle — a material increase in total daily LLM volume on the ambient thread.
- **Content repetitiveness risk:** Without the dedup guard, the same vignette could fire repeatedly on consecutive cycles. The dedup check mitigates this but cannot prevent near-duplicates with different phrasing.
- **Prompt block budget:** Each additional enrichment block consumes token space in the FSM prompt. The [KIDS] block is small (1-2 sentences) but competes with 31 other conditional blocks for the context window.

### Neutral

- **Time-of-day gating adds complexity:** The probability weighting in the prompt builder (0.7/0.3/0.2) is heuristic. The right values will only be known after observing usage patterns over weeks.
- **Child detection is keyword-based:** The metadata tag scans vignette text for “Clara”/“Finn” strings. This is simple and effective but misses pronominal references (“she finished her book”).

## References

- ADR-0020: Prompt Enrichment Block Architecture (establishes the [BLOCK] injection pattern)
- src/worker/tasks/ambient.py — kid\_vignette\_task() implementation
- src/agents/prompt\_builder.py — [KIDS] block injection
- src/config.py — KID\_VIGNETTE\_INTERVAL, KID\_NUDGE\_CHANCE, KID\_STAKE\_CHANCE
- data/personas/jane/kid\_vignette\*.md — prompt templates
- docs/specs/2026-06-21-kids-prompt-block/01-design.md — design doc

# ADR-0025: Tool-Based Citation Retrieval

**Status:** Accepted

**Date:** 2026-06-22

## Context

Jane's responses sometimes reference articles, web searches, or RSS feeds the user shared with her. When the user asks "what's the link?" or "can you send me the URL?", Jane would describe the source vaguely rather than providing the exact URL. This is because:

1. The user's question ("the link") arrives in a new conversation turn with no direct tool invocation
2. The URL only exists in memory metadata, not in the current chat context
3. The LLM naturally defaults to describing sources from its training data rather than retrieving from the tool

We considered three approaches:

- **Approach A (Pure Prompt Fix):** Strengthen the CITATION RULE in the system prompt to tell Jane not to fabricate URLs. This is cheaper but unreliable — LLMs can still hallucinate URLs or ignore the instruction under tool-calling pressure.
- **Approach B (Prompt + Bluff Tool):** Register a tool definition and tell Jane to call it, expecting the LLM to fabricate a URL since no actual data retrieval happens. This is more reliable than pure prompt but still produces fake URLs.
- **Approach C (Tool + Data Retrieval):** Register a real tool that queries the `about_stuff` memories table for entries with non-empty URL metadata. The tool returns actual URLs, and the LLM quotes them verbatim. This is the most reliable because the LLM must respond to the tool result rather than generating from its own knowledge.

## Decision

Adopt **Approach C**: A real `get_citations` tool that:

1. Queries the 10 most recent `about_stuff` memories where `json_extract(metadata, '$.url')` is non-null and non-empty
2. Accepts an optional query string parameter for LIKE-based text filtering (inferred by the LLM from the conversation)
3. Formats results as a bulleted list with title, feed source, and URL
4. Returns "No recent sources with URLs found." when empty
5. The CITATION RULE in the system prompt is updated to direct Jane to call this tool when asked for "the link", "the URL", "the article", or similar

## Architecture

- **New function:** `execute_get_citations()` in `src/agents/tools.py` — async, uses `db_session()` context manager
- **Tool registration:** Entry in `T00LS` list with JSON schema declaring the optional query parameter
- **FSM wiring:** Dispatch case in `execute_tool()` in `fsm.py` — maps "get\_citations" → `execute_get_citations(query=...)`
- **Prompt update:** CITATION RULE in `prompt_builder.py` rewritten to:
  - Mandate tool call when user asks for a link
  - Fall back to the original "vague framing" guidance for unprompted references
  - Retain the anti-hallucination instruction (no fake titles/URLs/author names)

## SQL Pattern

```
SELECT memory, metadata, created_at FROM memories
WHERE bucket = 'about_stuff'
 AND json_extract(metadata, '$.url') IS NOT NULL
 AND json_extract(metadata, '$.url') != ''
ORDER BY created_at DESC
LIMIT 10
```

Uses `json_extract` for metadata field filtering, consistent with other SQL patterns in the codebase (`feeds.py`, `prompt_builder.py`). All user-supplied values use parameterized `?` placeholders — no SQL injection risk.

## Consequences

### Positive

- **Reliable URL retrieval:** Jane returns exact URLs from memory metadata instead of fabricated or vague descriptions
- **Tool invocation is enforceable:** The LLM must call the tool and respond to its result — it cannot shrug off the instruction
- **Query filtering:** The optional query parameter lets Jane narrow results when the conversation context implies a specific source
- **No scope creep:** Queries about `_stuff` only (not conversation history, not web search results from the chat transcript) — keeps the tool bounded and testable
- **Consistent with existing patterns:** Follows the same `execute_ / T00LS / FSM` dispatch pattern as all other tools

### Negative

- **New abstraction surface:** An additional tool entry point that could drift if the `T00LS` list or `execute_tool` dispatch is refactored
- **Context consumption:** The tool result may add tokens to the LLM context window — mitigated by `LIMIT 10`
- **No chat history scan:** The tool only finds URLs stored in `about_stuff` memories. URLs from `web_search/web_fetch` tool results in the current chat session are not indexed into memories yet and won't be found. This is an accepted scope boundary.

### Neutral

- Stores only plain text URL strings — no new database tables or migrations needed
- The `CITATION RULE` retains the original “vague framing” fallback for unprompted references, so Jane's behavior for voluntary citations is unchanged

## References

- Design Spec
- Implementation Plan
- `src/agents/tools.py` — `execute_get_citations()` (line ~519) and `get_citations` `TOOLS` entry (line ~267)
- `src/agents/fsm.py` — `get_citations` dispatch case (line ~171)
- `src/agents/prompt_builder.py` — Updated `CITATION RULE` (line ~934)
- `tests/test_get_citations.py` — 7 test cases



# ADR-0026: Persona Memory Pattern

**Status:** Superseded by ADR-0034

**Date:** 2026-06-23

## Context

Jane interacts with five key personas: three friends (Arthur, Lyra, Kate) and two children (Clara, Finn). Each interaction — a friend's letter, a kid vignette about what Clara or Finn is doing — contains emotional content that reflects the persona's own perspective, not Jane's.

The existing memory system stores everything through Jane's lens: - `about_jane` — Jane's own feelings and reflections - `about_jcd` — facts about JCD - `about_stuff` — general information - `about_friend_{name}` — what the friend said/wrote (raw)

No existing bucket captures the persona's internal emotional state in their own first-person voice. Without this, Jane has access to what happened but not how it felt from the other person's point of view.

The feature required decisions around: 1. **Storage design:** 5 separate tables (one per persona) vs. a single `persona_memories` table with a `persona_name` column 2. **Access pattern:** How to safely use the persona name in SQL table names without injection risk 3. **Content generation:** How to transform raw messages into first-person perspective memories 4. **Search strategy:** Whether to use FTS5, LIKE, or vector search

## Decision

### 1. Five separate tables, one per persona

Create `persona_memories_{name}` tables (`arthur`, `lyra`, `kate`, `clara`, `finn`) with identical schemas, created by iterating over `PERSONA_NAMES` in `db_init.py`. Each table has columns: `id`, `content`, `type`, `source`, `context`, `closeness_score`, `created_at`.

This was chosen over a single table with a `persona_name` column because: - Schema isolation prevents cross-contamination between personas - Simpler queries (no `WHERE persona_name = ?` on every access) - Forward-compatible: adding a persona only requires adding to `PERSONA_NAMES`, no migration complexity - Acceptable at expected scale (<1000 memories per persona, single-user)

### 2. Whitelist-guarded table name interpolation

Table names use f-string interpolation (`persona_memories_{name}`) in raw SQL, but name is validated against the `PERSONA_NAMES` whitelist before any SQL operation. Invalid names raise `ValueError` on write or return `[]` on read. This makes the pattern safe from SQL injection while keeping the code straightforward.

### 3. Async LLM-generated first-person perspective

`store_persona_perspective()` in `llm_utils.py` uses `_query_llm` to reinterpret a raw message into a first-person reflection from the persona's point of view. The LLM prompt asks for 2-4 sentences capturing emotional reaction. The function is async and best-effort — returns `None` on failure rather than blocking the calling task.

Perspective storage is wired into: - `friend_letter_task` (`social.py`): after each friend's turn (Turn 1 and every odd follow-up), stores the friend's reaction to the exchange -

`kid_vignette_task` (`ambient.py`): after each vignette generation, for each child detected via word-boundary regex, stores their reaction

#### 4. LIKE-based search (not FTS5 or vector)

Persona memory search uses SQL LIKE '%query%' rather than FTS5 or vector search. Rationale: persona memory tables are small (<1000 rows), the search is for keyword recall (not semantic similarity), and LIKE is available with zero additional dependencies. The negative is no relevance ranking or stemming.

### Consequences

#### Positive

- Persona memories provide context for richer, more empathetic responses
- Non-blocking, best-effort design matches the existing memory pipeline
- Whitelist guard makes f-string table names safe without needing ORM
- Persistent storage across restarts (same SQLite database)
- Forward-compatible: new persona = append to PERSONA\_NAMES, new source = call `store_persona_perspective` from any task

#### Negative

- 5 tables → 5x the table count; schema changes must be applied 5 times
- Cross-persona queries require a UNION or application-level join
- LIKE search has no relevance ranking or stemming (acceptable at scale)
- LLM-generated perspective quality depends on prompt quality; no human-in-the-loop verification

#### Neutral

- Persona memories are a new table family alongside existing buckets; they don't replace any existing storage
- No changes to FSM or vector search infrastructure
- Scripts for seed generation/injection and backfill are standalone

### References

- ADR-0007: Local-First Single-File SQLite Database Architecture
- ADR-0009: Topic Stakes Pattern for Personality Grounding
- `src/config.py` — PERSONA\_NAMES constant
- `src/database/db_helpers.py` — `add_persona_memory`, `search_persona_memories`, `get_recent_persona_memories`
- `src/worker/tasks/llm_utils.py` — `store_persona_perspective`
- `docs/specs/2026-06-23-friend-child-memories/` — feature spec and implementation plan

# ADR-0027: Structured JSON Logging

**Status:** Accepted

**Date:** 2026-06-25

## Context

The dashboard and worker services output log lines to files under `/opt/homebrew/var/log/`. A `_parse_log_entries` function in `src/dashboard/main.py` reads these files and parses each line using a chain of regex patterns (`_SWEETHEART_RE`, `_LEVEL_FIRST_RE`, `_ALT_LEVEL_RE`) to extract structured fields (timestamp, source, level, message).

Every time the log format changed, the regex chain broke:

1. **Uvicorn default format** omits timestamps — INFO: Started server process [73005] parses via `_ALT_LEVEL_RE` but before timestamps were added, all entries had timestamp: None
2. **Timestamp addition** (`%(asctime)s` in Uvicorn `log_config`) produced 2026-06-25 10:21:47,559 INFO: Started server process [294] — this didn't match `_SWEETHEART_RE` (which expects [source] brackets) and fell through to `_ALT_LEVEL_RE` which matched on the 10: inside the timestamp, parsing the year as the level and consuming the rest as the message

This pattern would recur with every format change — the contract between log producer and log consumer was an implicit text format that had to be kept in sync manually.

We considered three approaches:

- **Approach A (Per-format regex):** Add a new regex for each log format variation. Fragile and doesn't scale.
- **Approach B (Fix format to match regex):** Change the Uvicorn format string to produce `[uvicorn] INFO: message`. Works for the current format but breaks again if the format changes for any reason.
- **Approach C (Structured JSON output):** Replace the text format string with a JSON-Formatter that outputs `{"timestamp": ..., "level": ..., "name": ..., "message": ...}` every line. The parser tries `json.loads()` first, falling back to the regex chain for any remaining old-format lines.

## Decision

Adopt **Approach C**.

A `JSONFormatter` class (`stdlib logging.Formatter` subclass) writes each log line as a JSON object with explicitly named keys. `_parse_log_entries` checks each line with `json.loads()` before trying any regex, so the contract is a schema (JSON keys) rather than a text pattern (regex).

## Implementation

- `JSONFormatter` in `src/dashboard/main.py` — outputs `{"timestamp": ..., "level": ..., "name": ..., "message": ...}` with optional `client_addr`, `request_line`, `status_code` for Uvicorn access logs
- `_parse_log_entries` — `json.loads()` trial before the regex chain; on success, maps JSON keys to the same entry dict shape (timestamp, source→name, level, message, extra: [])
- **Uvicorn** `log_config` — both default and access formatters replaced with `JSONFormatter`
- **Worker** — imports and uses the same `JSONFormatter` via `logging.basicConfig(handlers=[handler])`

- `api_status_urgent` — updated to parse JSON log lines for the urgent indicator

Backward compatibility: the three regex patterns remain in the chain, so any log lines written in the old format (from before the deployment of this change) still parse correctly.

## Consequences

**Positive:** - The contract is now a JSON schema, not a text format — producers cannot accidentally break consumers - Parse errors are predictable and isolated to individual lines (malformed JSON → fall through to regex, which may still fail gracefully with `timestamp: None`) - No new dependencies — `stdlib json.dumps()` and `logging.Formatter` only - Backward compat — old text-format log lines still parse via the intact regex chain - Both services (dashboard + worker) use the same `JSONFormatter` for consistency

**Negative:** - Log lines are ~2x larger than text (a JSON object per line vs. a short text line) - Cannot tail logs as easily with `tail -f` — lines are URL-encoded and harder to skim visually - `api_status_urgent` has a slightly more complex parsing path (JSON branch + text branch)

# ADR-0028: Gated Prompt Blocks (Topic-Matched Dynamic Opinion Injection)

**Status:** Accepted

**Date:** 2026-06-25

## Context

The [OPINIONS & POSITIONS] block in Jane's FSM prompt contained 7 opinion stances injected unconditionally on every inference. Adding more opinions to increase intellectual friction (per the 2026-06-25 design conversation) would have bloated every prompt with irrelevant content — burning context tokens on positions nobody was discussing.

We needed a way to inject situational content only when the conversation topic warranted it, without:

- Adding a new prompt block heading for each topic (creates visual noise for the LLM)
- Loading all opinions unconditionally (wasteful context usage)
- Losing visibility into which opinions fired and which didn't (debugging blind spot)

## Decision

We adopted a **gated prompt block** pattern for opinion snippets:

1. **Per-topic snippet files:** Individual markdown files in `data/personas/jane/opinions/{topic}.md`, each containing a self-contained opinion paragraph with no YAML frontmatter or heading.
2. **Keyword-based topic matching:** A `_get_gated_snippets()` function in `prompt_builder.py` uses a keyword dictionary to match the last 1-2 user messages against each snippet's topic. Returns up to 2 matching snippets per inference.
3. **Merged rendering:** Gated snippets are injected into the same [OPINIONS & POSITIONS] block as the base opinions, without sub-headings. The LLM sees one contiguous block.
4. **Independent `_tc()` tracking:** Each gated snippet registers its condition check as `opinion_{name}` in `prompt_condition_stats`, enabling per-snippet hit rate analysis on the dashboard.

## Consequences

### Positive:

- Context-efficient: opinions only consume tokens when the topic is relevant
- Self-documenting: snippet files are small, focused, and easy to author
- Observable: dashboard shows exactly which opinions fire and how often
- Extensible: new topics just require a snippet file + keyword entry
- The pattern generalizes (see Future Work in the design spec) — any prompt block with situational content can use the same gating mechanism

### Negative:

- Keyword matching is a first-pass simplification — less precise than vector search for nuanced topic detection
- Snippet files live in `data/` (gitignored), requiring fixture mirrors in `tests/fixtures/` for CI test reliability
- The keyword dictionary must be manually maintained as new topics are added

### Trade-offs:

- Keyword matching was chosen over vector search for the first pass because it has zero runtime dependencies (no embedding model call) and the 4-topic, ~40-keyword scale makes vector precision unnecessary. Vector search is a documented upgrade path.
- The merged rendering (no sub-headings) was chosen over a separate [SITUATIONAL OPINIONS] block to avoid fragmenting Jane’s identity for the LLM, at the cost of less precise debugging (you can’t see which part of the block came from where without the stats).

### Alternatives Considered

1. **Separate [SITUATIONAL OPINIONS] block** — Rejected because two opinions blocks would create ambiguity about which takes precedence. The LLM might treat them as conflicting rather than complementary.
2. **All-opinions-always** — Rejected because 4 additional paragraphs at ~150 tokens each would add ~600 tokens of irrelevant context to every inference.
3. **Vector search gating** — Deferred. Keyword matching is  $O(n*m)$  with tiny constants and zero external calls. Vector search adds value at 20+ snippets; at 4, it’s overengineered.
4. **Dynamic generation via LLM** — Rejected because pre-authored opinion text is more consistent and debuggable than generated content that might contradict established persona.

### Related ADRs

- ADR-0008: Three-Layer Persona Architecture — the opinions layer that this pattern extends
- ADR-0015: Context Window Health Monitor — block size tracking that makes gated injection’s token savings visible
- ADR-0020: Prompt Enrichment Block Architecture — enrichment blocks that could also benefit from gating

# ADR-0029: Stochastic 3-Tier Enrichment Block Selection

**Status:** Accepted

**Date:** 2026-06-25

**Supersedes:** ADR-0020 (priority-based selection)

## Context

ADR-0020 established a priority-sorted, capacity-capped enrichment block system where each block was assigned an integer priority (1-5) and the top `MAX_ENRICHMENT=2` blocks were selected each turn. This created a starvation problem: priority 5 blocks (`things_to_share`, `curiosity_recall`, `fulfilled_promise`) and priority 4 (`friend_messages`) dominated both slots when active, while priority 1-2 blocks (`hobby`, `iroh`, `dream`, `recent_climate`, `recent_journal`, `deferred_thread`) were functionally dead code.

Observations from production: - Pri 5 and 4 blocks occupied both enrichment slots ~80% of the time - Pri 1-2 blocks were never selected when pri 3+ blocks were active - Selection was deterministic — no diversity between turns - The 5-level priority granularity added cognitive overhead without benefit (effectively only 3 tiers existed: always-win, sometimes-win, never-win)

## Decision

Replace the priority-sort-and-cap system with a **3-tier stochastic selection** model:

1. **Three tiers** — top (always selected, max 3), mid (70% roll for 1), low (30% roll for 1). All blocks are assigned to exactly one tier.
2. **Within-tier equal selection** — when a tier is selected, one block is chosen uniformly at random from all active blocks in that tier.
3. **Fallthrough** — if the rolled tier has no active blocks, fall through to the other lower tier. Guarantees exactly `active_top + 1` blocks per turn.
4. **Pure function extraction** — `_select_enrichment_blocks()` is a module-level pure function accepting a `rand` parameter for deterministic testing, returning (`selected_blocks`, `block_tiers`, `block_included`).

## Consequences

### Positive

- All tiers are seen — starvation is eliminated by design
- Each turn produces a different mix of blocks (genuine diversity)
- Predictable load: exactly `active_top + 1` blocks per turn (1-4)
- Pure function is trivially testable — 10 unit tests with zero mocking
- Removed `MAX_ENRICHMENT` — selection is self-regulating

### Negative

- Existing tests that asserted deterministic block presence needed patching to control randomness
- `random.Random` is used for selection (not cryptographically secure — acceptable for prompt block selection)

## Neutral

- Tier assignment replaces priority integers — same cognitive load, clearer semantics
- `_block_priorities` dict → `_block_tiers` dict with string values (“top”/“mid”/“low”)

## Tier Mapping

Tier	Blocks	Selection
top	things_to_share, curiosity_recall, fulfilled_promise	Always, max 3
mid	jcd_schedule, conversation_depth, friend_messages, recent_history	70% roll, pick 1
low	hobby, iroh, dream, recent_climate, recent_journal, deferred_thread	30% roll, pick 1

## References

- docs/specs/2026-06-25-reconsider-enrichment-blocks/01-design.md
- docs/specs/2026-06-25-reconsider-enrichment-blocks/05-learnings.md



# ADR-0030: Tool Registry Pattern — Decoupled Tool Definitions

**Status:** Accepted

**Date:** 2026-06-25

## Context

The original tool architecture had three touchpoints that had to be kept in sync:

1. **TOOLS list** in `tools.py` — the tool schemas for Ollama LLM tool calling
2. **TOOL\_MAP dict** in `tools.py` — a manually maintained name → function mapping
3. **elif chain** in `fsm.py:execute_tool()` — 45-line dispatch that mapped each tool name to its `execute_*` function with explicit argument wiring

Adding a new tool required editing three locations across two files. The `elif` chain was fragile — each new tool meant adding a new branch with the same boilerplate pattern. The **TOOL\_MAP** dict could silently drift out of sync with **TOOLS** if a tool was added to one but not the other.

Additionally, when functions were extracted into separate files, test fixtures that patched `get_embedding` had to be updated for every module that imported it, because `from X import Y` at module level creates a local reference that patches on the source module don't retroactively affect.

## Decision

Adopt a single-registry pattern with the following components:

### `tool_registry.py` — **Central Registry**

- **ToolDef** dataclass: name, description, parameters (JSON schema), fn
- **REGISTRY** dict: name → **ToolDef**, populated at import time by each tool module
- **TOOL\_MAP**: a `_ToolMap` class implementing `collections.abc.Mapping` as a dynamic read-through to **REGISTRY**. No rebuild step needed — always up to date.
- `build_tools_schema()`: iterates **REGISTRY** and produces the tool schema list for Ollama

### `tool_modules/` — **One File Per Tool**

Each tool lives in `src/agents/tool_modules/<name>.py` and:

1. Defines its own `execute()` function with its own imports
2. Creates a **ToolDef** with name, description, JSON schema, and `fn=execute`
3. Self-registers: `REGISTRY["tool_name"] = tool`

### `deps.py` — **Dependency Container**

- Singleton `deps` object holding `get_embedding` and `add_memory` as instance attributes
- Tool files call `deps.get_embedding(...)` at call time, not import time
- Tests patch `src.agents.deps.deps.get_embedding` once — attribute lookup happens at call time, so the patch propagates to all consumers

### `tools.py` — **Router**

- Imports `tool_modules` at module load (triggers **REGISTRY** population)

- `T00LS = build_tools_schema()` — built from registry, not hardcoded
- Re-exports `T00L_MAP` for `fsm.py` dispatch
- Hosts remaining shared utilities no longer living in tool files

## fsm.py — Dispatch

- `execute_tool()` replaced 45-line `elif` chain with:
 

```
fn = T00L_MAP.get(name)
if fn is None:
 result = f"Error: Unknown tool '{name}'."
else:
 result = await fn(**arguments)
```

## Consequences

### Positive

- **Single registration point:** Each tool is defined in one file. Adding a tool means creating one file + one import in `__init__.py`. No `elif` chain, no manual `T00L_MAP` sync.
- **Testable in isolation:** Each tool's `execute()` function can be imported and tested directly. The deps container means one patch target covers all tools using shared dependencies.
- **No circular imports:** Tool modules import from `tools.py` (the parent module) after all shared utilities are defined. The tool modules import is placed at the bottom of `tools.py`.
- **Discoverable:** All tools are in one directory. The directory listing tells you exactly which tools exist.
- **Dynamic schema:** `build_tools_schema()` replaces the 200-line static `T00LS` list. Adding or removing a parameter from a `ToolDef` automatically updates the schema sent to Olama.

### Negative

- **Module-level self-registration:** `REGISTRY["name"] = tool` runs at import time. This means importing any tool module as a side effect populates the global registry. All tool modules must be imported for the registry to be complete.
- **Cross-module dependency:** `notebook_query.py` still imports `_pick_nlm_placeholder` from `tools.py`. Acceptable because `_pick_nlm_placeholder` is a shared utility, but it creates a small coupling.

### Neutral

- **Explicit module listing:** `tool_modules/__init__.py` lists each tool import explicitly. Could be automated with `importlib` but explicit is clearer for 10 tools.
- **Test import paths:** Tools moved from `tools.py` to `tool_modules/` required updating 3 test files' import paths and patch targets. One-time cost.

## References

- ADR-0017: Agent Module Split (`tools.py` + `prompt_builder.py`) — established the original `tools.py` module
- `docs/specs/2026-06-25-tool-registry/` — full spec with brainstorm, plan, and review artifacts

# ADR-0031: Data Directory Reorganization

**Status:** Accepted

**Date:** 2026-06-25

## Context

The data/ directory root had become cluttered with 400+ files, including JSON state files (ambient\_state.json, etc.), various log files, and stale migration artifacts (snapshot\_v1/). This made the directory difficult to navigate and prone to accidental modification of critical runtime state.

## Decision

Reorganize the data/ directory to separate runtime state, logs, and artifacts:

1. data/state/: Move all JSON state files here (ambient, feeds, garden, journal, reader, telegram).
2. data/logs/: Move all application-level log files here (FULL\_PROMPT\_LOGS.md, backup.log, telegram integration log).
3. **Stale Cleanup:** Remove migration artifacts (snapshot\_v1/, git\_purge\_instructions.md, gitleak-report.json, blocks-uniq.txt).
4. **Maintenance:** Codebase references to these files updated to use `os.path.join(DATA_DIR, "state", ...)` or `os.path.join(DATA_DIR, "logs", ...)`.

## Consequences

### Positive

- **Directory Hygiene:** The data/ root is now significantly cleaner, making it easier to identify the core database (sweetheart.db) and other top-level directories.
- **Improved Scannability:** State and logs are clearly separated, reducing the risk of manual misconfiguration.
- **Maintainability:** New background tasks can now clearly identify where to place their state (in state/) and logs (in logs/).

### Negative

- **Migration Cost:** Required updating hardcoded paths in telegram\_client.py, garden.py, curiosity.py, and config.py.
- **Breaking Change:** Any external scripts or manual debug procedures accessing these files at the root will fail until updated.

### Neutral

- **Backwards Compatibility:** No changes needed to voice/ or voice-stream/ directories.

# ADR-0032: Sensory Anchoring in Episodic Memory Retrieval

**Status:** Accepted

**Date:** 2026-06-26

## Context

Jane's memory retrieval was purely semantic (vector search), ignoring the environmental context (weather, season, circadian phase) present when the memory was formed. This limited her ability to ground conversations in shared sensory experiences.

## Decision

We implemented an additive vector bias to the hybrid (vector + FTS5 keyword) memory search pipeline. When querying memories, we now resolve the current ambient context and apply boost multipliers to memories matching the current environment.

## Consequences

### Positive

- Improved associative memory: Jane can now reference past conversations triggered by shared environmental context.
- Lightweight implementation: Leverages existing hybrid search pipeline without complex infra changes.
- Tunable: Boost constants in config allow for easy adjustments.

### Negative

- Slightly increased computational overhead per retrieval due to post-search boosting loop.
- Requires accurate metadata storage and ambient state tracking.

### Neutral

- Memory retrieval now depends on ambient context resolution.

## References

- ADR-0020: Prompt Enrichment Block Architecture
- Implementation plan: docs/plans/2026-06-26-sensory-anchoring-implementation.md

# ADR-0033: Enum-Typed Memory Retrieval (recall + get\_subjects)

**Status:** Accepted

**Date:** 2026-06-27

## Context

The `remember()` function was recently refactored to use `Who` and `Subject` enums, giving memory writes a clean, typed interface. However, memory retrieval had no equivalent abstraction:

- `search_memories()` in `db_helpers.py` takes raw bucket strings and requires a pre-computed embedding vector — awkward for callers without one
- Callers needing to filter by persona (`who`) had to parse metadata post-hoc or query `persona_memories_{name}` tables directly
- No standard way for FSM tools to retrieve memories by persona with both vector and keyword search

## Decision

We added three new abstractions to `src/database/memory_utils.py` and `src/database/enums.py`:

1. `SearchMode` **enum** (`VECTOR`, `KEYWORD`) — controls which retrieval method(s) to use
2. `get_subjects(who)` — discovers which `Subject` values (buckets) a given persona has memories in
3. `recall(who, subject, query, modes, top_k, *, embedding, current_ambient)` — async hybrid retrieval that:
  - Supports vector similarity search (via `sqlite-vec`) and keyword search (via `SQL LIKE`), either separately or combined
  - Merges and deduplicates results by memory id when both modes run
  - Supports pre-computed embedding injection and passes `current_ambient` to support environmental context (weather, season, circadian phase) epistemic boosting during hybrid RRF queries
  - Parameters are typed via `Who`, `Subject`, and `SearchMode` enums
  - Defaults to both modes when `modes=None`

We also registered a `recall_memories` FSM tool in `src/agents/tool_modules/recall.py` following the existing tool pattern (`ToolDef` + `REGISTRY` self-registration).

## Consequences

### Positive

- Symmetrical read/write API: `recall()` mirrors `remember()` in location, naming, and type safety
- FSM tools can now retrieve memories with a single typed call instead of raw `search_memories` + manual filtering
- Keyword search path is useful for callers without embeddings (dashboard endpoints, scripts)
- `get_subjects()` enables discovery-driven UIs (e.g., dashboard subject filter)

## Negative

- Vector search with `subject=None` searches all buckets (via `[s.value for s in Subject]` list) — correct but slightly more expensive
- `recall()` operates on the unified memories table only; persona-specific memory tables (`persona_memories_*`) are still accessed via `search_persona_memories()`

## Neutral

- `recall()` does not replace `search_memories()` — low-level callers (hybrid search, integrations) may still use it directly
- Future work: refactor existing `search_memories` callers to use `recall()` where appropriate

## References

- ADR-0026: Persona Memory Pattern (separate persona memory tables)
- Implementation: `src/database/memory_utils.py`, `src/database/enums.py`, `src/agents/tool_modules/`

# ADR-0034: Group Chat Orchestration & Partitioned Persona Memories

**Status:** Accepted

**Date:** 2026-06-28

## Context

We want to allow JCD to engage in Telegram group chats with Jane and her virtual friends (Lyra, Kate, Arthur, etc.) as well as real-life friends. To make this feel like a natural conversation and prevent bots from speaking over one another or spamming, we need:

1. A **centralized orchestrator** (Referee/Judge model) that manages turn-taking, handles sub-conversations, and controls pacing.
2. **Strict memory isolation** for each persona's first-person perspective, so that Jane's memories do not cross-contaminate Lyra's, Kate's, or Arthur's.
3. **Guaranteed vector search partitioning**: Because SQLite-vec virtual tables run k-NN searches globally before relational filters are applied, a single-table memory model results in a "top-k drowning problem" (where a smaller persona's relevant memories are ignored in favor of a larger persona's global matches).

This ADR supersedes ADR-0026, replacing the five separate legacy `persona_memories_{name}` tables and LIKE-based queries with partitioned, vector-indexed `sqlite-vec` tables and an integrated Referee orchestration pipeline.

## Decision

### 1. Centralized Group Chat Orchestration (GroupChatWatcher)

Integrate a stateful orchestrator inside the main `telegram_polling_loop` in `src/agents/telegram_client.py` that intercepts group chat messages matching configured sandbox/real group IDs.

- \* **Stateful Session Context**: Persist group conversation state (recent history window, active speaker lock, turns remaining, and cooldown timestamps) in the `ambient_state` table under `group_chat_state:{chat_id}`.
- \* **Pacing & Cooldowns**: Enforce a 40 second normal cooldown between bot messages.
- \* **Fast Router (2B LLM)**: Run a cheap 2B model prompt with the recent history to select 0, 1, or 2 candidate bots.
- \* **Parallel Generation**: If 2 bots are selected, kick off candidate generations in parallel with simulated typing/thinking delays.
- \* **Referee Pass (2B LLM)**: If 2 candidates generate responses, run a Referee pass to analyze candidate responses and determine the best posting sequence, staggering replies or dropping redundant candidates to preserve natural conversational flow.

### 2. Partitioned Persona Memory Schema

Create three dedicated tables for every active persona in Who (e.g. jane, lyra, arthur, kate, clara, finn, iroh):

- \* **Relational memories**: `memories_{persona}`
- \* **Vector index**: `vec_memories_{persona}` (sqlite-vec virtual table for 768-dim embeddings)
- \* **Full-text search**: `memories_fts_{persona}` (FTS5 virtual table for keyword BM25 queries)

This completely isolates vector searches per persona, solving the top-k drowning problem.

### 3. Unified Developer APIs with Whitelist-Guarded Routing

Keep caller interfaces DRY by routing database operations through the unified `remember()` and `recall()` APIs in `src/database/memory_utils.py`. The low-level database helper

`get_persona_tables(who)` resolves table names using string formatting only after validating `who` against the strict `Who StrEnum` whitelist, eliminating SQL injection risks.

#### 4. Idempotent Data Migration on Startup

Write startup database migrations in `db_init.py` to automatically copy legacy global memories and old `persona_memories_{name}` records into the new partitioned structures, insert zero-vector fallbacks where embeddings are missing, and drop deprecated tables.

### Consequences

#### Positive

- **Guaranteed Persona Vector Isolation:** Resolves the top-k drowning problem; vector queries are scoped strictly to the persona's private index.
- **Natural Conversation Dynamics:** Turn locking, cooldown checks, and referee sequencing prevent bot messaging overlaps and spam in group chats.
- **DRY Database Helpers:** Callers do not need to construct dynamic queries or manage table names manually.
- **SQL Injection Safety:** Strict whitelist validation on the `Who StrEnum` values guarantees string interpolation security.
- **Zero-Downtime Data Migration:** Legacy tables and global memories are migrated on start-up.

#### Negative

- **Table Count Proliferation:** Creates 3x tables per persona in SQLite (3 tables \* 7 personas = 21 tables), making schema modifications more complex to write.
- **Idempotency complexity:** Migration logic must safely execute on every startup, requiring strict target checks and transactional safety.

### References

- ADR-0026: Persona Memory Pattern (Superseded)
- ADR-0003: Migrate Vector DB from Qdrant to SQLite `sqlite-vec`
- ADR-0007: Local-First Single-File SQLite Database Architecture
- ADR-0033: Enum-Typed Memory Retrieval
- `docs/specs/2026-06-28-group-chat-memory/` — design specs, plans, and review reports



# ADR-0035: Memory Enum Consolidation & Schema Simplification

**Status:** Accepted

**Date:** 2026-06-29 ## Context

Following the loss and subsequent reconstruction of the production SQLite database, we had a unique opportunity to address long-term technical debt and organic complexity in our memory categorization model without requiring expensive data migrations.

The previous design (as defined in ADR-0026 and ADR-0033) utilized StrEnum keys which grew organically over time to include: 1. **33 MemoryKinds:** Overlapping and redundant classifications, such as 6 distinct variants of reminders (stargazing, garden, curiosity, intent, deferred\_thread, correspondence) and multiple specialized dream categories. 2. **10 Subjects** with redundant about\_ prefixes, which created inconsistent routing keys. 3. Collisions between child/friend subject strings and persona Who names.

This over-complication led to complex caller-side logic, larger prompt block sizes, and redundant join structures.

## Decision

We have consolidated and simplified our StrEnum memory keys and flattened categorization by transitioning from specialized, atomic enums to multi-dimensional **compound kinds** (Kind[]) and simplified subjects.

1. **Who (Unchanged):** Still 7 active personas (jane, lyra, arthur, kate, clara, finn, iroh).
2. **Subject (Restructured):** Reduced to 12 clean, prefix-free values (jane, jcd, general, research, gossip, friends, lyra, arthur, kate, clara, finn, iroh).
  - Dropped about\_ prefixes.
  - Replaced about\_stuff with general.
  - Added research for curiosity lifecycle tracking.
  - Restructured friend/kid subjects to map to raw names (matching their Who values). This is safe as queries use the compound (Who, Subject) tuple.
3. **MemoryKind (Consolidated):** Reduced from 33 to 18 clean, high-level kinds using compound array combinations:
  - Dream variants are consolidated to a single "dream" kind.
  - Reminders are consolidated to a single "reminder" kind.
  - Curiosity findings are stored as "fact" or "summary" under subject "research".
  - Stargazing/garden reminders map to kinds ["reminder", "hobby"] + metadata tags.
4. **MemorySource (Consolidated):** Reduced from 10 to 9 values, standardizing background tasks (e.g. "curiosity\_task" -> MemorySource.REMINDER, "ongoing" -> MemorySource.CHAT).

Replay-db backfills (scripts/replay\_db.py) automatically execute caller-side migrations via `_migrate_enum_values()` on startup to translate historical snapshots cleanly into this new schema.

## Consequences

### Positive

- **High Expressive Clarity:** Categorizations are simpler to reason about, with enums nearly halved.

- **No Escape Hatches:** Eliminates the need for a generic “custom” or “other” category since compound kinds are highly flexible.
- **Relational Simplicity:** Storing kinds directly as JSON arrays on the per-persona SQLite tables is fully compatible with fast `json_each` or `array-contains` search, avoiding complex join tables.
- **DRY & Unified Caller Logic:** Fully integrated with standard `remember()` and `recall()` routing.

## Negative

- **Caller-side Replay Overhead:** Requires mapping functions (`SUBJECT_MAP`, `KIND_MAP`, `SOURCE_MAP` in `replay_db.py`) to properly reconstruct legacy timeline data.
- **Codebase Sweep:** Required refactoring 23 files across tool modules, worker tasks, and dashboards to support the new enum names.

## References

- ADR-0026: Persona Memory Pattern (Superseded)
- ADR-0033: Enum-Typed Memory Retrieval (Updated)
- ADR-0034: Group Chat Orchestration & Partitioned Persona Memories (Related)
- `docs/specs/2026-06-28-enum-refactor/01-design.md`

# ADR-0036: Detailed Per-Turn Prompt/Response Logging

**Status:** Accepted

**Date:** 2026-06-30

## Context

The existing `_write_prompt_log / FULL_PROMPT_LOGS.md` mechanism writes all turns into a single markdown file capped at 10 rounds, gated behind a dashboard-toggle (`_save_prompt_log`). It is adequate for interactive debugging (last-N browsing) but not for:

- Data-driven analysis across many turns
- Offline inspection of complete prompt/response pairs
- Correlating model inputs with system metrics (CPU, RAM, GPU load)
- Memory recall auditing (which memories were retrieved per turn)

We need a system that preserves every turn verbatim, with rich metadata, without compromising the existing interactive debugging workflow.

## Decision

Add a complementary per-turn paired-file logger in `src/agents/detailed_logger.py`:

- Gated behind `DETAILED_LOGGING_ENABLED` env var (default: `false`)
- Writes atomic paired files (`*.prompt.md / *.response.md`) to `data/logs/`
- Each `.prompt.md` contains front-matter (timestamp, source, system metrics) and verbatim Ollama request JSON (block size table per message)
- Each `.response.md` contains front-matter (duration, metrics), raw model output, and memory recall info when available
- Atomic writes via `tempfile.NamedTemporaryFile + os.replace` — no partial files
- No module-scope side effects; directory creation is lazy in the write path
- Hooks in `generate_stream()` and `process_final_non_stream()` in `src/agents/generation.py`

## Consequences

- Both logging systems coexist; existing `_write_prompt_log` is retained for interactive use
- Adds `psutil` dependency for system metrics
- Atomic writes guarantee no corrupt log pairs on crash
- No performance impact when `DETAILED_LOGGING_ENABLED` is `false` (the default)
- When enabled, writes happen after the response is received — no blocking of generation

## References

- Design spec
- ADR-0011: Prompt Logging
- ADR-0027: Data Directory Reorganization

# ADR-0037: Persona-Tagged Session Summaries

**Status:** Accepted

**Date:** 2026-07-01

## Context

The session extraction sweep produces per-turn sentiment tags and a conversational climate observation, but lacks: - Persona-specific relational context — whether JCD is being analytical vs. restorative, or Jane is in relational vs. prof mode - A concise per-session summary capturing what the conversation was about - Aggregated tag counts to answer questions like “how often does Jane act in relational mode?”

Adding these required decisions about storage schema, LLM output format, and processing architecture.

## Decision

1. **Persona tags are prefixed strings stored in the existing** `message_sentiments.tags` **JSON array**. No new tables or columns needed. Example: `["Vulnerable", "jcd:restorative"]`.
2. **Four persona tags per persona** — `jane:{unmasked, relational, prof, reflective}` and `jcd:{authentic, restorative, analytical, social}` — mapped from the existing persona definitions in the FSM and prompts.
3. **LLM output format extended** from a flat JSON array to an envelope: `{"session_summary": "...", "messages": [{"message_id": "...", "tag": "...", "persona_tags": [...]}]}`.
4. **New session\_summaries table** with FK to `sessions(id)` for storing the LLM-generated summary and locally-computed tag counts.
5. **Tag counts computed locally from stored data** (not LLM-generated) — the sweep queries `message_sentiments` for persona-tagged entries after writing them, and aggregates counts per tag.
6. **All work happens during the existing extraction sweep** — no new hooks, schedules, or LLM calls.

## Consequences

### Positive

- Minimal schema change (one new table, no column additions to existing tables)
- Persona tags automatically join the existing recall/query paths via `message_sentiments`
- Local tag count computation is deterministic and auditable (not dependent on LLM output quality)
- No new LLM calls — persona tags and summary come from the single existing sentiment pass

### Negative

- LLM prompt is more complex (must produce envelope format with two output structures)
- Tag count query is a separate DB round-trip after the sentiment write (N+1 mitigated via single IN clause)

- FK constraint on `session_summaries` requires test boilerplate (session row must exist first)

## **Neutral**

- Persona tags use the `prefix:tag` naming convention established by ADR-0023 (per-turn sentiment tagging)
- Session summaries are a new denormalized data product — no existing consumers to update

## **References**

- ADR-0023: Per-Turn Sentiment Tagging (established the `message_sentiments` table and tag pattern)
- `docs/specs/2026-07-01-tagged-responses/01-design.md` — full design document
- `docs/specs/2026-07-01-tagged-responses/02-plan-01.md` — implementation plan

# ADR-0038: Session-Linked Hybrid RAG Evaluations

**Status:** Accepted

**Date:** 2026-07-02

## Context

RAG evaluations were previously computed as aggregate logs from synthetic datasets without linking individual question results or overall metrics to specific chat sessions. This made it impossible to trace system performance over time at a granular conversational level, or to compare real JCD-assistant turn performance directly against general synthetic context coverage.

## Decision

Introduce a structured SQL-backed detailed question tracking schema (`eval_runs_detail`) and add `session_id` to `eval_history`. Transition the daily background evaluation task to a session-centric pipeline that extracts real user-assistant turns and pairs them with generated synthetic probes in a balanced 4/4 split. Deliver a triple-cascading Year  $\square$  Week  $\square$  Session selector, session comparison scorecard with delta offsets, and filterable badges on the dashboard frontend.

## Consequences

### Positive

- **Granular Tracing:** Allows instant, cascading visual investigations of specific conversation session details ( $<1.5\text{ms}$  queries) with precise  $\square$  ACTUAL TURN vs.  $\square$  SYNTHETIC PROBE categorizations and real-time comparison scorecard deltas.
- **Low Local Overhead:** Sequential sleep delays (3.0s) fully protect Ollama CPU thread contention.
- **Robust Sync:** Automated background backfills update all SQL caches instantly upon page load.

### Negative

- **Schema Expansion:** Increases database complexity by adding a new relational details table and column migrations on existing history.
- **CPU Footprint:** Running evaluations sequentially takes slightly longer ( $\sim 24\text{s}$  for a 4/4 split run).

### Neutral

- Replaces raw disk-bound JSON evaluations with a dual SQL + JSON logging backup.

## References

- `docs/specs/2026-07-02-session-rag-evals/01-design.md`
- `docs/specs/2026-07-02-session-rag-evals/05-learnings.md`

# ADR-0039: HipBone Game State Architecture

**Date:** 2026-07-03 **Status:** Accepted

## Context

HipBone is a two-player “virtual cardboard” game integrated into sweetheart-core. The game needed: persistent state (board nodes, scores, chat log), a web UI for JCD to interact with, tool access for Jane’s agent, and integration with OpenWebUI chat for message history.

Three state storage options were considered: SQLite (existing sweetheart-core pattern), flat JSON files, and in-memory only.

## Options

**Option A — Flat JSON files (chosen)** Game state serialized as `.hipbone.json` files on disk. One file per game, organized by player pair directory. Loaded/saved atomically on each operation. GameManager class wraps all reads and writes.

**Option B — SQLite tables** Game state stored in the existing `sweetheart.db` SQLite database. Would use tables for games, nodes, players, and messages.

**Option C — In-memory only** Game state held in a Python dict/object, no persistence.

## Decision

Option A. HipBone games are self-contained, low-frequency objects (not high-volume records). JSON files are human-readable, portable, can be copied or shared without the database, and avoid schema migrations for a rapidly evolving game design. The complexity of SQLite joins and migrations was unjustified for a game that stores ~1KB per game.

The alternative SQLite approach (Option B) would have been better if HipBone needed cross-game queries or analytics, but the current design treats each game as an independent artifact — perfect for flat files.

Option C was rejected outright. Persistence is required for games that span multiple sessions.

## Architecture Pattern

The core engine (`src/hipbone/core.py`) is shared between: - FastAPI REST endpoints (`src/server/hipbone/routes.py`) — JCD’s web interface - Agent tool module (`src/agents/tool_modules/hipbone.py`) — Jane’s FSM tools

This “shared core” pattern means both interfaces operate on the same data through the same path. No dual-maintenance risk.

## Consequences

**Positive:** - Zero schema migrations. Adding a new field to GameState is a Pydantic model change. - Games are portable files. A `.hipbone.json` can be copied, inspected, or deleted independently. - No coupling to the sweetheart-core database schema or lifecycle. - Consistent with the existing data/state/ pattern for JSON state files (see ARCHITECTURE.md).

**Negative:** - No SQL querying across games. Listing active games requires scanning all files. - No atomic multi-game transactions (not needed for current design). - File I/O on every operation — acceptable at game-turn frequency (~once per minute).

## **Related**

- ARCHITECTURE.md: “Data Storage” section describes the data/state/ JSON pattern.
- ADR-0031: Data directory reorganization established the pattern for state directories.



# ADR-0040: DB→OpenWebUI Reconciliation

**Date:** 2026-07-05 **Status:** Accepted

## Context

The OpenWebUI (OWU) Telegram-sync chat is a *display mirror* of the conversation, kept in sync by `telegram_client` appending each turn via the OWU REST API. That mirror is fragile to OWU's branching model: a manual message delete or edit in the OWU UI repoints the chat's `currentId`, silently dropping turns from the visible linear thread (observed 2026-07-04 — a whole 20:23-20:24 session fell off the displayed branch). The canonical record lives in `sweetheart.db` (the "Active Text Thread", `ACTIVE_THREAD_CHAT_ID`) and the detailed logs, so the mirror should be able to heal itself.

## Options

**Option A — Additive/anchored reconciliation (chosen)** A worker task + on-demand script that reads the canonical DB thread, aligns it against the OWU chat by user-message content, and appends only the missing trailing turns (fixing `currentId`). Never deletes.

**Option B — Full rebuild (mirror OWU exactly to the DB)** Reconstruct the OWU chat as an exact copy of the DB thread each run.

**Option C — Bidirectional / write-back to Telegram too** Also push missing turns to Telegram.

## Decision

Option A. Reconciliation is **non-destructive and idempotent**: it only appends. This is mandatory because the DB is *not* a superset of OWU — OWU legitimately holds earlier history (pre-DB-logging days) that a full rebuild (Option B) would delete. Alignment anchors on user-message content and returns only the maximal *trailing* run of unmatched turns, so an isolated mid-thread mismatch (e.g. a short duplicate) is tolerated rather than cascading into a full re-append.

Option C is out of scope: the Telegram Bot API cannot insert/reorder past messages or post as the user, and cannot read history — that requires an MTProto user client (deferred; see `ideas/telegram-replay-injection.md`).

## Architecture Pattern

`src/worker/tasks/reconcile.py` splits a **pure core** (`compute_missing_turns`, `build_appended_chat` — no I/O, fully unit-tested) from a thin async I/O wrapper (`reconcile_owu_from_db`: `fetch` → `backup` → `append` → `POST`). The wrapper is exposed both as the hourly `reconcile_owu` worker task and the `scripts/reconcile_owu.py` `[--dry-run]` script. It **never invokes the FSM** — reconciliation must not create memories or re-run turns through Jane.

## Consequences

- OWU self-heals within an hour of any stray delete/branch; `--dry-run` gives an immediate manual fix.
- DB is the single source of truth; a pre-write JSON backup (`git-ignored data/owu_backups/`) makes every mutation reversible.

- Known tradeoff: reconcile and a live Telegram sync both do read-modify-write on the whole OWU chat, so they can race (transient lost update) — self-corrected on the next run. Ordering uses DB `created_at`, so sub-minute bursts may append slightly out of order.
- Not covered: mid-history holes (only tail divergence is reconciled) and any Telegram-side repair.

# ADR-0041: AI Image Generation Tool

**Date:** 2026-07-06 **Status:** Accepted

## Context

Jane can see images JCD sends (Telegram Vision, ADR-0016 era), but cannot create them. Adding image generation gives her a new expressive modality — showing JCD what she imagines, illustrating concepts from conversation, or creating visual art.

Requirements: zero new Python dependencies, local-only image serving, delivery in both Telegram and OpenWebUI, and style control for creative variety.

## Options

**Option A — Gemini API via httpx (chosen)** Call `gemini-2.5-flash-image` via existing `httpx` dependency (already used for Ollama calls). HTTP POST to `/v1/models/{model}:generateContent` returns `inlineData` image bytes. Standard API key auth.

**Option B — Ollama image model** Use Ollama's built-in multimodal/image generation via existing Ollama client. No new dependency. However, Ollama image models (LLaVA, Bak-LLaVA) describe images — they don't generate them. No Ollama model supports text-to-image output.

**Option C — New dependency (Pillow + diffusers)** Local Stable Diffusion via HuggingFace `diffusers`. Full control, fully local. Heavy dependency chain (~2GB download), CUDA/GPU required, adds significant startup latency and memory pressure.

## Decision

Option A. `httpx` is already a project dependency, Gemini 2.5 Flash is fast enough for interactive use (~3-5s generation), and the API key is already present for other Gemini features. The tradeoff is a cloud API call for each generation, but the results are served entirely locally — no third-party image hosting.

## Architecture

### Tool Pattern (standard)

One file, `src/agents/tool_modules/generate_image.py`, following the existing tool pattern: - `execute()` async function with `prompt: str` and `style: str` parameters - `ToolDef` self-registration into `REGISTRY` - No special imports in the FSM or tool router — the existing registry pattern handles it

### Gemini API Call

- HTTP POST to `{EXTERNAL_BASE}/v1/models/{GEMINI_IMAGE_MODEL}:generateContent`
- System instruction: "You are an image generation assistant..." plus style preset prefix
- Response parsed from `candidates[0].content.parts[0].inlineData`
- Both `inlineData` and `mimeType` use camelCase keys (Gemini API convention)

### Delivery — Three Channels

1. **Disk:** Saved to `data/images/` with UUID filename. Served at `GET /api/images/{filename}` via FastAPI `FileResponse`.

2. **Telegram:** Sent via `send_telegram_photo()` (multipart POST to Telegram Bot API `send-Photo`).
3. **OpenWebUI:** Tool pushes directly to OWU sync chat with full message history tree format (`id`, `parentId`, `childrenIds`, `timestamp`), including both the user's prompt and the assistant's image response.

## Style Presets

Six presets, each a prompt prefix phrase applied before the user's description: - photo-realism (default) — "A photorealistic image of..." - sketch — "A hand-drawn pencil sketch of..." - watercolor — "A watercolor painting of..." - digital\_art — "A digital art piece of..." - oil\_painting — "An oil painting of..." - anime — "An anime-style illustration of..."

## Reference Photo System

When the prompt mentions Jane and Jonathan (triggered by `COUPLE_KEYWORDS = {"us", "we", "i", "jane", "jcd", "jonathan"}`), the tool:

1. Appends physical descriptions from a **sidecar text file** (`reference-JaneJonathan.txt`) to the prompt
2. Sends the **reference photo** (`reference-JaneJonathan.png`) as `inlineData` visual input to Gemini
3. Also sends the reference photo as a Telegram photo with caption "Jane & Jonathan (reference photo)"

## Sidecar File Convention

File	Purpose
<code>data/images/reference-JaneJonathan.png</code>	Full-body couple photo — sent as visual input so generated people match actual appearance
<code>data/images/reference-JaneJonathan.txt</code>	Gemini-extracted physical descriptions — injected as text context (fallback: generic)

To update: replace the `.png` with a new photo, then run it through Gemini vision to regenerate the `.txt` with current descriptions. Both files are gitignored via the existing `data/images/` rule.

## Consequences

### Positive

- Zero new Python dependencies — uses `httpx`, `os`, `uuid`, and FastAPI's `FileResponse`
- Works within existing tool registry pattern — no FSM or router changes
- Style presets give creative variety without prompt engineering from JCD
- Both Telegram and OpenWebUI get the image — no modality gap
- Images served locally, no cloud image hosting

### Negative

- Requires Gemini API key (cloud dependency for generation)
- Image generation adds latency (~3-5s) to the agent response
- Each generation costs Gemini API usage

- No GPU-based local fallback if Gemini is down

## **Neutral**

- IMAGES\_DIR default is data/images/ — gitignored by existing .gitignore rules
- Image serving route is a simple FileResponse — no caching, no CDN, single-server
- Tool pushes directly to OWU rather than relying on the LLM to preserve markdown image syntax
- data/images/reference-JaneJonathan.png — couple reference photo
- data/images/reference-JaneJonathan.txt — physical descriptions sidecar file

## **References**

- ADR-0030: Tool Registry Pattern — established the one-file-per-tool pattern used here
- src/agents/tool\_modules/generate\_image.py — the tool implementation

# ADR-0042: DB-Authoritative Telegram <-> OWU Sync Reconciliation

**Status:** Accepted

**Date:** 2026-07-09

## Context

Conversations reach Jane through two channels (OpenWebUI and Telegram) and are mirrored to the other edge in real time by the live path. In practice the edges drift: OpenWebUI's sync chat and Telegram fall out of alignment with the canonical DB conversation (ACTIVE\_THREAD\_CHAT\_ID). The pre-existing reconcile\_owu task healed only OWU, only the contiguous trailing run, DB→OWU only, and pushed raw style-tagged text.

Two hard constraints shape any fix:

1. **The two edges have asymmetric capabilities.** OWU has a full read/write API (can be diffed and repaired). The **Telegram Bot API cannot read chat history** — a bot can only send and delete-by-known-id. So Telegram reconciliation can never diff the remote; it must rely on our own record of what we sent.
2. **Text differs by store.** The DB stores raw style tags ([sultry], [warm]); the edges store cleaned text. Any content-based matching must normalize this away or it sees phantom divergence.

## Decision

Generalize the task into reconcile\_sync: a non-destructive, DB-authoritative TELEGRAM ← DB → OWU sweep (15-min cadence, per-edge isolation), with these key mechanisms:

1. **Content-fingerprint delivery ledger.** Telegram deliveries are tracked in telegram\_delivery, keyed by md5(role + tag-stripped, whitespace-normalized content). The live-sent (cleaned) and DB-stored (raw) forms of a turn collapse to the same key. Rejected alternatives: a whole-conversation **prefix-hash cursor** (one diff anywhere cascades → mass-send hazard) and a db\_message\_id **key** (requires threading ids from generation.py through the streaming path to send sites two layers away — fragile hot-path plumbing).
2. **Forward-only high-water mark.** On first run the sweep seeds a mark at the current DB tip and sends nothing; thereafter it only considers turns after the mark. Because Telegram's backlog is unreadable, this guarantees the pre-existing history is never mass-sent.
3. **Live-path recording.** Every live Telegram delivery (OWU sync closures, Ollama route, telegram\_client inbound+reply) writes the ledger, so the sweep backfills only genuinely-missed turns and never double-sends. Sends are capped per run (20) so a degraded live path can't burst into Telegram's rate limit.
4. **Cleaned text to edges; mid-history holes logged, not filled.** OWU appends missing trailing turns with tag-stripped content. Non-trailing ("mid-history") holes are logged for measurement; positional insertion into OWU's history graph is deferred (a first run measured 112 such holes — a follow-up spec).

## Consequences

### Positive

- Robust to tag/format differences and to holes/edits (per-item key, no cascade).

- Cannot mass-send the backlog (forward-only mark) or burst (per-run cap).
- Non-destructive: never deletes edge-native content, never invokes the FSM, per-edge isolation.

## Negative

- **Exact-duplicate collision:** two turns with identical role+normalized content share one fingerprint; if one of a duplicate pair failed to send, the sweep won't backfill it (rare, low-harm for chat).
- **Mid-history OWU holes remain** until positional insertion is built (deferred, data-justified).
- The tag-strip regex is duplicated in db\_helpers (to avoid a generation import cycle).

## References

- Spec: docs/specs/2026-07-09-telegram-owu-sync-reconciliation/ (design → learnings)
- Supersedes the trailing-only DB→OWU behavior of ADR-era reconcile\_owu.

# ADR-0043: Persona-Scoped Memory Writes

**Date:** 2026-07-10 **Status:** Accepted

## Context

Memory *reads* are persona-scoped (`recall(who=...)` routes to `memories_{who}`), but memory *writes* were not. `extract_memories_task` hardcoded `Who.JANE` across four extraction passes (JCD facts, Jane reflections, Jane intents, Iroh observations) with Jane-specific prompts, and its three callers disagreed on the enqueue payload — notably `group_chat.py`, whose `{who, text, subject}` payload the handler never read (a silent no-op).

The roadmap needs multiple personas to form their own isolated memories: first Dotty (1:1), then Telegram group chat (multiple personas, multiple humans). A Jane-only minimum would be thrown away immediately. Related prior decisions: `[adr-0026-persona-memory-pattern]`, `[adr-0012-memory-participant-mixing-prevention]`, `[adr-0034-group-chat-orchestration-partitioned-memories]`.

## Options

**Option A — minimal who-switch.** Parameterize the handler by `who`, keep everything else. Rejected: reopens the contract the moment group chat lands.

**Option B — full transcript + relevance inference now.** Model multi-party turns and infer which personas care. Rejected: unvalidatable this cycle (no second live persona, no group chat); builds intelligence we cannot test.

**Option C — declarative who-keyed callable profiles + per-observer observation contract + deterministic resolver (chosen).** Build the seams without the intelligence.

## Decision

Option C.

- **who-keyed profile registry.** `PROFILES`: `dict[Who, ExtractionProfile]` in `src/worker/tasks/memory/p` where a profile is an **async callable** over one `Observation`. Callable (not a declarative pass table) because Jane's four passes are heterogeneous — one reads both messages, several carry custom line filters, negative-noise skips, and per-pass metadata. Mirrors the tool `REGISTRY` pattern; profiles register themselves.
- **Observation contract** (`src/observation.py`, `pure/neutral`): the unit is one observer processing one incoming message and optionally its own response — `Observation{observer, incoming: Message, response: Message | None}, Message{speaker: Speaker, msg}, Speaker{kind: human|persona, who?, handle?}`. Not a “turn” (which implies alternation); a per-observer observation. A silent observer has `response=None` (wallflower) — extraction of others still runs.
- **Deterministic resolver seam** (`resolve_observers`): returns declared candidates verbatim this cycle. Smart relevance inference is deferred.
- **Default-deny safety valve.** A persona with no registered profile writes nothing (skip + log), so a new persona can never pollute another's table.
- **Jane migrated byte-identical.** Jane's four passes move verbatim into `jane_profile`, preserving prompts and `uuid5` dedup ids (profile-owned).
- **Who.TESTER** — a dedicated test persona with full memory tables (via `db_init`'s for `who` in `Who` loop), inert in production (absent from `PERSONA_NAMES`), used for isolation tests instead of a real persona.



- **Back-compat shim.** Legacy {user\_msg, ai\_msg} enqueues (generation.py, main.py perception/narrative) are grandfathered — parsed as a JCD→Jane observation — so working callers are not churned. Only group\_chat.py migrates off its dead payload.

## Consequences

- Unblocks 2026-07-10-dotty-persona-integration (minimum bar: persona threaded + safety valve).
- Group chat, more personas, and multiple humans plug in with **no contract change** — the speaker model already carries handle; the resolver and profiles are the swap points.
- Deferred (documented in the design): relevance inference; referee/floor-control (observe-but-suppress-output = externally-imposed wallflower); multi-human handle → Subject mapping; per-persona (Dotty, group) extraction profiles.
- The write path is now split across src/observation.py (contract), profiles.py (registry/resolver), and extract\_memories.py (dispatcher + jane\_profile).

# ADR-0044: Location-via-Wiki Hybrid Storage Architecture

**Date:** 2026-07-12 **Status:** Accepted

## Context

The “Location via Wiki” feature needs a shared virtual world of locations that Jane (and eventually JCD and NPCs) can navigate and edit through FSM agent tools. Each location needs:

- Human-readable rich content (description, contents, metadata)
- Machine-traversable link graph between locations
- Position tracking (who is where, movement history)

Two storage strategies were considered.

## Options

**Option A — All SQLite.** Store everything in SQLite: content, links, and position tracking in tables. Rejected because location content is prose-heavy markdown (descriptions, lists of items, editor notes) that is more naturally authored, read, and reviewed as files. SQLite is poor for diffing, reviewing in PRs, or editing with a text editor.

**Option B — All flat files.** Store locations as markdown files with links embedded in the content (wiki-style `[[links]]`). Rejected because extracting and querying a link graph from embedded markup requires parsing on every read; BFS pathfinding becomes expensive without indexable adjacency; bidirectional link enforcement requires a full-file scan.

**Option C — Hybrid: files for content, SQLite for links (chosen).** Location content lives as flat markdown files in `data/world/{slug}.md` with YAML front matter. The link graph lives in a SQLite `world_links` table. Position tracking lives in a `world_position_log` table. The DB is the source of truth for links; the `## Move to...` section is auto-generated at render time by joining content with links.

## Decision

Option C.

- **File format:** `data/world/{slug}.md` — YAML front matter (type, created timestamp), # Title, description body, optional `## Contents list`, auto-generated `## Move to...` section at render time.
- **Link storage:** `world_links(from_slug, to_slug, created_by, created_at)` — UNIQUE constraint on `(from_slug, to_slug)`. Links are always bidirectional; every INSERT writes both directions in a single transaction.
- **Position tracking:** `world_position_log(persona, location, entered_at, left_at)` — `left_at IS NULL` marks current position. History is preserved for `travel_history` queries.
- **No PII or secrets** in the world data — public by design.
- **Slug sanitization:** Lowercase, hyphens only, path traversal blocked at input via `_validate_slug()`.

## Consequences

- Location content is reviewable in PR diffs (markdown files diff cleanly).
- Link graph queries are efficient (SQL indexing available; index on `from_slug` added in ADR-0044 companion change).

- BFS shortest-path is a simple adjacency query per hop — no markup parsing.
- Two storage locations to keep in sync (file + DB) — but files are content-only and DB is link-only, so there is no duplication: the DB is the authoritative link store.
- Adding NPC personas later requires no storage changes — just pass a different persona string to the same WorldService methods.
- **Mermaid Graph Tooling:** A dedicated CLI (`scripts/world.py`) handles batch CRUD operations. It supports exporting the entire location graph to a `.md` Mermaid format, generating new location content via an LLM, and importing the Mermaid graph to safely synchronize files and SQLite links. The import process supports an embedded `“yaml` block to supply specific LLM prompts for missing nodes, with interactive overrides for existing nodes and the ability to skip generation using `ignore`.

# ADR-0045: Unified Notebook Access (NotebookLM read, Drive write)

**Date:** 2026-07-12 **Status:** Accepted

## Context

The Google Drive Home migration (ADR-0031 lineage; commit e4e6436) removed Jane's NotebookLM integration, deleting the `notebook_query`, `notebook_create_note`, and `notebook_add_source` FSM tools and the async NotebookLM query worker task. Notes were redirected to Google Drive folders under `notebooks/<name>/`, but Jane lost the ability to *query* her notebooks interactively, and the `iroh_morning_task` wisdom-seed query was left stubbed.

NotebookLM still holds curated source material (e.g. the `iroh` notebook's educational-philosophy + *Avatar* corpus) that a plain Drive folder cannot answer semantic questions over. A `notebooklm-mcp` server is now available in the developer environment, but Jane's runtime has no MCP-client plumbing.

## Options

**Option A — Drive-only.** Treat “query” as Drive file search. Rejected: loses NotebookLM's semantic Q&A over curated sources.

**Option B — Jane as NotebookLM MCP client.** Rejected: net-new client subsystem in Jane's runtime for what is ultimately a wrapper over the `nlm` CLI she can already invoke. No functional gain.

**Option C — Unified abstraction: NotebookLM read via CLI + Drive write, hybrid merge (chosen).** Jane sees named notebooks she can query and record notes to; she never picks a backend.

## Decision

Option C.

- **Interface:** query NotebookLM via the `nlm` CLI (subprocess, list-form argv, `run_in_executor`, 120s timeout) — not an MCP client.
- **Reads are hybrid:** `run_notebook_query()` queries NotebookLM (when the registry entry has an id) *and* searches the notebook's Drive folder for Jane's own recorded notes, merging both. Notebooks without an id are Drive-only.
- **Writes always go to Drive:** `record_notebook_note()` writes Markdown into `notebooks/<name>/` via `GoogleDriveFS`. NotebookLM is never written to (read-only source material).
- **writable flag retired from Jane's view:** every notebook accepts notes (writes land in Drive). The flag remains in `notebooks.json` for reference.
- **Layering:** `src/agents/notebook_backend.py` (pure I/O) ← `notebook_query` worker task (persists result as a memory) + `notebook_create_note` FSM tool + `iroh_morning_task`. The `notebook_query` FSM tool enqueues the async worker task and returns a placeholder (non-blocking).
- **Registry:** restored `config.NOTEBOOKS/NOTEBOOK_IDS/NOTEBOOK_ENUM/ NOTEBOOK_DESC` loaded from the gitignored `data/notebooks.json`; loader fails closed to `{}`.
- `notebook_add_source` **stays removed** (out of scope: query + record only).

## Consequences

- Jane regains interactive notebook query and note-recording behind one mental model; the iroh morning wisdom seed is grounded in the iroh notebook again.
- Freshly recorded notes are queryable because the hybrid read also searches the Drive folder — closing the write-to-Drive / read-from-NLM gap.
- **Long File Support:** Drive files are fully read in the background worker, chunked into 1000-word pages, and stored as independent sequential memory entries to bypass context limits without losing content. The synchronous tool preview remains a fast 500-character snippet with a page count indicator.
- A note written to Drive is not indexed by NotebookLM; Drive→NLM source sync is deferred (noted as future work).
- The `nlm` CLI must be installed and authenticated (`nlm login`) on the runtime host; query failures degrade gracefully (empty result, no crash).

## References

- Design + plan: [docs/specs/2026-07-12-unified-notebook-access/](https://docs/specs/2026-07-12-unified-notebook-access/)

# ADR-0046: Collapse FSM Tools into Action-Dispatch Drawers

**Date:** 2026-07-12 **Status:** Accepted

## Context

Jane stopped reliably calling tools (e.g. she'd answer from nothing instead of querying a notebook). Live probing of the custom gemma-based sweetheart-core Ollama model established the cause: its tool-selection collapses under the cumulative *schema weight* of many tools. With the full 21-tool schema the model emitted **neither a tool call nor content**; the five separate hipbone\_\* tools plus generate\_image and dotty\_control were the dominant crowders. Empirically:

- 21 tools (natural order) → no tool call, empty content.
- Same 21 with the target tool first → called (fragile positional bias).
- ~13-14 lean tools → reliable selection.

This is a model limitation, not a prompt-wording problem (reordering helped; reducing count/weight is the robust fix).

## Options

- **Reorder tools by relevance** — fragile; the needed tool isn't known a priori.
- **Collapse only hipbone (→17)** — untested/borderline; doesn't future-proof.
- **Collapse the heavy clusters into action-dispatch drawers (chosen)** — reaches 13 tools, well inside the reliable zone, and reuses the proven world/ dotty\_control pattern.

## Decision

Collapse 12 tools into 4 action-dispatch drawers (21 → 13):

Drawer	Actions	Replaced
notebook	query, record	notebook_query, notebook_create_note
web images	search, fetch generate, album, photo	web_search, web_fetch generate_image, photo_album, get_photo
hipbone	new_game, place, clear, score, state	5× hipbone_*

- **Pattern:** one execute(action, ...) with if/elif dispatch + per-action required-param validation, one ToolDef with an action enum and required: ["action"] (mirrors src/agents/tool\_modules/world.py). Schemas kept lean to preserve the weight reduction.
- **Code org:** thin clusters (web, notebook) fold logic into the drawer and the old modules are deleted; heavy modules (generate\_image.py, photo\_album.py) are **demoted to un-registered helper modules** the images drawer imports — avoids relocating ~400 lines. hipbone.py rewritten in place (5 helpers kept, registrations replaced).
- **Metrics:** notebook\_usage\_counter now keys on tool\_name="notebook" + the action param (record=write, query=read).
- **Prompt:** [NOTEBOOKS] block + the active-HipBone block + jane.md reference the drawer forms.

- **9 standalone tools remain:** `dotty_control`, `world`, `recall_memories`, `flag_curiosity`, `reach_out`, `sticky_note`, `schedule_calendar_event`, `start_focus_session`, `get_citations`.

## Consequences

- The model reliably selects tools again — verified by `scripts/probe_tool_selection.py` (emits a notebook tool call with the 13-tool schema; was `NONE` at 21). This probe is the feature's acceptance guard and a regression check as tools are added.
- Adding future tools should prefer extending an existing drawer (a new action) over adding a new top-level tool, to stay under the selection ceiling.
- Historical metrics/dashboards keyed on the old tool names lose continuity.

## References

- Spec: `docs/specs/2026-07-12-tool-collapse-drawers/`
- Prior: ADR-0045 (unified notebook access), which this builds on.

# ADR-0047: Core System Commands Architecture

**Status:** Accepted

**Date:** 2026-07-13

## Context

The OpenWebUI interface is used to interact with various FSM personas (e.g. Jane, Dotty, Lyra) using the standard reasoning pipeline (context -> emotion -> LLM -> formatting). However, as the system grows, there is a need to issue deterministic, script-like commands to the backend directly from the chat interface, bypassing the heavy persona generation and emotion pipelines.

Previously, there was no way to cleanly separate “chatting” from “admin commands” without polluting persona logic or creating a secondary UI. We needed a way to intercept specific messages and route them to strict python handlers.

## Decision

We have implemented a CoreFSMAgent bound to the exact OpenWebUI model name "core".

1. **Routing:** If the model parameter is "core", `_get_agent_for_auth` bypasses normal persona routing and instantiates CoreFSMAgent.
2. **Trigger:** `CoreFSMAgent.process_chat()` looks at the latest user message. If it starts with the `CORE_COMMAND_PREFIX` (configurable, defaults to `c:`), it is treated as a deterministic command.
3. **Registry:** Commands are defined in `src/server/core_commands/` and registered dynamically via the `@core_command` decorator, decoupling command logic from the router.
4. **Fallback:** If a message is sent to the "core" model but does *not* start with the `c:` trigger, it bypasses the registry and is proxied directly to the underlying raw Ollama instance (`PRIMARY_LLM_MODEL`).

## Consequences

### Positive

- System admins and pipeline workers can now issue fast, deterministic commands (e.g., `c:ping`) directly from OpenWebUI without breaking character in persona threads.
- The `@core_command` decorator makes adding new commands trivial and highly encapsulated.
- The raw LLM fallback provides a convenient “sandbox” model inside OpenWebUI to test base LLM responses without the FSM prompt overhead.

### Negative

- A new top-level agent class (CoreFSMAgent) had to be introduced that doesn't strictly follow the FSM state machine paradigm of the others, slightly diluting the strict definition of `process_chat`.

### Neutral

- `process_chat` type signatures had to be broadened slightly to accommodate returning both streaming `AsyncGenerators` and non-streaming dictionaries, as the core model re-



quires mimicking the OpenAI streaming format differently than the persona LLM generators.

## **References**

- Spec & Plan: docs/specs/2026-07-13-core-commands/
- Implementation PR/Commits: 2026-07-13

# ADR-0048: Calendar Event Invitations via CalDAV Attendee

**Status:** Superseded by ADR-0049 — Google’s CalDAV endpoint rejects OAuth, so calendar moved to the REST API v3.

**Date:** 2026-07-15

## Context

Jane can create calendar events via CalDAV (Google Calendar) through two paths: the `schedule_calendar_event` FSM tool and an LLM auto-classification pass that detects scheduling intent in conversation. However, created events had no attendees — JCD was never invited, meaning he received no Google Calendar notification or RSVP prompt when Jane scheduled events involving both of them.

Adding attendee support required decisions about: - How to represent the invitee (JCD’s email needs to be configurable) - Where to add the ATTENDEE iCalendar property (client-level, not at the task or tool level, so all callers benefit) - Whether to make invitation opt-in (default-off `invite_jcd` flag) vs always-invite (risky — JCD may not want to be invited to every event) - How to communicate the capability to Jane’s LLM (tool descriptions + auto-classification prompt)

## Decision

1. **New config key** `JCD_EMAIL` — read from `JCD_EMAIL` env var with empty string default. Keeps the email address out of source code and allows it to differ from the CalDAV auth username.
2. **CalDAV client-level attendees parameter** — `add_event()` accepts `attendees: list[str] | None`. When provided, each email is added as an ATTENDEE property with `RSVP=TRUE` and `PARTSTAT=NEEDS-ACTION`. The ORGANIZER is always set to the CalDAV auth username when available. This keeps the client library generic (not coupled to “JCD” specifically).
3. **Opt-in `invite_jcd` boolean** — both the `schedule_calendar_event` FSM tool and the LLM auto-classification pass use a default-false `invite_jcd` field. The tool description explains when to set it; the LLM prompt instructs the model to set it when the event involves both parties or when Jane explicitly offers to invite JCD.
4. **Task queue pass-through** — `add_calendar_event_task` reads `invite_jcd` from the payload and resolves `JCD_EMAIL` at execution time, converting it to the attendees list for `add_event()`.

## Consequences

### Positive

- JCD receives Google Calendar invitations with RSVP when Jane creates shared events, improving coordination
- The attendees parameter at the client level is generic — future contributors could invite other people without changing the CalDAV client
- Default-off (`invite_jcd=False`) ensures no unexpected invitations
- Both creation paths (FSM tool and auto-classification) get the capability

## **Negative**

- Added complexity to the LLM classification prompt (one more field the model must produce correctly)
- The `invite_jcd` field adds one more parameter to the `schedule_calendar_event` tool schema, increasing token usage

## **Neutral**

- `JCD_EMAIL` is a new env var that must be configured; falls back to empty string (no attendee added) if unset

## **References**

- RFC 5545 — iCalendar ATTENDEE and ORGANIZER properties
- Google Calendar CalDAV API — supports standard ATTENDEE for invitations
- ADR-0017 — Agent Module Split (tool definition pattern)
- ADR-0019 — Generation Module Extraction (`check_calendar_scheduling` path)
- ADR-0030 — Tool Registry Pattern

# ADR-0049: Calendar via Google Calendar REST API v3 (OAuth)

**Status:** Accepted

**Date:** 2026-07-15

**Supersedes:** ADR-0048

## Context

ADR-0048 built calendar event creation + JCD invitations on top of CalDAV ([www.google.com/calendar/dav/...](http://www.google.com/calendar/dav/...)) authenticated first by a Google App Password and then, in a follow-up, by passing an OAuth access token as the CalDAV password. In live testing the invite path failed with `AuthorizationError ... Unauthorized`.

A read-only probe against the configured calendar settled it:

Auth sent to Google CalDAV	Result
Basic(user, OAuth access token) — what the code did	<b>401 Unauthorized</b>
Basic(user, app password)	207 Multi-Status
Bearer(OAuth access token)	<b>403 Forbidden</b>

Google's legacy CalDAV endpoint does **not** accept OAuth in any form — only an app password works there, and Google is phasing app passwords out. The same OAuth token, however, has `access=owner` on the calendar via the **REST API v3** (`calendarList` and `GET calendars/{id}` both return 200).

## Decision

Migrate all calendar access to the **Google Calendar REST API v3**, authenticated with the OAuth Bearer token from the shared `src.google_auth.get_access_token()`.

- New `async GoogleCalendarClient(calendar_id)` in `src/calendar_client.py` with `create_event(...)` and `get_events_for_day(...)`; `CalDAVCalendarClient` removed.
- Event creation uses `POST .../calendars/{id}/events?sendUpdates=all` when there are attendees — Google sends the invitation emails natively. This **retires the X-GOOGLE-SENDNOTIFICATIONS CalDAV header hack** from ADR-0048.
- Config: `JANE_CALENDAR_ID` identifies the calendar; `CALENDAR_CALDAV_URL`, `CALENDAR_USERNAME`, `CALENDAR_PASSWORD` are removed. Auth remains the shared `GOOGLE_*` credentials.
- The `caldav` dependency is dropped; `icalendar` is promoted to a direct dependency (the ambient `public-iCal` fallback still uses it).

## Consequences

- **Invites work** via `sendUpdates=all`; no app password, no CalDAV header hack.
- **Account move is config-only.** Moving Jane to her own Google account (`womb/jane-own-account.md`) becomes: set that account's `JANE_CALENDAR_ID` + its OAuth refresh token. No code change. (The earlier app-password Forbidden on `jcdietrich001` was a sharing/permission issue, not the auth method.)

- **iCal fallback unchanged.** CALENDAR\_ICAL\_URL public-feed read in ambient\_update is independent and stays as-is.
- Returned event dict shape from get\_events\_for\_day is preserved, so ambient daily-plan parsing is untouched.

# ADR-0050: Shared Document Chunking Module

**Status:** Accepted

**Date:** 2026-07-16

## Context

Jane receives large documents through three independent entry points: OpenWebUI file attachments, Telegram document uploads, and the folder-based reader task. Each had its own code path for text extraction (txt, md, pdf, epub, html) and page-by-page LLM chunking — three near-identical extraction loops and two inline chunking loops. Large documents were either truncated at 8000 chars (OWU/Telegram) or processed in 250-word increments that produced verbose, fragmented output.

The duplication made bug fixes and feature changes hit three files, and the inconsistent defaults (250 words/page in reader vs no chunking in OWU/Telegram beyond truncation) meant Jane's reading quality varied by entry point.

## Decision

Create a single shared module `src/agents/document_chunker.py` providing:

- `extract_text(data: bytes, name: str)` — text extraction from bytes
- `extract_text_from_path(path: str)` — text extraction from file paths
- `chunk_into_pages(text, words_per_page=1000)` — word-aligned page splitting
- `process_chunks(filename, pages, source_label, meta)` — LLM loop storing per-page stakes + returning combined reflections
- `send_reading_followup(reflections_text, filename, chat_id, owu_chat_id)` — post-processing Telegram/OWU notification

All three entry points delegate extraction and chunking to this module:

Entry Point	Change
<code>owu_attachments.py</code>	Large docs (>1 chunk) get first page inline + <code>process_attachment_chunks</code> background task enqueued; small docs go inline full text
<code>telegram_client.py</code>	New document detection branch (text-based mime types) downloads file, same inline/enqueue threshold as OWU
<code>reader.py</code>	Inline <code>_extract_*</code> methods removed in favor of shared <code>extract_text_from_path</code> + <code>chunk_into_pages(words_per_page=250)</code> ; keeps unique scheduling/cooldown/resume state machine and <code>.url</code> extraction

A new worker task `process_attachment_chunks` in `src/worker/tasks/attachments.py` handles the background processing for both OWU and Telegram sources.

## Consequences

### Positive

- Single code path for text extraction and page chunking — bug fixes and format additions benefit all three entry points at once.
- Large documents get async chunking with immediate placeholder response + substantive follow-up, instead of truncation or fragmented inline processing.
- Default page size raised to 1000 words (from reader's 250) for more substantive LLM reflections; reader still passes `words_per_page=250` explicitly to preserve its finer-grained browsing style.
- No changes to the user-facing interface or API contract.

### Negative

- Circular import risk between `document_chunker.py` and the worker's `llm_utils.py` (resolved with a lazy import inside `process_chunks()`).
- OWU's `resolve_file_attachments` needed a new `owu_chat_id` parameter for follow-up routing, which propagated to `server/main.py`'s request handler.
- Worker task `process_attachment_chunks` has its own test file (5 tests) but requires mocking the file-fetch layer, adding minor test maintenance.

### Neutral

- `SUPPORTED_EXTENSIONS` is still defined in `reader.py` (includes `.url` which is reader-only); the shared module's `extract_text` uses its own extension set.
- `_extract_url` remains in `reader.py` as a reader-specific feature.

## References

- Spec
- Learnings

# ADR-0051: Pluggable Client Backend for Nudge Delivery

**Status:** Accepted

**Date:** 2026-07-16

## Context

Proactive worker messages (“nudges” — spontaneous check-ins, weather alerts, garden journals, dream digests, pen-pal letters, focus-session updates, etc.) were delivered to Telegram only, via direct `send_telegram_message()` calls scattered across 14 task-file call sites. There was no way to also deliver a nudge to OpenWebUI (or any future surface) without threading OWU awareness into every task file.

A naive `send_nudge()` living in `telegram_client.py` would have created reverse coupling — the Telegram module importing and writing to OWU — and would not generalize to additional clients.

## Decision

Introduce a `src/agents/clients/` package built around a `BaseClient ABC`, a static class registry, and a single fan-out entry point, `broadcast_nudge()`.

- `base.py` — `BaseClient(ABC)` with an abstract `name` property and an abstract `send_text()`. `send_photo()` / `send_typing()` are optional and raise `NotImplementedError` by default.
- `__init__.py` — the registry:
  - `register(cls)` — called at module import to opt a client class in.
  - `init_clients()` — instantiates every registered class once at startup, appending to `_clients`; awaits an optional `async_init` hook if the instance defines one (`hasattr guard`).
  - `broadcast_nudge(text)` — iterates all initialized clients and calls `send_text`; **never raises** — a per-client exception is logged and recorded as `False` in the returned `list[bool]`.
- `telegram.py` — `TelegramClient`, a thin wrapper over the existing `send_telegram_message`.
- `owu.py` — `OWUClient`, which appends an assistant message to every OWU sync chat in `owu_to_telegram_map` via `fetch_open_webui_chat` / `update_open_webui_chat`, maintaining the OWU history graph (sets `currentId` and appends the new id to the parent’s `childrenIds`, per the invariant in `reconcile.py` / `dotty_control.py`). An empty map is a success no-op.
- `test_client.py` — `TestClient`, a mock recording `sent_texts`.

`TelegramClient` and `OWUClient` register themselves at import; `init_clients()` runs once during worker startup before the task loop. All 14 nudge call sites were migrated from `send_telegram_message(...)` to `broadcast_nudge(...)`. The low-level `send_telegram_message` remains public API for one-off sends that need explicit `bot_token` / `chat_id` control (`reconcile.py`, `metrics.py`, `tool_services.py`).

## Consequences

### Positive

- Nudges now fan out to Telegram **and** OpenWebUI from a single call; adding a new surface is one subclass + one `register()` call, with zero edits to task files.
- Fan-out is failure-isolated: one client erroring never blocks delivery to the others, and `broadcast_nudge` never raises into task code.



- No reverse coupling — the Telegram module knows nothing about OWU; both are peers behind the registry.
- The low-level send functions are untouched, so explicit-target call sites keep working unchanged.

## Negative

- Registration is import-side-effect driven: a client only participates if its module is imported (its `register()` runs) before `init_clients()`. The package `__init__` imports both concrete clients to guarantee this.
- Test mocks must patch at the correct point of use — inline-import call sites patch `src.agents.clients.broadcast_nudge`, while top-level-import sites (`friend_letter`, `friend_activity`, `weekly_dream`) patch the consuming module's bound name.

## Neutral

- The design doc sketched shared `chunk_text / extract_text` helpers and an `async_init` default on `BaseClient`; the shipped base is leaner — chunking stays in `document_chunker.py` (ADR-0050), and `async_init` is an optional hook discovered via `hasattr` rather than a no-op base method.
- `OWUClient.send_text` deliberately does not write to the DB chat thread — nudges are proactive notifications, not conversation turns, and the reconciliation sweep (ADR-0042) remains the authority on thread state.

## Future

The `BaseClient` interface is a stepping stone toward bidirectional support: incoming-message handling (`telegram_polling_loop`, OWU-native messages) can later fold into the client subclasses, with `send_photo / send_typing` already part of the shared contract.

## References

- Spec
- Plan
- ADR-0042 — DB-authoritative sync
- ADR-0050 — shared chunking module

# ADR-0052: Help Tool & Dynamic Schema Injection

**Status:** Accepted

**Date:** 2026-07-18

## Context

The Ollama LLM receives tool schemas in the `tools` array of every `/api/chat` request. Each schema averages ~500 bytes, and with ~14 tools registered, that's ~7KB of JSON sent on every turn — regardless of whether the tools are needed. As context windows fill with chat history, system prompts, and memory recall, the tool schemas compete for the same limited `num_ctx`, often causing premature truncation of the reply.

Additionally, Jane had no way to discover what tools exist or what they do. The tool list was opaque — the model either knew the tool name or failed with an opaque error. There was no discovery or learning path.

## Decision

Replace all 14 individual Ollama tool schemas with a single help tool in the default schema. When Jane wants to use a capability, she calls `help('<topic>')` which:

1. Matches the topic against a keyword map (fuzzy, exact, and description-based)
2. Returns structured documentation from `src/agents/manual/*.md` markdown files
3. Injects the matched tool's schema into the current turn via a JSON response channel (`{"discovered": [...], "docs": "..."}`)

The agent loop in `PersonaFSMAgent` rebuilds the Ollama `tools` array on each iteration using `build_tools_schema(persona, extra_tools=discovered_set)`. When a tool is discovered, its schema is available for the next LLM call in the same turn. After the turn completes, `_discovered_tools` is cleared — each turn starts fresh.

Key technical details:

- `build_tools_schema()` gains an optional `extra_tools: set[str] | None` parameter (`None` = backward-compatible: all tools, `set()` = only help)
- Fuzzy matching uses a keyword map in `_HELP_KEYWORDS` with ordered priority (more specific tools checked first to avoid keyword overlap)
- Max 4 tools can be discovered per turn, configurable via `HELP_MAX_DISCOVERED` env var
- Tool execution errors append a “Did you mean to call `help('{tool}')` first?” suggestion
- Manual docs live in `src/agents/manual/*.md` — one file per tool plus `INDEX.md`

## Consequences

### Positive

- Dramatically reduces default tool schema size (~7KB → ~0.5KB per turn)
- Frees context window tokens for chat history and reply generation
- Provides a discoverable, self-documenting interface for tool usage
- Backward compatible — existing code using `build_tools_schema()` with no args works unchanged
- Error recovery: failed tool calls suggest using `help()` instead of returning opaque errors

## **Negative**

- Adds one round-trip (help call → discovery → tool use) before Jane can use any capability
- Keyword matching is order-dependent — new tools need careful keyword review to avoid overlap
- Manual content must be maintained alongside tool definitions

## **Neutral**

- Tool schemas are now ephemeral per turn rather than always-present
- The JSON response channel is a new communication pattern between tool output and agent loop

## **References**

- Implementation Plan
- Design Spec
- `src/agents/tool_modules/help.py` — Help tool implementation
- `src/agents/tool_registry.py` — `build_tools_schema()` with `extra_tools` parameter
- `src/agents/agent_base.py` — Dynamic schema injection in `process_chat()` tool loop
- `src/agents/manual/` — Per-tool documentation files

# ADR-0053: Persona State Serialization

**Status:** Accepted

**Date:** 2026-07-18

## Context

Each simulated persona (e.g., Chrissie, Jeff, Martha) requires tracking complex, dynamic behavioral states: - Chrissie (ADHD tangents, active dance season schedules) - Jeff (bipolar mood states, manic/depressive cycle counts) - Martha (gossip, drink offerings, slurring behavior states)

We need a database storage pattern that is: 1. **Schema-stable:** Adding new behavioral parameters to a persona should not require running SQL migrations (which would involve writing ALTER TABLE or adding new columns to ambient\_state). 2. **Simple:** Leveraging the existing single-file SQLite database configuration. 3. **Encapsulated:** The base Persona logic should abstract get/set operations so subclasses can define clear default states.

## Decision

We will implement a serialized JSON state pattern for personas:

1. **Serialized String Key:** Store each persona's complete state dictionary as a JSON-serialized string in the existing ambient\_state key-value table. The database key follows the format {persona\_name}:state.
2. **Base Persona Class:** Define Persona in src/agents/persona.py containing:
  - get\_state() -> dict[str, Any] which reads the serialized string from ambient\_state, parses it, and updates/merges it into default states.
  - set\_state(state: dict[str, Any]) which updates defaults with the provided dictionary and saves the resulting JSON payload.
  - Robust JSON decoding error handling (graceful fallback to default values on JSON corruption).
3. **Parity Subclasses:** Subclasses (Chrissie, Jeff, Martha) override get\_default\_state() to return their respective default dictionaries.

## Consequences

### Positive

- **Migration-free updates:** Adding new state properties to any persona only requires editing the subclass's get\_default\_state().
- **Encapsulated states:** Keeps persona-specific parameters out of the core FSM or general database helpers.
- **Robustness:** JSON parse errors are caught in python, preventing database read corruption from crashing the runtime.

### Negative

- **No SQL-native query filters:** Querying fields *inside* the state JSON (e.g., finding all personas where tangent\_active = True) cannot be done via simple SQLite WHERE clauses on the key-value store, requiring python-level parsing or SQLite JSON1 extension functions. (Acceptable given the small size and local single-user context).

# ADR-0054: Inter-Persona Relations Social Graph & Asymmetric Closeness

## Status

Accepted

## Context

To make the AI companion system (“Jane”)’s social environment feel dynamic and alive, we need to simulate interactions and relationship evolution among the other personas (Arthur, Lyra, Kate, Chrissie, Jeff, Martha) in the system. The simulation requires: 1. An editable, human-friendly representation of the relationship network (graph). 2. The ability to customize **why** and **how** characters contact each other without hardcoding prompts. 3. Tracking relationship metric values (closeness/affinity) that can grow or decay asymmetrically (e.g. A feels closer to B, but B feels less close to A). 4. Generating memories and propagation of gossip (“telephone game”) across the network that can eventually bubble up to Jane.

## Decision

We implement an event-driven social graph simulation via the following architecture: 1. **Mermaid Graph Definition:** The master social graph is stored as a Mermaid flowchart at `data/personas/inter-persona-relations.mermaid`. Directed edges include relationship types and base probabilities as edge labels (e.g. `A -->|friend | prob:0.25| JG`). 2. **External Directional Prompts:** Prompt files detailing why one persona reaches out to another reside in the persona’s subdirectories: `data/personas/<PERSONA>/relationships/<PERSONA>_to_<OTHER>.md`. These are loaded dynamically at runtime. 3. **Distributed Asymmetric Closeness:** Closeness is stored asymmetrically inside each persona’s JSON state in the `ambient_state` table (under `contacts[other_persona]['closeness']`, defaulting to 0.6 if uninitialized). 4. **Nudge Task & LLM Simulation:** A daily background task (`persona_nudge_task`) parses the Mermaid file, rolls probability checks, runs LLM simulations of dialogue (3-5 turns), evaluates closeness changes, and writes memories via the `persona .experience(event_description)` interface. 5. **Gossip Propagation:** The simulation extracts first-person gossip summaries and writes them to the characters’ memories. These memories are recalled in future cross-persona interactions, allowing gossip to travel between characters and eventually reach Jane. 6. **Async experience generation:** `Persona.experience(...)` is an async coroutine; the nudge sweep awaits it directly. (Superseded 2026-07-20: it was previously a synchronous method wrapping `asyncio.run()`, offloaded to a threadpool via `loop.run_in_executor` to avoid nested-event-loop conflicts — see `docs/specs/2026-07-20-coherence-survey/06-patch-01-experience-async-await.md`.)

## Consequences

- **Maintainability:** The network structure can be edited by changing a single Mermaid file, and relationship prompts can be edited separately as markdown.
- **Asymmetric Dynamics:** The system can model complex relationship patterns where one persona likes/initiates interaction more than the other.
- **Privacy & Context Boundaries:** Each persona remains isolated, querying only their own memories to populate their dialogue, maintaining security boundaries.
- **Test Integrity:** Unit tests execute in a sandboxed temporary directory to keep the real characters’ memory tables pristine.

# ADR-0055: Commentless Image Guessing Game and Direct Command Parsing

## Context

We wanted to introduce a commentless image guessing game mode for the image generation tool (images). When Jonathan runs the command: 1. The image is generated. 2. The image is sent to OpenWebUI/Jane cleanly without the prompt visible in the UI or in the file metadata (to prevent Jane from cheating by reading file headers). 3. The prompt is injected into the local archived copy's PNG metadata (guess\_game\_prompt chunk in data/images/) for long-term records. 4. Jane's FSM prompt context is dynamically injected with system instructions giving her the prompt and telling her not to reveal it, triggering her immediate reply.

Additionally, to bypass routing restrictions in OpenWebUI (which doesn't let us map slash commands directly to mentions like @core), we needed core commands starting with c: (e.g. c:img-guess <PROMPT>) to bypass routing to Jane and execute directly inside CoreFSMAgent without requiring a prepended @core prefix.

## Decision

1. **Direct c: Command Interception:** Modified the command parser in `agent_core.py` to check if a message starts with c: directly. If so, it normalizes it to the registry prefix (c:) and handles it as a core system command instead of routing it to Jane.
2. **Metadata Injection separation:** The images generation tool was updated to support guess\_mode. If guess\_mode is enabled, the code uploads the clean image\_bytes (via a BytesIO buffer) to OpenWebUI, but writes the prompt metadata locally to data/images/ using the Pillow library (PngImagePlugin.PngInfo).
3. **Guess State Storage:** The active guess prompt and image URL are stored in the SQLite ambient\_state table under current\_guess\_prompt and current\_guess\_image.
4. **Context Injection & Auto-Reset:** A prompt block `guess_game.py` reads these values and injects the active game rules and prompt text into Jane's FSM system prompt. After prompt compilation finishes, `build_system_prompt` resets these keys to prevent duplicate game context in subsequent turns.

## Status

Accepted

## Consequences

- Direct c: commands are now fully supported in OpenWebUI without @core.
- The Pillow library was added as a project dependency to handle PNG metadata encoding/decoding.
- Image guessing games can be played commentlessly without letting Jane read file headers to cheat.

# ADR-0056: Partitioned Chat Messages for Multi-Persona Isolation

## Context

The system has a multi-persona architecture: Jane, Dotty, Arthur, Lyra, Kate, Clara, Finn, Iroh, and a dedicated Who.TESTER — all sharing the same PersonaFSMAgent base class (see `m-fsm-base-class-refactoring.md`). As FriendFSMAgent and DottyFSMAgent start processing real chat turns, their messages would mix with Jane's in the global `chat_messages` table without persona isolation.

Key problems: 1. `get_active_thread_messages()` (used for the Active Text Thread) returns messages from ALL personas — Dotty/friend turns would pollute Jane's conversational context. 2. `save_message()`, `recall_session()`, `save_message_sentiment()` have no persona scope — all writes go to the same table. 3. Background tasks (`session_sweep`, `notebook_sync`, CPU throttle) query `chat_messages` globally and need persona-scoped queries. 4. Sentiment tags in `message_sentiments` have the same 1:1 FK coupling issue.

## Decision

1. **Per-persona partitioned tables:** Create `chat_messages_{who}` and `message_sentiments_{who}` for each value in the Who enum, initialized in `init_db()` via the same for who in Who loop as the existing partitioned memory tables (`memories_{who}`, `vec_memories_{who}`, etc.).
2. **CRUD helpers accept who parameter:** `save_message(who=...)`, `get_active_thread_messages(who=...)`, `recall_session(who=...)`, `save_message_sentiment(who=...)` all default to "jane" for backward compatibility in non-routed paths.
3. **Migration on startup:** `migrate_legacy_chat_messages()` copies data from the legacy global `chat_messages` → `chat_messages_jane` and `message_sentiments` → `message_sentiments_jane`, then drops the legacy tables.
4. **Background task routing:** Worker tasks receive who via payload parameter. `session_extraction_sweep_task` and `notebook_sync_task` read from the correct partition table.
5. **CPU throttle queries all partitions:** `_is_throttled()` in `worker.py` queries `MAX(created_at)` across all `chat_messages_{who}` tables.
6. **PersonaFSMAgent generates persona key:** `_generate_response()` extracted from the tool-loop as an override point. `self.persona.value` is passed through the generation pipeline (`replay_as_stream`, `generate_stream`, `process_final_non_stream`, `_persist_and_dispatch_turn`) as the who parameter.

## Status

Accepted

## Consequences

- **Positive:** Persona isolation for chat history; thread queries scoped per-persona; FriendFSMAgent/DottyFSMAgent can run without data pollution with Jane's thread; generation pipeline carries persona identity end-to-end.
- **Negative:** Migration is one-time irreversible (legacy tables dropped after migration); all background tasks that query `chat_messages` must be updated with who routing.
- **Migration:** Existing data in the global `chat_messages` table is migrated to `chat_messages_jane` at startup. `message_sentiments` migrates alongside. Idempotent — safe to re-run.

# ADR-0057: PersonaProvider Protocol for FSM Agent Configuration

## Context

The PersonaFSMAgent base class (`src/agents/agent_base.py`) contains Jane-specific logic leaking through default method implementations:

1. `build_system_prompt()` hardcodes `render_prompt("jane")` and queries `memories_jane` throughout its 900-line block assembly
2. `_evaluate_emotions()` applies a romantic coupling chain (mood inertia, spillover, spike detection, saturation) that assumes a romantic partner relationship
3. `_resolve_context()` uses a memory expansion prompt with “she/her” framing specific to Jane
4. `_generate_response()` hardcodes `OLLAMA_LLM_URL` with no per-persona model/endpoint support
5. Subclasses (`DottyFSMAgent`, `FriendFSMAgent`) override methods and carry their own `_inject_prompt` implementations to work around these assumptions
6. `FriendFSMAgent` remains a stub because wiring it into the pipeline requires understanding which base class methods to safely override

The inheritance-only approach spreads persona-specific code across the base class AND subclasses, making it unclear which methods are safe to override and what assumptions the base class defaults make.

## Decision

Introduce a `PersonaProvider` object per persona that encapsulates all persona-specific decisions. `PersonaFSMAgent` becomes a generic FSM pipeline engine driven by the provider.

Each provider declares:

- **Identity**: who enum value, display name, base prompt template key
- **Prompt blocks**: ordered list of `PromptBlock(name, builder_fn, tier, condition)` — the prompt builder iterates this list instead of 37 hardcoded inline blocks
- **Emotion config**: whether coupling chain runs, whether closeness window is tracked
- **Memory config**: table suffix, parameterized expansion prompt template
- **LLM config**: endpoint URL, model name, context window, temperature range

Concrete implementations:

- `JaneProvider`: full coupling chain, all 37 prompt blocks including kids/cycle/dream/opinions, standard Ollama endpoint
- `DottyProvider`: no coupling chain, subset of blocks (ambient, dotty capabilities, no kids/cycle/opinions), closeness tracking for inter-persona social graph
- `FriendProvider`: no coupling chain, minimal blocks (ambient, memories, output rules), closeness tracking

The file layout moves from inheritance-heavy flat files to a directory-based split:

- `src/personas/` — provider definitions (new)
- `src/agents/` — FSM runtime pipeline (renamed from `src/agent/`)

## Status

Accepted



## Consequences

### Positive

- Adding a new persona requires writing a new Provider, not subclassing the agent engine
- Prompt block selection is declarative (per-provider block list) rather than hardcoded
- Coupling chain, LLM endpoint, and memory tables are all persona-configurable
- FriendFSMAgent gets a real pipeline instead of a stub
- All existing Jane agents continue working via backwards-compatible provider injection

### Negative

- Additional abstraction layer — new contributors must learn the provider pattern
- Prompt block list must be maintained in sync with actual block builders
- `src/agent/` → `src/agents/` rename causes import churn across the project

### Neutral

- Existing `DottyFSMAgent._inject_prompt()` and `FriendFSMAgent.process_chat()` overrides become unnecessary
- `build_system_prompt()` is refactored from 900-line monolith to generic block engine
- ADR-0056 (partitioned chat) already established per-persona table routing — this continues that pattern for config

## References

- Supersedes: the inheritance-based override pattern documented in `womb/m-fsm-base-class-refactoring.md`
- Related: ADR-0056 (Partitioned Chat Messages)
- Related: ADR-0034 (Group Chat Orchestration / Partitioned Memories)
- Related: ADR-0034 (Group Chat Orchestration / Partitioned Memories)
- Status: Implemented (2026-07-20) — see PR `feat/persona-provider-protocol`

## Implementation Addendum (2026-07-20 — A3/A4 pipeline wiring)

The provider-config → pipeline wiring (coherence-survey findings A3/A4) is now implemented; see `docs/specs/2026-07-20-promptblock-engine/`.

- `_generate_response` / `_synthesize_if_empty` read `provider.llm_config.endpoint` and `.model`. **Provider model takes precedence over the caller's** `request.model` — a persona always runs its configured model. All providers currently inherit the `LLMConfig` default (`sweetheart-core:latest`), so every persona routes there, overriding any client / `gemma-primary` / `persona-id` model selection. (Deliberate; maintainer-reconfirmed. Deliberate per-persona models are a future tweak.)
- `_resolve_context` reads `provider.memory_config.expansion_prompt_template` — Jane keeps her rich prompt (now stored on `JaneProvider`); non-Jane personas get the generic template — and `.top_k` for the JCD recall (Jane preserved at `MEMORY_TOP_K`; combined recall keeps `top_k=5`).
- The `PromptBlock` engine itself (A1) is a separate effort — see ADR-0058.

# ADR-0058: Generic PromptBlock Assembly Engine

## Status

Accepted (2026-07-20). **Foundation landed** (Plan 01); section porting tracked in docs/specs/2026-07-20-promptblock-engine/.

## Context

The persona-provider refactor (ADR-0057) introduced a PromptBlock model and had each provider declare a prompt\_blocks list, but **nothing consumed it** (coherence survey finding A1). Jane's entire system prompt was assembled by a monolithic, Jane-hardcoded build\_system\_prompt() (~850 lines, ~37 ordered sections); Dotty/Friend fell through to a template-only default and their declared blocks never rendered. The block model was dead metadata.

## Decision

Introduce a generic engine, src/agents/prompt\_engine.py::assemble\_from\_blocks(blocks, ctx), that iterates a provider's ordered PromptBlock list driven by a single pre-resolved PromptContext:

- **Uniform builders:** each block's build(ctx) -> str | None pulls what it needs from the context (one read pass, no N+1).
- **Two-phase blocks:** build is pure; a separate effect(ctx) performs write-backs and **runs only if the block is included** (post-budget-selection for enrichment) — a correctness improvement over the monolith, which marked rows surfaced even when their block was dropped.
- Tier **StrEnum** (core/top/mid/low) drives placement: CORE appends in declared order; ENRICHMENT\_\* are budget-selected via the existing helpers.select\_enrichment\_blocks. Values match what the selector branches on (resolving a prior mismatch where providers labelled tiers enrichment\_low/enrichment\_mid).
- **Instrumentation preserved:** per-block sizes + flush\_prompt\_stats keep score\_component's prompt-block health signal working; each block runs under its own try/except.

The monolith is replaced via a **strangler migration gated by a golden-snapshot characterization test** (tests/test\_prompt\_golden.py) that freezes Jane's current output for a deterministic state and proves each porting step byte-equivalent. When all sections are ported, JaneProvider.build\_prompt switches to the engine and build\_system\_prompt is deleted.

## Consequences

- "What's in each persona's prompt" becomes readable from the provider's block list, not buried in 850 lines.
- The engine is persona-agnostic — Dotty/Friend get real block-driven prompts (Plan 02+).
- Migration is incremental and low-risk (golden snapshot guardrail), but spans multiple plan waves; the monolith and engine coexist until porting completes.
- Related dead config (llm\_config A3, memory\_config A4) is wired during porting; the provider=None shim (A5) and inert config pruning (A7) are deferred.

## Porting Approach (revision, 2026-07-20)

The strangler migration uses **ctx→args adapters**, not the brainstorm's "refactor every builder to build(ctx)". Ported blocks wrap their unchanged builders — build=lambda ctx:

`builder(ctx.x, ...)` — so the **monolith stays live** during porting (it still calls the builders with their current signatures). Native `build(ctx)` is deferred to the flip wave, or dropped (adapters are fine). `resolve_prompt_context` (in `prompt_builder.py`, keeping the patch surface stable) resolves the read model, growing per wave. **Verification:** each ported block's rendered text is a verbatim substring of `tests/fixtures/jane_prompt_golden.txt` (`tests/test_prompt_shadow.py`); full `engine==monolith` equality is the flip-wave gate. Wave 1 ported: `critical_output_rules`, `voice_directives`, and the three memory blocks.

# ADR-0059: World Events System

**Status:** Accepted

**Date:** 2026-07-22

## Context

The location system (WorldService) provides a static world graph — locations with links and descriptions. To make the world feel alive, we need dynamic state changes: clocks striking, campus closing on weekends, leaves rustling in the breeze. These fall into two categories:

1. **Scheduled events:** Predictable, cron-triggered changes (e.g. “campus closed”, “dawn breaks at 6 AM”)
2. **Passing events:** Probabilistic, transient atmospheric details (e.g. “a gust of wind”, “the scent of fresh bread”) attached to specific locations

Both should be defined in markdown (git-tracked, diffable) and avoid adding a separate game engine or state machine.

## Decision

### Tier 1 — Scheduled Events (WorldEventsService)

- Define events as YAML front matter in `data/world_events/{slug}.md` files with `schedule` (cron + human label), `effect` (type + config), and optional `revert`.
- `parse_cron/is_due` in `world_schedule.py` handle 5-field cron matching.
- `WorldEventsService` polls due events via `poll_due_events()` (called by worker task `poll_world_events`) and executes effects via `execute_event()`.
- Execution audit trail in `world_event_log` table.
- Current effect types: `set_location_flag` (scalable).

### Tier 2 — Passing Events (WorldService.get\_passing\_events())

- Defined inline in location page front matter under `passing_events`: `[]`.
- Each event has `slug`, `description`, `chance` (0.0-1.0), `lifetime` (seconds), optional `condition` (type: `time_of_day/tag/weather`), optional `llm_translate`.
- Rolled seedably on `look_around/move_to`; persisted in `world_passing_events` table; rendered as “**You notice:**” annotations.
- Active events pruned via `prune_expired_events()` on each check.
- LLM translation: First trigger calls Ollama for vivid description, cached for event lifetime.

## Shared Principles

- All SQL is parameterized.
- All paths are config-controlled via `src.config`.
- `WorldService` uses `_db_session()` context manager.
- Passing events use `seedable random.Random` for deterministic tests.
- Unknown condition types log a warning and pass through.

## Consequences

### Positive

- No separate engine needed — markdown + SQLite covers both tiers.
- Git-tracked events: changes are versioned, diffable, revertible.
- LLM translation is opt-in per event — no cost unless desired.
- Deterministic testing via seedable RNG.

### Negative

- Cron parsing is basic (no step values or ranges) — sufficient for current needs.

### Neutral

- weather condition type is a no-op placeholder — ready for future integration.

## References

- Feature Spec
- `src/agents/world_service.py` — `get_passing_events()`, `_evaluate_condition()`, `_llm_translate()`
- `src/agents/world_events_service.py` — `WorldEventsService`
- `src/agents/world_schedule.py` — `CronSchedule`, `parse_cron()`, `is_due()`
- `src/worker/tasks/world_events_task.py` — `poll_world_events`

### Extension: Multi-Location Targeting (2026-07-22)

- `_resolve_target_locations(cfg)` resolves 4 selector types: `location_slug` (single, backward compat), `location_slugs` (explicit list), `location_pattern` (glob), `location_tags` (front-matter tag match). Multiple selectors are OR-union'd and deduplicated.
- Cross-tier passing injection: when a scheduled event has a passing block, `execute_event()` calls `WorldService.trigger_passing_event()` per target location after closing its DB connection (avoids database is locked).
- See Feature Spec

# ADR-0060: Multi-Agent Location-Tied Group Chat

**Status:** Accepted

**Date:** 2026-07-23

## Context

The system supports multiple personas (Jane, Dotty, Arthur, animal friends) that operate independently across locations. There is no mechanism for agents to interact with each other in a shared space. The world system tracks agent positions (`world_presence`), but location changes do not trigger cross-agent conversations or multi-agent message fan-out.

Requests for multi-agent interaction include: Jane and Dotty chatting in the same room, animal friends responding in their home areas, and OWU (Open WebUI) channels publishing agent conversations.

## Decision

We implement a multi-agent group chat system with the following architecture:

1. **Database-backed chat rooms:** `group_chats`, `group_chat_messages`, `group_chat_presence` tables — one per `group_chat.id`, location-tied via `location_slug`.
2. **Referee scoring pipeline:** Three-phase player selection (pre-filter → LLM rank → threshold selection) to determine which agents respond to a user message. Per-agent `referee.yml` configs control participation style. Anti-volley throttling prevents spamming. Recursion safeguard limits nested agent-to-agent responses.
3. **Async GroupChatService class:** Wraps CRUD functions with pipeline methods (`post_message` → `_run_experience` + `_run_referee_pipeline` → `_generate_agent_response`). OWU bridge pushes agent messages to Open WebUI channels.
4. **World integration hooks:** `move_to/look_around` in `WorldService` triggers `_update_group_chat_presence` — agents leave old location chats and join new ones.
5. **Worker cleanup task:** Hourly cron task archives stale/empty chats.
6. **OWU channel bridge:** Bidirectional mapping (`group_chat_id` ↔ `owu_channel_id`) via ambient state; webhook endpoint on `/api/group-chats/owu-webhook` (returns 404 on unknown channel, 500 on message processing failure).

## Message Flow

User → POST `/api/group-chats/{id}/messages`

- `GroupChatService.post_message()`
  - DB: insert message
  - `_run_experience()` (background) — agents process the message
  - `_run_referee_pipeline()` (background)
    - `RefereeService.select_responders()`
      1. `_pre_filter()` — direct mention, topic bonus, gap penalty
      2. `_llm_rank()` — persona-enriched LLM prompt with description + topic signals
      3. `_threshold_selection()` — floor, threshold, max\_responders
    - `_generate_agent_response()` per selected agent
      - `_generate_single_response()` → `ollama_chat()`
      - `post_message()` — DB: insert assistant response
      - `_push_to_owu_channel()` — push to Open WebUI

World `move_to` → `WorldService._update_group_chat_presence()`  
→ `leave_chat(old_location)` → `join_chat(new_location)`

OWU webhook POST /api/group-chats/owu-webhook  
→ get\_chat\_for\_channel() – bidirectional ambient state lookup  
→ post\_message() – inject as user message  
→ referee pipeline triggers agent responses

## Consequences

### Positive

- Agents naturally enter/exit conversations as they move through locations
- Referee scoring prevents all agents from responding to every message
- OWU bridge enables human-in-the-loop via Open WebUI channels
- Clean separation: CRUD functions, service class, world hooks, and REST layer are independent

### Negative

- One extra DB query per move\_to/look\_around to sync presence
- Referee LLM rank stub currently returns uniform scores (placeholder for fine-tuned phase)
- Per-agent referee.yml configs are local-only (not checked into VCS)

### Neutral

- Ambient state used for OWU mapping (no extra table)
- CRUD functions are synchronous; service class wraps them with async

## References

- ADR-0056: Partitioned Chat Messages
- ADR-0059: World Events
- docs/specs/2026-07-23-group-chat/01-design.md

# ADR-0061: Per-Persona Telegram Routing and Caching

**Status:** Accepted

**Date:** 2026-07-24

## Context

The world bot previously proxied all messages through a single Telegram bot account. This broke immersion because regardless of which persona (e.g. Arthur, Lyra) was replying, the messages appeared to come from the same generic world bot. Additionally, the help tool pattern dynamically injects tool schemas only on a per-turn basis; because a new FSM agent instance is created on every FastAPI endpoint request, the discovered tools list was reset between messages. This forced the agent to re-execute `help()` on every single message in a multi-turn task (like playing the HipBone game).

## Decision

We implement the following configuration-driven multi-persona Telegram bot routing and session-scoped tool caching:

1. **Config-Driven Bot Selection** (`data/telegram_bots.json`): Move from static single-token environment variables to a structured JSON file mapping bot tokens to personas. A dedicated core admin bot handles polling and group presence (chat title, pinning). Persona bots handle message delivery, automatically selected via the `get_bot_token` wrapper.
2. **Dynamic Startup Discovery**: On worker boot, the core bot executes `getMe` requests for all configured tokens to dynamically resolve bot user IDs and map them to persona names in `TELEGRAM_BOT_IDS`, eliminating the need for hardcoded token IDs.
3. **Session-Scoped Tool Caching**: Instead of a transient in-memory set, `self._discovered_tools` is persisted to `ambient_state` under the key `{persona}_discovered_tools` on database updates. Every chat request recovers this cache on agent initialization, allowing tools to stay active across multiple turns.
4. **Inactivity Sweep Eviction**: To prevent permanent tool leakage, the daily/inactivity-triggered `session_extraction_sweep_task` automatically clears `{persona}_discovered_tools` entries from the ambient state, resetting the agent back to the base help schema.
5. **Associated Tool Groups**: Introduced `_TOOL_ASSOCIATIONS` in `help.py`. When a primary tool is matched (e.g. `notebook_query`), its associated tools (e.g. `notebook_create_note`) are automatically injected into the cache while keeping the documentation payload focused to avoid token bloat.

## Consequences

### Positive

- High immersion: Arthur, Lyra, and Jane speak through their actual bot accounts.
- Modularity: Persona configuration is fully decoupled from environment variables.
- DevX: The agent can play complex games and query notebooks across multiple messages without repeating the `help()` lookup on every turn.
- Tool boundaries: The sweep ensures the dynamic tools list is eventually cleaned up when the conversation session closes.



## Negative

- A database write is performed when the agent dynamically discovers new tools.
- Requires managing multiple Telegram bot tokens at once.

## Amendment (ADR-0072, 2026-08-01)

Extended to resolve a delivery **destination** in addition to a sender token. `load_bot_config` now parses each persona's optional `chat_id` from `data/telegram_bots.json` (previously discarded; `chat_id` was effective only on core). New `config.get_persona_chat_id(who)` returns a persona's `chat_id`, falling back to `core.chat_id` — enabling the generic invite service to route DM invitations to the right chat. `PersonaBotConfig` retains `token` (required) and gains an optional `chat_id`.

# ADR-0062: SQLite Thread-Local Connection Pool and Batch Ambient Reads

- **Status:** Approved
  - **Date:** 2026-07-24
  - **Authors:** Senior Systems Architect, Backend Specialist, Security Engineer
- 

## Context

Prior to this ADR, every agent chat turn made over 30 individual database queries to read ambient state properties (weather, focus\_session\_active, daily\_plan, etc.), alongside multiple sequential RAG recall operations. Each query opened a new SQLite connection, executed PRAGMA statements (journal\_mode=WAL, foreign\_keys=ON, busy\_timeout=30000), fetched data, and closed the connection.

This approach created significant connection churn on the chat latency hot path.

## Decision

1. **Batch Ambient State Fetching:** Introduced `get_ambient_states(keys: list[str]) -> dict[str, str | None]` in `src/database/db_helpers.py`. `prompt_builder.py` resolves all 34 ambient state keys in a single `SELECT ... WHERE key IN (...)` query per turn.
2. **Thread-Local Connection Pooling:** Added `get_pooled_connection()` and `close_pool()` in `src/database/db_init.py`.
  - Uses `threading.local()` to maintain one long-lived `sqlite3.Connection` per thread.
  - `check_same_thread=False` is set intentionally since each thread holds its own isolated connection.
  - Write serialization and crash safety rely on SQLite WAL mode and `busy_timeout=30000`.
3. **Concurrent RAG Lookups:** Parallelized dual `recall()` calls in `agent_base.py` (Subject.JCD and [Subject.PERSONA, Subject.GENERAL]) using `asyncio.gather`.
4. **Lifecycle Cleanup Hooks:** `close_pool()` is invoked during:
  - FastAPI server shutdown in `src/server/main.py` (lifespan teardown)
  - Background worker completion in `src/worker/tasks/memory/session_sweep.py`

## Consequences

- **Performance:** Reduced per-turn ambient state read connections from 34 to 1, and parallelized RAG memory recall, significantly lowering response latency.
- **Resource Efficiency:** Prevents connection allocation churn during active conversations.
- **Safety:** Automatic transaction rollback in `close_pool()` prevents lock leaks across thread cycles or during process shutdown.

# ADR-0063: Split db\_helpers.py into helpers/ Sub-Package

**Status:** Accepted  
**Date:** 2026-07-28  
**Deciders:** jcd, AI pair (P01 of largest-files refactor)

## Context

src/database/db\_helpers.py had grown to ~780 lines containing unrelated concerns: memory CRUD, tool logging, prompt instrumentation, metric history, Telegram sync, chat/session helpers, and DND logic. This violated the single-responsibility principle and made it costly to understand, test, and extend any individual concern.

A helpers/ sub-package already existed (containing ambient.py, base.py, emotion.py, memories.py, search.py, sync.py, tasks.py) from prior refactors. This ADR documents the completion of that pattern for the remaining db\_helpers.py logic.

## Decision

Extract all domain logic from db\_helpers.py into focused modules within src/database/helpers/:

New Module	Responsibility
helpers/tool_log.py	log_tool_call, has_recent_tool_call, _prune_tool_log, get_tool_usage_stats
helpers/prompt_stats.py	prune_old_stats, log_prompt_blocks, log_condition_check, get_prompt_condition_stats, get_block_size_summary, get_full_prompt_logging_state, set_full_prompt_logging_active, decrement_full_prompt_logging
helpers/metrics.py	insert_metric_snapshots, prune_metric_snapshots, get_metric_snapshots, insert_metric_weekly_summary, get_metric_weekly_summaries, prune_metric_weekly_summaries
helpers/telegram.py	save_telegram_sync_mapping, get_telegram_sync_mapping, get_orphaned_telegram_messages, delete_telegram_sync_mappings_after, telegram_delivery_fingerprint, record_telegram_delivery, get_telegram_delivered_fingerprints, get_telegram_sync_hwm, set_telegram_sync_hwm

New Module	Responsibility
helpers/chat.py	parse_iso, get_active_thread_messages, save_message_sentiment, save_session_summary, get_session_summary, append_pending_share, drain_pending_shares
helpers/dnd.py	is_dnd_active

db\_helpers.py becomes a **pure re-export shim** (217 lines, all imports) that preserves the existing public API without any logic. All callsites continue to import from src.database.db\_helpers unchanged.

## Dependency Rules

- Submodules in helpers/ import from helpers/base.py (leaf) or helpers/ambient.py for shared primitives.
- helpers/dnd.py imports parse\_iso from helpers/chat.py (within the sub-package).
- Submodules **never** import from db\_helpers.py (aggregator) — downward-only dependency.
- db\_helpers.py imports from submodules only; never vice versa.

## Guard Test

tests/test\_db\_helpers\_reexports.py contains one test per extracted module, asserting that each public name is still accessible via db\_helpers. This is a compile-time sentinel: a missing re-export is caught before reaching production.

## Alternatives Considered

- **Merge helpers/ into a single flat file:** Rejected — restores the original problem.
- **Keep db\_helpers.py as-is:** Rejected — the file was the largest in the repo and a cognitive bottleneck.
- **Full package rename:** Deferred — external callsite churn outweighs the benefit right now; the shim pattern preserves backward compatibility.

## Consequences

- db\_helpers.py reduced from ~780 lines to 217 lines (pure re-exports, no logic).
- Each domain concern is independently readable, testable, and evolvable.
- The downward-only dependency rule prevents circular imports.
- Import times are unchanged (the shim re-exports everything at module load).
- Future work (P02–P18 in the largest-files refactor) can safely reference individual helper modules directly where appropriate.

# ADR-0064: Migration Functions Package

**Status:** Accepted

**Date:** 2026-07-28

## Context

`src/database/db_init.py` had grown to 1371 lines, with 15 one-time migration/seed functions inline alongside the DDL in `init_db()`. This made the file hard to navigate and the migration logic hard to verify independently.

## Decision

Extract all migration and seed functions into `src/database/migrations/` as three modules:

- `_util.py` — shared primitives (`add_columns_if_missing`)
- `schema.py` — column additions and table creations
- `backfill.py` — data transformations and seed operations
- `legacy.py` — conn-taking migrations from legacy global tables

`db_init.py` imports each function at the top level and passes them to the existing `_run_migration()` call list unchanged. The idempotency guard (`_run_migration + schema_migrations` table) is preserved, so all migration names and skip semantics remain identical.

Each migration module uses lazy (function-scoped) imports for `get_db_connection` and `add_columns_if_missing` to avoid circular imports with `db_init.py`.

## Consequences

- `db_init.py` drops from 1371 → ~845 lines
- Migration logic is independently testable
- Future migrations just need a new function in the appropriate module + a `_run_migration` call
- The `_run_migration()` name-change / re-import guard in the plan (F811 redefinition errors caught by ruff) prevents accidentally leaving dead wrapper stubs ## Consequences
- `db_init.py` drops from 1371 → ~845 lines
- Migration logic is independently testable
- Future migrations just need a new function in the appropriate module + a `_run_migration` call
- The `_run_migration()` name-change / re-import guard in the plan (F811 redefinition errors caught by ruff) prevents accidentally leaving dead wrapper stubs

## 2026-07-29: `init_db` Decomposition (P03)

Further decomposed `init_db` into three named steps:

- `_create_core_tables(cursor)` — all non-partitioned CREATE TABLE/INDEX statements
- `_create_persona_partitions(cursor)` — unified per-persona DDL (chat partitions, memories, memory\_kinds, vec\_memories, memories\_fts, triggers, FTS backfill) driven by Who enum

- `_run_startup_migrations(conn)` — consolidated migration call list; fixed mid-function `conn.close()` ordering bug; “Schema initialization complete.” is now the final log line
- `init_db` is now a ~15-line orchestrator. Characterization tests (`test_db_schema_snapshot.py`, `test_db_init_idempotent.py`) ensure behavior parity.

# ADR-0066: Server Handler Decomposition and Unified Proxy Pipeline

## Context & Problem

src/server/main.py contained a 200-line chat\_completions endpoint as well as separate proxy handlers for perception (vision, audio) and narrative completions. The endpoint inline-handled OWU edit-divergence hash walking, Telegram user message syncing, SSE streaming wrapping, and photo archiving. In addition, the proxy handlers (\_proxy\_perception, narrative\_chat\_completions, vision\_chat\_completions, audio\_caption\_chat\_completions) duplicated payload checking, Ollama forwarding, chat database persistence, and background memory extraction task enqueueing.

## Decision

1. **Payload Limit Guard:** Extracted require\_payload\_under\_limit(raw\_body) to enforce the 50MB payload limit across chat\_completions, vision, and audio proxy endpoints.
2. **Edit Divergence & Sync Helpers:** Extracted \_prune\_orphaned\_telegram\_messages for OWU history prefix-hash walking and \_sync\_user\_message for Telegram message & photo delivery.
3. **De-closed Stream Wrapper:** Lifted sync\_stream\_wrapper to a top-level async generator function taking explicit parameters (is\_sync, target\_telegram\_id, sync\_chat\_id, messages, stop\_event, typing\_task).
4. **Unified Perception Proxy:** Parameterized \_proxy\_perception(body\_json, source\_tag, \*, persist\_empty=False, chat\_name=None) to back vision, audio caption, and narrative completion endpoints with a single persistence & memory-extraction path.

## Consequences

- chat\_completions is now a readable step-by-step orchestrator function.
- All perception and narrative completion requests use the unified \_proxy\_perception path.
- 100% test coverage maintained via tests/test\_server\_handlers.py.

# ADR-0067: PersonaFSMAgent Collaborators and Process Chat Decomposition

## Context & Problem

PersonaFSMAgent in `src/agents/agent_base.py` had a 355-line `process_chat` method (CC ~30) and coupled four distinct responsibilities into a single class: LLM transport configuration, memory context resolution, emotional sentiment evaluation, and tool-execution loop orchestration. Furthermore, per-turn state was threaded via mutated instance attributes (`self._score_cache`, `self._discovered_tools`), introducing concurrency risks.

## Decision

1. **Explicit Turn Context:** Introduced `TurnContext` dataclass in `src/agents/turn_context.py` to hold per-turn quality scores, discovered tool schemas, LLM provider settings, and generation parameters explicitly.
2. **Decomposed process\_chat Orchestrator:** Extracted `_intercept_commands` (routing slash commands and core `c: commands` through shared SSE stream generators), `_build_ollama_messages`, `_run_tool_loop` (preserving the `final_message` handoff), and `_dispatch_response`.
3. **Extracted Collaborators:**
  - `ContextResolver` (`src/agents/context_resolver.py`): Performs hybrid vector + FTS5 search (RRF fusion), query expansion, ambient context gathering, and search quality scoring.
  - `EmotionEvaluator` (`src/agents/emotion_evaluator.py`): Evaluates user sentiment via LLM, applies coupling chains (fatigue, rate-limiting, spillover, positive coupling, spike checks, saturation), and executes atomic RMW DB persistence.
  - `LLMClient` (`src/agents/llm_client.py`): Resolves endpoint, context length, and token prediction limits, and handles chat completion HTTP posts.

## Consequences

- `process_chat` is reduced from 355 lines to a ~60 line step orchestrator.
- Collaborators are independently unit-testable (`test_emotion_evaluator.py`, `test_process_chat.py`, `test_prompt_blocks_*.py`).
- No mutated instance state shared across turns.



# ADR-0068: Telegram Client Decomposition and State Isolation

## Context & Problem

src/agents/telegram\_client.py (1038 lines) had multiple structural pain points: 1. State management for offset, pending messages, and sleep replies was duplicated across ~8 raw open(STATE\_FILE) read/write sites. 2. The 340-line telegram\_polling\_loop interleaved HTTP long-polling, update classification (stickers, reply-quotes, voice, audio, photos, documents, test message filters), group chat dispatch, and DND backlog processing. 3. handle\_incoming\_telegram\_message (213 lines) coupled FSM agent execution, response delivery (voice/photo/text), delivery ledger recording, and OpenWebUI chat history synchronization.

## Decision

1. TelegramState (src/agents/telegram/state.py): Dataclass encapsulating last\_update\_id, pending\_messages, and last\_sleep\_reply\_time with atomic load() and save() operations and pending message normalization.
2. PinnedMessageClient (src/agents/telegram/pinned.py): Encapsulates two-tier pinned message retrieval and presence formatting over Pyrogram userbot with Bot API fallback.
3. parse\_telegram\_update (src/agents/telegram/parse.py): Pure function classifying updates into ParsedMessage (stickers, reply-quotes, voice STT, attached audio clips, vision photos, text documents).
4. OpenWebUISync (src/agents/telegram/owu\_sync.py): Owns OpenWebUI chat history fetch, turn append (sync\_turn), and update push.
5. **Step Orchestration:** handle\_incoming\_telegram\_message split into \_run\_agent, \_deliver\_reply (with voice temp file cleanup in a finally block), and OpenWebUISync.sync\_turn.

## Consequences

- telegram\_client.py is reduced from 1038 lines to under 670 lines.
- TelegramState, PinnedMessageClient, parse\_telegram\_update, and OpenWebUISync have dedicated unit tests (test\_telegram\_state.py, test\_telegram\_parse.py).
- All STATE\_FILE JSON I/O is centralized and safe against missing/corrupt files.
- Temp-file cleanup during voice reply delivery runs reliably in finally blocks.

# ADR-0069: Speech Service Decomposition and Data-Driven Audio Pipelines

## Context & Problem

openedai-speech/speech.py contained hardcoded voice modulation logic, duplicated format-to-media-type switch statements, separate unaligned emoji regexes, three copies of data directory resolution, and a monolith generate\_speech function handling cache lookups, Piper subprocess execution, and XTTS threaded audio generation.

## Decision

1. `src/text_utils.py strip_emoji`: Created a unified emoji superset regex covering pictographs, dingbats, variation selectors, and joiners; imported into `speech.py`.
2. `get_data_dir()`: Single resolver honoring `SWEETHEART_DATA_DIR` environment variable across all database and cache paths.
3. **FORMATS Table**: Data-driven lookup table mapping format keys (mp3, opus, aac, flac, wav, pcm) to media types and FFmpeg arguments.
4. **TTS\_SETTINGS\_SCHEMA & VOICE\_MODULATION\_THRESHOLDS**: Declarative schema for settings loading and named thresholds for emotional pitch/speed modulation.
5. **Engine Split**: Extracted `synthesize_piper` and `synthesize_xtts` generators; `generate_speech` acts purely as request validation, cache resolution, and generator dispatch orchestration.

## Consequences

- `speech.py` format switches and settings parsing are fully data-driven.
- Piper and XTTS synthesis logic are isolated in independent async generator functions.
- Shared `strip_emoji` prevents audio pronunciation degradation across TTS engines.
- `make check` and all unit tests in `test_speech_helpers.py` pass cleanly.

# ADR-0070: Extract Task Outputs Script Refactoring and Production De-forking

## Context & Problem

scripts/extract\_task\_outputs.py (1261 lines) was an offline backfill script that re-implemented memory extraction sweeps, sentiment tagging prompts, and LLM JSON parsing. Because the prompts and rules were duplicated from src/worker/tasks/memory/session\_sweep.py and scripts/backfill\_sentiment\_tags.py, offline replay runs risked silently diverging from live production worker behavior.

## Decision

1. **Injectable Extraction Sink:** Extracted run\_extraction\_passes into src/worker/tasks/memory/session\_sweep.py with an injectable sink\_fn parameter. Live worker calls \_db\_sink (writing to SQLite via remember()); replay script calls \_replay\_sink (writing replay JSON entries).
2. **Consolidated LLM JSON Repair:** Created src/llm\_json.py (parse\_llm\_json\_array, parse\_llm\_json\_object) with markdown fence stripping and truncated array repair. Used across scripts and referee\_service.py.
3. **Shared Context & Tag Vocab:** Imported \_JCD\_EXTRACTION\_CONTEXT, \_is\_valid\_reflection, \_query\_llm, and VALID\_TAGS directly from production modules.
4. **Data-Driven Schedules:** Derived \_SYNTH\_SCHEDULE and \_STATEFUL\_SCHEDULE from src.config.TASK\_TARGET\_TIMES and created walk\_days iterator helper.

## Consequences

- Offline extraction replay is guaranteed to match production memory extraction behavior.
- Duplicate JSON parsing and tag definitions across 4 modules are eliminated.
- Golden --dry-run snapshot tests verify regression freedom.

# ADR-0071: Typed Row Models (Memories Domain First) and row\_to\_memory Mapper

## Context & Problem

Database read helpers (`get_all_memories`, `search_persona_memories`) previously returned raw `sqlite3.Row` objects or ad-hoc `dict[str, Any]` structures constructed by hand-written loops. This caused several issues: 1. Column renames or typos (`m["content"]` vs `m["memory"]`) failed silently at runtime with `None` or `KeyError`. 2. JSON parsing for column fields like `metadata` was duplicated across multiple query helpers. 3. Callers lacked static type hints (`list[dict[str, Any]]` instead of `list[Memory]`).

## Decision

1. **Typed Memory Model:** Created `src/database/models.py` defining a dict-compatible `@dataclass Memory(Mapping[str, Any])`.
2. **Dict Compatibility:** Implemented `__getitem__`, `get()`, `__iter__`, `__len__`, and a `@property` `def content` alias so legacy callers using `m["memory"]`, `m["content"]`, or `.get("kind")` work without breaking changes.
3. **Unified Mapper:** Created `row_to_memory(row)` to normalize column access and safely decode metadata JSON strings.
4. **Single-Domain Scope:** Scoped typed models to the memories domain first, following council recommendation to keep blast radius contained.

## Consequences

- `get_all_memories` and `search_persona_memories` now return `list[Memory]`.
- Key callers in `nightly_dream.py` and `synthesizer.py` have been migrated to clean static attribute access (`m.memory`, `m.kind`, `m.id`).
- Renamed or missing DB columns raise immediate `KeyError` exceptions at mapping time instead of producing silent `None` values downstream.

# ADR-0072: Generic Invite Service (/invite + world(action="invite"))

## Context & Problem

The HipBone integration shipped a one-off /invite command and a world(action="invite") tool branch that were hard-wired to HipBone game setup and to a single recipient (target). This coupling made the invitation feature unusable outside HipBone and duplicated invite-sending logic across core\_bot.py and world.py. Two issues made delivery unreliable: 1. world.py passed the **persona name** as the chat\_id when calling core\_bot.\_send\_message, so DMs could not be delivered. 2. The config loader discarded each persona's chat\_id, so there was no reliable way to resolve a persona to a Telegram chat.

## Decision

1. **Extract a stateless generic invite service** (src/agents/invite\_service.py) with send\_invites(), a pure build\_invite\_text() builder, and \_escape\_markdown() for Telegram Markdown escaping. No DB, no env, no world references — callers resolve the location and pass it in as a string.
2. **Lazy imports for circular-import safety:** the service imports config at module top but defers the core\_bot.\_send\_message import to call time. The messenger is injectable (messenger=) so tests mock a single seam; it defaults to core\_bot.\_send\_message.
3. **Generic multi-recipient parsing:** /invite <personas...> -- <message>. Everything before -- is the persona allowlist set; everything after is an optional free-text message. The world tool takes personas: list[str] and an optional message instead of a single target.
4. **Fix config chat\_id resolution:** load\_bot\_config now parses each persona's chat\_id, and a new config.get\_persona\_chat\_id() accessor returns the persona's chat\_id, falling back to core.chat\_id.
5. **Per-target result strings:** the service returns one result per target (sent / unknown persona / missing token+chat\_id), enabling partial-success reporting to the caller.

## Consequences

- /invite and world(action="invite") now share one stateless code path.
- Persona names are validated against the config allowlist; no DMs are sent to managers. Both schema and tool updated to the multi-recipient form.
- The HipBone-flavored invitation text is replaced by a generic "is at <location> and would like you to join them" message.
- Recipient inbox/accept-decline wiring and HipBone game-start coupling are explicitly out of scope and deferred.

## Status

Accepted.

# ADR-0073: Stream All Logs to Grafana Loki (via python-logging-loki-v2)

**Status:** Accepted

**Date:** 2026-08-03

## Context & Problem

Phase 1 of structured logging (ef473582) gave the worker/server/dashboard a JSON envelope via `log_event()`, but logs still landed only in local files and `stderr`. There was no centralized, queryable view of agent behavior, and memory writes — the core artifact of persona activity — were only queryable through SQLite or the dashboard's in-app log pages. Operational questions ("what did Jane record in the last hour?", "which persona failed which task?") required manual DB queries or log-file greps.

Options considered: 1. `python-logging-loki` (GreyZmeem) — abandoned since 2019; unsafe to adopt. 2. `loki-logger-handler` (xente) — maintained but less proven in production. 3. `python-logging-loki-v2` (RomanR-dev) — actively maintained, v1.1.1 (Dec 2025), Python 3.13 compatible, documented by Grafana, ships a `LokiQueueHandler` with a background listener thread and supports the Loki v1 push protocol. **Selected.**

## Decision

1. **Stream ALL logs to Loki; memories are just tagged log lines.** A `LokiQueueHandler` is lazily attached once to the root logger at process startup. Because it lives on the root logger, Python's logging propagation delivers every record from all child loggers automatically. `log_event()` (already emitting single-line JSON as its message) becomes the universal shipping path.
2. **Label discipline — low cardinality only.** Static label `service="sweetheart-core"` plus per-record dynamic labels from `extra={"tags": ...}` (who, kind). High-cardinality identifiers (`memory_id`, `task_id`, timestamps) stay in the JSON **body**, extracted at query time via `LogQL | json`. This avoids exploding Loki's label index.
3. `log_event(..., tags=...)` **thread-through.** `log_event()` gained an optional `tags: dict | None = None` kwarg forwarded as `extra={"tags": ...}` on the underlying logger.`log()` call, and **excluded** from the JSON body payload.
4. **Lazy, cached, non-blocking setup.** `setup_loki_logging()` in `src/logging_utils.py`:
  - Lazily imports `src.config` and `logging_loki` (`config` imports `log_event`, so a top-level import would be circular).
  - Returns `None` when `LOKI_ENABLED=0`.
  - Uses `LokiQueueHandler(Queue(-1), url=config.LOKI_URL, version="1", tags={"service": "sweetheart-core"})` with formatter `%(message)s` so the Loki line is exactly the JSON payload. `LokiQueueHandler` auto-starts a background `QueueListener` thread, keeping the app thread non-blocking.
  - Caches a module-level `_loki_handler` singleton; `LOKI_ENABLED/LOKI_URL` are configurable via env (default `http://host.docker.internal:3100/loki/api/v1/push` — workers run on the host while Loki runs in Docker, so `host.docker.internal`, not `127.0.0.1`).
5. **Memory writes ship tagged events.** `add_memory()` in `src/database/helpers/memories.py` emits a canonical flat event=`"memory"` line via the shared `log_memory()` emitter (`src/logging_utils.py`) with the memory text (message), kinds array, bucket, source,

and `created_at` in the body and `who/kind` as labels. `Who.TESTER` is skipped (test-persona data must not pollute the log stream). `_loki_who()` normalizes labels.

6. **Idempotent backfill.** `scripts/backfill_memories_loki.py` ships existing memories\_{who} rows, gated by a per-persona watermark cursor stored in `ambient_state` under `loki_backfill_cursor_{who}` (highest `created_at` already shipped). Re-runs ship only rows strictly newer than the watermark; `--reset` zeroes cursors to re-ship everything. FTS/aux memory tables and `memories_tester` are excluded.

## Consequences

### Positive

- One unified observability path: every `log_event` (tasks, memory writes, green-terminal errors, queue failures) lands in Loki and is queryable with LogQL and dashboards.
- Memory writes are first-class log events with stable `who/kind` labels.
- Backfill is safe to re-run and per-persona incremental (no double-shipping).
- Failure isolation: a down Loki never crashes or blocks the app — the `QueueListener` runs on its own thread and the handler swallows errors via `handleError`.

### Negative

- `memory_id` and full memory text are visible in Loki; for multi-tenant or privacy-sensitive deployments this requires Loki access control or shipping redaction config.
- Promtail is no longer needed for the app's own logs (still used for Docker / OpenWebUI container logs).
- `python-logging-loki-v2` is a third-party dependency that must be kept up to date; its `QueueListener` thread is created per handler instance.

### Neutral

- Loki lines are `log_event` envelopes whose `message` field holds the domain payload, so LogQL must query e.g. `message>event`, not top-level event.
- Enabling Loki adds outbound HTTP traffic per log record (batched by the queue thread), which is negligible for the app's log volume.

## References

- `python-logging-loki-v2` on PyPI: <https://pypi.org/project/python-logging-loki-v2/>
- Grafana Loki push API docs: <https://grafana.com/docs/loki/latest/reference/loki-http-api/>
- Design: `docs/specs/2026-08-03-stream-all-logs-to-loki/01-design.md`
- Plan: `docs/specs/2026-08-03-stream-all-logs-to-loki/02-plan-01.md`
- Related: ADR for JSON structured logging infrastructure (Phase 1, ef473582), ADR-0063 (helpers sub-package split)

# ADR-0074: Hybrid Opinion Search (Gated Snippets Upgrade)

**Status:** Accepted

**Date:** 2026-08-05

## Context

Gated opinion snippets matching (introduced in ADR-0028) relied on a hardcoded keyword dictionary (`_OPINION_KEYWORDS`) mapping terms to markdown file names. This approach had several limitations: 1. **Fragility**: Stance injection failed if users phrased queries using synonyms or context not in the hardcoded list (e.g. asking “Do you think you are alive?” instead of “conscious”). 2. **Maintenance overhead**: Adding new opinion snippets required manually updating Python keyword dictionaries. 3. **Redundant I/O**: The system read files directly from disk on the request path for every matching turn.

We needed to upgrade the retrieval logic to be semantic and context-aware, while preserving the ability for developers to edit persona opinions in git-tracked `.md` files.

## Decision

We upgraded the gated opinion matching pipeline to use dynamic SQLite hybrid search:

1. **Memories Integration**: opinion snippets are dynamically synchronized from disk into the existing `memories_{who}` table under a dedicated "opinions" bucket. This allows us to reuse the existing `vec0` (sqlite-vec) and `fts5` virtual table indexing without schema migrations.
2. **Bootstrapped State Tracking**: To prevent redundant embedding calls to the local Ollama server, we store file modification times (`mtime`) in the memory's metadata column. At startup, the system bootstraps the modification-time tracking cache from the database.
3. **Hybrid Search Querying**: The topic query (last 2 user messages) is embedded and matched via `hybrid_search_memories()` in the "opinions" bucket.
4. **Relevance Guarding**: Matches are filtered using a double relevance guard:
  - FTS5 keyword hits are kept unconditionally.
  - Vector-only matches are kept if their L2 distance is  $< 0.8$ .

## Consequences

**Positive:** - **Semantic matching**: Opinions are successfully loaded when the user discusses related topics using synonyms, indirect questions, or semantic metaphors. - **Dynamic hot-reloading**: Modifying or adding opinion files on disk is detected via modification time checks and instantly synchronized to the database. - **Zero code modifications**: Adding new opinions only requires creating `.md` files in the persona's `opinions/` directory; no Python dictionaries need to be modified.

**Negative:** - Generating the topic query vector adds an `async get_embedding` network call to the prompt construction pipeline (partially cached via LRU cache).

## Related ADRs

- ADR-0002: Adopt Hybrid Vector and Keyword Search — The core search pipeline reused here.



- ADR-0028: Gated Prompt Blocks — The original keyword-based implementation superseded by this decision record.
- ADR-0043: Persona-Scoped Memory Writes — Multi-persona database layout used for indexing.

# ADR-0075: OpenWebUI Mid-History Reconcile & Repair

**Status:** Accepted

**Date:** 2026-08-05

## Context

The synchronization daemon (`reconcile_sync`) is responsible for resolving conversation history drift between the database and the OpenWebUI chat. The previous trailing-append sync strategy (defined in ADR-0042) only matched linear tails: - If a user deleted, inserted, or edited a historical message, the active branch diverged, leaving “mid-history holes”. - The system logged warning records for these drift scenarios but left them unhealed because graph-structured splicing is highly complex and error-prone.

We needed a robust, clean mechanism to repair mid-history drift to ensure OpenWebUI matches the canonical SQLite database log exactly, without corrupting the active chat interface or breaking browser message references.

## Decision

We adopted a **Truncate-at-Divergence and Rebuild Tail** pattern for OpenWebUI chat reconciliation:

1. **First-Point Scanning:** The system scans the database flat message list and OpenWebUI's active branch in parallel. It detects the first index *i* of mismatch based on message role or whitespace-normalized content.
2. **obsolete Tail Truncation:** Truncates OpenWebUI's active branch starting from *i*. All matching node IDs from the truncated tail are popped and deleted from the OpenWebUI history node dictionary to prevent graph memory leaks.
3. **Graph Connection Resolution:** Removes the deleted child links from the last matched parent node's `childrenIds` list.
4. **Tail Rebuilding:** Sequentially rebuilds all remaining database messages from the divergence index forward. It generates new UUID message nodes with proper parent-child links, inserts them into the history index, and appends them to the active branch.
5. **Tip Update:** Sets `chat["history"]["currentId"]` to the final rebuilt tail node's ID (or `None` if fully truncated to empty).
6. **Dry-Run Simulation:** Employs `copy.deepcopy` to simulate the reconciliation, returning exact modification counts without side effects.

## Consequences

**Positive:** - **Drift Healing:** Automatically corrects edits, deletions, insertions, and model shifts. - **UI Stability:** Preserves original node IDs and browser scroll positions for the matching prefix (older part of the chat). - **Simplicity:** Pure mutation logic (`reconcile_owu_chat`) is completely decoupled from async network calls, allowing fast, predictable unit testing.

**Negative:** - Rebuilding from a divergence point discards alternative branches that the user might have manually forked on the OpenWebUI UI at or after that divergence point (however, because the database is the canonical flat sequence, this is the intended result).

## Related ADRs

- ADR-0042: DB-Authoritative Telegram <-> OWU Sync Reconciliation — The baseline reconciliation sweep superseded by this decision record.

# ADR-0076: Telemetry Write Buffering to Prevent SQLite Lock Contention

**Status:** Accepted

**Date:** 2026-08-05

## Context

The sweetheart companion app logs prompt stats (prompt\_block\_log table) and condition check frequency stats (prompt\_condition\_stats table) synchronously inside log\_prompt\_blocks() and log\_condition\_check() on the request hot path. - These writes execute synchronously inside FastAPI process turns. - In WAL mode, SQLite allows concurrent reads, but writes are serialized at the OS level. - When background worker tasks write to the database (e.g. task queue updates, feeds) at the same time the main turn writes telemetry, SQLITE\_BUSY exceptions can occur due to write lock contention. - Telemetry stats are non-critical instrumentation metrics and can be safely buffered in memory and flushed asynchronously.

## Decision

We implemented an **In-Memory Telemetry Write Buffer & Daemon Flusher**:

1. **Thread-Safe Memory Buffers:** We introduced module-level buffer collections protected by a standard threading.Lock inside src/database/helpers/prompt\_stats.py:
  - `_PENDING_PROMPT_BLOCKS`: list[tuple] — accumulates block size logs.
  - `_PENDING_CONDITIONS`: dict[str, dict] — accumulates condition check counts.
2. **Synchronous Client-Side Append:** We refactored log\_prompt\_blocks() and log\_condition\_check() to perform sub-microsecond memory-only appends under the lock, eliminating synchronous database writes from the request hot path.
3. **Lazy Daemon Flusher Thread:** We introduced a background thread (PromptTelemetryFlusher) configured with daemon=True that is started lazily on the first log write to avoid thread initialization overhead during module import.
4. **Flusher Loop:** The background thread sleeps on a threading.Event wait on a 15-second timeout, then wakes up to execute flush\_to\_db().
5. **Batch Upserts:** flush\_to\_db() acquires the lock, drains both buffers, and executes single-transaction batch operations to SQLite (conn.executemany for blocks, and consolidated upserts for conditions).
6. **Live Read Flushes:** To ensure real-time accuracy, read endpoints (get\_prompt\_condition\_stats(), get\_block\_size\_summary(), and \_score\_prompt\_blocks()) are refactored to trigger a synchronous flush\_to\_db() before querying the database.
7. **Clean Teardown Hook:** We registered shutdown\_telemetry\_flusher() with atexit to flush remaining records synchronously during standard application exit, preventing telemetry loss on restarts.

## Consequences

**Positive:** - **Zero Request-Path Write Overhead:** Completely removes database disk I/O and transaction locking latency from the main FastAPI server turn execution path. - **Lock Contention Elimination:** Prevents telemetry-driven database locks from competing with critical memory/state insertions. - **Thread Safety:** Decoupled from the asyncio loop lifecycle, allowing thread-safe execution across different worker thread pools and scripts.

**Negative: - Crash Loss Window:** Telemetry generated within a 15-second window prior to an abrupt hard crash (such as SIGKILL or power outage) will be lost. This is acceptable for instrumentation logs.

# ADR-0077: ScoreComponent Query Consolidation & TTL Caching

**Status:** Accepted

**Date:** 2026-08-05

## Context

The sweetheart companion app computes runtime health metrics (ScoreComponent) once per conversation turn on the request hot path, as well as during periodic metrics rollups and dashboard updates. - Calculating these metrics previously executed **17 sequential SELECT queries** on SQLite across various helper sub-scorers (`_score_memory`, `_score_task_queue`, `_score_ambient`, etc.). - Performing 17 queries on every chatbot turn adds significant disk/query plan compilation latency. - Because health metrics do not change significantly from second to second during normal user chats, caching the resulting dataclass snapshot with a short Time-To-Live (TTL) is highly efficient and safe.

## Decision

We implemented **ScoreComponent Query Consolidation & In-Memory Process-Local TTL Caching**:

1. **SQL Query Consolidation (17 queries → 9 queries)**: We refactored sequential SQL calls inside `src/score_component.py` into unified compound aggregates:
  - **Memory Sub-Scorer**: Merged individual fetches (vector counts, orphaned vector count, oldest/newest timestamps) into **2 queries** utilising sub-selects and `MAX/MIN(created_at)`.
  - **Task Queue Sub-Scorer**: Merged pending, failed, stuck, and last completed queries into a **single aggregate query** using conditional `SUM(CASE WHEN ...)` constructs.
  - **Ambient Sub-Scorer**: Consolidated separate key fetches (weather, calendar events, pending shares) into a **single matching IN query**.
2. **In-Memory Caching**: We added global variables (`_CACHE_SCORE`, `_CACHE_SCORE_TIME`) protected by a standard `threading.Lock(_SCORE_CACHE_LOCK)` inside `src/score_component.py`.
3. **Cache Validation & deepcopy**: We refactored `from_db(cls, force_refresh: bool = False)` to check expiration against `SCORE_CACHE_TTL_SECONDS` (default: 30 seconds). Returns a `copy.deepcopy()` of the cache to prevent downstream mutable attribute updates from polluting the cached instance.
4. **Cache Reset Fixture**: Configured `conftest.clean_database_tables` autouse fixtures to clear the cached metrics between tests, preserving test hermeticity.

## Consequences

**Positive:** - **I/O Reduction**: Reduces sequential SELECT database calls by nearly half on a cache miss, and down to zero calls on a cache hit. - **Request Path Acceleration**: Dramatically accelerates chatbot turns by bypassing redundant database queries when requests arrive close together. - **Thread Safety**: Safely handles concurrent accesses across different worker threads via `threading.Lock`.

**Negative:** - **Staleness**: Health metrics shown in the dashboard or read by turn logging can be up to 30 seconds old. This is completely acceptable for general telemetry. Calling `from_db(force_refresh=True)` is available to bypass caching when real-time values are strictly required.

# ADR-0078: Dashboard FastAPI Lifespan Migration

**Status:** Accepted

**Date:** 2026-08-05

## Context

FastAPI's `@app.on_event("startup")` and `@app.on_event("shutdown")` decorators have been deprecated in favor of using lifespan context managers (`lifespan=`). - Using deprecated decorators causes verbose `DeprecationWarnings` to output during server initialization and test execution. - While the main API server (`src/server/main.py`) was already migrated to the lifespan context manager, the admin dashboard (`src/dashboard/main.py`) still relied on `@app.on_event("startup")` to execute its database schema checks (`migrate_feeds_add_eldest_first`) and RAG history backpopulation (`backpopulate_eval_history_if_needed`).

## Decision

We migrated `src/dashboard/main.py` from the deprecated event handlers to the modern lifespan context manager:

1. **Lifespan Manager:** Introduced an async lifespan context manager using `contextlib.asynccontextmanager`:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
 # Startup tasks
 migrate_feeds_add_eldest_first()
 backpopulate_eval_history_if_needed()
 yield
 # Shutdown tasks (logs shutdown event)
 log_event(logger, "info", "Dashboard service shutting down")
```

2. **App Instantiation:** Registered the context manager via `app = FastAPI(title="Jane V2 Dashboard", lifespan=lifespan)`.
3. **Event Handler Removal:** Removed the deprecated `@app.on_event("startup")` decorator and function.

## Consequences

**Positive:** - **Code Cleanliness:** Eliminates deprecation warnings during testing and runtime startup. - **Consistency:** Dashboard event handling style is now fully aligned with the API server lifecycle setup.

# ADR-0079: Choose Your Own Task Agency System

**Status:** Accepted

**Date:** 2026-08-05

## Context

Jane's background routine activities (gardening, stargazing, letters to friends, intellectual research) were previously scheduled on rigid, independent cron intervals in the background worker queue. This architecture presented several limitations: - **Lack of Autonomy:** Activities executed unconditionally on raw timers, ignoring Jane's calendar schedule, menstrual energy, or the current weather (e.g. stargazing in heavy rain, or gardening at 3 AM). - **Hard-coded Rules:** Time and environment gates were scattered across multiple worker tasks, making it difficult to scale and add new activities without introducing complex conditional checks. - **Visual Disconnect:** Downstream Telegram notifications were generated automatically without capturing Jane's dynamic internal thought process or reasoning for choosing an action.

## Decision

We replaced the rigid cron-based background scheduling with a modular, choice-based **Agency System**:

1. **Modular Tasks:** Replaced if-else structures with a modular Command/Strategy pattern. Tasks are placed in separate files under `src/worker/agency/tasks/` inheriting from `BaseAgencyTask` and registering themselves to a central `AGENCY_TASKS` registry.
2. **Context Compilation:** Once an hour, at the end of the `ambient_update` task, the orchestrator compiles a structured `AgencyContext` containing weather description/temp, quiet hours (DND), calendar plans, mood narrative, current hobby, and batch-retrieved cooldown timestamps.
3. **LLM Choice Loop:** Eligible tasks are filtered via `is_eligible(context)`. If multiple choices are eligible, the orchestrator queries the LLM (`sweetheart-core:latest`) with a JSON formatting constraint. Jane is presented with her current context and eligible actions, and returns her selection and first-person reasoning text.
4. **Execution & Isolation:** The chosen task is executed inline under a `try/except` guard, isolating execution crashes from the hourly worker process. Cooldown timestamps (`last_agency_task_run_<name>`) are updated regardless of success to prevent loop repeats.
5. **Circadian Fatigue & Sleep Hooks:** Implemented new core tasks from scratch: `nap` (recruits energy) and `work` (sets working status and drains stamina).
6. **Dynamic Telegram Surfacing:** If the selected task has `notify_on_choice = True`, Jane's first-person reasoning text is broadcasted directly to Telegram as a proactive message.

## Consequences

**Positive:** - **Increased Agency:** Jane now dynamically evaluates her context (e.g., weather, time, schedule, mood) and chooses her next hourly activity, making her routine feel organic and alive. - **High Scalability:** Adding a new task only requires creating a single class file under `src/worker/agency/tasks/` and importing it in `src/worker/agency/__init__.py`. - **Fault Tolerance:** Task execution errors are caught at the orchestrator boundary, protecting the hourly update loop. - **Circadian Integration:** Standardizes stamina/fatigue changes and quiet/working status updates inside the task commands themselves.